Intel 8086 ile Mikroişlemci Programlamaya Giriş

Şadi Çağatay Öztürk

Dr. Oğuz Ergin

© 2026

Intel 8086 ile Mikroişlemci Programlamaya Giriş
Şadi Çağatay Öztürk ve Dr. Oğuz Ergin
İlk Baskı, 2026

Tüm hakları saklıdır.

Önsöz

Bu kitap, Intel 8086 mikroişlemcisi üzerinden çevirici dili programlamayı öğretmeyi amaçlamaktadır. Bilgisayar mühendisliği ve elektrik-elektronik mühendisliği öğrencilerine yönelik hazırlanan bu kaynak, soyut kavramları somut ve çalışan kod örnekleriyle buluşturmayı hedeflemektedir.

8086 mikroişlemcisi, 1978 yılında Intel tarafından piyasaya sürülmüş ve bugün kullandığımız x86 mimarisinin temelini oluşturmuştur. Modern masaüstü işlemciler geriye dönük uyumluluk nedeniyle 8086 komut kümesini desteklemeye devam etmektedir; dolayısıyla 8086'yı anlamak, çağdaş işlemcilerin işleyişini kavramanın en sağlam yoludur.

Kitap, sayı sistemleri ve bilgisayar mimarisi temellerinden başlayarak veri taşıma, aritmetik-mantık, denetim akışı, dizgi işleme ve kesme yönetimi konularını adım adım ele almaktadır. Son bölümde ise önceki bölümlerde öğrenilen tüm kavramları bir araya getiren ve disket üzerinde çalışan basit bir işletim sistemi projesi sunulmaktadır. Bu proje; önyükleme, menü, hesap makinası, metin düzenleyici ve seri iletişim yazılımlarından oluşmakta ve öğrenciye gerçek bir sistem programlama deneyimi kazandırmaktadır.

Kitabın hazırlanmasında görüş ve önerileriyle katkıda bulunan meslektaşlarımıza ve bu dersi yıllar içinde birlikte şekillendiren öğrencilerimize teşekkür ederiz.

Şadi Çağatay Öztürk ve Oğuz Ergin
Ankara, 2026

İçindekiler

Önsöz **v**

1 Bilgisayar Sistemlerine Giriş **1**
1.1 Bilgisayar Nedir? . . . 1
1.2 Çağdaş Anlamda Bilgisayarın Tarihçesi . . . 1
1.3 Bilgisayarın Bileşenleri . . . 3
1.3.1 Donanım . . . 3
1.3.2 Yazılım . . . 10
1.4 8086 Neden? . . . 15
Bölüm Sonu Soruları . . . 15

2 Sayı Sistemleri **17**
2.1 Rakamlar . . . 17
2.2 Rakamlar ve Bilgisayarlar . . . 18
2.2.1 İkilik Taban . . . 19
2.2.2 Onaltılık Taban . . . 21
2.3 İkilik Tabanda Tümleyenler . . . 24
2.3.1 1'e Tümleyen Yöntemi . . . 24
2.3.2 2'ye Tümleyen Yöntemi . . . 25
2.4 İkilik Tabanda Aritmetik İşlemler . . . 25
2.4.1 İkili Tabanda Toplama İşlemi . . . 25
2.4.2 İkili Tabanda Çıkarma İşlemi . . . 26
2.5 Onaltılık Tabanda Aritmetik İşlemler . . . 30
2.5.1 Onaltılık Tabanda Toplama İşlemi . . . 30
2.5.2 Onaltılık Tabanda Çıkarma İşlemi . . . 30
2.6 İşaretli Sayılar . . . 33
2.7 Kayan Nokta Veri Türü . . . 34
2. Bölüm Soruları . . . 36

3 8086 Mimarisi **39**
3.1 8086 Öncesi . . . 39
3.2 8086 . . . 40
3.2.1 İğne Açıklamaları . . . 42
3.3 8086 İç Mimarisi: EU ve BIU . . . 48
3.3.1 Veri Yolu Arabirimi Birimi (BIU) . . . 48
3.3.2 Yürütme Birimi (EU) . . . 48

3.4 Bellek . . . 49
3.4.1 Komut (Kod) Belleği . . . 50
3.4.2 Veri Belleği . . . 52
3.4.3 Yığıt Belleği . . . 52
3.4.4 Ayrılmış Bölgeler . . . 52
3.5 Kesmeler . . . 52
3.6 Girdi/Çıktı Kapıları . . . 52
3.7 Yazmaçlar . . . 53
3.7.1 CS Yazmacı . . . 53
3.7.2 SS Yazmacı . . . 53
3.7.3 DS Yazmacı . . . 53
3.7.4 ES Yazmacı . . . 53
3.7.5 AX Yazmacı . . . 54
3.7.6 BX Yazmacı . . . 54
3.7.7 CX Yazmacı . . . 54
3.7.8 DX Yazmacı . . . 54
3.7.9 SP Yazmacı . . . 54
3.7.10 BP Yazmacı . . . 54
3.7.11 SI Yazmacı . . . 54
3.7.12 DI Yazmacı . . . 55
3.7.13 IP Yazmacı . . . 55
3.7.14 Bayrak Yazmacı . . . 55
3.8 Komutlar . . . 56
3.9 Adresleme Kipleri . . . 56
3.9.1 Yazmaç Adresleme Kipleri . . . 56
3.9.2 Bellek Adresleme Kipleri . . . 57
Bölüm Sonu Soruları . . . 60

4 Çevirici Diline Giriş **63**
4.1 Veri Türleri . . . 64
4.2 Bölütler . . . 65
4.2.1 Kod Bölütü . . . 66
4.2.2 Veri Bölütü . . . 67
4.2.3 Yığıt Bölütü . . . 69
4.2.4 Ek Bölüt . . . 71
4.3 Bölütlerin Örtüşmesi . . . 71
4.4 Çevirici, Bağlayıcı, Konumlandırıcı ve Hata Ayıklayıcı . . . 71
4.4.1 Çevirici . . . 72
4.4.2 Bağlayıcı . . . 72
4.4.3 Konumlandırıcı . . . 72
4.4.4 Intel Hex Format Çeviricisi . . . 72
4.4.5 Hata Ayıklayıcı . . . 73
4.4.6 FASM . . . 74
4.5 Makine Diline Çevrim . . . 74
4.5.1 Yazmaç ve R/M Bellek Adreslemeleri . . . 74
4.5.2 Özel Adresleme . . . 75

4.5.3 Anlık Veri Adreslemeleri . . . 76
4.5.4 Bölüt-Yazmaç Komutları . . . 76
4. Bölüm Soruları . . . 77

5 Veri Taşıma Komutları 79
5.1 MOV . . . 79
5.2 PUSH . . . 80
5.3 POP . . . 80
5.4 XCHG . . . 82
5.5 IN . . . 83
5.6 OUT . . . 83
5.7 XLAT . . . 84
5.8 XLATB . . . 84
5.9 LEA . . . 85
5.10 LDS . . . 85
5.11 LES . . . 86
5.12 LAHF . . . 87
5.13 SAHF . . . 87
5.14 PUSHF . . . 88
5.15 POPF . . . 88
5.16 Veri Taşıma Komutlarının Bayrak Etkileri . . . 89
Bölüm Sonu Soruları . . . 89

6 Aritmetik ve Mantık Komutları 93
6.1 Aritmetik Komutları . . . 93
6.1.1 ADD ve ADC . . . 93
6.1.2 INC . . . 96
6.1.3 AAA . . . 97
6.1.4 DAA . . . 98
6.1.5 SUB ve SBB . . . 99
6.1.6 DEC . . . 101
6.1.7 NEG . . . 101
6.1.8 CMP . . . 102
6.1.9 AAS . . . 103
6.1.10 DAS . . . 104
6.1.11 MUL ve IMUL . . . 104
6.1.12 AAM . . . 107
6.1.13 DIV ve IDIV . . . 108
6.1.14 AAD . . . 111
6.1.15 CBW . . . 111
6.1.16 CWD . . . 112
6.2 Mantık Komutları . . . 113
6.2.1 NOT . . . 113
6.2.2 SHL . . . 113
6.2.3 SAL . . . 114
6.2.4 SHR . . . 114
6.2.5 SAR . . . 115

6.2.6 ROL . . . 116
6.2.7 ROR . . . 117
6.2.8 RCL . . . 117
6.2.9 RCR . . . 118
6.2.10 AND . . . 119
6.2.11 TEST . . . 120
6.2.12 OR . . . 121
6.2.13 XOR . . . 122
6.3 Mantık İşlemleri . . . 123
6.4 Aritmetik ve Mantık Komutlarının Bayrak Etkileri . . . 126
6. Bölüm Soruları . . . 127

7 Denetim Komutları 129
7.1 Denetim Aktarma Komutları . . . 129
7.1.1 CALL . . . 129
7.1.2 RET/RETF . . . 130
7.1.3 JMP . . . 131
7.1.4 JE/JZ . . . 132
7.1.5 JL/JNGE . . . 133
7.1.6 JLE/JNG . . . 133
7.1.7 JB/JNAE . . . 134
7.1.8 JBE/JNA . . . 135
7.1.9 JP/JPE . . . 135
7.1.10 JO . . . 136
7.1.11 JS . . . 137
7.1.12 JNE/JNZ . . . 137
7.1.13 JNL/JGE . . . 138
7.1.14 JNLE/JG . . . 139
7.1.15 JNB/JAE . . . 139
7.1.16 JNBE/JA . . . 140
7.1.17 JNP/JPO . . . 141
7.1.18 JNO . . . 141
7.1.19 JNS . . . 142
7.1.20 LOOP . . . 143
7.1.21 LOOPZ/LOOPE . . . 143
7.1.22 LOOPNZ/LOOPNE . . . 144
7.1.23 JCXZ . . . 144
7.1.24 INT . . . 145
7.1.25 INTO . . . 145
7.1.26 IRET . . . 146
7.1.27 Koşullu Atlama Komutları Özet Tablosu . . . 146
7.1.28 CALL ve RET Yığıt Diyagramı . . . 147
7.2 İşlemci Denetimi Komutları . . . 147
7.2.1 CLC . . . 147
7.2.2 CMC . . . 148
7.2.3 STC . . . 148

7.2.4 CLD . . . 148
7.2.5 STD . . . 148
7.2.6 CLI . . . 149
7.2.7 STI . . . 149
7.2.8 HLT . . . 149
7.2.9 WAIT . . . 149
7.2.10 ESC . . . 150
7.2.11 LOCK . . . 150
7.2.12 NOP . . . 150
7. Bölüm Soruları . . . 151

8 Dizgi İşleme Komutları 155
8.1 DF Bayrak Biti . . . 156
8.2 REP/REPE/REPNE/REPNZ/REPZ . . . 156
8.3 MOVSB/MOVSW . . . 157
8.4 CMPSB/CMPSW . . . 159
8.5 SCASB/SCASW . . . 160
8.6 LODSB/LODSW . . . 162
8.7 STOSB/STOSW . . . 164
8.8 BCD-ASCII İşlemleri . . . 165
8.8.1 BCD Sayı Sistemi . . . 165
8.8.2 ASCII Sayılar . . . 167
8.8.3 Paketlenmemiş BCD – ASCII Çevirimi . . . 167
8.8.4 Paketlenmiş BCD – ASCII Çevirimi . . . 168
8.8.5 ASCII – Paketlenmemiş BCD Çevirimi . . . 168
8.8.6 ASCII – Paketlenmiş BCD Çevirimi . . . 169

9 Kesmeler ve Çevre Birimlerinin Kullanımı 171
9.1 Kesmeler . . . 171
9.1.1 Donanım Kesmeleri . . . 171
9.1.2 Yazılım Kesmeleri . . . 172
9.1.3 Kural Dışı Kesmeler . . . 172
9.1.4 BIOS Kesmeleri . . . 172
9.1.5 Kesme Vektör Tablosu . . . 174
9.2 INT 00H . . . 175
9.3 INT 10H ve Monitör . . . 175
9.4 INT 15H . . . 178
9.5 INT 09H ve Klavye . . . 179
9.6 INT 16H ve Klavye Servisleri . . . 179
9.6.1 00H Alt Yordamı . . . 180
9.6.2 01H Alt Yordamı . . . 180
9.6.3 02H Alt Yordamı . . . 182
9.7 INT 14H ve Seri Kapı . . . 182
9.8 INT 17H ve Yazıcı . . . 184
9.9 INT 19H . . . 185
9.10 INT 1AH ve RTC . . . 185
9.11 INT 11H . . . 187

9.12 INT 13H – Alt Seviye Disk Servisleri . . . 187
9.12.1 Disket Sürücü . . . 188
9.12.2 INT 13H Alt Yordamları . . . 188
9.12.3 FAT Dosyalama Sistemi . . . 189

10 Girdi/Çıktı İşlemleri 195
10.1 Seri İletişim . . . 195
10.2 8250 . . . 195
10.2.1 Örnek . . . 196
10.2.2 Paralel İletişim . . . 197
10.2.3 Tek Yönlü İletişim . . . 199
10.2.4 Vidyo Belleği Girdi/Çıktı İşlemleri . . . 199
10.2.5 BIOS Vidyo Kipleri . . . 200
10.2.6 Vidyo Kartı Yazmaçları . . . 202
10.2.7 Dikey Tekrar Bekleme . . . 202
10.2.8 Yazı Modunda Metin Yazdırma . . . 203
10.2.9 VGA Grafik Modunda Çizim . . . 204

11 Basit Bir İşletim Sistemi 209
11.1 İşletim Sistemi Nedir? . . . 209
11.2 Basit İşletim Sistemimiz . . . 209
11.2.1 Disket Yükleyici . . . 210
11.2.2 Önyükleme Yazılımı . . . 213
11.2.3 Genel Menü Yazılımı . . . 215
11.2.4 Hesap Makinası Yazılımı . . . 218
11.2.5 Metin Düzenleyici Yazılımı . . . 219
11.2.6 Resim Çizdirme Yazılımı . . . 220
11.2.7 Seri İletişim Alıcı ve Verici Yazılımları . . . 220
Bölüm Sonu Soruları . . . 220

A x86 Komutlarının Listesi 223

B Derleme Araçları 233

C DOS ve Linux Kesmeleri 237

D BIOS Kesmeleri 241

E ASCII Kodları ve Klavye Tarama Kodları 247

Şekil Listesi

Bölüm 1 — Bilgisayar Sistemlerine Giriş

1.1 Von Neumann mimarisi. 3

Bölüm 2 — Sayı Sistemleri

2.1 2'ye tümleyen yöntemiyle ikili çıkarma akış çizgesi 28
2.2 1'e tümleyen yöntemiyle ikili çıkarma akış çizgesi 29
2.3 16'ya tümleyen yöntemiyle onaltılık çıkarma akış çizgesi 31
2.4 15'e tümleyen yöntemiyle onaltılık çıkarma akış çizgesi 32

Bölüm 3 — 8086 Mimarisi

3.1 8086 CPU iğne diyagramı 41
3.2 8086 EU–BIU iç mimari blok diyagramı 48

Bölüm 4 — Çevirici Diline Giriş

4.1 Yığıt büyümesi: PUSH işlemleri 70

Bölüm 5 — Veri Taşıma Komutları

Bölüm 6 — Aritmetik ve Mantık Komutları

6.1 ROL döndürme işlemi 116
6.2 ROR döndürme işlemi 117
6.3 RCL döndürme işlemi 118
6.4 RCR döndürme işlemi 119

Bölüm 7 — Denetim Komutları

7.1 CALL ve RET yığıt işlemleri 147

Bölüm 8 — Dizgi İşleme Komutları

Bölüm 9 — Kesmeler ve Çevre Birimlerinin Kullanımı

9.1 IVT bellek düzeni 174

Bölüm 10 — Girdi/Çıktı İşlemleri

Bölüm 11 — Basit Bir İşletim Sistemi

11.1 Hesap makinası ekran görüntüsü. 219

11.2 Metin düzenleyici ekran görüntüsü. 219

Ek A — x86 Komutlarının Listesi

Ek B — Derleme Araçları

Ek C — DOS ve Linux Kesmeleri

Ek D — BIOS Kesmeleri

Ek E — ASCII Kodları ve Klavye Tarama Kodları

Tablo Listesi

Bölüm 1 — Bilgisayar Sistemlerine Giriş

Bölüm 2 — Sayı Sistemleri

2.1 Romen rakamları ve büyüklük tablosu . . . 17
2.2 0–9 arası sayıların ikili kodları . . . 19
2.3 Onaltılık taban rakamları . . . 22
2.4 1'e tümleyen örnekleri . . . 25
2.5 2'ye tümleyen örnekleri . . . 25
2.6 Dört bitlik işaretli sayı gösterimleri . . . 34
2.7 IEEE 754 32 bitlik kayan nokta biçimi . . . 35

Bölüm 3 — 8086 Mimarisi

3.1 8086 iğne tanımları . . . 42
3.2 A17/S4 ve A16/S3 iğne durumları . . . 43
3.3 BHE' ve A0 iğne durumları . . . 44
3.4 S2', S1', S0' durum bitleri . . . 45
3.5 QS1 ve QS0 değerleri . . . 46
3.6 16 bitlik veri saklama düzeni . . . 49
3.7 32 bitlik adres saklama düzeni . . . 49
3.8 Çevirici kodunun makine dilindeki karşılığı . . . 51
3.9 8086 bayrak yazmacı düzeni . . . 55

Bölüm 4 — Çevirici Diline Giriş

4.1 Veri türleri ve değer aralıkları . . . 64
4.2 Komutların bellekte gösterimi . . . 67
4.3 Makine dili komutu bölümleri . . . 74
4.4 MOV komutu türevleri . . . 75
4.5 Bölüt yazmaç kodları . . . 76

Bölüm 5 — Veri Taşıma Komutları

5.1 MOV komutu türevleri . . . 79
5.2 PUSH komutu türevleri . . . 80
5.3 POP komutu türevleri . . . 81
5.4 Örnek 5.2 sonrası yazmaç değerleri . . . 81
5.5 XCHG komutu türevleri . . . 82
5.6 IN komutu türevleri . . . 83

5.7 OUT komutu türevleri . . . 83
5.8 XLAT komutu türevleri . . . 84
5.9 LEA komutu türevleri . . . 85
5.10 LDS komutu türevleri . . . 86
5.11 LES komutu türevleri . . . 86
5.12 LAHF komutu türevleri . . . 87
5.13 SAHF komutu türevleri . . . 87
5.14 PUSHF komutu türevleri . . . 88
5.15 POPF komutu türevleri . . . 88
5.16 Veri taşıma komutlarının bayrak etkileri . . . 89

Bölüm 6 — Aritmetik ve Mantık Komutları

6.1 ADD komutunun makine dili kodlaması . . . 93
6.2 Örnek 6.2 çıktısı – yazmaç değerleri . . . 94
6.3 Örnek 6.3 çıktısı – yazmaç değerleri . . . 95
6.4 INC komutunun makine dili kodlaması . . . 97
6.5 AAA komutunun makine dili kodlaması . . . 97
6.6 AAA komutunun iç işlemleri . . . 98
6.7 DAA komutunun makine dili kodlaması . . . 99
6.8 SUB komutunun makine dili kodlaması . . . 99
6.9 SBB komutunun makine dili kodlaması . . . 100
6.10 DEC komutunun makine dili kodlaması . . . 101
6.11 NEG komutunun makine dili kodlaması . . . 101
6.12 CMP komutunun makine dili kodlaması . . . 102
6.13 CMP komutu bayrak bit durumları (Örnek 6.13) . . . 102
6.14 AAS komutunun makine dili kodlaması . . . 103
6.15 DAS komutunun makine dili kodlaması . . . 104
6.16 MUL komutunun makine dili kodlaması . . . 105
6.17 IMUL komutunun makine dili kodlaması . . . 105
6.18 Örnek 6.20 çıktısı – yazmaç değerleri . . . 107
6.19 AAM komutunun makine dili kodlaması . . . 107
6.20 DIV komutunun makine dili kodlaması . . . 108
6.21 IDIV komutunun makine dili kodlaması . . . 108
6.22 AAD komutunun makine dili kodlaması . . . 111
6.23 CBW komutunun makine dili kodlaması . . . 112
6.24 CWD komutunun makine dili kodlaması . . . 112
6.25 NOT komutunun makine dili kodlaması . . . 113
6.26 SHL komutunun makine dili kodlaması . . . 114
6.27 SAL komutunun makine dili kodlaması . . . 114
6.28 SHR komutunun makine dili kodlaması . . . 115
6.29 SAR komutunun makine dili kodlaması . . . 115
6.30 ROL komutunun makine dili kodlaması . . . 116
6.31 ROR komutunun makine dili kodlaması . . . 117
6.32 RCL komutunun makine dili kodlaması . . . 118
6.33 RCR komutunun makine dili kodlaması . . . 119
6.34 AND (VE) işleminin doğruluk tablosu . . . 119

6.35 AND komutunun makine dili kodlaması . . . 120
6.36 TEST komutunun makine dili kodlaması . . . 120
6.37 OR (VEYA) işleminin doğruluk tablosu . . . 121
6.38 OR komutunun makine dili kodlaması . . . 121
6.39 XOR (ÖZEL VEYA) işleminin doğruluk tablosu . . . 122
6.40 XOR komutunun makine dili kodlaması . . . 122
6.41 1 bitlik tam toplayıcı doğruluk tablosu . . . 123
6.42 A0 için Karnaugh haritası . . . 123
6.43 T0 için Karnaugh haritası . . . 124
6.44 Tam toplayıcı doğruluk tablosu . . . 124
6.45 T_n için Karnaugh haritası . . . 124
6.46 A_{n+1} için Karnaugh haritası . . . 125
6.47 Aritmetik ve mantık komutlarının bayrak etkileri . . . 127

Bölüm 7 — Denetim Komutları

7.1 CALL komutu işlem kodları . . . 129
7.2 RET/RETF komutu işlem kodları . . . 131
7.3 JMP komutu işlem kodları . . . 131
7.4 JE/JZ komutu işlem kodu . . . 132
7.5 JL/JNGE komutu işlem kodu . . . 133
7.6 JLE/JNG komutu işlem kodu . . . 134
7.7 JB/JNAE komutu işlem kodu . . . 134
7.8 JBE/JNA komutu işlem kodu . . . 135
7.9 JP/JPE komutu işlem kodu . . . 136
7.10 JO komutu işlem kodu . . . 136
7.11 JS komutu işlem kodu . . . 137
7.12 JNE/JNZ komutu işlem kodu . . . 137
7.13 JNL/JGE komutu işlem kodu . . . 138
7.14 JNLE/JG komutu işlem kodu . . . 139
7.15 JNB/JAE komutu işlem kodu . . . 139
7.16 JNBE/JA komutu işlem kodu . . . 140
7.17 JNP/JPO komutu işlem kodu . . . 141
7.18 JNO komutu işlem kodu . . . 141
7.19 JNS komutu işlem kodu . . . 142
7.20 LOOP komutu işlem kodu . . . 143
7.21 LOOPZ/LOOPE komutu işlem kodu . . . 143
7.22 LOOPNZ/LOOPNE komutu işlem kodu . . . 144
7.23 JCXZ komutu işlem kodu . . . 144
7.24 INT komutu işlem kodları . . . 145
7.25 INTO komutu işlem kodu . . . 146
7.26 IRET komutu işlem kodu . . . 146
7.27 Koşullu atlama komutları özeti . . . 147
7.28 CLC komutu işlem kodu . . . 148
7.29 CMC komutu işlem kodu . . . 148
7.30 STC komutu işlem kodu . . . 148
7.31 CLD komutu işlem kodu . . . 148

7.32 STD komutu işlem kodu . . . 149
7.33 CLI komutu işlem kodu . . . 149
7.34 STI komutu işlem kodu . . . 149
7.35 HLT komutu işlem kodu . . . 149
7.36 WAIT komutu işlem kodu . . . 150
7.37 ESC komutu işlem kodu . . . 150
7.38 LOCK komutu işlem kodu . . . 150
7.39 NOP komutu işlem kodu . . . 151

Bölüm 8 — Dizgi İşleme Komutları

8.1 Dizgi komutları özeti . . . 155
8.2 REP ön ekinin makine kodu . . . 156
8.3 MOVS komutunun makine kodu . . . 157
8.4 CMPS komutunun makine kodu . . . 159
8.5 SCAS komutunun makine kodu . . . 161
8.6 LODS komutunun makine kodu . . . 162
8.7 STOS komutunun makine kodu . . . 164
8.8 Paketlenmemiş BCD sayı sistemi . . . 166
8.9 Paketlenmiş BCD sayı sistemi . . . 166
8.10 ASCII rakam kodları . . . 167

Bölüm 9 — Kesmeler ve Çevre Birimlerinin Kullanımı

9.1 BIOS kesme listesi . . . 173
9.2 INT 10H alt yordamları . . . 176
9.3 INT 10H renk kodları . . . 177
9.4 INT 16H alt yordamları . . . 179
9.5 INT 14H alt yordamları . . . 182
9.6 INT 17H alt yordamları . . . 185
9.7 INT 1AH alt yordamları . . . 186
9.8 INT 11H çıktı AX yazmacı anlamları . . . 187
9.9 INT 13H alt yordamları . . . 189

Bölüm 10 — Girdi/Çıktı İşlemleri

10.1 8250 yazmaç adresleri . . . 195
10.2 Paralel port iğne işlevleri . . . 197
10.3 LPT yazmaç adresleri . . . 198
10.4 20×5 piksel ızgara numaralandırması . . . 199
10.5 `INT 10H` vidyo kipleri . . . 200
10.6 Yazı modu öznitelik baytı . . . 200
10.7 VESA SVGA vidyo kipleri . . . 200
10.8 VESA `INT 10H` işlevleri . . . 202
10.9 VGA yazmaç grupları . . . 202

Bölüm 11 — Basit Bir İşletim Sistemi

11.1 İşletim sistemi dosyaları . . . 210

Ek A — x86 Komutlarının Listesi

A.1 8086 komut kümesi 223
A.2 80186/80188 komutları 226
A.3 80286 komutları 227
A.4 80386 komutları 227
A.5 80486 komutları 228
A.6 Pentium komutları 228
A.7 P6 CMOVxx komutları 229
A.8 Pentium III SSE komut listesi. 230

Ek B — Derleme Araçları

B.1 FAT12 öznitelik baytı 235
B.2 FAT12 zaman alanı 235
B.3 FAT12 tarih alanı 235

Ek C — DOS ve Linux Kesmeleri

C.1 Fare düğme durum sözcüğü 238

Ek D — BIOS Kesmeleri

D.1 BIOS kesme vektörleri 241
D.2 `INT 10H` alt yordamları 242
D.3 VGA 16 renk 243
D.4 `INT 11H` AX bit anlamları 244
D.5 `INT 13H` alt yordamları 244
D.6 `INT 14H` alt yordamları 244
D.7 `INT 16H` alt yordamları 245
D.8 `INT 17H` alt yordamları 245
D.9 `INT 1AH` alt yordamları 245

Ek E — ASCII Kodları ve Klavye Tarama Kodları

E.1 ASCII karakter tablosu 247
E.2 ASCII denetim karakterleri 247
E.3 Klavye tarama kodları 250

1

Bilgisayar Sistemlerine Giriş

1.1 Bilgisayar Nedir?

Bilgisayar, günümüzde hemen hemen tüm alanlarda kullanılan ve şu anda dünyanın en önemli aygıtı durumuna gelmiş bir araçtır. Tarihsel gelişim süreci göz önünde bulundurulduğunda, mevcut kaynakların ve bu konuda uzman kişilerin çoğu bilgisayar tarihinin antik çağlarda abaküsün kullanımı ile başladığını iddia ederler. Abaküs, bilindiği üzere boncuk dizilerinden oluşan ve basit toplama işlemlerini yapmaya yarayan bir araçtır. Çok basit bir hesap makinesi olarak da düşünülebilecek olan abaküs, genel olarak insanoğlunun ürettiği ilk bilgisayar olarak tasvir edilir. Bunun nedeni, insanların sayısal işlemleri yaparken hayatlarını kolaylaştırmada abaküsü kullanmalarıdır.

Bilgisayar, ilk bulunduğundan beri birçok kereler tanımlanmaya çalışılmıştır. Günümüzde kullanılan en doğru tanım ise "aritmetik ya da mantık işlemlerini önceden yüklenmiş programlar sayesinde çözebilen araç" tır. Bilgisayar sistemleri sürekli kendini geliştirmekte ve yenilemekte olduğundan bu tanım kalıcı bir tanım değildir. Yakın gelecekte bilgisayarların kendi kendilerini programlaması ve yapay usları sayesinde kendi kararlarını — kısıtlı da olsa — verebilmeleri mümkün olacaktır. Geçmişten abaküs olarak gelen bilgisayar, gelecekte yapay sinir ağlarıyla donatılmış "zeki" bir alet haline dönüşecektir.

1.2 Çağdaş Anlamda Bilgisayarın Tarihçesi

Çağdaş anlamda üretilen ilk bilgisayar olarak 1642'de Blaise Pascal tarafından üretilen hesap makinesidir. Bu makine, üzerinde bulunan tuşlar sayesinde girilen sayılar üzerinde toplama ve çıkarma yapabilmekteydi. Yaygın olarak kullanılmayan, kolay taşınamayan, hantal ve hayli ağır olan bu araç pek rağbet görmemiştir. Daha sonra 1671'de Gottfried Wilhelm von Leibniz bir bilgisayar tasarlamış ancak bu tasarladığı bilgisayarı 1694 yılına kadar üretememiştir. Günümüzde kullanılan bilgisayarlardan oldukça farklı olarak ağır mekanik aksam ve dişliler kullanılarak üretilmiş bu bilgisayar dört işlem yapabilmekteydi. Ancak bu bilgisayar da Pascal'ın hesap makinesinin akıbetine uğramış, kullanımının zor olması nedeniyle pek ilgi görmemiştir.

1820'de Xavier Thomas ticari amaçlı ilk hesap makinesini üretene kadar bilgisayar tasarımı konusunda bir durgunluk yaşanmıştır. Dört işlem yapabilen Xavier Thomas'ın hesap makinesi,

gelişen sanayinin karmaşık hesaplamaların yapılmasına olan gereksinimi nedeniyle önceki bilgisayarlara göre daha fazla ilgi görmüştür. Bu noktadan sonra bilgisayarın gelişimi gözle görülür bir şekilde hızlanmış, 1823'te buharla çalışan ilk tam otomatik hesap makinesi yapılmıştır. Bu aracın en büyük özelliği olan sabit talimat programıyla kumanda edilmesi çağdaş anlamda ilk programlama örneği olarak görülebilir.

1890 yılı bilgisayar tarihi için bir dönüm noktasıdır. Bu tarihte Herman Hollerich, yaklaşık 80 yıl kadar dünyanın çeşitli ülkelerinde kullanılacak olan delikli kart teknolojisini bulmuş ve bu mantığı kullanan bir bilgisayar tasarlamıştır. Delikli kart teknolojisi ile delikli kartlar hem bilgisayara kod girişinin yapılması hem de bellek olarak kullanılmıştır. Böylece bilgisayara kod girişi kolaylaşmış, bilgisayarlara bellekten alınan bir veriyi kullanabilme özelliği eklenmiştir. Delikli kart kullanan ilk analitik araç nüfus sayımını yaparken hesaplamalarında kolaylık sağlamak amacıyla üretilmiştir. Bu tarihten sonra bilgisayarın gelişimi savaşa giren ülkelerin teknolojide otomasyon kullanmak istemesi ve bilime verdikleri önemin artması sonucu daha da hızlanmıştır. Örneğin Amerikalı bilim adamı Howard Hathaway Aiken'in yönettiği bir araştırma takımı, 2. Dünya Savaşı'nın öncesinde ülkelerin arasındaki rekabetin en üst düzeyde olduğu 1937'de, Mark–1 adındaki ilk tam otomatik sayısal bilgisayarı yapmayı başarmıştır. Bu bilgisayar 1890'da üretilen atası gibi delikli kartları kullanmaktaydı ve sonradan üretilecek olan ENIAC'a göre epey yavaş çalışmaktaydı. 1945'de, 2. Dünya Savaşı'nın bitmek üzere olduğu bir zamanda, askeri amaçla üretilen ENIAC bir futbol sahası büyüklüğündeydi ve teknisyenler ENIAC'ın patlayan radyo lambalarını tamir edebilmek ya da bozulan rölelerini değiştirebilmek için golf arabaları kullanmak zorundaydılar. ENIAC'ın muazzam büyüklüğünden dolayı böcekler içine yuva yapabilmekte ve elektrik kablolarını kemirmekteydiler. Bu yüzden ENIAC sık sık hata yapmakta ya da bozulmaktaydı. Bilgisayar tarihinde "böcek" (bug) kavramının en bilinen örneği 1947'de Harvard Mark II bilgisayarında gerçek bir güve böceğinin devre arızasına neden olmasıdır; bu olayı belgeleyen Grace Hopper ve ekibi, log defterine "İlk gerçek böcek bulundu" notunu düşmüştür. Ancak "bug" terimi bilgisayarların çok öncesinden mühendislik jargonunda mevcuttu. Günümüzde yazılım ve donanım hatalarını tanımlamak için bu terim yaygın biçimde kullanılmaya devam etmektedir. ENIAC ile mekanik donanımın yerine elektronik devreler kullanılmaya başlanarak bilgisayar teknolojisinde yeni bir devrim yapılmıştır.

Ticari amaçlarla kullanılabilen ve seri halde üretimi yapılan ilk bilgisayar UNIVAC I olmuştur. Bilgisayarın giriş–çıkış birimleri manyetik bant olarak tasarlanmış ve bir yazıcı bilgisayara eklenmiştir. Aynı yıllarda IBM 701 bilgisayarı piyasaya çıkmış, bu bilgisayar vakum tüplü ve kolayca programlanabilen yapısı sayesinde çok büyük ilgi görmüştür. IBM firması 1958'den itibaren bilgisayarda vakum tüpleri yerine diyot ve transistörleri kullanmaya başlamış, buna bağlı olarak daha küçük, daha hafif ve daha az ısınan bilgisayarlar pazarlanmaya başlamıştır. Ayrıca bilgi depolama ortamları olarak delikli kartlar yerine teker (disk) ve tamburlar kullanılmaya başlanmıştır.

1964 yılından itibaren transistörler ve diğer devre elemanları ile oluşturulan devrelerin yerini bu devrelere göre daha küçük kalan bütünleşik devrelerin (IC – Integrated Circuit) alması bilgisayar alanındaki gelişmelere ivme kazandırmış; daha hızlı, güvenilir ve daha ucuz bilgisayarlar üretilmeye başlanmıştır. 1970 yılından itibaren geniş çapta bütünleşik devrelerin kullanılmaya başlanmasıyla bilgisayar tasarımı yeni boyutlar kazanmıştır. Özellikle 1993 yılından itibaren geniş bellekli ve hızlı bilgisayarların yanı sıra güçlü programlama dilleri ve işletim sistemleri ortaya çıkmıştır. Artık eski bilgisayarlarda kullanılan çekirdek bellek yerine daha ucuz manyetik iç bellekler kullanılmakta ve bilgisayar maliyetleri gün geçtikçe düşmek-

tedir. HTML dili Tim Berners-Lee tarafından 1989–1991 yılları arasında CERN'de geliştirilmiştir. 1993'te Mosaic tarayıcısının piyasaya sürülmesiyle İnternet geniş kitlelerce kullanılabilir hale gelmiş ve bilgisayarlar tam olarak günlük hayatımızda yerlerini almaya başlamışlardır. Bilgisayar alanında bu tarihten sonra oluşan her gelişim günlük hayatı büyük ölçüde etkilemiştir.

1.3 Bilgisayarın Bileşenleri

Bilgisayarların bileşenleri donanım ve yazılım olmak üzere ikiye ayrılabilir. Bileşenler arasında böyle bir ayrım yapılsa da bilgisayarların düzgün olarak çalışabilmeleri için donanım ve yazılımın birbirleriyle uyum içinde olmalıdır.

1.3.1 Donanım

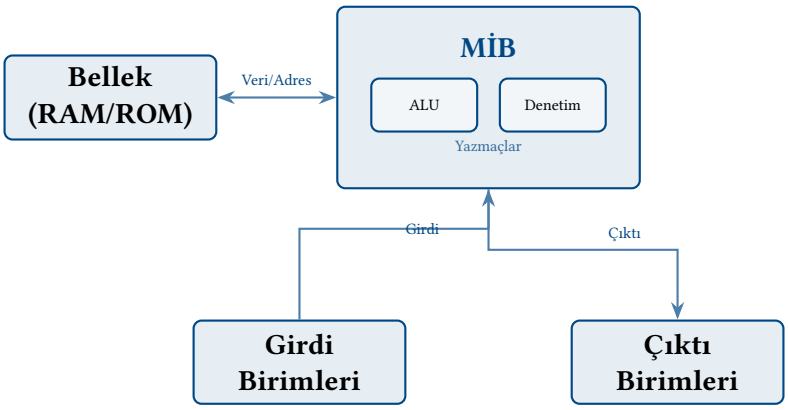
Donanım, bilgisayarın sahip olduğu tüm elektronik ve mekanik aksama verilen genel addır. Bilgisayarın gereksinim duyduğu gücü sağlamak, bilgi aktarımını düzenlemek, ekrana görüntü yansıtmak gibi işler donanımın denetimindedir. Donanım, birçok farklı görevdeki parçanın birleşmesi ile oluşur. Donanımın düzgün çalışması için donanımı oluşturan tüm bileşenlerin yazılımla uyum içinde çalışması gerekir.

Donanım, Temel Birimler ve Çevre Birimleri olmak üzere ikiye ayrılıp incelenebilir.

Temel Birimler

Temel birimler, bir bilgisayarın sorunsuz çalışması için bulunması şart olan birimlerdir. Bu birimlerin herhangi birisi herhangi bir şekilde bir sorun yaratırsa bilgisayarın çalışması aksar, bilgisayar kendi kendine zarar verebilir.

Merkezi İşlem Birimi Von Neumann mimarisi, modern bilgisayarların temel tasarım şemasıdır: işlemci, bellek ve girdi/çıktı birimleri ortak bir veri yolu üzerinden birbirleriyle iletişim kurar. Şekil 1.1 bu mimariyi özetlemektedir.



Şekil 1.1: Von Neumann mimarisi. İşlemci (MİB), bellek ve girdi/çıktı birimleri ortak bir yol üzerinden haberleşir.

Merkezi İşlem Birimi — MİB (Central Processing Unit – CPU) bilgisayarın beynidir. Bir bilgisayarın işlemcisinin başlıca görevi kullanıcı programının komutlarını alarak işlemek ve bu

komutların işlenmesi sonucunda ortaya çıkan geçici verileri saklamaktır. İşlemci aynı zamanda belleğin tutarlı biçimde güncellenmesinden sorumludur. İşlemciler bellekte bulunan komutlara göre işlem yaparlar. Komutlar, o işlemci için belirlenmiş bit dizileridir. Mikroişlemci üreticileri farklı mimari yapılar kullandıklarından her işlemci ailesi yeni komut kümelerine sahip olmak zorunda kalmıştır. Evrensel bir komut kümesi bulunmamaktadır.

Günümüzde bir MİB, çok küçük ölçekli devreler sınıfında tasarlanıp üretilmektedir. İşlemcilerin içerisinde milyonlarca transistör bulundurur. Örneğin Intel Core Duo işlemcisi yaklaşık olarak 151 milyon transistör içerir. Bir MİB, 1847'de George Boole'nin geliştirmiş olduğu Boole cebiri mantığı ile çalışır. Boole cebiri, 1 ve 0'lardan oluşan değerlerle işlem yapma mantığı üzerine kurulmuştur.

MİB, bilgisayarda gerekli olan tüm düzenlemeleri yapan donanım aracı olarak da tanımlanabilir. İşlemci yapılan işlemleri düzene sokar ve tüm diğer elemanların birbirleriyle eşgüdüm içinde çalışmasını sağlar. Merkezi işlem birimlerinde yarı iletken malzemeler kullanılmaktadır. Günümüz transistörleri büyük ölçüde silisyum tabanlıdır; germanyum ise yalnızca bilgisayar tarihinin ilk dönemlerinde kullanılmış, daha sonra yerini silisyuma bırakmıştır. Bir MİB'in iç mimarisi oldukça karmaşıktır.

MİB'in en önemli kısımlarından birisi ön bellektir (cache). Ön bellek, bilgisayara herhangi bir biçimde girilen bir verinin kullanıldıktan sonra gelecekteki kullanımlarında MİB'in veriye ulaşımının zaman ve işlem bakımından uzun süreceği düşünüldüğünden üretilmiş bir bilgisayar donanımıdır. Ön bellek, bir tür RAM olarak da düşünülebilir, ancak RAM'dan çok daha hızlıdır. Ön bellek, temel olarak kullandığı verilerin kaynaklarını — bilgisayarın sabit disklerinde saklanmış olan bir veri için — kendisine kaydeder, bu özelliği sayesinde gelecekte bilgisayarın veriye ulaşımı çok daha kolay olur. Böylece MİB'in veriye ulaşım süresi azaltılır ve bu sayede işlem süresi bakımından büyük bir kazanç sağlanır. Bilgisayarın kullanımında ön belleğin çok gerekli olduğu yapılan araştırmalarla da kanıtlanmıştır. Ön bellek, sabit diskte aynı fiziksel adrese sahip olmayan ve birlikte gruplanmış olarak bulunmayan verilerin ulaşımında bir nevi kestirme yol kullanmaktadır.

Ön bellek, veri alımı için yapılan girişleri kendi hafızasında bir havuzda toplar. Her giriş, bir dizine ve etikete sahiptir. Etikette gerçek adres bulunur. Dizin ise önbelleğin göreli bir adresidir. MİB bir veri okumak istediğinde öncelikle önbelleğe başvurulur. Eğer istenen veri önbellekte etiket aracılığıyla bulunabiliyorsa önbellekten veri okunur. Bu duruma bilgisayar literatüründe "cache hit" (önbellek bulma) denir. Aksi halde "cache miss" (önbellek bulamama) durumu oluşur. Bu durum oluştuğunda veri sabit diskette veya başka bir depolama biriminde aranır.

Eğer ön bellek kayıt limitine ulaşırsa az kullanılan veriler önbellekten silinir; kullandıkları dizinler boşaltılır ve yeni verilere yer açılır. Bu işleme önbelleğin yer değiştirme işlemi denir. En popüler yer değiştirme işlemi teknolojilerinden olan LRU (Least Recently Used – En az sıklıkta kullanılan) en az sıklıkta kullanılan girişi silerek yerine yeni gelen girişi kaydeder. Daha yeni teknolojilere sahip olan yer değiştirmelerde ise, verinin girişi yerine sahip olduğu frekans kullanılır. Bu teknolojilerin eskilere göre daha etkili olduğu kesinlikle söylenebilir.

Ana kart Ana kart, kasanın içinde bulunan donanım parçalarının tümünün üstüne monte edildiği dev bir veri yolu ağı olarak düşünülebilir. Bilgisayarın ayarlama merkezi olan BIOS da ana kart üzerinde bulunmaktadır.

Peki BIOS nedir? BIOS, bilgisayarcılıkta basit giriş/çıkış sistemi olarak bilinir. Bilgisayar çalıştırıldığında saklama birimlerinde yüklü bulunan diğer programların çalışabilmesi için BIOS

aktive edilir ve bilgisayar, BIOS'da bulunan yönetim kodları sayesinde belli bir düzende çalışmaya başlar. BIOS'un bir çipin üzerine yazılmış olan ve bilgisayarın tüm çalışma sistemlerini denetimi altında tutan küçük bir kod dizilim sistemi olduğu da söylenebilir. BIOS, bilgisayar ilk açıldığında ROM, EPROM veya bir flash diskten başlatılır. Bu işlemden sonra BIOS'un içinde barındırdığı kodlar belleğe yüklenir ve işlemci bu kodları işlemeye başlar. BIOS bilgisayar açıldığında otomatik bir denetim işlemine başlayıp bilgisayara bağlı olan tüm aletleri tarar ve denetler. Bilgisayarın BIOS ayarlarının rastgele değişmesi tehlikeli sonuçlar doğurabileceğinden, bu ayarlarda bir hata oluştuğunda fabrika çıkış ayarlarına geri dönülmelidir.

Ana kart üzerindeki parçalar genellikle kart yuvalarına (slot) yerleşirken, bazen bazı parçalar ana kart üzerine entegre edilmiş olarak gelebilir. Bu durumdaki parçalara bütünleşik (on board) parçalar denmektedir. Özellikle Ethernet kartları bütünleşik üretilirken, bazen ekran ve ses kartlarının ve hatta işlemcinin bile bütünleşik olarak üretildiği görülebilir. Bütünleşik üretime genellikle dizüstü bilgisayarlarda rastlanmaktadır.

Ana kart için basit bir örnek vermek gerekirse, bir ülke düşünelim, MİB bu ülkenin başkenti, diğer parçalar ülkenin şehirleri olsun. O ülkede lojistik anlamda ve haberleşme anlamında gereken tüm uygulamaları ana kart yapar; başka bir deyişle ana kart bir nevi ülkenin yol, telefon ve haberleşme ağı olarak görülebilir. Ana kart, bilgisayarın başarımını en çok etkileyen parçalardandır. Hızlı bir işlemci eğer yeterli hızda veri yollarına sahip değilse tam hızda çalışamaz. Aynı durum diğer parçalar için de geçerlidir. Eğer veri yollarında bir aksama veya bir yavaşlama olursa, bilgisayarın başarımı bir anda düşer ve bilgisayarda olası bozukluklara neden olur.

Sistem yolları, bilgisayarın bileşenlerinin birbirine bağlandığı, veri alışverişinin yapıldığı yollardır. x86 ailesi işlemcilerde sistem yolları; adres, veri ve denetim yolları olmak üzere üç bölümden oluşur. Bir yol, sistemin bileşenleri arasında elektrik sinyallerinin geçişini sağlayan teller bütünüdür. Bu yolların yapıları ve aktarımı yerine getirme özellikleri işlemciden işlemciye değişirler. Sistem yollarında standart mantık seviyeleri kullanılır. Bu yoldaki her telin 0 ve 1 ile temsil edilen bir voltaj seviyesi kullandığı anlamına gelir. Bu özellik de işlemciden işlemciye değişir ve bu yüzden 0 ve 1 ile temsil edilirler.

Veri yolu, işlemcinin bilgisayarın diğer bileşenleriyle bilgi alışverişi yapmak için kullandığı yoldur. Veri yolunun boyutu işlemcinin boyutunu belirler diyebiliriz. 16 bitlik veri yolu, işlemcinin işleyeceği veri boyutunu sınırlamaz; ancak her bir saat çevriminde 16 bitlik veri aktarılabileceğini gösterir. 8086 işlemcileri 16 bitlik veri yoluna sahipken, Pentium işlemcilerde bu 64 bittir.

Adres yolu, veri yolunda taşınan verinin işlemci, bellek ve girdi/çıktı birimlerinden hangisine gideceğini ve gideceği yerdeki adresini belirler. Adres yolunda belirtilen adres bir bit dizisidir ve örneğin verinin bellekte yazılacağı veya bellekten okunacağı yer bu diziye göre belirlenir.

Denetim yolu, işlemcinin nasıl iletişim kuracağını belirler. Örneğin veri yolundaki bir veriyi ele alalım. CPU bu bilgiyi belleğe gönderiyor mu, yoksa bellekten alıyor mu sorusunu denetim yolu cevaplar. Denetim yolunda iki hat vardır. Bunlar okuma ve yazma hatlarıdır. Aynı zamanda işlemcilerin daha küçük veri yığınlarıyla ilgilenmesini de denetim yolu sağlar.

Ana kart, aynı zamanda bilgisayarda güç dağıtımını yapan parçadır. Güç girişi de diğer tüm dış girişler gibi ana kartın üzerindedir. Düzgün bir sistem başarımı elde edebilmek için sistemin güç beslemesinin düzenli ve yeterli olması gereklidir. Günümüzde ana kartlar olası voltaj dalgalanmalarına karşı bilgisayarın diğer birimlerini korumak amacıyla özel donanım parçalarıyla donatılmıştır.

Ekran Kartı Ekran kartı tanım olarak, hafızada sayısal durumda saklanmış olan bir imgenin mantıksal gösterimini elektriksel sinyallere dönüştürüp herhangi görüntü ortamına (genellikle bir monitöre) çıktı olarak gönderilmesini sağlayan araçtır. Ekran kartları, hafızalarına ve işlem frekanslarına göre başarım gösterip ona göre görüntünün kalitesini yükseltip alçaltabilirler. Yeni nesil ekran kartları içlerinde kendi işlemcilerini barındırmakta olup özellikle oyun oynama amaçlı kullanılan kişisel bilgisayarlarda bu kartlar yüksek başarım göstermektedirler.

Ekran kartı, her görüntüyü çıkış ünitesine piksellerine ayırarak gönderir. Tabii ki burada başarım açısından çıkış ünitesinin desteklediği azami çözünürlük de önemlidir.

Bir görüntü alınması gerektiğinde basit olarak şu işlem gerçekleşir:

Kullanıcının herhangi bir giriş ünitesi yardımıyla girdiği ham veri, ana kart yardımıyla işlemciye gönderilir ve burada işlemci, sabit disk veya RAM'dan gerekli bilgiyi çağırır. İşlemci gerekli uygulamaları ve karşılaştırmaları yaptıktan sonra veriyi işlemeyi tamamlar. Veri işlemesi tamamlanınca ana kart, işlemciden aldığı komutlarla işlenmiş veriyi ekran kartına transfer eder. Bu andan itibaren devreye giren ekran kartı, işlemciden gelen sayısal bilgileri çözümleyerek pikselleri ayırmayı kendi mantığına göre tanımlar ve elde ettiği çıktıyı çıkış birimine gönderir. Bir piksel, tüm sayısal görüntülerin en küçük parçasına eşittir; tek bir veri ile tanımlanabilecek en küçük görüntü parçacığıdır. Çözünürlük ise, bir aygıtın desteklediği en büyük piksel dizilimi miktarıdır. Çözünürlük büyüdükçe görüntü kalitesi yükselir ama gösterilen verinin bellek veya sabit diskette kapladığı alan ve gerektirdiği işlem sayısı piksel sayısı arttığından artar.

Ekran kartının (çağdaş adıyla grafik işleme biriminin) tarihsel gelişim basamaklarına değinelim. Yeni nesil ekran kartları 1970 ve 1980'li yılların tek parça üretilen (monolithic) ekran kartlarının geliştirilmiş modelleridir. Eski kartlar, kısıtlı BitBLT (Bit Block Transfer) desteğine sahiptirler. Biraz daha açmak gerekirse, iki boyutlu grafik ve animasyon destekleri çok azdır ve genellikle şekil çizim desteğine sahip değildirler. Bu nesildeki bazı grafik işlemciler, bir ekrana yazdırma işlem tablosunda birkaç işlemi aynı anda yapıp DMA (Direct Memory Access – Direk Hafıza Erişimi) kullanarak görüntü yollayan sunucunun üstündeki yükü azaltabilirler. Bu tip ekran kartlarına önceki nesillerden bir örnek vermek istersek, ANTIC yardımcı işletim sistemi, Atari 800 ve Atari 5200 modellerinde kullanılmıştır. 1980'lerin sonunda ve 1990'ların başında yüksek hızlı ve genel amaçlı mikroişlemciler yüksek çıkışlı grafik işlemcilere entegre edilmeye ve popülerliği iyice artmaya başlamıştır.

Çip işlemci teknolojisi geliştikçe, BitBLT fonksiyonlarını ve çizimleri aynı tabana hatta aynı çipe taşımak mümkün olmuştur. Bu teknolojinin asıl kaynağı VGA'nın (Video Graphics Array – Video Grafik Dizisi) kullanılmasıyla başlar. Bu 2D hızlandırıcılar mikroişlemci tabanlı ekran kartları kadar esnek olmasalar da, yapımları kolay olduğu için ve dünya çapında çok satıldığı için popüler olmuşlardır.

1980'lerde ise durum biraz daha değişmiştir. Commodore Amiga, video donanımında bir belleğe ve bellekten veri transferi için özel olarak üretilen ek işlemcisi ve IBM'in yeni teknolojisi olan 8514 grafik sistemini içeren 2D tabanlı bilgisayarını piyasaya sürmüştür. 1985'de ortaya çıkan "oyun konsolları krizinin" (The Console Crashdown) nedenlerinden biri Amiga'nın oyun konsollarından daha ucuza satılması ve oyun dışında da kullanıma müsait olmasıdır. Amiga, ilk tam video hızlandırıcıya sahip olması dolayısıyla tamamen özgündü. Tüm video işlemlerini uygulamada donanıma yüklemesi de zamanında onu tercih edilen bir sistem yapan özelliklerindendir. Nintendo firmasının ürettiği NES oyun konsolu ise oyun konsolları krizini aşan ilk konsol olarak görülmektedir. İçinde grafik işlemler için özel bir PPU (Picture Processing Unit – Resim İşleme Birimi) bulunmakta idi. Günümüzde üretilen gelişmiş konsollarda

kutu tasarımına kadar NES etkisini görebiliriz.

Günümüzde simülasyon teknolojilerine gereksinimlerin artması, oyun sektörünün büyük ölçüde gelişmesi ve sürekli daha gerçekçi grafiklerin oyunlarda kullanılmak istenmesi ve sanal gerçeklik teknolojisinin geliştirilmeye çalışılması nedeniyle ekran kartları bir hayli gelişmiştir. Günümüzde ekran kartları başlı başına "ana kartı" olan, işlemci, RAM ve giriş/çıkış birimlerine sahip olan bir bilgisayar şeklini almıştır. Bazı ekran kartları koşut bağlanarak daha fazla başarım elde edilebilmektedir.

Sabit Disk Sabit disk (HDD – Hard Disk Drive) bir bilgisayarda bulunan tüm verilerin depolandığı manyetik bir kayıt ortamıdır. Günümüzde dakikada 10000 devir/dakika (rpm) hıza kadar çıkan sabit diskler üretilmiş olsa bile, yapıları mekanik olduğu için diğer bellek elemanlarından daha yavaştırlar. Sabit diskler, bilgisayarlarda yardımcı ve kalıcı depo alanı olarak kullanılırlar. Bir program işletilmeye başlandığında, programın çalışması için gerekli olan bilgiler işlemci tarafından sabit diskten okunarak çok daha hızlı ve sabit disketin aksine elektronik yapıda olan RAM belleğe aktarılır. Gereksinim duyulan kısım RAM'a sığmayacak kadar büyükse, bilgisayar sabit diskin bir bölümünü RAM bellek gibi kullanabilme seçeneğine sahiptir.

Sabit disklerde veri yazımı; metal, cam veya plastikten yapılmış, yüzeyi FeO ya da başka manyetik özellikteki malzeme ile kaplı diskler üzerine yapılır. Bu diskler katmanlar şeklinde birbirlerinin üzerine binmişlerdir. Bu kayıt ortamlarında veriler mıknatıslanma yolu ile kaydedildiğinden kullanıcı tarafından silinene kadar sabit kalırlar, elektrik kesintileri gibi durumlarda bilgisayar bellek parçalarındaki gibi kaybolmazlar, bu nedenle sabit disk şeklinde adlandırılmışlardır.

Sabit disk içerisinde bulunan verileri okumak için gerekli olan sistem, hareketli bir kafa olarak tasarlanmıştır. Bu kafanın sabit disk yüzeyine uzaklığı bir saç telin kalınlığından daha az olduğundan (yeni nesil sabit disklerde bu uzaklık genel olarak birkaç nanometre boyutundan daha fazla değildir) bilgisayar çalışırken asla hareket ettirilmemelidir. Eğer sabit diski okuyan kafa manyetik yüzeye temas ederse sabit disk üzerinde bozuk alan (bad sector) oluşturabilir veya daha kötü ihtimalle sabit diski parçalayabilir. Masaüstü bilgisayarlarda 3,5", dizüstü bilgisayarlarda ise 2,5" çaplarındaki sabit diskler kullanılır. Daha küçük sabit diskler ise taşınabilir mp3 çalar, taşınabilir diskler ve hafızalı telefon gibi aletlerde kullanılırlar.

Genel olarak sabit diske manyetik bir cisim yaklaştırıldığında verilerin kaybolduğundan bahsedilir. Gerçekte veriler büyük ölçüde kaybolmaz. Sadece küçük değişiklikler yüzünden dosyalar okunamaz. Bazı özel yazılımlar sayesinde bu okunamayan dosyaların bozuk verileri tespit edilebilmekte ve düzeltilebilmektedir. Bu tür yazılımlar genellikle alt seviye işlemler gerektirdiğinden çevirici dili kullanılarak yazılır.

Sabit diskler, bağlantı ara yüzlerine göre çeşitli türlere ayrılır: ATA/IDE, Serial ATA (SATA), SCSI ve daha eski ST-506/ESDI standartları bunların başlıcalarıdır. Tarihsel olarak ST-506 (5 MB kapasite, 5 Mbit/sn) ve ESDI gibi erken standartların yerini zamanla IDE ve ardından SATA almıştır; SCSI ise uzun yıllar sunucu ve iş istasyonlarında yüksek başarımlı seçenek olarak kullanılmıştır. Günümüzde SATA (ortalama 3,0 Gbit/sn) ve NVMe tabanlı katı hal sürücüleri (SSD) yaygın kullanımdadır.

RAM RAM (Random Access Memory – Rastgele Erişimli Bellek), bilgisayarda kayıtlı bir veriye herhangi bir emirle rastgele olarak ulaşılmasını sağlayan bellek tipidir. Diğer hafıza araçlarında (örneğin bir manyetik sabit diskte) o diskin sahip olduğu mekanik tasarıma göre

şekillendirilmiş bir yolu izleyerek veriye ulaşılabilirken, RAM'da veriye hızlı ve kısa yoldan ulaşmak mümkündür.

Genel olarak bir bilgisayarda RAM ana hafıza veya ana depo birimi olarak adlandırılmaktadır. Bunun sebebi, RAM'ın bilgisayarda o anda yapılan tüm programları hafızasına alıp çalıştırmasıdır. Çalıştırdığınız tüm programlar RAM'ın üzerine yüklenir. RAM'de fazla yüklenme olduğu zaman bilgisayar başarımı düşer. Buradan RAM'ın bilgisayarın başarımını en çok etkileyen parçalardan biri olduğu görülebilir. İşlemciniz istediği kadar hızlı olsun, sabit diskiniz istediği kadar yüksek devir sayısına sahip olsun, eğer RAM'ınızın kapasitesi düşükse, çalıştırabileceğiniz program sayısı ve seviyesi her zaman sınırlı kalacaktır. Ana kart üzerindeki RAM veri yolunun hızı da çok önemlidir.

Bilgisayarların sistemi çalıştırırken RAM'a gerekli programları yüklemesi nedeniyle RAM'ın sabit diske göre çok daha hızlı olması gerekmektedir. Bu nedenle sabit disk gibi bir bellek tipi olan RAM, sabit diskten farklı olarak manyetik/mekanik karışımı bir sistem yerine tamamen elektronik bir sistem üzerine tasarlanmıştır. Bu özellik sayesinde RAM üzerinde bulunan tüm program kodlarına, hangi bellek noktalarında bulunurlarsa bulunsunlar, hemen hemen aynı hızda ulaşılabilmektedir.

RAM'ın en önemli özelliği ise geçici bir bellek olmasıdır. Bilgisayar çalışmaya başladığı andan itibaren RAM'a gerekli veriler yüklenir, güç kapatıldığı anda ise RAM üzerinde bulunan tüm veriler silinir. Burada RAM, diğer bellek tiplerinden dezavantaj sayılabilecek bir farkla ayrılmaktadır. Sizin üzerinde çalıştığınız herhangi bir dosya sistem açıkken anlık olarak RAM'a kaydedildiğinden, eğer dosya üzerinde yaptığınız değişiklikleri kaydetmediyseniz güç kesintisi anında sizin yaptığınız tüm kaydedilmemiş çalışmalar kaybolacaktır. Günümüzde RAM'ların bu dezavantajını ortadan kaldırmak için çalışmalar yürütülmektedir. Bazı RAM üreticisi şirketler, gelecekte üretilebilecek bir teknoloji olarak geçici olmayan RAM üretmeye çalışmaktadırlar. Klasik elektrik devreleriyle böyle bir sistem çok karmaşık ve maliyetli olacağından, güç kesildiğinde bile üzerinde bulunan verileri koruyacak şekilde bir RAM üretebilmek için bilim adamları değişik teknolojilere başvurmayı denemektedirler. Şu anda bu iş için kullanılması denenen teknolojiler ise karbon nanotüp teknolojisi ve manyetik tünel etkisidir.

RAM'ların tek özelliği veri saklamak değildir. Bazı yeni nesil RAM'lar sistemin çalışması esnasında ortaya çıkabilecek olan kayıtlı verideki rastgele hataları RAM paritesi kullanarak algılayabilmekte ve algıladıkları bu hataları düzeltebilmektedir. Bu sistem sayesinde bilgisayar çalışırken ortaya çıkabilecek olan olası sistem hataları engellenmektedir.

2003 yazında, 128 KB'lık ve 0,18 µm teknolojisiyle üretilmiş ilk manyetik RAM bilgisayar piyasasına tanıtılmıştır. Bu tasarımdaki manyetik RAM'ın çekirdek dizaynı tamamen manyetik tünel etkisi teknolojisi kullanılarak yapılmıştır. 128 KB elbette günümüzde kullanılan yüksek kapasiteli RAM'lara karşı komik bir rakam olarak kalmaktadır, ancak sabit disk gibi kalıcı belleğe sahip olan RAM teknolojisinin henüz geliştirme aşamasının bitmediği de göz önünde bulundurulmalıdır. Zaten 2004 yılına gelindiğinde Infineon Technologies şirketinin tamamen aynı tasarımda ve aynı boyuttaki 18 MB hafızaya sahip bir manyetik RAM geliştirdiği görülebilir. Manyetik tünel etkili RAM teknolojisi, karbon nanotüplü kalıcı RAM teknolojisine göre biraz daha az tercih edilmektedir. Geleceğin nanoteknolojide olacağı düşüncesi göz önünde bulundurularak kalıcı RAM üretimin Ar-Ge'sinde öncelik karbon nanotüplerle yapılan araştırmalara verilmiştir. Örnek vermek gerekirse, 10 GB'lık karbon nanotüp serisine sahip bir kalıcı RAM tasarımı için Ar-Ge çalışmaları 2004 yılının sonlarında başlatılmıştır.

Kalıcı RAM teknolojisi, bilgisayar için yapılan Ar-Ge çalışmalarından belki de en önemlisidir. Bu teknoloji üretildiği an bilgisayarda yapılan hiçbir işlemin siz istemedikçe kaybolmaya-

cağı, bilgisayarın elektrik verildiği anda yüklemeye gerek kalmadan çalışacağı aşikardır. Nanoteknoloji yardımıyla üretilecek olduğu için, günümüzde kullanılan RAM'lara göre oldukça hızlı bir teknolojiye sahip olacaktır. Kullanılacak kurtarma sistemleri ile sık sık rastlanan sistem hataları hemen hemen sıfıra inecek, hata meydana gelse de RAM kalıcı olduğu için gerekli dosyalar zarar görmeyecektir.

Çevre Birimleri

Bilgisayarın temel birimlerine bir kullanıcı ara yüzü sağlamak amacıyla çevre birimleri kullanılır. Çevre birimlerinden bazılarını tanıyalım.

Klavye Delikli kartların ardından ilk klavye tipleri ortaya çıkmaya başladı ve 1980'den sonra neredeyse tüm bilgisayarlar klavye teknolojisi ile donatılmaya başlandı. Klavyeden sonra birçok kullanıcı girdisi almaya yarayan çevre birimi piyasaya sürülmesine rağmen klavye günümüzde bile hala bir numaralı girdi birimidir.

Klavyenin çalışma mantığı çok basittir. Klavyenin bir tuşuna basıldığında tuşun altında bulunan bir tel bu tuş için ayrılmış ana iletim teline değer ve iletim sağlanır. İletim sağlandığında ana iletim telinin bağlı olduğu deneyletici çipe bir sinyal gider ve bu çip gelen sinyalin hangi veri yolundan geldiğine bakarak bir kod oluşturur ve bu kodu bilgisayara gönderir. Bilgisayardaki bir çip ise bu kodu çözer ve basılan tuşun görevi yerine getirilir.

Fare Fare Douglas Engelbart tarafından 1963'de icat edilmiştir. Fare aslında fare benzeri ama vücudun farklı bölgeleri ile yönetilen bir dizi icat içinden seçilmiştir. Fare en kolay ve alışılmış kullanıma sahip olduğundan tercih edilmiştir.

Fare ile ekrandaki bir imlecin x ve y koordinatları değiştirilebilmektedir. Fare ileri yönde hareket ettirildiğinde y koordinatı azaltılır, geri hareket ettirildiğinde y koordinatı artar. Sağa hareket ettirildiğinde x koordinatı artar ve sola doğru hareket ettirildiğinde x koordinatı azalır. Burada dikkat edilmesi gereken şey koordinat sisteminin sol üst köşeden başlamasıdır. Aslında bu bir uzlaşımdır. Tasarladığınız sistemde fare ile çok farklı işlemler yapabilirsiniz.

Yazıcı Bir yazıcı ile bilgisayar ortamındaki bir verinin basılı kopyası çıkarılabilir. Farklı yazıcı çeşitleri bulunmaktadır.

"Karakter yazıcılar" ilk otomatik yazıcı türüdür. Tasarım olarak daktilolara çok benzemektedirler. Baskı şeridi üzerine vuruşlar yapılarak karakter yazılmaktadır. İlerleyen zamanlarda daha farklı türleri de piyasaya sürülmüştür. Örnek vermek gerekirse aynı anda bir satırı dolduran karakter yazıcılar da üretilmiştir. Karmaşık ve yavaş olduklarından artık üretilmemektedirler.

"Nokta vuruşlu yazıcılar" ise bilgisayardan gönderilen verileri baskı iğnelerini uygun sırada kağıdın üstünde bulunan şeride vurarak baskı yapan yazıcılardır. Hızlarının düşük ve çok sesli çalışmalarına rağmen harf ve rakam basmak için uygun olduklarından günümüzde halen kullanılmaktadırlar.

"Mürekkep püskürtmeli yazıcılar" mantık olarak nokta vuruşlu yazıcılar ile aynı şekilde çalışmaktadırlar. İğne ile şeride vurup kağıda iz bırakmak yerine kağıdın üzerindeki küçük noktalara yüksek hızda boya püskürtmektedirler. Bu yüzden grafik baskısında da kullanılabilmektedirler. Çözünürlükleri çok yüksektir. Daha eski yazıcılara göre sessiz ve hızlıdırlar. Farklı

boya püskürtme yöntemleri kullanılmaktadır. Isıl püskürtmeli yazıcılar mürekkebi yüksek sıcaklıklara ısıtıp ortaya çıkan patlamanın verdiği hız ile kağıda püskürtmektedirler. Piezoelektrik püskürtmeli yazıcılarda ise piezoelektrik kristallerin titreşimi ile mürekkep püskürtülmektedir. Kullanılacak olan mürekkepte herhangi bir ısıl özellik aranmadığından ısıl püskürtmeli yazıcılara göre daha yaygın olarak kullanılmaktadırlar.

"Lazer yazıcılar" sessiz, yüksek baskı kalitesine sahip ve hızlıdır. Bilgisayardan gelen veriler lazer yardımıyla ışığa duyarlı baskı davuluna çizilir. Bu sayede çizilen bölgeler statik elektrik ile yüklenir ve baskı tozları bu bölgelere yapışır. Davul dönerken ortam ısıtılarak kağıdın üzerine bu tozların yapışması sağlanır ve ardından da soğutulur.

"Isıl yazıcılar" ise ısıya duyarlı kağıtlar üzerine baskı yapabilen yazıcılardandır. Isıl yazıcılarda ince çizgi olarak dizilmiş sabit ısıtıcı birimler bulunmaktadırlar. Kağıt geçerken bu ısıtıcı birimler çalıştırılarak kağıdın birim altında bulunan bölgesine baskı yapılabilmektedir. Hızlı ve sessiz çalışmaktadırlar. Yüksek baskı kalitesine sahip değildirler bu yüzden de bilet, ATM çıktıları ve fişlerde kullanılırlar.

1.3.2 Yazılım

Yazılım bir bilgisayarın en önemli parçalarından biridir. Donanımın düzgün, koordineli ve istenen şekilde çalışmasını sağlar. Günümüzde bilgisayarlardaki en temel yazılım işletim sistemidir. İşletim sisteminin denetimi altında ise diğer yazılımlar çalışır. Yazılım geliştirmek için programlama dilleri icat edilmiştir.

Günümüzde birçok programlama dili mevcuttur. Farklı alanlar için programlama dilleri geliştirilmiştir. Programlama dilleri sınıflandırılırken ise aslında genel geçer bir kural yoktur. Bazı kitaplarda işleve göre, bazılarında derleyicilerine göre, bazılarında ise tarihe göre sınıflandırılır. Biz ise burada programlama dillerini tarihsel açıdan getirdikleri yeniliklere göre gruplara ayırarak listeleyeceğiz.

1. Grup – Makine Dilleri

Makine dilleri 0 ve 1'lerden oluşan en alt seviyeli dillerdir. Bu diller tamamen işlemci ve donanıma bağlı olarak değişmektedir. Bu yüzden de günümüzde direk olarak kullanılmamaktadır.

Makine diline verilebilecek ilk örnek ilk bilgisayardaki makine dilidir. 1943 senesinde ENIAC makinesi için geliştirilen makine dili buna bir örnektir.

Günümüzde üretilen her bilgisayar için özel bir makine dili vardır.

Programlamada oluşturduğu zorluklardan dolayı makine dili kullanılmamaktadır. Makine dili yerine günümüzde kullanılan en alt seviyeli diller çevirici dilleridir.

2. Grup – Çevirici Dilleri

Her işlemci platformu için farklı çevirici dilleri mevcuttur. Bir Motorola işlemci için farklı bir çevirici dili varken bir Intel işlemci için farklı bir çevirici dili vardır. PIC gibi mikrodenetleyiciler için de çevirici dilleri mevcuttur.

Bu kitapta x86 mimarisi için geliştirilmiş olan çevirici dili incelenmektedir. Aşağıda "Merhaba Dünya!" iletisini veren birkaç farklı çevirici dili örneği bulabilirsiniz.

x86 ASM – İşletim Sistemsiz

```
MOV     AH, 03H
```

```
INT     10H
MOV     AL, 01H
MOV     BH, 00H
MOV     BL, 01001111B
MOV     CX, MESAJ_SON-MESAJ_BAS
PUSH    CS
POP     ES
MOV     BP, MESAJ_BAS
MOV     AH, 13H
INT     10H
JMP     SON
MESAJ_BAS   DB  'Merhaba Dunya!'
MESAJ_SON   DB   00H
SON:    RET
```

Kod 1.1: x86 Assembly – İşletim Sistemsiz

Yukarıdaki kod herhangi bir işletim sistemi gerektirmeden çalışabilmektedir. Aşağıdaki örnekler DOS ve Windows'da çalışan örnekler olduğundan DOS, Windows ve Linux için de örnekler verelim.

DOS

```
MOV     AX, CS
MOV     DS, AX
MOV     AH, 09H
MOV     DX, OFFSET MESAJ
INT     21H
XOR     AX, AX
INT     21H
MESAJ:
DB      "Merhaba Dunya!", 13, 10, "$"
```

Kod 1.2: x86 Assembly – DOS

Windows

```
TITLE Merhaba Dunya ASM. Tasm
VERSION T310
Model use32 Flat, StdCall
start_code segment byte public 'code' use32
Begin:
    Call MessageBox, 0, ofset sMesaj, ofset sBaslik, 0
    Call ExitProcess, 0
start_code Ends
start_data segment byte public 'data' use32
sMesaj  DB 'Merhaba Dunya', 0
sBaslik DB "Merhaba!", 0
start_data Ends
End begin
```

Kod 1.3: x86 Assembly – Windows

Linux

```
SECTION .data
mesaj       db "Merhaba Dunya!", 0xa;
uzunluk     equ $ - mesaj
```

```
SECTION .text
global main
main:
    mov     eax, 4
    mov     ebx, 1
    mov     ecx, mesaj
    mov     edx, uzunluk
    int     0x80
    mov     eax, 1
    mov     ebx, 0
    int     0x80
```

Kod 1.4: x86 Assembly – Linux

68000 Amiga ASM

```
move.l      #DOS
move.l      4.w, a6
jsr         -$0198(a6)
move.l      d0, a6
beq.s       .Out
move.l      #MerhabaDunya, d1
jsr         -$03B4
move.l      a6, a1
move.l      4.w, a6
jsr         -$019E(a6)
.Out    rts
DOS         dc.b 'dos.library', 0
MerhabaDunya dc.b 'Merhaba Dunya!', $A, 0
```

Kod 1.5: 68000 Amiga Assembly

3. Grup – Orta Seviyeli Diller

Orta seviyeli dillere örnek olarak C, C++, D, Java, C# ve ADA gibi dilleri verebiliriz. Java aslında orta seviyenin biraz daha üstünde bulunmaktadır. Bunun nedeni Java'nın derlendikten sonra farklı bir makine diline (Java Byte Code) çevrilmesidir.

Bu dillere örnek olarak aşağıdaki kodları verebiliriz.

```c
#include <stdio.h>
int main(void)
{
    printf("Merhaba␣Dunya!\n");
    return 0;
}
```

Kod 1.6: C

```c
#include <iostream>
#include <ostream>

int main()
{
    std::cout << "Merhaba␣Dunya!" << std::endl;
    return 0;
```

```
}
```

Kod 1.7: C++

```
import std.stdio;
void main(char[][] argumanlar)
{
    writef("Merhaba Dunya!\n");
}
```

Kod 1.8: D

```
import java.io.*;
class MerhabaDunya{
    static public void main( String args[] )
    {
        System.out.println("Merhaba␣Dunya");
    }
}
```

Kod 1.9: Java

```
class MerhabaDunya
{
    static void Main()
    {
        System.Console.WriteLine("Merhaba␣Dunya!");
    }
}
```

Kod 1.10: C#

```
with Ada.Text_IO;
procedure Merhaba is
begin
    Ada.Text_IO.Put_Line("Merhaba Dunya!");
end Merhaba
```

Kod 1.11: ADA

4. Grup – Yüksek Seviyeli Diller

Bu diller algoritmik dillerdir. Fortran, Pascal, Cobol ve Basic gibi diller dördüncü gruba giren dillerdendir.

Bu dillere örnek olarak aşağıdaki kodları verebiliriz.

```
WRITE (6,7)
FORMAT(14H MERHABA DUNYA)
STOP
END
```

Kod 1.12: Fortran

```
program MerhabaDunya(output);
    begin
        writeln('Merhaba Dunya!')
    end
```

Kod 1.13: Pascal

```
IDENTIFICATION DIVISION.
PROGRAM-ID MERHABA-DUNYA.
PROCEDURE DIVISION.
PARAGRAPH-1.
    DISPLAY "Merhaba Dunya!".
    STOP RUN.
```

Kod 1.14: Cobol

```
10 REM Merhaba Dunya BASIC
20 PRINT "Merhaba Dunya!"
```

Kod 1.15: Basic

5. Grup – Çok Yüksek Seviyeli Diller

Bu diller insan ile etkileşim açısından insana en yakın olan dillerdir. Görsel programlama gibi nitelikleri vardır. Bu dillere örnek olarak Visual Basic, Access, Paradox, XBase ve Foxpro verilebilir.

```
Private Sub Command1_Click()
    Print "Merhaba Dunya!"
End Sub
```

Kod 1.16: Visual Basic

6. Grup – İnternet Dilleri

İnternet dilleri aslında "Çok Yüksek Seviyeli Diller" grubu ile "Yüksek Seviyeli Diller" grubu arasındaki dillerdir. HTML, Javascript, PHP bunlara örnektir.

Bu dillere örnek olarak aşağıdaki kodları verebiliriz.

```
<html>
<body>
<p>Merhaba Dunya!</p>
</body>
</html>
```

Kod 1.17: HTML

```
document.write('Hello␣World');
```

Kod 1.18: Javascript

```
<?php
echo 'Merhaba␣Dunya!';
?>
```

Kod 1.19: PHP

7. Grup – Betik Dilleri

Betik dilleri Yüksek Seviyeli Diller seviyesinde ama betik amaçlı kullanılan dillerdir. Bu dillerin en büyük özelliği diğer diller ile beraber kullanılabilmeleridir. Bu dillere örnek olarak Python, Ruby ve Perl verilebilir.

Bu dillere örnek olarak aşağıdaki kodları verebiliriz.

```
print("Merhaba␣Dunya!")
```

Kod 1.20: Python

```
puts "Merhaba Dunya!"
```

Kod 1.21: Ruby

```
#!/usr/bin/perl
print "Merhaba␣Dunya!\n";
```

Kod 1.22: Perl

1.4 8086 Neden?

Bu bölümde bilgisayarın tarihsel gelişimi, bileşenleri ve programlama dillerinin genel yapısı ele alındı. Peki bu denli geniş bir tablonun ardından neden özellikle Intel 8086 mikroişlemcisini inceliyoruz?

8086, 1978 yılında Intel tarafından piyasaya sürülen 16 bitlik bir mikroişlemcidir ve bugün kullandığımız x86 mimarisinin doğrudan atasıdır. Modern masaüstü işlemciler olan Intel Core ve AMD Ryzen serileri hâlâ geriye dönük uyumluluk adına 8086 komut kümesini desteklemektedir. Dolayısıyla 8086'yı anlamak, çağdaş işlemcilerin temel işleyişini kavramak demektir.

8086 ayrıca öğretim açısından da idealdir: yazmaç sayısı sınırlıdır, bellek modeli somuttur ve komut kümesi karmaşık mimarilere kıyasla daha öğretilebilir bir boyuttadır. Bu kitapta 8086 çevirici dilini öğrenerek donanım ile yazılımın nasıl kesiştiğini, belleğin nasıl yönetildiğini ve düşük düzey programlamanın neden hâlâ önemini koruduğunu anlayacaksınız.

Bölüm Sonu Soruları

Alıştırma 1.1. Bir hoca öğrencilerine 50 Hz sıklığında mektup atıyorsa, öğrenciler 1 günde kaç mektup alır?

Alıştırma 1.2. Kasırga işlemcisinin saatinin vuruş sıklığı 100 MHz ise ve her saat vuruşunda bir buyruk işliyorsa, iki buyruğun işlenmesi arasında geçen süre ne kadardır?

Alıştırma 1.3. Saat sıklığı 2,5 GHz olan bir işlemcide iki saat vuruşu arasında geçen süre kaç pikosaniyedir?

Alıştırma 1.4. Sanal bellek nedir? Mikroişlemcilerin sanal belleğin işletilmesi konusunda üzerine düşen görevi açıklayın. Sanal bellek ne amaçla kullanılır? Sanal bellek kullanılmayan bir bilgisayar olabilir mi? Eğer böyle bir bilgisayar olabilirse hangi bileşenlerin görevi değişir? Eğer böyle bir bilgisayar olmazsa neden olmaz?

Alıştırma 1.5. Onluk tabandaki 25 sayısını ikili tabanda gösterin. Kaç basamak gerekir?

Alıştırma 1.6. Merkezi İşlem Birimi'nin (MİB) temel bileşenleri nelerdir? Her bir bileşenin görevi kısaca açıklayın.

Alıştırma 1.7. Donanım ile yazılım arasındaki fark nedir? Bir bilgisayarın çalışabilmesi için her ikisinin de bulunması neden zorunludur? Yalnızca donanımı ya da yalnızca yazılımı olan bir sistem ne yapabilir?

Alıştırma 1.8. Von Neumann mimarisinin temel özellikleri nelerdir? Günümüz bilgisayarlarında bu mimari hâlâ kullanılıyor mu? Açıklayın.

Alıştırma 1.9. Yüksek düzeyli diller varken çevirici (assembly) dilinin öğrenilmesinin ve kullanılmasının gerekçelerini en az üç madde halinde açıklayın. Özellikle gömülü sistemler ve donanım sürücüsü geliştirme bağlamında değerlendirin.

Alıştırma 1.10. RAM ile sabit disk (HDD/SSD) arasındaki temel farklar nelerdir? Bir program çalıştırıldığında verilerin sabit diskten RAM'e neden taşındığını açıklayın.

2

Sayı Sistemleri

2.1 Rakamlar

Sayılar ilk çağlardan beri insanlık için önemlidir. Sayılar sayesinde insanlar ellerinde hangi varlıktan ne kadar bulunduğunu bilebilmişler ve sahip oldukları şeylerin denetimini sağlayabilmişlerdir.

İlk insanlar büyüklükleri gösterebilmek için çizgileri kullanmışlardır. Örneğin bir elma için bir çizgi çizen insan, on elma için on çizgi çizmiştir. Bu düzen içerdiği tek rakam çizgi olan bir sayı sistemidir. Tek rakamlı bu sayı sistemine birli sayı sistemi denir. Birli sistemde, gösterilecek sayının büyüklüğü arttığında yazılan ifadenin okunması zorlaşmış ve bazı karışıklıklar ortaya çıkmıştır. Bu sorunu aşmak için I ile gösterilen çubuklardan öbekler oluşturulmaya başlanmıştır. Böylece sayıların gösterimi daha düzenli hale getirilmiştir.

Çizgilerin art arda eklenmesinin ya da basitçe gruplandırılması bazı durumlarda pek de uygulanabilir değildir. Gruplamaların büyük sayıları göstermede yetersiz kalması üzerine bir adım daha ileriye gidilmiş ve yaratılan gruplandırmalar da aralarında gruplandırılmaya başlanmıştır. Ayrıca gruplar kolaylık açısından artık simgelerle gösterilmeye başlanmıştır. "Bir" büyüklüğü "I" ile ifade edilirken, "Beş" büyüklüğü beş tane "I" yerine tek bir "V" ile "On" büyüklüğü ise on tane "I" ya da iki tane "V" yerine bir tane "X" ile gösterilmeye başlanmıştır.

Bu gruplama sistemini ilk olarak Romalılar kullandığından bu simgelere "Romen Rakamları" denir. Aşağıdaki tabloda Romen rakamlarının büyüklük tablosu verilmiştir:

Tablo 2.1: *Romen rakamları ve onluk tabandaki karşılıkları.*

Romen Rakamı	Büyüklük
I	1
V	5
X	10
L	50
C	100
D	500
M	1000

Bazı büyüklükler Romen rakamları kullanılarak aşağıdaki gibi gösterilir:

$$\text{MCMLXXXVI} \rightarrow \text{M} - \text{CM} - \text{LXXX} - \text{VI} \rightarrow 1000 + 900 + 80 + 6 = 1986$$
$$\text{MMVII} \rightarrow \text{M} - \text{M} - \text{VII} \rightarrow 1000 + 1000 + 7 = 2007$$

Romen sayı sistemi toplama ve çıkarma üzerine kuruludur. Örneklerde görüldüğü gibi herhangi bir sıfır kavramı bulunmamaktadır. Basamak gibi bir kavram da yoktur. Bu yüzden Romen rakamları çarpma, bölme gibi aritmetik işlemlerin yapılması için pek uygun değildir.

Bütün bu sorunlar yüzünden bugün kullanılan rakamlar ve sayı sistemi geliştirilmiştir. Günümüzde kullanılan 0, 1, 2, 3, 4, 5, 6, 7, 8 ve 9 rakamları Hint-Arap Rakamları'dır. Daha önceki sayı sistemlerinde bulunmayan sıfır ilk olarak Hintliler tarafından bulunmuştur. Hint-Arap sayı sisteminin temelleri büyük ölçüde Arap matematikçiler tarafından geliştirilmiş ve ardından tüm dünyaya yayılmıştır.

2.2 Rakamlar ve Bilgisayarlar

İçinde bulunduğumuz teknoloji çağının "Sayısal Çağ" adını almasında şüphesiz en büyük etken artık sayısal teknolojinin hayatın her evresine girmiş olmasıdır. Sayısal bilgisayarların en önemli özelliği yazılım adı verilen bir dizi komutu izleyebilmesi ve verilen girdi üzerinde işlem yapabilmesidir. Kullanıcı ise girdileri ya da yazılımı gereksinimlerine göre değiştirebilmektedir.

Sayısal bir sistemdeki bu "yazılımlar" aslında bir rakam topluluğudur ve bu rakamlar "veri" veya "komut" öbeklerini oluşturur. Yazılımları oluşturan verilerin ve komutların gerçek elektrik devrelerinde gerilim ve akım denetimi kullanılarak yaratılır. Günümüzde bütün sayısal bilgisayarlarda yüksek ve düşük olmak üzere iki çeşit sinyal kullanılmaktadır. Bu yüzden de bu sinyallere ikili sinyaller adı verilir. İkili düzendeki sayılar, 1 (yüksek) ve 0 (düşük) değerlerinden oluşur ve bu düzendeki sayıların 0 ve 1'den oluşan her bir basamağına "bit" denir.

Bitler (0 ve 1'lerden oluşan diziler) kullanılarak her türlü veri ikili düzende gösterilebilir. İkili düzende oluşturulan bu dizilerin uzunlukları farklı olabilir. Oluşturulan bu dizilere aynı zamanda "kod" da denilebilir. Örneğin bilgisayarlarda sıfırdan dokuza kadar olan sayılar dört bit ile kodlanmıştır. Onluk düzende gösterilen sayıların ve diğer türde verilerin bu kodlanma biçimine ikili kodlama denir.

Tablo 2.2: *0'dan 9'a kadar olan sayıların ikili düzendeki kodları.*

Rakam	İkilik Tabandaki Kod
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Sayısal sistemlerde veri olarak tanımlanabilen her şey bu bir ve sıfırlardan oluşmaktadır. Harfler, resimler, karakterler, sayılar, girdi ve çıktı gibi her türlü bilgi yalnızca iki tane rakamdan oluşan bir sayı sistemi kullanılarak gösterilmektedir.

Günlük hayatta gereksinim duyulan her türlü büyüklük 0, 1, 2, 3, 4, 5, 6, 7, 8 ve 9 rakamları kullanılarak gösterilir. Toplam on rakamdan oluştuğu için günlük hayatta kullanılan bu tabana onluk taban denir. Bilgisayarlar ise yalnızca 0 ve 1 rakamlarını kullandıklarından ikili tabana göre çalışırlar.

Özellikle çevirici dili (assembly) ile program yazarken sayı tabanlarının ve birbirleri arasındaki çevrimlerin bilinmesi çok önemlidir. Yazılımlar oluşturulurken ikili taban dışında onaltılık sayı tabanı da sıklıkla kullanılır. Onaltılık sayı tabanı uzun 0 ve 1 dizilerini kısaltmak amacıyla kullanılır. Onaltılık tabandaki bir sayıyı ikili tabana çevirmek ya da ikili tabandaki bir sayıyı onaltılık tabana çevirmek diğer tabanlar arasında çevrim yapmaktan daha kolaydır.

2.2.1 İkilik Taban

Onluk tabandaki her türlü sayı büyüklüğünün gösterilmesi için 0'dan 9'a kadar rakamlar kullanılır. İkili tabanda ise her sayı 0 ve 1 rakamları kullanılarak oluşturulur. Aşağıda ikilik tabanda gösterilen bazı sayı örnekleri verilmiştir:

$$(10011)_2$$
$$(111)_2$$
$$(101111)_2$$
$$(101001)_2$$
$$(11111011011)_2$$
$$(01110011)_2$$
$$(10011111)_2$$

Değişik sayı tabanındaki sayılar arasında dönüşüm yapmak mümkündür. Onluk tabandaki sayılar ikili tabandaki sayılara, ikilik tabandaki sayılar onaltılık tabandaki sayılara kolayca dönüştürülebilir.

İkilik – Onluk Taban Çevrimleri

Aşağıda onluk tabanda bir sayı gösterilmiştir:

$$12345$$

İnsanlar onluk tabanda işlem yaptıklarından sayıların onluk tabanda olduğunu belirtmek için herhangi bir işaretleme yapılmaz. Eğer bir işaretleme yapılması gerekirse aşağıdaki gibi yapılabilir:

$$(12345)_{10}$$

Yukarıda gösterilen sayı onluk tabandaki 12345 değerine eşittir. Bu sayının gerçek değeri diğer tüm tabanlardaki gibi taban dönüşümü kuralları kullanılarak bulunur. Aşağıda onluk düzendeki bu sayının değerinin basamaklarının ayrıştırılması yoluyla nasıl bulunduğu gösterilmiştir.

$$\begin{aligned}(12345)_{10} &= 5 \times 10^0 + 4 \times 10^1 + 3 \times 10^2 + 2 \times 10^3 + 1 \times 10^4 \\ &= 5 \times 1 + 4 \times 10 + 3 \times 100 + 2 \times 1000 + 1 \times 10000 \\ &= 5 + 40 + 300 + 2000 + 10000 \\ &= 12345\end{aligned}$$

Görüleceği gibi sayının en sağındaki haneden başlanarak basamaktaki rakam taban değerinin ilgili basamağın kuvveti ile çarpılarak sayıyı oluşturan rakamların sayıya kattıkları değer bulunmaktadır. Sayının toplam değeri her bir basamağın sağladığı değerin toplamıdır. Aynı mantığı kullanarak aşağıdaki ikili tabandaki sayının ondalık tabandaki değeri aşağıdaki gibi bulunur:

$$\begin{aligned}(11000000111001)_2 &= 1 \times 2^0 + 0 \times 2^1 + 0 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 + 1 \times 2^5 \\ &\quad + 0 \times 2^6 + 0 \times 2^7 + 0 \times 2^8 + 0 \times 2^9 + 0 \times 2^{10} + 0 \times 2^{11} \\ &\quad + 1 \times 2^{12} + 1 \times 2^{13} \\ &= 1 + 0 + 0 + 8 + 16 + 32 + 0 + 0 + 0 + 0 + 0 + 0 + 4096 + 8192 \\ &= 12345\end{aligned}$$

Diğer tüm tabanlardaki sayıların onluk tabandaki karşılığının bulunması için yukarıda gösterilen yol aynı biçimde kullanılır.

Onluk – İkili Taban Çevrimleri

Onluk tabandan ikili tabana geçerken, ikili tabandan onluk tabana geçme yolundan farklı bir yol izlenerek art arda bölme işlemi uygulanır ve bu bölmenin sonucunda ortaya çıkan kalana bakılır. Aşağıda verilen onluk tabandaki sayı üzerinde onluk tabandan ikilik tabana geçme yöntemi gösterilecektir.

$$(123)_{10}$$

Onluk tabandaki bu sayının ikili tabana çevrilmesi için aşağıdaki adımlar izlenmelidir:

1. Sayı çevrilmek istenen tabana bölünür. Kalan belirlenir. $123 \div 2 = 61$, kalan $= 1$
2. Bulunan kalan sayının çevrilmek istenen tabandaki değerinin ilk basamağını gösterir. Bulunan kalan 1 olduğundan sayının ikili tabandaki son basamağı 1 olmalıdır ($s_0 = 1$).
3. Bölüm yeniden ikiye bölünür. Kalan belirlenir. $61 \div 2 = 30$, kalan $= 1$
4. $s_1 = 1$. Bölüm ikiye bölünür ve kalan belirlenir. $30 \div 2 = 15$, kalan $= 0$
5. $s_2 = 0$. Bölüm ikiye bölünür ve kalan belirlenir. $15 \div 2 = 7$, kalan $= 1$
6. $s_3 = 1$. Bölüm ikiye bölünür ve kalan belirlenir. $7 \div 2 = 3$, kalan $= 1$
7. $s_4 = 1$. Bölüm ikiye bölünür ve kalan belirlenir. $3 \div 2 = 1$, kalan $= 1$
8. $s_5 = 1$. Bölüm daha fazla ikiye bölünemediği için çevirme işlemi sona erer ve bölümün değeri ikili düzendeki sayının en anlamlı basamağı olan s_6'nın değerini oluşturur. $s_6 = 1$.

Bu çevirme işlemi sonucunda aşağıdaki sayı bulunur:

$$(s_6 s_5 s_4 s_3 s_2 s_1 s_0)_2 = (1111011)_2$$

Sonucun doğrulanması için $(1111011)_2$ sayısı yeniden onluk tabana çevrilirse:

$$\begin{aligned}(1111011)_2 &= 1 \times 2^0 + 1 \times 2^1 + 0 \times 2^2 + 1 \times 2^3 + 1 \times 2^4 + 1 \times 2^5 + 1 \times 2^6 \\ &= 1 + 2 + 0 + 8 + 16 + 32 + 64 \\ &= 123\end{aligned}$$

Onluk tabandaki sayılar diğer sayı tabanlarına çevrilirken de aynı yöntem kullanılır. Sayı bu durumda ikiye değil dönüştürülmek istenen sayı tabanına, bölüm bu sayı tabanından büyük olduğu sürece, bölünür ve kalanlar ile bölüm yeni sayı tabanındaki sayıyı oluşturur.

2.2.2 Onaltılık Taban

Onluk tabanda sayıların gösterilmesi için 0'dan 9'a kadar olan 10 adet rakam kullanılır. Onaltılık tabanda ise sayıların gösterilmesi için 16 ayrı rakam gerektiğinden 9'dan sonraki rakamları göstermek için harflerden yararlanılır. Aşağıdaki tabloda onaltılık tabandaki sayılar ve bu sayıların onluk tabandaki değerleri karşılıklı olarak gösterilmiştir.

Tablo 2.3: *Onaltılık tabandaki rakamlar ve onluk tabandaki değerleri.*

Onaltılık Rakam	Onluk Değer
0	0
1	1
2	2
3	3
4	4
5	5
6	6
7	7
8	8
9	9
A	10
B	11
C	12
D	13
E	14
F	15

Onaltılık – Onluk Taban Çevrimleri

Onaltılık tabandaki sayılar onluk tabana çevrilirken ikili taban ile onluk taban arasındaki çevrim yöntemi aynı biçimde uygulanır. Örneğin onaltılık tabandaki `A12F` sayısı onluk tabana aşağıdaki gibi çevrilir:

$$(\texttt{A12F})_{16} = \texttt{F} \times 16^0 + 2 \times 16^1 + 1 \times 16^2 + \texttt{A} \times 16^3$$

Onaltılık tabanda kullanılan harfler onluk tabandaki değerleri ile değiştirilirse:

$$\begin{aligned}(\texttt{A12F})_{16} &= 15 \times 16^0 + 2 \times 16^1 + 1 \times 16^2 + 10 \times 16^3 \\ &= 15 \times 1 + 2 \times 16 + 1 \times 256 + 10 \times 4096 \\ &= 15 + 32 + 256 + 40960 \\ &= 41263\end{aligned}$$

Görüldüğü gibi sayılar ikili tabandan onluk tabana çevrilirken kullanılan çevrim mantığı burada da geçerlidir.

Onluk – Onaltılık Taban Çevrimleri

Onluk tabandaki sayılar onaltılık tabana çevrilirken onluk düzendeki sayı bölüm 16'dan küçük oluncaya dek onaltıya bölünür. Bölme işlemi tamamlandıktan sonra bölüm onaltılık düzendeki sayının en anlamlı basamağını, kalanlar ise diğer basamaklarını gösterirler. Onluk tabandaki değeri 41263 olan sayı onaltılık tabana aşağıdaki biçimde çevrilir:

1. $41263 \div 16 = 2578$, kalan $= 15$, $s_0 = 15 = \texttt{F}$
2. $2578 \div 16 = 161$, kalan $= 2$, $s_1 = 2$
3. $161 \div 16 = 10$, kalan $= 1$, $s_2 = 1$
4. İşlemin sonucunda ortaya çıkan 10 sayısı 16'ya daha fazla tam olarak bölünemeyeceğinden $s_3 = 10 = \texttt{A}$ olarak belirlenir.

Basamakları oluşturan rakamlar bir araya getirildiğinde $(41263)_{10} = (\texttt{A12F})_{16}$ sonucu bulunur.

İkili – Onaltılık Taban Çevrimleri

İkili tabandaki sayıların boyutu çok hızlı büyüdüğünden gösterilen sayılardaki basamak sayısı hızlı bir biçimde artar. Bu durum ikili tabandaki sayıların okunurluğunu azalttığından bilgisayar işlemlerinde yüksek basamaklı sayılar onaltılık tabanda gösterilir.

İkili tabandaki sayıların onaltılık tabana dönüştürülmesi için sayı önce onluk tabana daha sonra da onluk tabandan onaltılık tabana dönüştürülebilir. Bu işlemi daha kısa zamanda yapmayı sağlayan ve arada sayıyı onluk düzene çevirmeyi engelleyen kolay bir yöntem vardır. İkili düzendeki sayının basamakları sol yandan başlayarak dörderli öbeklere ayrılır. Daha sonra bu öbeklerin onaltılık tabandaki karşılıkları bulunur ve öbekler bulunan rakamlarla değiştirilir. Böylece ortaya çıkan sayı ikili tabandaki asıl sayının onaltılık tabana dönüştürülmüş biçimidir.

Örneğin $(11111010101010110)_2$ sayısının 16'lık tabandaki karşılığının bulunması için sayının basamakları önce aşağıdaki gibi dörderli öbeklere ayrılır:

$$(1)_2 \quad (1111)_2 \quad (0101)_2 \quad (0101)_2 \quad (0110)_2$$

İkili düzendeki sayı dörderli öbeklere ayrıldıktan sonra her öbeğin onluk düzendeki değeri aşağıdaki gibi yazılır:

$$\begin{aligned}(0001)_2 &= 1\\ (1111)_2 &= 15\\ (0101)_2 &= 5\\ (0101)_2 &= 5\\ (0110)_2 &= 6\end{aligned}$$

Ondalık tabandaki bu değerler onaltılık tabana çevrilirse:

$$\begin{aligned}1 &= (1)_{16}\\ 15 &= (\texttt{F})_{16}\\ 5 &= (5)_{16}\\ 5 &= (5)_{16}\\ 6 &= (6)_{16}\end{aligned}$$

Öbeklerin onaltılık tabandaki karşılıkları yerine yazılırsa $(\texttt{1F556})_{16}$ sayısı elde edilir.

Çevirici diliyle program yazılırken çokça kullanıldığından bu iki taban arasında geçiş yapma kurallarının bilinmesi oldukça önemlidir.

Onaltılık – İkili Taban Çevrimleri

Onaltılık tabandaki bir sayının ikili tabana çevrilmesi için yukarıda açıklanan yöntemin tersi kullanılabilir. Bu durumda onaltılık sayının her bir basamağının ikili tabandaki dört bitlik karşılığı yazılır. Örneğin bir önceki bölümde sonuç olarak bulunan onaltılık tabandaki `1F556` sayısının yeniden ikili tabana çevrilmesi için her basamak ayrı ayrı ikili tabana çevrilir. Eğer rakamın ikili tabana çevrildiği durumda dört basamaktan daha az basamak bulunuyorsa, sayı dört basamaklı olana kadar sayının soluna sıfır eklenir.

$$
\begin{aligned}
(6)_{16} &= (0110)_2 \\
(5)_{16} &= (0101)_2 \\
(5)_{16} &= (0101)_2 \\
(\mathtt{F})_{16} &= (1111)_2 \\
(1)_{16} &= (0001)_2
\end{aligned}
$$

Rakamlar ikili tabana çevrildikten sonra dört basamaklı öbekler bir araya getirilerek ikili tabandaki sayı oluşturulur:

$$(00011111010101010110)_2$$

Çıkan sayının sol tarafındaki 0'lar anlamsız olduğu için sayıdan atılır:

$$(11111010101010110)_2$$

Bulunan ikili tabandaki bu değer $(\mathtt{1F556})_{16}$ sayısının karşılığıdır.

2.3 İkilik Tabanda Tümleyenler

Tümleyenler bilgisayarlarda çıkarma işlemini ve mantık işlemlerini yalın bir biçimde gerçekleştirmek amacıyla kullanılır. Bilgisayarlarda eksi sayılar, tümleyen yöntemleri kullanılarak yaratılır ve saklanır. Bu nedenle sayıların tümleyenlerinin nasıl alındığını ve bu düzende gösterilen sayılarla nasıl işlem yapıldığını bilmek önemlidir.

2.3.1 1'e Tümleyen Yöntemi

İkili tabanda gösterilen bir sayının 1'e tümleyenini bulmak için her bitin (ikili sayının her bir basamağının) değeri terslenir. Eğer tümleyeni alınan bitin değeri sıfır ise sonuç bir, bir ise sonuç sıfır olur. Aşağıda ikili tabandaki bazı sayılar ve bu sayıların 1'e tümleyenleri örnek olarak verilmiştir:

Tablo 2.4: *İkili tabandaki sayılar ve 1'e tümleyenleri.*

Sayı	1'e Tümleyeni
1	0
0	1
101	010
1111	0000
11011010101001111	00100101010110000

Devre düzeyinde bir sayının 1'e tümleyeni alınırken her bir basamağın değerinin tersi DEĞİL (Evirici) kapısı kullanılarak bulunur.

2.3.2 2'ye Tümleyen Yöntemi

İkili tabanda gösterilen bir sayının 2'ye tümleyenini bulmak için sayının önce 1'e tümleyeni alınır ve bulunan sonuç bir ile toplanır. Toplama sonucunda bulunan sayının basamak sayısı tümleyeni alınan sayının basamak sayısından fazla olursa, sonuçta bulunan sayının en soldaki basamağı atılır. Aşağıda ikili tabandaki bazı sayılar ve bu sayıların 2'ye tümleyenleri örnek olarak verilmiştir:

Tablo 2.5: *İkili tabandaki sayılar ve 2'ye tümleyenleri.*

Sayı	2'ye Tümleyeni
1	1
0	0
101	011
1111	0001
1101101	0010011

İkiye tümleyen almanın bir başka yolu da aşağıda adımları verilen yöntemi kullanmaktır:

1. Sayının sağından başlayıp soluna doğru ilerleyerek değeri 1 olan ilk basamağa kadar bulunan tüm 0'ları kopyala. (Eğer sayının en sağdaki biti bir ise bu adım atlanır)
2. 0'ların ardından gelen ilk 1'i kopyala.
3. Geri kalan basamakların tamamının tersini al.

2.4 İkilik Tabanda Aritmetik İşlemler

2.4.1 İkili Tabanda Toplama İşlemi

İkili tabandaki toplama işlemi günlük hayatta onluk tabandaki toplama işleminin yapıldığı gibi iki sayının her bir basamağının ayrı ayrı toplanması yoluyla yapılır. Tabana göre elde var ise (iki basamağın toplamı birden büyük çıktıysa) elde bir soldaki basamağa eklenir. Aşağıda ikili tabandaki toplama işlemine iki örnek verilmiştir:

İkili tabandaki `0110101` ile `011` sayıları aşağıdaki gibi toplanır:

$$\begin{array}{r} 0110101 \\ +\quad 011 \\ \hline 0111000 \end{array}$$

İkili tabandaki `1101101` ile `1101101` sayıları aşağıdaki gibi toplanır:

$$\begin{array}{r} 1101101 \\ +1101101 \\ \hline 11011010 \end{array}$$

2.4.2 İkili Tabanda Çıkarma İşlemi

Bilgisayarlarda çıkarma işlemi yapılırken tümleyenler kullanılır. Bir sayı diğerinden çıkarılırken çıkarılan sayının tümleyeni (bir diğer deyişle eksilisi) alınır ve bulunan bu sayı diğer sayı ile toplanır. Böylece çıkarma işlemi gerçekleşmiş olur.

Örnek Örnek 2.1

2'ye tümleyen yöntemi kullanılarak ikili tabandaki `110001101` sayısından `011110101` sayısının çıkarılması işlemi aşağıdaki biçimde gösterilebilir:

$$110001101 - 011110101$$

Bu çıkarma işlemi ikinci sayının ikiye tümleyeni alınıp ilk sayı ile toplanması yoluyla toplama işlemine çevrilir. İkinci sayı `011110101`'in 2'ye tümleyeni `100001011`'dir. Böylece yukarıdaki çıkarma işlemi aşağıdaki toplama işlemine dönüşür:

$$\begin{array}{r} 110001101 \\ +100001011 \\ \hline 1010011000 \end{array}$$

Toplama işleminin sonucu görüldüğü gibi ikili tabanda `1010011000` bulunmuştur. Çıkarma işleminin sonucunu ise bulunan bu sonuçla aynı değildir. Çıkarma işlemi yapılırken sonuç, işlenenlerin basamak sayısı en çok olanından daha geniş olamaz. Bu nedenle yukarıdaki sonuçta ortaya çıkan en soldaki bit fazladır ve bu fazlalık silinerek çıkarma işleminin doğru sonucu bulunur. Çıkarma işlemi için yapılan toplamanın sonucunda basamak sayısının bir artması bulunan sonucun sıfırdan büyük olduğunu gösterir. Fazladan bulunan basamak silindiğinde çıkarma işleminin sonucu aşağıdaki gibi bulunur:

$$010011000$$

Bu sonuç çıkarma işleminin sonucudur ve en soldaki biti sıfır olduğu için bulunan değer sıfırdan büyük bir değerdir.

Örnek Örnek 2.2

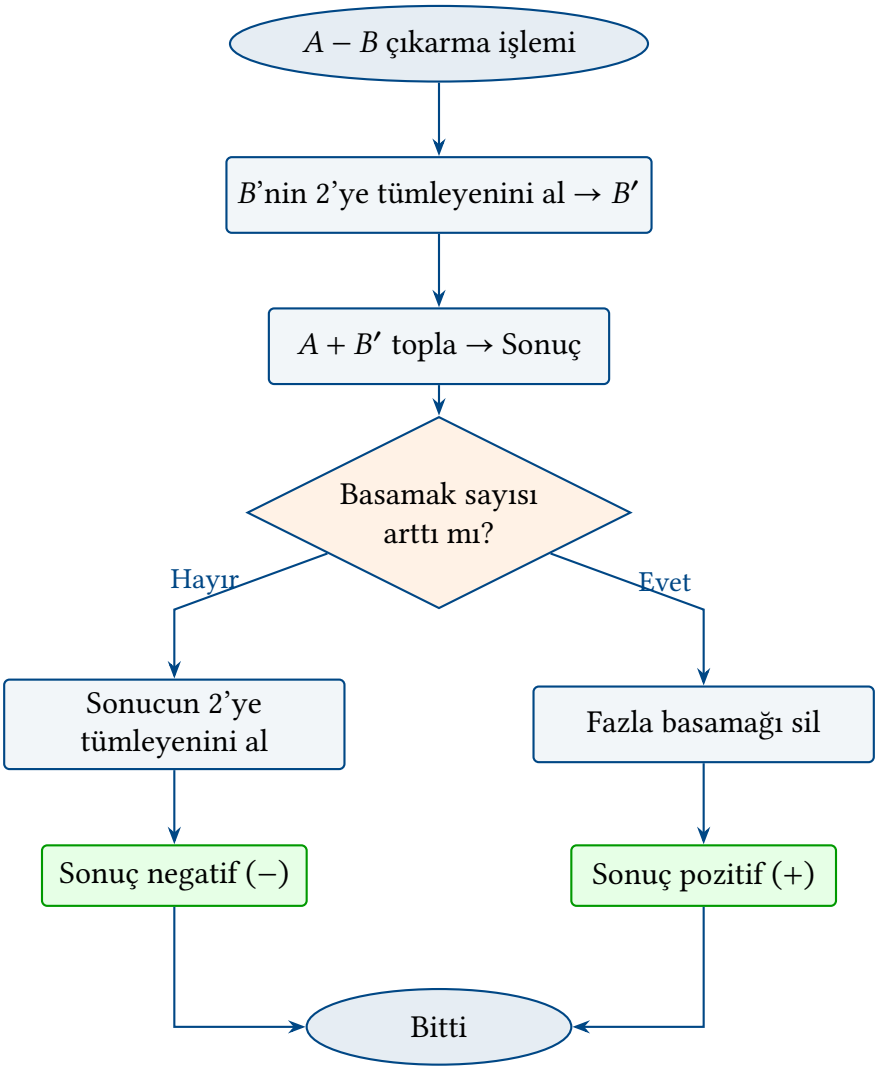
`011110101` sayısından `110001101` sayısının ikiye tümleyen yöntemi kullanılarak çıkarılması işlemi aşağıda gösterilmiştir.

Çıkarılacak olan `110001101` sayısının ikiye tümleyeni `001110011` sayısıdır. Elde edilen bu sayı ile `011110101` sayısı aşağıdaki biçimde toplanır:

$$\begin{array}{r} 011110101 \\ +001110011 \\ \hline 101101000 \end{array}$$

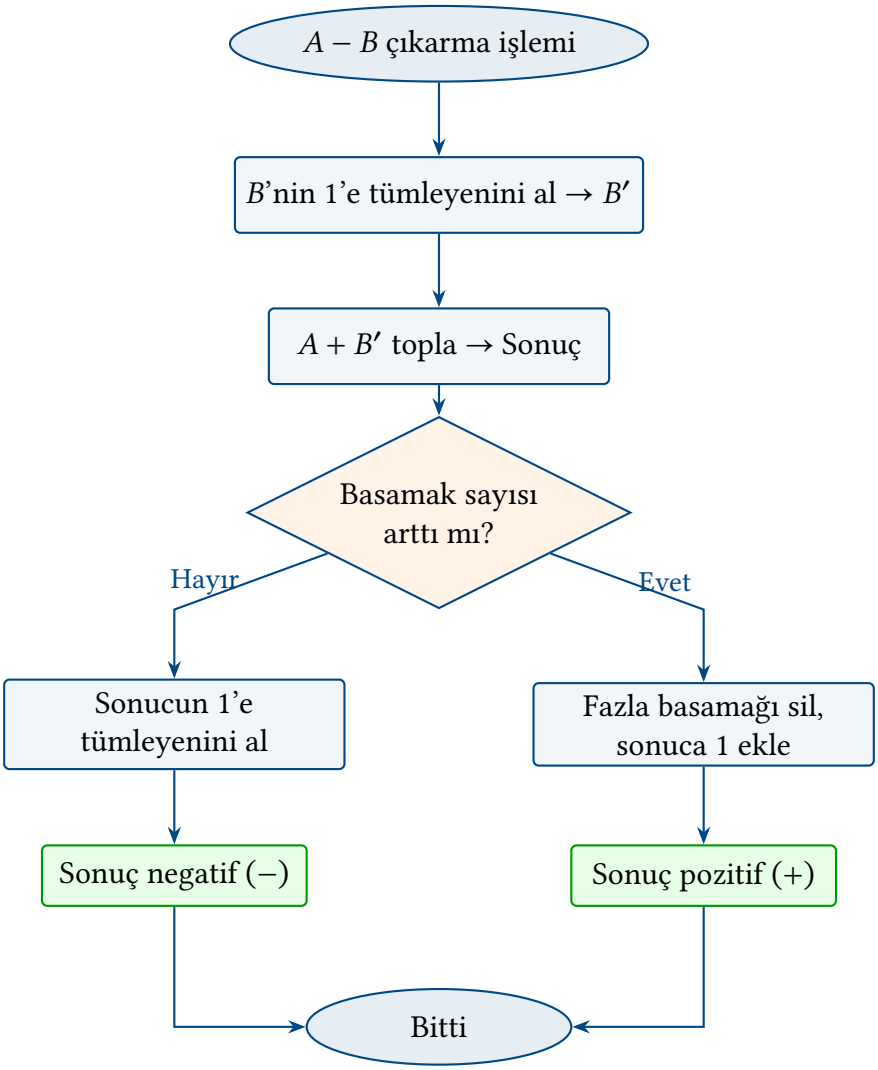
Görüldüğü gibi toplama işleminin sonucu `101101000` sayısıdır. Bir önceki örneğin aksine basamak sayısında bir artış olmadığından bu konuda bir işlem yapılmasına gerek yoktur. İşlemde küçük bir sayıdan büyük bir sayı çıkarıldığı için sonucun sıfırdan küçük olması beklenir. Çıkarma işlemi için yapılan toplamanın sonucunda basamak sayısında artış olmaması bulunan sayının eksi olduğunu gösterir. Bu yüzden çıkarma işleminin sonucunu bulabilmek için bulunan `101101000` sayısının ikiye tümleyeninin alınması gerekir. `101101000` sayısının ikiye tümleyeni `010011000` sayısı olduğundan çıkarma işleminin sonucu -010011000 sayısıdır.

İkili tabanda 2'ye tümleyen yöntemi kullanılarak yapılan çıkarma işlemi aşağıdaki akış çizgesi ile özetlenebilir:



Şekil 2.1: İkili tabanda 2'ye tümleyen yöntemi kullanılarak yapılan çıkarma işleminin akış çizgesi.

Örneklerde verilen çıkarma işlemlerinin aynıları çıkarılan sayının 1'e tümleyeni kullanılarak da yapılabilir. Bu yapılırken 2'ye tümleyen yöntemine benzer biçimde çıkarılan sayının 1'e tümleyeni alınır ve bu tümleyen birinci sayı ile toplanır. İkili tabanda 1'in tümleyeni kullanarak yapılan çıkarma işlemini aşağıdaki akış çizgesi özetlemektedir:



Şekil 2.2: İkili tabanda 1'e tümleyen yöntemi kullanılarak yapılan çıkarma işleminin akış çizgesi.

Örnek Örnek 2.3

İkili tabandaki `110001101` sayısından `011110101` sayısının 1'e tümleyen yöntemi kullanılarak çıkarılması aşağıdaki biçimde yapılır:
Bu çıkarma işleminin yapılmasının ilk adımı ikinci sayının 1'e tümleyeninin bulunmasıdır. İkinci sayının tüm basamaklarının tersi alındığında 1'e tümleyeni `100001010` sayısı olarak bulunur. Çıkarma işleminin sonucunun bulunması için bulunan bu tümleyen birinci sayı ile toplanır:

$$\begin{array}{r} 110001101 \\ +100001010 \\ \hline 1010010111 \end{array}$$

Toplama işlemi sonunda bulunan sonuç `1010010111` sayısıdır. Ortaya çıkan sonucun basamak sayısı işlenenlerin basamak sayısından büyük olduğu için, sonucun en soldaki "artan" basamağı silinir ve kalan sayı bir ile toplanır. Yapılan silme ve toplama işlemi

sonucunda ortaya çıkan sonuç `10011000` sayısıdır.
Bire tümleyen yöntemiyle yapılan çıkarma işlemlerinde bulunan sonuçların yanlış olmaması için artan basamak silinip kalan sayıya eklenerek çıkan sonuç düzeltilir.

Örnek Örnek 2.4

2'ye tümleyen yöntemi kullanılarak çözülen ikinci çıkarma örneğinde yapılan 011110101 − 110001101 işleminin ikinci sayının bire tümleyeni alınarak yapılışı aşağıda gösterilmiştir.
`110001101` sayısının 1'e tümleyeni `001110010` sayısıdır. Bu sayı ile birinci sayı aşağıdaki gibi toplanır:

$$\begin{array}{r} 011110101 \\ +001110010 \\ \hline 101100111 \end{array}$$

Görüldüğü gibi çıkan `101100111` sonucunda herhangi bir basamak artışı olmamıştır. Bu nedenle çıkarma işleminin doğru sonucunun bulunması için, bulunan toplama sonucunun 1'e tümleyeni alınır ve başına sayının sıfırdan küçük olduğunu belirten "−" işareti konur. Böylece çıkarma işleminin sonucu −010011000 sayısı olarak bulunur.

2.5 Onaltılık Tabanda Aritmetik İşlemler

2.5.1 Onaltılık Tabanda Toplama İşlemi

Onaltılık tabandaki toplama işlemleri ile ikili tabandaki toplama işlemlerinin yapılma yöntemleri aynıdır. Diğer tabanlarda da olduğu gibi basamaklar ayrı ayrı toplanır. Basamakların toplamı 15'ten büyük olduğunda, oluşan artan kısım bir sonraki basamağa eklenir.

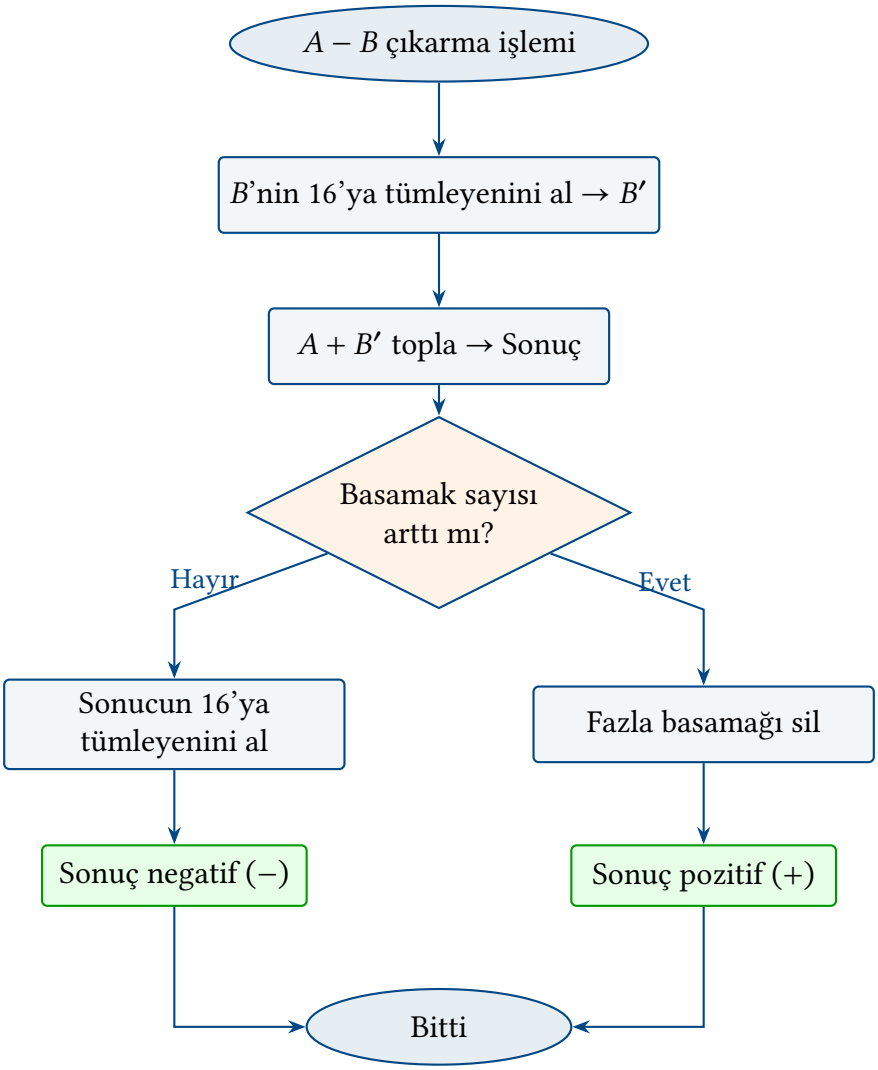
Örnek Örnek 2.5

Onaltılık tabandaki `AA1234F0` sayısı ile `1234ABCD` sayısı aşağıdaki biçimde toplanır:

```
 AA1234F0
+1234ABCD
---------
 BC46E0BD
```

2.5.2 Onaltılık Tabanda Çıkarma İşlemi

Çıkarma işlemi de ikili tabandaki çıkarma işlemleri ile aynı mantık kullanılarak yapılır. İkili tabandan farklı olarak yalnızca 2'ye tümleyen yerine 16'ya tümleyen ve 1'e tümleyen yerine 15'e tümleyen kullanılır. 16'nın tümleyeni yöntemi kullanılarak yapılan çıkarma işleminin akış çizgesi aşağıda gösterilmiştir.



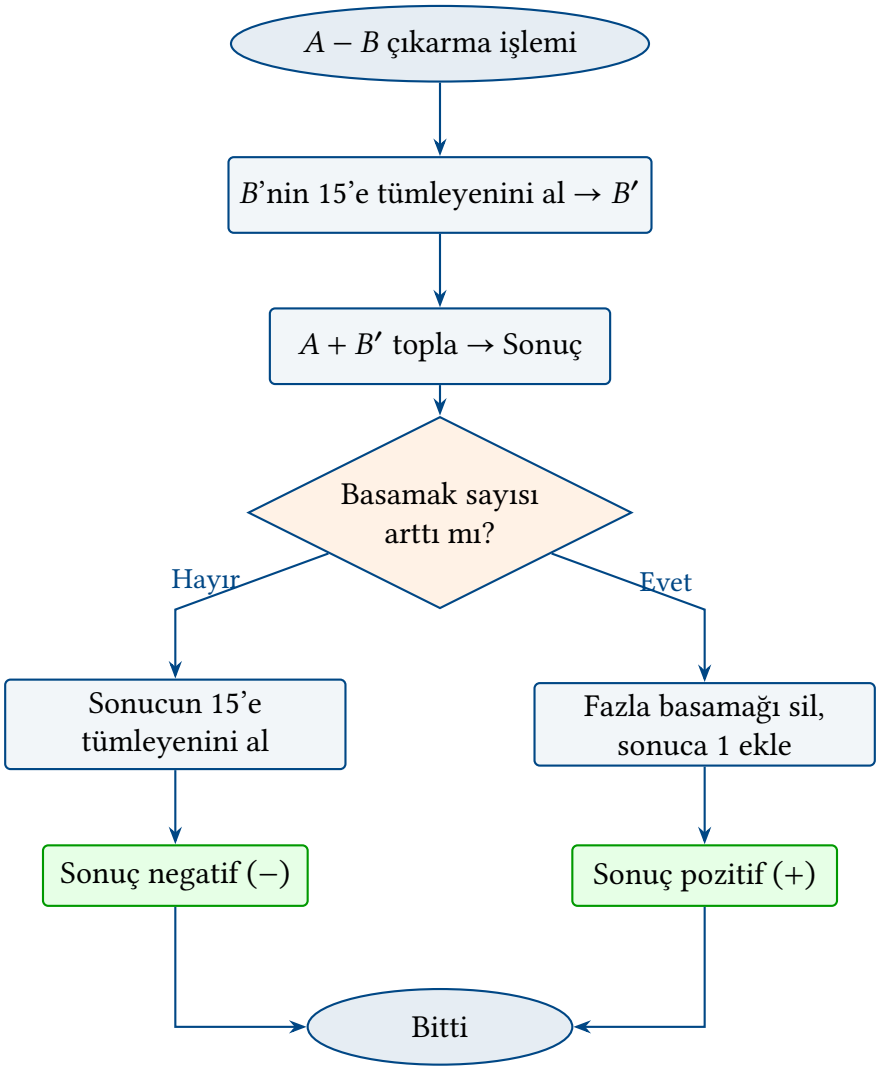
Şekil 2.3: 16'lık tabanda 16'ya tümleyen yöntemi kullanılarak yapılan çıkarma işleminin akış çizgesi.

Örnek Örnek 2.6

Onaltılık tabandaki `1234ABCD` sayısı `AA1234F0` sayısından 16'ya tümleyen yöntemi kullanılarak aşağıdaki biçimde çıkarılır:
Bu işlem yapılırken, akış çizgesine göre öncelikle ikinci sayının 16'ya tümleyeni alınır. `1234ABCD` sayısının 16'ya göre tümleyeni `EDCB5433` sayısıdır. Bu sayı ile birinci sayı toplanır:

```
  AA1234F0
 +EDCB5433
 ---------
 197DD8923
```

Basamak sayısı arttığından en soldaki basamak silinir ve çıkarma işleminin sonucu $(\texttt{97DD8923})_{16}$ olarak bulunur.



Şekil 2.4: 16'lık tabanda 15'e tümleyen yöntemi kullanılarak yapılan çıkarma işleminin akış çizgesi.

Örnek Örnek 2.7

Onaltılık tabandaki `AA1234F0` sayısı `1234ABCD` sayısından 16'ya tümleyen yöntemi kullanılarak aşağıdaki biçimde çıkarılır:
`AA1234F0` sayısının 16'ya tümleyeni `55EDCB10` sayısıdır. Bulunan bu sayı ile `1234ABCD` sayısı toplanarak çıkarma işlemi gerçekleştirilir:

```
  1234ABCD
 +55EDCB10
 ---------
  682276DD
```

Bulunan sonuçta basamak artışı olmadığından, `682276DD` sayısının 16'ya göre tümleyeni alınıp, başına sayının sıfırdan küçük olduğunu belirten "−" işareti konularak çıkarma işleminin sonucuna ulaşılır. Çıkarma işleminin sonucu −`97DD8923` sayısıdır.

Örnek Örnek 2.8

Onaltılık tabandaki `1234ABCD` sayısı `AA1234F0` sayısından 15'e tümleyen yöntemi kullanılarak aşağıdaki biçimde çıkarılır:
Bu işlem yapılırken, akış çizgesine göre öncelikle ikinci sayının 15'e tümleyeni alınır. `1234ABCD` sayısının 15'e tümleyeni `EDCB5432` sayısıdır. Bulunan bu sayı ile birinci sayı toplanarak çıkarma işlemi gerçekleştirilir:

```
  AA1234F0
 +EDCB5432
 ---------
 197DD8922
```

Toplama işleminin sonucunda basamak sayısı arttığından en soldaki basamak silinir ve toplamanın sonucuna bir eklenerek çıkarma işleminin sonucu $(\texttt{97DD8923})_{16}$ olarak bulunur.

Örnek Örnek 2.9

Onaltılık tabandaki `AA1234F0` sayısı `1234ABCD` sayısından 15'e tümleyen yöntemi kullanılarak aşağıdaki biçimde çıkarılır:
`AA1234F0` sayısının 15'e tümleyeni `55EDCB0F` sayısıdır. Bu sayı ile `1234ABCD` sayısı toplanarak çıkarma işleminin sonucu bulunur:

```
  1234ABCD
 +55EDCB0F
 ---------
  682276DC
```

Toplama işleminin sonucunda basamak artışı olmadığından bulunan `682276DC` sayısının 15'e tümleyeni alınıp başına sayının sıfırdan küçük olduğunu belirten "−" işareti konularak çıkarma işleminin sonucuna ulaşılır. 15'e tümleyen yöntemiyle yapılan bu çıkarma işleminin sonucu −`97DD8923` sayısıdır.

2.6 İşaretli Sayılar

Bu bölüme kadar anlatılan aritmetik işlemler ve sayı gösterimleri sıfırdan büyük sayılar için geçerlidir. Örneklerde bulunan tüm sayılar ya sıfırdan büyüktür ya da sıfırdan küçük oldukları başlarına konulan "−" işareti ile gösterilmiştir. Ancak bilgisayar donanımları yalnızca 0 ve 1 değerleriyle çalışabildiği için sayının eksi olduğunu gösteren bu "−" işaretlerinin donanımın anlayacağı biçime sokulmaları gerekir.

İşaretli sayılar belirtilen bu soruna çözüm olarak ortaya atılmıştır. Bilgisayarlarda gösterilen ikilik düzendeki işaretli sayılarda en soldaki bit sayının işaretini tutar. Sayının en başındaki bu "işaret bitinin" içeriğine göre sayının sıfırdan büyük ya da küçük olduğu anlaşılır.

Genel kural olarak işaret biti 1 olan sayı sıfırdan küçük, işaret biti 0 olan sayı sıfırdan büyük kabul edilir. Kullanılan işaretleme yöntemine göre en soldaki işaret biti sayının değerine dahil olabilir ya da olmayabilir. 1'e tümleyen ve 2'ye tümleyen gösterimlerinde işaret biti sayının değerine dahildir.

Aşağıdaki tabloda dört bit kullanılarak değişik yöntemlerle gösterilebilecek tüm işaretli

sayılar sıralanmıştır.

Tablo 2.6: *Dört bit kullanılarak farklı yöntemlerle gösterilebilen işaretli sayılar.*

İkilik Tabanda Sayı	İşaret ayrıysa değeri	1'e tümleyen	2'ye tümleyen
0000	+0	+0	0
0001	1	1	1
0010	2	2	2
0011	3	3	3
0100	4	4	4
0101	5	5	5
0110	6	6	6
0111	7	7	7
1000	−0	−7	−8
1001	−1	−6	−7
1010	−2	−5	−6
1011	−3	−4	−5
1100	−4	−3	−4
1101	−5	−2	−3
1110	−6	−1	−2
1111	−7	−0	−1

İşaret bitinin ayrı olduğu durumda ilk bit her zaman işaret için ayrılmıştır ve sayıya dahil değildir. İşaret biti sayıdan ayrıldıktan sonra elde kalan 3 bit sayının değerini belirler. 1'e tümleyen düzeninde işaret biti 1 olduğunda sayının sıfırdan küçük olduğu anlaşılır ve değerinin anlaşılması için sayının işaret biti dahil 1'e tümleyeni alınarak değeri bulunur ve başına "−" işareti konulur. 2'ye tümleyen düzeninde de benzer biçimde, işaret biti 1 olduğunda sayının 2'ye tümleyeni alınarak değeri bulunup başına eksi işareti konulur.

İşaret bitinin değerden ayrı olarak kullanıldığı durumda ve 1'e tümleyen yönteminde +0 ve −0 olmak üzere iki ayrı sıfır değeri olduğundan bu gösterim yöntemlerinde gösterilebilecek sayı sayısı 2'ye tümleyen yöntemiyle gösterilebilecek sayı sayısından bir eksiktir. Bu nedenle aynı sayıda bitle işaretli sayıları göstermek için 2'ye tümleyen yöntemi daha verimlidir.

2.7 Kayan Nokta Veri Türü

Bu kitabın kapsamında anlatılan konular içinde verilen örneklerde genellikle 16 bitlik (2 bayt) sayılardan büyük sayılar ile işlemler yapılmayacaktır. Eğer bu 16 bitin en solundaki bit işaret biti olarak kullanılırsa -2^{15} (−32768) ile $2^{15} - 1$ (32767) arasındaki sayılar gösterilebilir. Bir başka deyişle 15 bit duyarlılığında sayı gösterme aralığı 2^{15}'tir.

Bilgisayarlarda bazı durumlarda ondalıklı sayılarla ya da çok büyük sayılarla işlem yapmak gerekebilir. Örneğin $1{,}01101 \times 2^{23}$ gibi bir sayı ile işlem yapılması gerekebilir. Böyle durumlarda verilerin gösterilmesi için kayan nokta veri türü kullanılır. Değişik çeşitleri olan kayan nokta veri türünün günümüz bilgisayarlarında genellikle 32 bitlik bir çeşidi kullanılmaktadır. Bu 32 bitin ilk biti işaret, ilk bitten sonraki sekiz bit üs ve bu dokuz bitten sonra gelen yirmi üç bit ise anlamlı kısım için kullanılmaktadır. Anlamlı kısım, gösterilmek istenen sayı 1,aaa × 2

biçimine getirildiğinde virgülden sonra gelen *aaa* kısmı, üs ise ikinin kuvvetini belirten *ü*'dür. Bu gösterim biçimine IEEE 754 biçimi denir.

Tablo 2.7: *32 bitlik IEEE 754 kayan nokta saklama biçimi.*

İşaret (1 bit) = A	**Üs (8 bit) = B**	**Anlamlı Kısım (23 bit) = C**
Sayının Değeri $= (-1)^A \times 1{,}C \times 2^{B-127}$		

IEEE 754 biçiminde gösterilen 32 bitlik bir verinin değeri aşağıdaki gibi hesaplanabilir:

$$\text{Sayının Değeri} = (-1)^A \times 1{,}C \times 2^{B-127}$$

Bilgi Notu 2.1: Örtülü 1 Biti (Hidden Bit)

IEEE 754 biçiminde normalleştirilmiş bir sayı her zaman $1{,}xxx \times 2^n$ biçiminde yazılır; yani virgülden önce her zaman bir "1" bulunur. Bu öncü 1, bellekte *saklanmaz*; çünkü zaten orada olduğu bilinir. Bu bit, **örtülü 1 biti** (hidden bit veya implicit leading 1) olarak adlandırılır. Bu sayede anlamlı kısım için ayrılan 23 bitin tamamı virgülden *sonraki* basamaklara tahsis edilebilir ve böylece duyarlılık bir bit artırılmış olur. Örneğin anlamlı kısım alanı `01101000000000000000000` ise gerçek anlamlı kısım 1,01101 olarak yorumlanır. Yalnızca sıfır ve sonsuz gibi özel değerlerde bu kural farklı biçimde uygulanır.

Örnek Örnek 2.10

$1{,}01101 \times 2^{23}$ sayısı IEEE 754 kayan nokta veri türü biçiminde 32 bitle gösterilirken:

- İşaret biti (A): Sayı pozitif olduğundan $A = 0$
- Üs (B): 2^{23} için $B - 127 = 23 \Rightarrow B = 150 = (10010110)_2$
- Anlamlı kısım (C): $01101\,\underbrace{00\ldots0}_{18}$ (23 bit)

Sonuç olarak 32 bitlik IEEE 754 gösterimi:

$$\underbrace{0}_{A}\,\underbrace{10010110}_{B}\,\underbrace{01101000000000000000000}_{C}$$

Örnek Örnek 2.11

IEEE 754 kayan nokta veri türü biçiminde gösterilen `10101010101111101111000000000000` sayısının değeri aşağıdaki gibi bulunur:
Sayının işaretini belirten otuz birinci bitin (en soldaki bitin) değeri 1'dir. Bu bitin değerinin 1 olması sayının eksi (sıfırdan küçük) olduğunu gösterir. Ardından gelen 8 bitin değeri `01010101` olduğundan $B = 85$ alınır ve buradan $B - 127$'nin değeri -42 olarak bulunur.
Üs bitlerinden sonra gelen bitler sayının virgülden sonra gelen anlamlı kısmını gösterdiği için `01111110111000000000000` sayısı virgülden sonra gelen kısmıdır.
Böylece sayının değeri:

$$(-1)^1 \times 1{,}01111110111 \times 2^{-42} = -1{,}01111110111 \times 2^{-42}$$

olarak bulunur.

2. Bölüm Soruları

Alıştırma 2.1. Aşağıdaki onluk tabandaki sayıları ikili tabana çevirin:

a) 47
b) 200
c) 1023

Alıştırma 2.2. Aşağıdaki ikili tabandaki sayıları onluk tabana çevirin:

a) `10110101`
b) `01001100`
c) `11111111`

Alıştırma 2.3. Aşağıdaki onaltılık tabandaki sayıları hem onluk hem de ikili tabana çevirin:

a) `3AH`
b) `FFH`
c) `1B4H`

Alıştırma 2.4. Aşağıdaki 8 bitlik sayıların 1'e tümleyenlerini bulun:

a) `01101010`
b) `10000001`

Alıştırma 2.5. Aşağıdaki 8 bitlik sayıların 2'ye tümleyenlerini bulun ve bu 2'ye tümleyen gösteriminin onluk tabandaki işaretli değerini belirtin:

a) `00110100`
b) `10101010`

Alıştırma 2.6. İkili tabanda aşağıdaki toplama işlemlerini gerçekleştirin ve varsa elde bitini belirtin:

a) `01110011` + `00101101`
b) `11001100` + `01010101`

Alıştırma 2.7. 2'ye tümleyen yöntemini kullanarak aşağıdaki ikili çıkarma işlemlerini gerçekleştirin:

a) `10110000` − `01001100`
b) `01111111` − `10000001`

Alıştırma 2.8. Onaltılık tabanda aşağıdaki toplama işlemini gerçekleştirin:

$$\texttt{A4F3H} + \texttt{5B2EH}$$

Alıştırma 2.9. $-18{,}75$ sayısını 32 bitlik IEEE 754 kayan nokta biçiminde gösterin. İşaret, üs ve anlamlı kısım bitlerini ayrı ayrı hesaplayın.

Alıştırma 2.10. Aşağıdaki 32 bitlik IEEE 754 kayan nokta sayısının onluk tabandaki değerini bulun:

$$\underbrace{0}_{A}\,\underbrace{10000001}_{B}\,\underbrace{01000000000000000000000}_{C}$$

(İşaret biti: 0, Üs: `10000001`, Anlamlı kısım: `01000000000000000000000`)

3

8086 Mimarisi

3.1 8086 Öncesi

Günümüzdeki mikroişlemci tanımına uyan ciddi anlamda ilk mikroişlemci projeleri 1968 yılında Intel firmasının yürüttüğü 4004, Texas Instruments firmasının yürüttüğü TMS 1000 ve Garrett AiResearch firmasının yürüttüğü Central Air Data Computer projeleridir. Bu projeler hemen hemen aynı zamanlarda tamamlandığından tam olarak çalışır halde bir mikroişlemcinin ilk olarak hangi firma tarafından üretildiği bilinmemektedir.

1971'de bitirilen ve Intel firmasının ilk mikroişlemcisi olan 4004 mikroişlemcisi aynı anda yalnızca 4 bitlik veriler ile çalışabilen 4 bitlik bir işlemci idi. Saat sıklığı 740 kHz olan bu işlemcinin 16 iğnesi bulunmakta ve sistem ile bağlantısı bu iğneler aracılığıyla yapılmaktaydı. 4004 işlemcisinde iki saat vuruşu arasında geçen süre (çevrim süresi) $1/740000 = 1{,}35$ mikro saniye tutmaktaydı. Günümüzde kullanılan ve saat sıklığı 3 GHz olan işlemciler, yalnızca saat sıklığı düşünüldüğünde, 4004 mikroişlemcisinden yaklaşık 4000 kat daha hızlıdır. Çağdaş işlemcilerde, başarımın artırılması için pek çok değişik yöntem kullanıldığından, aradaki hız farkı gerçekte çok daha fazladır. 4004 işlemcisinin 4 bitlik veri yoluna karşın, günümüzdeki işlemciler 32 veya 64 bitlik veri yoluna sahip olduğundan aynı anda daha çok veri alabilmekte ve işleyebilmektedir. Bugünkü işlemcilerde daha sonra da değineceğimiz gibi kullanılan bazı yöntemler sayesinde aynı anda birden fazla işlem yapılabilmektedir. Örneğin bazı öngörü birimleri sayesinde işlemciler programın ne yönde akacağını öngörüp ona göre hareket edebilmektedir.

4004 işlemcisinde komutlar 8 bitlik idi ve tam olarak 46 tane komut bulunmaktaydı. 4004 her biri 4 bitlik genişlikte veri tutabilen 16 tane yazmaç içeriyordu. Bellek adreslemesi için 12 bitlik adresler kullanılabildiği için 4 kilobaytlık belleklere erişmek mümkündü. Tasarımında 2300 transistor kullanılan 4004'ün altyordam yığını 3 düzeyli olduğundan iç içe 3 tane alt yordam girebiliyordu.

4004 işlemcisinin ardından Intel firması 8008 mikroişlemcisini piyasaya sürdü. Bu işlemci aynı anda 8 bitlik verilerle çalışabilmekteydi ve 0,8 MHz'lik bir saati vardı. 14 bitlik adresleme olanağı bulunduğundan 16 kilobayt bellek erişimi yapılabiliyordu. 8008 işlemcisinin bir saniyede işleyebildiği komut sayısı 4004 mikroişlemcisinden daha az olmasına rağmen, 8 bitlik 8008, aynı anda işleyebildiği veri sayısının yüksek olması dolayısıyla 4 bitlik 4004'e göre daha hızlıydı.

8008'den sonra Intel firması kullanılabilirliği yüksek olan 8080 mikroişlemcisini piyasaya sürdü. 8080 8 bitlik bir mikroişlemci idi ve 2 MHz'lik bir saati vardı. 8080 işlemcisinin 8 bitlik 7 adet yazmacı vardı ve bu 7 yazmaçtan 6 tanesi birleştirilip 16 bitlik yazmaç olarak da kullanılabilmekteydi. 16 bitlik bellek adreslemesi ve 8 bitlik veri yolu sayesinde 8080 mikroişlemcisiyle 64 kilobaytlık belleklere erişilebilmekteydi. Tasarımında 6000 transistör kullanılan 8080'de 8 bitlik birikeç (akümülatör), 16 bitlik yığıt işaretçisi ve 16 bitlik program sayacı bulunmaktaydı. Bu terimlerin anlamları bölümün sonlarına doğru açıklanacaktır.

8080'den sonra üretilen 8085'in kullanımı, beraberinde kullanılması gereken devre elemanı sayısı daha az olduğu için 8080'den daha kolaydı. Örneğin 8085 besleme olarak yalnızca 5 V'luk bir güç kaynağı gerektirirken, 8080 işlemcisine +5 V, −5 V ve +12 V'luk gerilimlerin sağlanması gerekmekteydi. Kullanım kolaylığından ötürü 8085 işlemcisi mikrodenetleyici olarak uzun süre kullanıldı ve halen bazı uygulamalarda kullanımı sürmektedir.

Bu işlemcilerin ardından Intel günümüzde yaygın olarak kullanılan x86 mikroişlemci mimarisinin ilk örneğini üretti: 8086.

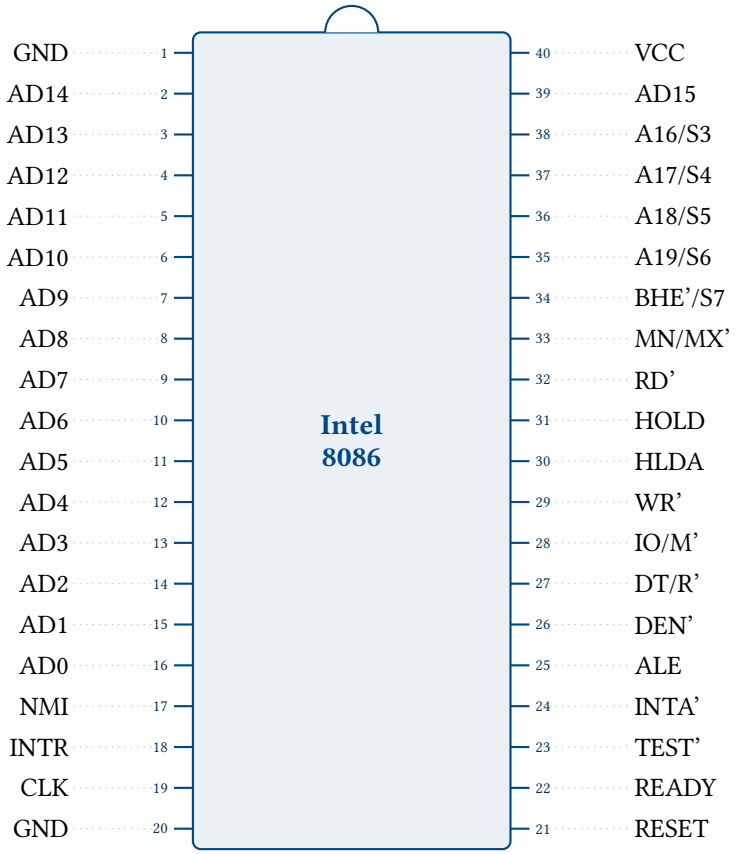
3.2 8086

1978 yılında Intel firması 16 bitlik 8086 mikroişlemcisini piyasaya sürdü. 8086 işlemcisinin bazı temel teknik özellikleri aşağıdaki gibidir:

- 16 bit veri yolu; tek seferde 16 bit veri işleyebilme özelliği (16 bitlik mikroişlemci)
- 4,7 MHz – 10 MHz arası saat sıklığı
- 20 bitlik adresleme özelliği; $2^{20} = 1$ MB'lık belleğe erişebilme
- 8 adet 16 bitlik yazmaç

x86 komut kümesini kullanan bütün mikroişlemciler (örneğin Intel Pentium ve AMD Athlon serisi) 8086 ile geriye dönük olarak uyumludur. 8086 işlemcisi için yazılan tüm komutlar bugünkü x86 tabanlı bilgisayarlarda çalışabilmektedir. Ancak günümüz bilgisayarları için yazılmış olan ve genişletilmiş x86 komut kümesi kullanan yazılımlar 8086 mikroişlemcisi üzerinde çalışamamaktadırlar.

8086 mikroişlemcisi aşağıdaki iğne diyagramından da görüldüğü gibi 40 iğnelik bir tümleşik devre şeklindedir. Şeklin altındaki tabloda iğnelerin işlevleri açıklanmıştır.



Şekil 3.1: 8086 CPU iğne diyagramı. Sol tarafta 1–20, sağ tarafta 21–40 arası iğneler bulunmaktadır.

Tablo 3.1: *8086 mikroişlemcisinin 40 iğnesinin tanımları.*

İğne	Kısaltma	Tür	İşlev
1	GND	—	Topraklama iğnesi
2	AD14	G/Ç	On beşinci adres ve veri yolu iğnesi
3	AD13	G/Ç	On dördüncü adres ve veri yolu iğnesi
4	AD12	G/Ç	On üçüncü adres ve veri yolu iğnesi
5	AD11	G/Ç	On ikinci adres ve veri yolu iğnesi
6	AD10	G/Ç	On birinci adres ve veri yolu iğnesi
7	AD9	G/Ç	Onuncu adres ve veri yolu iğnesi
8	AD8	G/Ç	Dokuzuncu adres ve veri yolu iğnesi
9	AD7	G/Ç	Sekizinci adres ve veri yolu iğnesi
10	AD6	G/Ç	Yedinci adres ve veri yolu iğnesi
11	AD5	G/Ç	Altıncı adres ve veri yolu iğnesi
12	AD4	G/Ç	Beşinci adres ve veri yolu iğnesi
13	AD3	G/Ç	Dördüncü adres ve veri yolu iğnesi
14	AD2	G/Ç	Üçüncü adres ve veri yolu iğnesi
15	AD1	G/Ç	İkinci adres ve veri yolu iğnesi
16	AD0	G/Ç	Birinci adres ve veri yolu iğnesi
17	NMI	G	Maskelenemeyen kesme iğnesi
18	INTR	G	Kesme istemi iğnesi
19	CLK	G	Saat darbesi iğnesi
20	GND	—	Topraklama iğnesi
21	RESET	G	Yeniden başlatma iğnesi
22	READY	G	Hazır belirteci iğnesi
23	TEST’	G	Deneme iğnesi
24	QS1	Ç	İkinci sıra durum iğnesi
25	QS0	Ç	Birinci sıra durum iğnesi
26	S0’	Ç	Birinci durum iğnesi
27	S1’	Ç	İkinci durum iğnesi
28	S2’	Ç	Üçüncü durum iğnesi
29	LOCK’	Ç	Kilitleme iğnesi
30	RQ’/GT1’	G/Ç	İkinci istem ve atama iğnesi
31	RQ’/GT0’	G/Ç	Birinci istem ve atama iğnesi
32	RD’	Ç	Okuma iğnesi
33	MN/MX’	G	Minimum/maksimum kipi seçici iğnesi
34	BHE’/S7	Ç	Veri ve adres yolu etkinleştirme ve sekizinci durum iğnesi
35	A19/S6	Ç	Adres yolu ve yedinci durum iğnesi
36	A18/S5	Ç	Adres yolu ve altıncı durum iğnesi
37	A17/S4	Ç	Adres yolu ve beşinci durum iğnesi
38	A16/S3	Ç	Adres yolu ve dördüncü durum iğnesi
39	AD15	G/Ç	On altıncı adres ve veri yolu iğnesi
40	VCC	—	+5 V besleme iğnesi

3.2.1 İğne Açıklamaları

2 – 16. ve 32. iğneler (ADDRESS/DATA BUS – ADRES/VERİ YOLU) 2–16 ve 32. iğneler adres ve veri yolu iğneleridir. Bu iğneler zamanda çoğullanmış T1 adres yolu ile T2, T3, TW

ve T4 veri yollarını oluşturmaktadır.

AD0, D7 – D0 veri yolunu seçtirdiğinden BHE' iğnesine benzer bir işlev görmektedir. AD0 T1 süresince mantıksal 0 iken D7 – D0 veri yolundan veri aktarımı yapılabilir. 8 bitlik aygıtlar D7 – D0 veri yoluna bağlı ise AD0 kullanılarak seçme işlemi yapılabilir.

Bu iğneler mantıksal 1 iken etkindirler ve ayrıca kesme alındığında ya da bekletme istemi alındığında ise üç durumlu "kapalı" durumuna geçerler.

Bilgi Notu 3.1: T1, T2, T3, T4, TW ve TI

İşlemcinin her veri yolu çevrimi en az dört tane CLK çevriminden oluşur. Bu çevrimlere T1, T2, T3 ve T4 denir. T1 süresince işlemci adresi verir. T3 ve T4 süresince ise veri alış-verişi gerçekleştirilir. T2 süresi ise okuma işlemleri gerçekleştirilirken veri yolunun yönünün değiştirilmesinde kullanılır.
T1 süresince işlemci tarafından adreslenen birim "Hazır Değilim" sinyali yollarsa T3 ve T4 sürelerinin arasına bir CLK çevrimi uzunluğunda bir TW (Wait – Bekle) süresi eklenir.
Veri yolu çevrimleri arasında bazı boş çevrimler oluşabilir. Bu çevrimler TI (Idle – Boş) ile belirtilir. Bu "boş" çevrimler işlemci tarafından iç işlemlerin yapılmasında kullanılır.

35 – 38. iğneler (ADDRESS/STATUS – ADRES/DURUM) T1 süresince en anlamlı dört adres iğnesi bu iğnelerdir. Girdi/Çıktı işlemleri yapılırken bu iğneler mantıksal 0 durumundadır. T2, T3, TW ve T4 süresince bellek ve girdi/çıktı işlemleri yapılırken bu iğnelerden durum bilgisi okunabilir. Kesme etkinleştirilme bayrak biti S5 her CLK çevrim başlangıcında güncellenmektedir.

A17/S4 ve A16/S3 iğnelerinin farklı durumları için yüklendikleri farklı anlamlar aşağıdaki tabloda verilmiştir. A17/S4 ve A16/S3 iğneleri hangi yer değiştirme yazmacının veri erişiminde kullanılacağını belirtir. S6 iğnesi aşağıdaki her durum için mantıksal 0 durumunda olması gerekmektedir.

Tablo 3.2: *A17/S4 ve A16/S3 iğnelerinin farklı durumları ve anlamları.*

A17/S4	A16/S3	Tip
0	0	Dönüşümlü Veri
0	1	Yığıt
1	0	Kod ya da boş
1	1	Veri

Bu iğneler mantıksal 1 iken etkindirler ve ayrıca kesme alındığında ya da bekletme istemi alındığında ise üç durumlu kapalı durumuna geçerler.

34. İğne (BHE/STATUS – BHE/DURUM) T1 süresince D15 – D8 veri yolunun etkinleştirilmesi için BHE' (Bus High Enable' – Üst Veri yolu Etkinleştirme') iğnesi kullanılmalıdır. 8 bitlik aygıtlar D15 – D8 veri yoluna bağlı ise BHE' kullanılarak seçme işlemi yapılabilir. BHE' iğnesi T1 süresince okuma, yazma ve kesme alma çevrimleri için mantıksal 0 durumunda olmalıdır. S7 durum bilgisi T2, T3 ve T4 süresince alınabilir.

BHE' iğnesi mantıksal 0 konumunda iken etkindir ve üç durumlu kapalı durumuna "bekleme" sinyali alındığında geçer. T1 süresince ilk "kesme alındı" çevrimi için mantıksal 0 durumundadır.

Tablo 3.3: *BHE' ve A0 iğnelerinin farklı durumları ve anlamları.*

BHE'	A0	Tip
0	0	Tam sözcük
0	1	Üst bayt ↔ Tek Adres
1	0	Alt bayt ↔ Çift Adres
1	1	—

32. İğne (READ – OKU) RD' iğnesine gelen tetikleme darbesi, işlemcinin bellek ya da girdi/çıktı okuma çevriminde olduğunu belirtir. Bellek ya da girdi/çıktı okuma çevrimi olması S2 iğnesi tarafından belirlenir.

RD' iğnesi T2, T3 ve TW süresince mantıksal 0 durumunda etkindir ve T2 süresince işlemcinin yerel hattı kayana kadar mantıksal 1 durumunu korumaktadır. Üç durumlu kapalı durumuna "bekleme" sinyali alındığında geçer.

22. İğne (READY – HAZIR) Adreslenen bellek ya da girdi/çıktı işlemi yapan aygıttan işlemin yapılacağına dair alındılama geldiğinde mantıksal 1 durumuna geçer ve bu durumda etkindir. Alınan sinyalin anuyumlu olması 8284A Saat Darbesi Üreteci tarafından sağlanmaktadır.

Bu iğne anuyumlu değildir ve düzgün işlem yapabilmek için kurulum ve bekleme zamanlarının denetlenmesi gerekmektedir.

18. İğne (INTERRUPT REQUEST – KESME İSTEMİ) Her komutun son saat çevriminde işlemcinin kesme alındı işlemi yapmasına veya yapmamasına karar verebilmek için kullandığı bir sinyaldir. Yazılım tarafından kesme etkinleştirme biti sıfırlanarak maskelenebilir. INTR tamamen anuyumludur. Mantıksal 1 konumunda iken etkindir.

23. İğne (TEST – DENEME) `WAIT` komutu işlenirken bu girişe bakılır. Eğer TEST' mantıksal 0 ise işlemci yaptığı işi sürdürür; aksi halde "bekleme alındı" kipine geçer. Bu giriş her saat çevriminin artan ucunda anuyumlu hale getirilir.

17. İğne (NMI) Bu giriş sayesinde ikinci tip bir kesme yaratılabilir. NMI (Non Maskable Interrupt – Maskelenemeyen Kesme) yazılım tarafından maskelenemez. Mantıksal 1 ve 0 geçişi işlenen komutun ardından bir kesme oluşmasına neden olur. NMI girişi anuyumludur.

21. İğne (RESET – BAŞTAN BAŞLAT) İşlemcinin yaptığı işlemi hemen sonlandırmasına neden olur. Bunun için iğnenin en az dört saat çevrimi için mantıksal 1 konumda aktif olması gerekmektedir. İğne mantıksal 0 konumuna getirildiğinde ise işlemci baştan başlatılır. RESET girişi anuyumludur.

19. İğne (CLOCK – SAAT) İşlemci ve veri yolu denetleyicisi için gerekli saat darbesini sağlar. Asimetriktir ve %33 doluluk boşluk oranına sahiptir. Bu sayede en iyi iç zamanlama oluşturulabilmektedir.

33. İğne (MN/MX') Minimum/Maksimum' (MN/MX') iğnesi ile işlemcinin çalıştığı kip seçilebilir. Minimum kipi için mantıksal 1, maksimum kipi için ise mantıksal 0 uygulanmalıdır.

Aşağıda önce maksimum kip için geçerli iğne bilgileri verilecektir. Ardından minimum kip için geçerli iğneler işlenecektir.

26. – 28. İğneler (STATUS – DURUM) T1, T2 ve T4 süresince aktiftir. T3 süresince pasif veya TW süresince READY mantıksal bir ise pasif. Pasif durumda S2', S1' ve S0' mantıksal bir durumundadır.

Bu durum bitleri 8288 Veri Yolu Denetleyicisi tarafından bellek ve girdi/çıktı erişimleri için gerekli sinyalleri üretmede kullanılır.

T4 süresince S2', S1' veya S0' iğnelerinde meydana gelen bir değişiklik bir veri yolu çevriminin başladığının göstergesidir. T3 veya TW süresince pasif duruma geçiş ise veri yolu çevriminin sonlandığını gösterir.

Bu iğneler "bekleme alındılandı" durumu oluştuğunda üç durumlu kapalı durumunu alır.

Tablo 3.4: *S2', S1' ve S0' durum bitlerinin değerleri ve anlamları.*

S2'	S1'	S0'	Tip
0	0	0	Kesme İstemi Alındı
0	0	1	Girdi/Çıktı Kapısı Oku
0	1	0	Girdi/Çıktı Kapısına Yaz
0	1	1	Dur
1	0	0	Kod Erişimi
1	0	1	Bellek Oku
1	1	0	Bellek Yaz
1	1	1	Edilgin

30. ve 31. İğneler (REQUEST/GRANT – İSTEM/ATAMA) Bu iğneler yerel ana veri yolları tarafından işlemcinin devam eden veri yolu çevriminin bitiminde kullanılan veri yolunu salmasını sağlamak için kullanılır. RQ'/GT0' iğnesinin RQ'/GT1' iğnesine göre önceliği vardır.

RQ'/GT' iğneleri kaynak gerilimine bağlı dirençlere (pull-up) sahiptirler ve gerektiğinde boş bırakılabilirler.

İstem ve atama dizisi aşağıdaki biçimde gerçekleşir:

1. Yerel ana veri yolundan gelen bir CLK genişliğindeki sinyal 8086 mikroişlemcisine bir "yerel veri yolu istemi" olduğunu belirtir.
2. T1 veya T4 süresince 8086 mikroişlemcisi bu istemciye bir CLK genişliğinde bir sinyal yollar ise istem kabul edilmiştir ve bir sonraki CLK darbesinde "bekleme alındılandı" durumuna geçecektir. İşlemcinin veri yolu ara yüzü "bekleme" durumuna geçildiğinde mantıksal olarak veri yolundan ayrılır.

3. İstemci tarafından 8086 mikroişlemcisine gönderilen bir CLK genişliğindeki sinyal "bekleme" durumunun biteceğini ve 8086 mikroişlemcisinin bir sonraki CLK darbesinde yerel veri yolunu bağlayabileceğini belirtir.

Her veri yolu değişimi yukarıdaki biçimde üç tane darbe ile yapılmaktadır. Her değişim arasında ise bir CLK genişliğinde bir darbe gönderilmelidir. Mantıksal 0 durumunda darbeler etkindir.

Eğer istem mikroişlemci bir bellek çevriminde iken yapılırsa, mikroişlemci T4 süresince aşağıdaki durumların hepsi gerçeklendiğinde yerel veri yolunu salar:

1. İstem T2 süresince ya da T2'den önce oluşursa,
2. İstem anındaki çevrim bir kelimenin alt baytı değil ise ya da tek adres değil ise,
3. İstem anındaki çevrim bir kesme alındı dizisinin ilk alındılaması değil ise,
4. Kilitlenmiş bir komut işlemci tarafından çalıştırılmamaktaysa.

Eğer yerel veri yolu istem yapıldığında boşta ise iki farklı durum oluşabilir:

1. Bir sonraki saat darbesinde yerel veri yolu salınabilir.
2. Bir bellek çevrimi üç saat darbesi içinde başlayabilir. Yukarıdaki dört kural geçerlidir.

29. İğne (LOCK – KİLİTLE) LOCK' etkin (mantıksal 0) iken yerel ana veri yolları arasında değişim yapılamaz. LOCK' sinyali LOCK komutu tarafından etkinleştirilir ve bir sonraki komut işlenene kadar etkin olarak kalır.

Sinyal etkin iken mantıksal 0 durumundadır ve "bekleme alındılandı" durumuna geçildiğinde üç durumlu kapalı durumunu alır.

24. ve 25. İğneler (QUEUE STATUS – SIRA DURUMU) Sıra işlemi yapıldıktan sonraki CLK çevrimi süresince bu iğne geçerlidir. QS1 ve QS0 iğneleri dışarıdan 8086 mikroişlemcisinin komut sırasını takip etmek için kullanılmaktadır.

Tablo 3.5: *QS1 ve QS0 değerleri ve anlamları.*

QS1	QS0	Tip
0	0	İşlem yok
0	1	İşlem kodunun ilk baytı
1	0	Sırayı boşalt
1	1	Sıradaki bir sonraki bayt

Minimum kip için geçerli iğneler aşağıda tanıtılmıştır.

28. İğne (STATUS LINE – DURUM HATTI) Mantıksal olarak maksimum kipteki S2 iğnesine eşdeğerdir. Bellek erişimi mi yoksa girdi/çıktı erişimi mi yapıldığını anlamak için kullanılır. M/IO' veri yolu çevriminden sonra gelen T4 süresince geçerlidir ve çevrim sonuna kadar T4 süresince geçerlidir.

"Bekleme alındılandı" durumuna geçildiğinde M/IO' üç durumlu kapalı durumunu alır.

29. İğne (WRITE' – YAZ) Bu iğne sayesinde işlemcinin belleğe yazma işlemi yaptığını veya girdi/çıktı sisteminde bir yazma işlemi yaptığı anlaşılır. Bir yazma işlemi yapılırken WR' iğnesi T2, T3 ve TW süresince etkindir.

WR' iğnesi mantıksal 0 durumunda iken etkindir ve "bekleme alındılandı" durumunda üç durumlu kapalı durumunu alır.

24. İğne (INTERRUPT ACKNOWLEDGE – KESME ALINDILANDI) INTA' iğnesi kesme alındı çevrimlerinin belirlenmesinde kullanılır. T2, T3 ve TW süresince her kesme alındılandı çevriminde etkindir.

Mantıksal 0 durumunda iken etkindir.

25. İğne (ADDRESS LATCH ENABLE – ADRES MANDALI ETKİNLEŞTİRME) 8282/8283 adres mandalına işlemci tarafından adreslerin mandallanabilmesi için kullanılır. Mantıksal 1 durumunda iken etkindir.

T1 süresince veri yolu çevrimlerinde etkindir.

27. İğne (DATA TRANSMIT/RECEIVE – VERİ İLETİM/ALIŞ) 8286/8287 alıcı-verici yongaları kullanılmak istenen ve minimum kipte çalışan sistemlerde kullanılan iğnedir. Alıcı-verici yongasından geçen verilerin yönünün ayarlanmasında kullanılır.

Mantıksal olarak DT/R' iğnesi maksimum kipteki S1' iğnesine eşdeğerdir ve zamanlaması ise M/IO' iğnesine eşdeğerdir. İğne yongayı mantıksal 1 durumunda iken verici, mantıksal 0 durumunda ise alıcı olarak ayarlar.

"Bekleme alındılandı" durumunda üç durumlu kapalı durumunu alır.

26. İğne (DATA ENABLE – VERİ ETKİNLEŞTİR) 8286/8287 alıcı-vericisi kullanan minimum kipteki sistemler tarafından kullanılır. Alıcı-verici tümleşik devresinin çıkışını etkinleştirmek için kullanılır. Bellek ve girdi/çıktı işlemleri süresince ayrıca INTA çevrimlerinde mantıksal sıfır durumundadır ve etkindir.

Okuma ve INTA' çevrimlerinde T2'nin ortasından T4'ün ortasına kadar etkindir. Yazma çevrimlerinde ise T2'nin başından T4'ün ortasına kadar etkindir.

"Bekleme alındılandı" durumunda üç durumlu kapalı durumunu alır.

30. ve 31. İğneler (HOLD/HLDA – BEKLETME/BEKLETME ALINDILANDI) Başka bir ana veri yolunun bir yerel veri yolu "beklemesi" istemi gönderdiğini belirtir. Alındılanma yapılabilmesi için HOLD'un mantıksal 1 durumunda olması gerekmektedir. HOLD mantıksal 1 durumunda iken etkindir. "Bekletme" istemini alan mikroişlemci T4'ün ya da T1'in ortasında HLDA iğnesini etkinleştirir. HLDA iğnesi mantıksal 1 durumunda iken etkindir. HLDA etkinleştirildikten sonra mikroişlemci yerel veri yolunu ve denetim hatlarını boşa alır.

HOLD mantıksal 0 durumuna getirildiğinde ise mikroişlemci HLDA iğnesini mantıksal 0 durumuna getirir. İşlemci bundan sonra bir çevrime girerse yerel veri yolunu ve denetim hattını yeniden bağlar.

HOLD ve HLDA iğneleri kaynak gerilimine bağlanan dirençlerine sahiptirler.

RQ'/GT' için yerel veri yolunun boşa alınması için açıklanan kuralların hepsi HOLD ve HLDA iğneleri için de geçerlidir.

HOLD eşzamansız bir giriş değildir. Eğer sistem belirli bir kurulum süresini garantileyemiyorsa dışarıdan bir eşzamanla uygulanması gerekebilmektedir.

3.3 8086 İç Mimarisi: EU ve BIU

8086 mikroişlemcisi işlevsel olarak iki ana birime ayrılır: **Yürütme Birimi** (EU – Execution Unit) ve **Veri Yolu Arabirimi Birimi** (BIU – Bus Interface Unit). Bu iki birimin koşut çalışması 8086'nın başarımını artıran temel tasarım kararlarından biridir.

3.3.1 Veri Yolu Arabirimi Birimi (BIU)

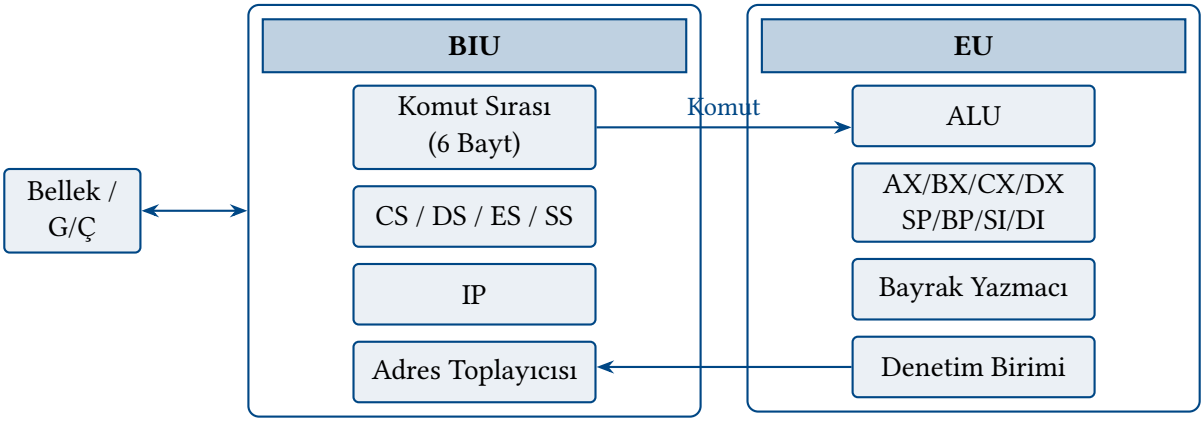
BIU, 8086 ile bellek ve girdi/çıktı aygıtları arasındaki tüm veri alışverişini yönetir. BIU'nun başlıca bileşenleri şunlardır:

- **Komut Sırası (Instruction Queue):** 6 bayt kapasiteli bir ön yükleme tamponu. BIU, veri yolu boş olduğu sürece bellekten komut baytlarını önceden okuyarak bu sıraya yerleştirir.
- **Bölüt Yazmaçları:** CS, DS, ES ve SS yazmaçları burada yer alır.
- **IP (Komut İşaretçisi):** İşlenecek bir sonraki komutun göreceli konum adresini tutar.
- **Adres Toplayıcısı (Address Adder):** Bölüt adresi ile göreceli konum adresini birleştirerek 20 bitlik gerçek adresi hesaplar.

3.3.2 Yürütme Birimi (EU)

EU, BIU'nun komut sırasından aldığı komutları çözümler ve çalıştırır. EU'nun başlıca bileşenleri şunlardır:

- **ALU (Aritmetik Mantık Birimi):** Toplama, çıkarma, VE, VEYA gibi aritmetik ve mantık işlemlerini gerçekleştirir.
- **Genel Amaçlı Yazmaçlar:** AX, BX, CX, DX, SP, BP, SI, DI yazmaçları.
- **Bayrak Yazmacı:** İşlem sonuçlarına göre güncellenen CF, ZF, SF, OF gibi durum bitlerini tutar.
- **Denetim Birimi:** Komutları çözümler ve diğer birimleri yönlendirir.



Şekil 3.2: 8086 mikroişlemcisinin EU (Yürütme Birimi) ve BIU (Veri Yolu Arabirimi Birimi) iç mimari blok diyagramı.

EU ile BIU'nun koşut çalışması sayesinde EU bir komutu yürütürken BIU aynı anda bellekten sonraki komut baytlarını önceden okuyabilir. Bu yaklaşıma **komut ön yükleme** (instruction prefetch) denir ve 8086'nın etkin saat kullanımını artırır.

3.4 Bellek

8086'da komutlar (kod), veri ve yığıt aynı bellek alanlarında tutulur. 8086'nın 20 bitlik adres yolu sayesinde $2^{20} = 1$ MB'lık bellek erişimi sağlanabilir. Ancak işlemcide işlenebilen komutlarının pek çoğunun adresleme alanı 16 bitlik olduğundan, çoğu komutla $2^{16} = 65536$ bayt = 64 kB bellek erişimi yapılabilmektedir. 64 kB bellek erişiminin üzerine çıkabilmek için özel bölüt yazmaçları kullanılmaktadır.

16 bitlik veriler ve işaretçiler aşağıdaki gibi tutulur:

Tablo 3.6: *8086'da 16 bitlik verilerin bellekte saklanma düzeni.*

ADRES + 0000H	ADRES + 0001H
Alt Bayt	Üst Bayt

Dikkat edilirse adresin alt baytı ilk, üst baytı bir sonraki adreste tutulmaktadır. Böyle bir durumun oluşmasının nedeni "Küçüğü Başta" ya da "Sağdan Anlamlı" (Little endian) Kuralı'dır. Bu kurala ileriki bölümlerde değinilecektir.

32 bitlik adresler `BÖLÜT ADRESİ : GÖRECELİ KONUM ADRESİ` biçiminde gösterilir. Bu ikili adresleme şekli ile gerçek bir adrese işaret edilir. 8086'da 32 bitlik adres aşağıdaki şekilde tutulur:

Tablo 3.7: *8086'da 32 bitlik adreslerin bellekte saklanma düzeni.*

ADRES + 0000H	ADRES + 0001H	ADRES + 0002H	ADRES + 0003H
Alt Bölüt Baytı	Üst Bölüt Baytı	Alt Göreceli Konum Baytı	Üst Göreceli Konum Baytı

Bölüt ve göreceli konum kavramları çevirici dilinin komutları incelenirken daha ayrıntılı işlenecektir.

Bilgi Notu 3.2: Gerçek Adres, Sanal Adres ve Göreceli Konum Adresi

8086 işlemcisinde gerçek adres, 20 iğnelik adres kapısının iğnelerine atanan 20 bitlik adrestir. Bu 20 bitlik adres bellek arayüz devrelerindeki kod çözücüler tarafından çözülür. Böylece 8086'da `00000H`'dan başlayarak `FFFFFH`'ye kadar olan ROM ya da RAM'daki 1 MB'lık bellek bölgesi adreslenebilmektedir.
Göreceli konum adresi bir bölüt içinde kalan ve `0000H`'dan başlayarak `FFFFH`'ye kadar olan bölgeyi adreslemektedir.
Sanal adres 2 kısımdan oluşur: birinci kısım bölüt adresini, ikinci kısım ise göreceli konum adresini belirtir. `0200H:0000H` sanal adresinde `0200H` bölüt adresi ve `0000H` göreceli konum adresidir. Sanal adres verildiğinde gerçek adres bulunabilir. Bunun için bölüt adresi bir basamak sola kaydırılır (ya da `10H` ile çarpılır) ve göreceli konum adresiyle toplanır. Örneğin `0200H:0000H` sanal adresine denk gelen gerçek adres:

$$0200\text{H} \times 10\text{H} = 02000\text{H}$$
$$02000\text{H} + 0000\text{H} = 02000\text{H}$$

şeklinde hesaplanır ve gerçek adresin `02000H` olduğu bulunur.
Burada dikkat edilmesi gereken nokta iki ayrı sanal adresin aynı gerçek adresle eşleşebileceğidir. Örneğin `0000H:2000H` sanal adresine denk gelen gerçek adres de `02000H`'dır. Değişik bölüt adresi – göreceli konum adresi karışımları aynı gerçek adrese işaret edebilir.

3.4.1 Komut (Kod) Belleği

Yazılan kodlar belleğin herhangi bir yerinde tutulabilir. Zorunlu olarak yazılması gereken bir yer yoktur. Aslında kod mikroişlemcinin anlayabildiği komutlar ve bu komutlar ile birlikte gelen verilerdir.

Örneğin aşağıdaki çevirici kodu incelenirse:

```
1  MOV AL, 'S'
2  MOV BL, AL
3  MOV AL, 'C'
4  MOV BX, 1234H
```

Kod 3.1: Bellek içeriği örnek kodu

Birinci satırda AL yazmacına 'S' ASCII karakterinin değeri olan `53H` sayısı kaydedilir.

Ardından ikinci satırda BL yazmacına AL yazmacının değeri kaydedilir. AL yazmacının o andaki değeri birinci satırdaki işlemden dolayı 'S' ASCII karakterinin değeri olduğundan BL yazmacına 'S' karakterinin ASCII değeri yazılır.

Üçüncü satırda AL yazmacına 'C' ASCII karakterinin değeri olan `43H` sayısı kaydedilir. Dördüncü satırda ise `MOV BX, 1234H` komutu ile BX yazmacına `1234H` değeri yüklenir; bu işlem sonucunda BH yazmacına `12H`, BL yazmacına ise `34H` değeri yazılır.

Yukarıdaki kodun işlevinin anlaşılması bu aşamada önemli değildir. Asıl önemli nokta yukarıdaki çevirici kodunun nasıl bir koda çevrileceği ve mikroişlemcinin bu kodu nasıl işleyeceğidir.

Yukarıda yazdığımız çevirici kodu herhangi bir çevirici ile çevrilip belleğin `0000:0000` adresine kaydedilirse aşağıdaki tablodaki koda dönüşür:

Tablo 3.8: *Çevirici kodunun `0000:0000` adresinden itibaren bellekteki gösterimi.*

Bellek Adresi	Adreste Saklanan Veri
0000:0000	B0H
0000:0001	53H
0000:0002	8AH
0000:0003	D8H
0000:0004	B0H
0000:0005	43H
0000:0006	BBH
0000:0007	34H
0000:0008	12H

Mikroişlemcinin komutları okumaya `0000:0000` adresinden başladığını varsayarak kod mikroişlemcinin işlediği biçimde aşağıdaki gibi işlenir:

1. `0000:0000` bellek adresi okunur.
2. Okunan veri `MOV AL, DEĞER` komutuna karşılık gelmektedir.
3. AL yazmacı 8 bit olduğundan 1 baytlık veri okunacağı anlaşılır.
4. Bir sonraki bellek adresi olan `0000:0001` adresi okunur.
5. `53H` verisi okunur. Bu veri, komutun özelliği gereği AL yazmacına yazılmalıdır.
6. `0000:0002` bellek adresi okunur.
7. Okunan veri `8AH`'dır. Bu veri `MOV BL, YAZMAÇ` komutuna karşılık gelmektedir.
8. BL yazmacının 8 bitlik bir yazmaç olduğu bilindiğinden 1 baytlık veri okunur.
9. `0000:0003` bellek adresindeki veri okunur.
10. `D8H` verisi AL yazmacını simgeler. Bu yüzden AL yazmacının değeri BL yazmacına kaydedilir.
11. `0000:0004` bellek adresi okunur.
12. Okunan kod `B0H`'dır. Bu kod `MOV AL, DEĞER` komutuna karşılık gelmektedir.
13. AL yazmacı 8 bitlik olduğundan 1 bayt veri okunmalıdır.
14. `0000:0005` bellek adresindeki veri okunur.
15. `43H` verisi AL yazmacına kaydedilir.
16. `0000:0006` bellek adresi okunur.
17. Okunan veri `BBH`'dır. Bu veri `MOV BX, DEĞER` komutuna karşılık gelmektedir.
18. BX yazmacı 16 bitlik olduğundan 2 bayt okunmalıdır.
19. `0000:0007` adresindeki veri okunur ve BX yazmacının alt baytına yazılır.
20. `0000:0008` adresindeki veri okunur ve BX yazmacının üst baytına yazılır.

Yukarıdaki adımlar izlenerek çevirici (assembly) dilinde yukarıdaki çevirici kodu ile ifade edilen işlemler yapılmıştır.

Aslında mikroişlemci için kod, veri ya da yığıt gibi kavramlar yoktur. Mikroişlemci yalnızca birler ve sıfırlarla ilgilenir. Mikroişlemci, sistem ilk açıldığında belleğin bir hücresine erişerek

1 bayt okur. Bu baytın değeri eğer mikroişlemcide bir komut olarak algılanıyorsa, bu algılanan komutun işlevine göre bir sonraki bayt okunur ya da okunmaz. Sistemin tüm işleyişi okunan ilk veriye bağlıdır.

3.4.2 Veri Belleği

Mikroişlemci kod bölütüne, veri bölütüne, yığıt bölütüne ve ek bölütteki verilere erişebilir. Her bölüt 64 kB'lık bir bellek alanını adresleyebildiğinden toplamda 256 kB veriye erişilmesi mümkündür. Yığıt, kod, veri ve ek bölütün tam olarak ne olduğu bir sonraki bölümde açıklanacaktır.

Veri belleğine veri kaydederken ya da bu verilere erişirken dikkat edilmesi gereken bir durum vardır. Eğer 16 bitlik bir veri bellek adresi tek sayı olan bir bellek hücresinden başlanarak kaydedilecekse işlemci bunu iki bellek erişimiyle tamamlayabilir. 16 bitlik veri adresi çift sayı olan bir bellek hücresinden başlanarak kaydedilecekse tek bellek erişimiyle kayıt tamamlanır.

3.4.3 Yığıt Belleği

Yığıt belleği, belleğin herhangi bir yerinde bulunabilir. Yığıt bir sonraki bölümde açıklanacağı gibi geçici kayıt işlemlerinde kullanılabilmektedir. Yığıta erişim sağlayan komutlar bazı yazmaç erişim komutlarından çok daha hızlı olduğundan geçici kayıt işlemlerinde kullanılabilmektedir.

Veri belleği erişiminde olduğu gibi yığıt erişiminde sonu sıfırla biten adresten başlamak başarımı artırır.

3.4.4 Ayrılmış Bölgeler

Belleğin bazı bölgeleri, bir takım özel verileri tutmak için ayrılmıştır. Örneğin belleğin `0000H` ile `03FFH` adresleri arasında kalan bellek alanı kesme yöneyleri (vektörleri) için ayrılmıştır. `FFFF0H` ya da `FFFFFH` adresleri bilgisayar ilk açıldığında ya da yeniden başlatıldığında başlangıç adresi olarak kullanılır.

3.5 Kesmeler

Kesmeler, işlemcinin içinde çalışan programın akışını durdurup denetimi özel bir kesme yordamına aktaran özel komutlar ya da sinyallerdir. Çoğu kesme komutunun işlenmesinin ardından kesme yordamı işini tamamladığında program kaldığı yerden çalışmayı sürdürür. Kesmeler yazılım ya da donanım kullanılarak uygulanabilir. Kesmeler konusu yedinci bölümde ayrıntılı olarak incelenecektir.

3.6 Girdi/Çıktı Kapıları

8086'da girdi/çıktı işlemleri bellek adresleri ile aynı biçimdedir ve 65536 tane bayt yazmacını ya da 32768 tane sözcük yazmacını adresleyebilir. İşlemcinin dış dünya ile olan bağlantısını sağlayan girdi/çıktı kapıları birçok işlemde kullanılır. Girdi/çıktı kapıları ve bu kapıların kullanımları sekizinci bölümde ayrıntılı olarak incelenecektir.

3.7 Yazmaçlar

8086'nın sahip olduğu yazmaçlar aşağıda sıralanmıştır:

- **Genel Amaçlı Yazmaçlar:** AX, BX, CX, DX
- **Dizin Yazmaçları:** DI, SI
- **İşaretçi Yazmaçları:** SP, BP
- **Bölüt Yazmaçları:** CS, DS, ES, SS
- **Özel Yazmaçlar:** IP, Bayrak Yazmacı

Bilgi Notu 3.3: Genel Amaçlı Yazmaçlar

AX, BX, CX, DX yazmaçlarına genel amaçlı yazmaçlar da denir. Bunun nedeni yalnızca belirtilen işler için değil, tamamen farklı işler için de kullanılabilmeleridir. Örneğin bu yazmaçlar geçici program değişkenlerinin değerlerini tutmak için de kullanılırlar. Benzer biçimde, özel yöntemler kullanılarak, bütün diğer yazmaçlar da başka işler için kullanılabilir.

8086'nın yukarıda sıralanan yazmaçları aşağıda tek tek açıklanmıştır:

3.7.1 CS Yazmacı

CS (Code Segment – Kod Bölütü) yazmacı 16 bitlik bir yazmaçtır. CS yazmacı işlemci tarafından IP yazmacı ile birlikte işlemcinin işleyeceği bir sonraki komutun adresi olan CS:IP adresini belirtir. CS yazmacının değeri herhangi bir atama ile doğrudan değiştirilemez. Uzun mesafeli CALL (CALL FAR) komutu, uzun mesafeli geri dönme RET komutu ve uzun mesafeli JMP (JMP FAR) komutları ile değeri dolaylı olarak değiştirilebilir.

3.7.2 SS Yazmacı

SS (Stack Segment – Yığıt Bölütü) yazmacı CS yazmacı gibi 16 bitliktir. İşlemci SP ve BP yazmaçları ile adreslenen verilerin yığıt bölütünde olduğunu kabul eder; kodda yazılmasa bile SP SS:SP ve BP SS:BP olarak kullanılır. SS yazmacının değeri POP komutu ile değiştirilebilir.

3.7.3 DS Yazmacı

DS (Data Segment – Veri Bölütü) yazmacı 16 bitliktir. İşlemci AX, BX, CX, DX, SI, DI yazmaçları ile adreslenen verilerin veri bölütünde olduğunu kabul eder. POP ya da LDS komutları ile değeri değiştirilebilir.

3.7.4 ES Yazmacı

ES (Extra Segment – Ek Bölüt) yazmacı 16 bitliktir. Dizgi işlemlerinde işlemci DI yazmacı ile adreslenen verinin ek bölütte bulunduğunu varsayar. POP ya da LES komutları ile değeri değiştirilebilir.

Bilgi Notu 3.4: Bölüt Yazmaçları

İşlemcinin CS, ES, DS ve SS'de yapıldığını varsaydığı adreslemeler, yazılan kodun başına ES, CS, DS ya da SS yerleştirilmesi yoluyla kullanılan bölüt açıkça belirtilerek ortadan kaldırılabilir.

3.7.5 AX Yazmacı

AX (Accumulator Register – Birikeç Yazmacı) 16 bitlik bir yazmaçtır. Üst ve alt 8 biti sırasıyla AH ve AL yazmaçları olarak kullanılır. Birikeç yazmacı olan AX genellikle dizgi işlemlerinde ve girdi/çıktı işlemlerinde kullanmaya uygundur.

AX (16 bit)	→	AH (8 bit)	AL (8 bit)

3.7.6 BX Yazmacı

BX (Base Register – Taban Yazmacı) 16 bitlik bir yazmaçtır. Üst ve alt 8 biti sırasıyla BH ve BL yazmaçları olarak kullanılır. BX yazmacının değeri genellikle adreslemelerde taban adresi olarak kullanıldığı için taban yazmacı adını almıştır.

3.7.7 CX Yazmacı

CX (Count Register – Sayaç Yazmacı) 16 bitlik bir yazmaçtır. Üst ve alt 8 biti sırasıyla CH ve CL yazmaçları olarak kullanılır. Sayaç yazmacı genellikle dizgi işlemleri gibi işlemlerde sayaç olarak kullanıldığından bu adı almıştır.

3.7.8 DX Yazmacı

DX (Data Register – Veri Yazmacı) 16 bitlik bir yazmaçtır. Üst ve alt 8 biti sırasıyla DH ve DL yazmaçları olarak kullanılır. Girdi/çıktı komutlarında kapı numarası ve matematiksel işlemlerde ek veri tutulan yazmaçtır. Bu yüzden veri yazmacı adını almıştır.

3.7.9 SP Yazmacı

SP (Stack Pointer – Yığıt İşaretçisi) yazmacı 16 bitlik bir yazmaçtır. İçerdiği değer yığıtı işaretlemektedir.

3.7.10 BP Yazmacı

BP (Base Pointer – Taban İşaretçisi) yazmacı 16 bitliktir. İçerdiği değer yığıttaki bir veriyi işaretlemektedir.

3.7.11 SI Yazmacı

SI (Source Index – Kaynak Dizini) yazmacı 16 bitliktir. Adreslemelerde kaynak adresi olarak kullanılır.

3.7.12 DI Yazmacı

DI (Destination Index – Hedef Dizini) yazmacı 16 bitliktir. Adreslemelerde hedef adresi olarak kullanılır.

3.7.13 IP Yazmacı

IP (Instruction Pointer – Komut İşaretçisi) daha önce CS yazmacında da belirtildiği gibi CS yazmacı ile birlikte işlenecek olan komuta işaret eder.

3.7.14 Bayrak Yazmacı

Bayrak yazmacı 16 bitliktir ve içerdiği bitlerden 9'u bayrak biti olarak kullanılabilir.

Tablo 3.9: *8086 bayrak yazmacının 16 bitlik düzeni.*

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
—	—	—	—	OF	DF	IF	TF	SF	ZF	—	AF	—	PF	—	CF

OF (Overflow Flag – Taşma Bayrağı) biti işaretli sayılar üzerinde yapılan işlemlerin sonucunda sonuç −128 ile 127 sayıları arasında değilse 1 değerini alır. Aksi halde 0 değerindedir.

Örnek Örnek 3.1

`64H` (−100) ve `3CH` (−60) işaretli sayıları toplandığında sonuç `FF60H` (−160) olarak bulunur. −160 sayısı −128'den küçük olduğu için bir taşma oluşur ve OF bitinin 1 durumuna geçmesi gerekir.

DF (Direction Flag – Yön Bayrağı) biti bazı komutlar tarafından kullanılmaktadır. Eğer bu bayrak bitinin değeri 0 ise işlemler ileriye doğru, 1 ise işlemler geriye doğru yapılır. Dizgi işlemleri yapan komutlarda kullanılır.

IF (Interrupt Flag – Kesme Bayrağı) biti 1 olduğu zaman işlemci çevre birimlerinden gelen kesmelere cevap verir. Aksi halde kesmeler yanıtlanmaz.

TF (Trap Flag – Yakalama Bayrağı) biti bazı özel uygulamalarda kullanılır. Örneğin bir kod adım adım çalıştırılmak isteniyorsa bu bayrak bitine 1 değeri verilir. Böylece bir sonraki komutun ardından bir yakalama kesmesi oluşur.

SF (Sign Flag – İşaret Bayrağı) bitinin değeri 1 ise yapılan son işlemin sonucu sıfırdan küçük, 0 ise yapılan işlemin sonucu sıfırdan büyüktür.

ZF (Zero Flag – Sıfır Bayrağı) biti 1 ise yapılan işlemin sonucu 0'dır. Bitin değeri 0 ise son yapılan işlemin sonucu 0'dan farklıdır.

AF (Auxiliary Flag – Yardımcı Bayrak) biti 1 ise yapılan işlemin sonucunun alt 4 bitinde bir taşma oluşmuş demektir. Yalnızca işaretsiz sayılar için geçerlidir.

PF (Parity Flag – Eşlik Bayrağı) biti 1 ise yapılan işlemin sonucunda ortaya çıkan değerde çift sayıda 1 bulunmaktadır. PF biti 0 ise yapılan işlemin sonucunda tek sayıda 1 bulunmaktadır.

CF (Carry Flag – Elde Bayrağı) bitine işaretsiz sayılarla işlem yapılırken elde oluştuğu zaman 1 değeri kaydedilir. Aksi halde değeri 0 değerine sahiptir.

Örnek Örnek 3.2

`7FH` (onluk tabanda 127) ile `C8H` (onluk tabanda 200) sayılarını toplarsak: 127 + 200 = 327 = `0147H`. Görüldüğü gibi bu işlemin sonucunu tutmak için 1 bayt (2 onaltılık basamak) yetmemiştir ve bir üst bayt kullanılması gerekmiştir. Böyle durumlarda CF biti 1 değerini alır.

3.8 Komutlar

8086 mikroişlemcisinin komut kümesi şu alt başlıklara ayrılabilir:

Veri Taşıma Komutları: `MOV, PUSH, POP, XCHG, IN, OUT, XLAT, XLATB, LEA, LDS, LES, LAHF, SAHF, PUSHF, POPF`

Aritmetik İşlem Komutları: `ADD, ADC, INC, AAA, DAA, SUB, SBB, DEC, NEG, CMP, AAS, DAS, MUL, IMUL, AAM, DIV, IDIV, AAD, CBW, CWD`

Mantık İşlemi Komutları: `NOT, SHL, SAL, SHR, SAR, ROL, ROR, RCL, RCR, AND, TEST, OR, XOR`

Dizgi İşleme Komutları: `REP / REPE / REPNE / REPNZ / REPZ, MOVSB / MOVSW, CMPSB / CMPSW, SCASB / SCASW, LODSB / LODSW, STOSB / STOSW`

Denetimi Aktarma Komutları: `CALL, JMP, RET / RETF, JE / JZ, JL / JNGE, JLE / JNG, JB / JNAE, JBE / JNA, JP / JPE, JO, JS, JNE / JNZ, JNL / JGE, JNLE / JG, JNB / JAE, JNBE / JA, JNP / JPO, JNO, JNS, LOOP, LOOPZ / LOOPE, LOOPNZ / LOOPNE, JCXZ, INT, INTO, IRET`

İşlemci Denetimi Komutları: `CLC, CMC, STC, CLD, STD, CLI, STI, HLT, WAIT, ESC, LOCK, NOP`

8086 mikroişlemcisinin komut kümesi 4. Bölüm'de ayrıntılı olarak incelenecektir.

Yukarıda verilen komutlar makine diline çevrilerek işlemcinin iç elektronik donanımı tarafından kullanılmaya uygun hale getirilir. İşlemci için bir komutun MOV olması herhangi bir anlam taşımaz. İşlenen komutun 0 ve 1'lerden oluşan bir makine dili karşılığı olmalıdır. Makine dili 4. Bölüm'de ayrıntılı olarak açıklanacaktır.

3.9 Adresleme Kipleri

8086 mikroişlemcisinin kullandığı birçok adresleme kipi vardır. 8086'nın ve bu işlemcinin kullandığı çevirici dilinin kullanılabilmesi için adresleme kiplerinin çok iyi anlaşılması gerekir.

8086 mikroişlemcisinin adresleme kipleri 2 ana başlığa ayrılabilir:

- Yazmaç Adresleme Kipleri
- Bellek Adresleme Kipleri

3.9.1 Yazmaç Adresleme Kipleri

Yazmaçlar, bellek bölgelerine göre işlemciye daha yakın olduğundan erişim hızı belleğe göre çok yüksektir. Yazılımlar yazılırken bu yazmaçların bu özelliği göz önüne alınırsa başarım ciddi oranda artırılabilir.

Daha önce de belirtildiği gibi 8086 mikroişlemcisinde 16 bitlik yazmaçlar bulunmaktadır. 8086 mikroişlemcisinin komutları bu yazmaçlarla çalışabilmektedir. İşlenen kullanan bir komutta işlenen olarak bir yazmaç kullanılarak yazmaç adreslemesi yapılabilir.

Her komut çeviriciden (assembler) geçip makine diline çevrilir. Genelde çevirici dilindeki komutların tek bir makine dili karşılığı yoktur. Komutların farklı adresleme kipleri ve bu adresleme kiplerinde kullanılan yazmaçlar için değişik türevleri mevcuttur. Örneğin yazılımlar çevirici dili kullanılarak yazıldığında, MOV komutu yazmaçların verilerini bir yerden başka bir yere taşımada kullanılabileceği gibi belleğin bir adresindeki veriyi başka bir bellek adresine taşımada ya da bu işlemlerin hepsini karışık olarak yapmada kullanılabilir.

Aşağıdaki kod parçasında MOV komutunun yazmaç adreslemesi yapan bazı örnekleri verilmiştir:

```
MOV AL, CH      ; CH yazmacindaki veri AL yazmacina kaydedilmektedir.
MOV BX, DX      ; DX yazmacindaki veri BX yazmacina aktarilir.
MOV SI, DX      ; DX yazmacindaki veri SI yazmacina aktarilir.
MOV SP, BP      ; BP yazmacindaki veri SP yazmacina kaydedilir.
```

Kod 3.2: Yazmaç adresleme örnekleri

Birinci satırdaki kod (`MOV AL, CH`) ile CH yazmacındaki veri AL yazmacına kaydedilmektedir.

İkinci satırdaki kod (`MOV BX, DX`) ile DX yazmacındaki veri BX yazmacına aktarılır.

Üçüncü satırdaki kod (`MOV SI, DX`) ile ise DX yazmacındaki veri SI yazmacına aktarılır.

Dördüncü satırdaki kod (`MOV SP, BP`) ile ise BP yazmacındaki veri SP yazmacına kaydedilir.

Yukarıda yapılan tüm işlemler yazmaç adreslemesine örnektir. Görüldüğü gibi MOV komutunun buradaki kullanım biçimiyle yalnızca yazmaçlar arasında işlem yapılmıştır.

3.9.2 Bellek Adresleme Kipleri

8086 mikroişlemcisinin değişik bellek adresleme kipleri bulunmaktadır.

Doğrudan Adresleme Kipi

Doğrudan adresleme kipinde 16 bitlik sabit bir değer kullanılır. Bu sabit değer adres olarak kullanılarak belleğe erişilir.

Örnek Örnek 3.3

Aşağıdaki kod parçası doğrudan adreslemeye örnektir:

```
MOV AL, DS:[1234H]    ; DS:1234 adresinden 1 baytlik veri AL'ye kaydedilir.
MOV DS:[1234H], AX    ; AX yazmacindaki 2 baytlik veri DS:1234 adresine yazilir.
```

Kod 3.3: Doğrudan adresleme örneği

Kod parçasının birinci satırıyla (`MOV AL, DS:[1234H]`) DS:1234 adresinden 1 baytlık veri AL yazmacına kaydedilmektedir.
İkinci satırda AX yazmacındaki iki baytlık veri DS:1234 adresine yazılmaktadır.
Yukarıda da görüldüğü gibi 16 bitlik sabit `1234H` değeri adres olarak kullanılarak bellek adreslemesi yapılmıştır.

Yazmaç Değeriyle Dolaylı Adresleme Kipi

Yazmaç değeriyle dolaylı adresleme kipi kullanıldığında bir yazmacın değeri adres olarak kullanılarak herhangi bir bellek adresindeki veriler değiştirilebilir. Bu adresleme kipinin 4 farklı çeşidi vardır. Aşağıdaki kod parçasında bu dört farklı çeşit adresleme kipine örnekler sıralanmıştır:

```
1  MOV AL, [BX]    ; AL <-- DS:[BX] adresindeki veri
2  MOV AL, [BP]    ; AL <-- SS:[BP] adresindeki veri
3  MOV AL, [SI]    ; AL <-- DS:[SI] adresindeki veri
4  MOV AL, [DI]    ; AL <-- DS:[DI] adresindeki veri
```

Kod 3.4: Yazmaç değeriyle dolaylı adresleme örnekleri

Yukarıdaki kod parçasında ilk satırda gösterilen adresleme yönteminde (`MOV AL, [BX]`), bellek erişimi için kullanılacak adresin oluşturulması için bölüt adresi değeri DS yazmacının o andaki değeri, göreceli konum adresinin değeri ise BX yazmacının değeri olarak alınır. DS yazmacının değeri ile BX yazmacının değerinin birlikte oluşturduğu 32 bitlik adres erişilecek bellek adresidir. Başka bir deyişle, `MOV AL, [BX]` satırındaki adresleme yöntemiyle AL yazmacına DS:[BX] adresindeki veri kaydedilmiştir.

İkinci satırdaki adresleme yönteminde (`MOV AL, [BP]`), göreceli konum adresini BP yazmacının değeri belirlerken, bölüt adresi olarak SS yazmacının değeri kullanılır. Böylece AL yazmacına SS:[BP] adresindeki veri kaydedilmiştir.

Üçüncü satırda gösterilen adresleme yönteminde (`MOV AL, [SI]`), göreceli konum adresi olarak SI yazmacının değeri, bölüt adresi olarak da [BX] adreslemesinde olduğu gibi DS bölüt yazmacının değeri kullanılır. Böylece AL yazmacına DS:[SI] adresindeki veri yazılır.

Dördüncü satırda gösterilen adreslemede (`MOV AL, [DI]`) ise göreceli konum adresi olarak DI yazmacının değeri ve bölüt adresi olarak DS kesim yazmacının değeri kullanılır. Böylece AL yazmacına DS:[DI] adresindeki veri yazılır.

Eğer varsayılan bölüt adresi yazmacı ([BP] için SS yazmacı, [BX], [SI] ve [DI] için DS yazmacı) yerine başka bir bölüt adresi yazmacının kullanılması istenirse satırın başında istenilen bölüt yazmacı açıkça belirtilmelidir. Örneğin [BX] için CS bölüt yazmacı, [SI] için DS bölüt yazmacı, [DI] için SS bölüt yazmacı ve [BP] için ES bölüt yazmacının kullanılması için kod parçası aşağıdaki biçimde yazılmalıdır:

```
1  MOV AL, [CS:BX]     ; CS:BX adresindeki veri
2  MOV AL, [ES:BP]     ; ES:BP adresindeki veri
3  MOV AL, [DS:SI]     ; DS:SI adresindeki veri
4  MOV AL, [SS:DI]     ; SS:DI adresindeki veri
```

Kod 3.5: Bölüt yazmacı atama örnekleri

Varsayılan bölüt yazmaçlarının yerine bölüt yazmacı atanmasının uygulaması çeviriciden çeviriciye farklılık gösterir. FASM çeviricisinde bölüt geçersiz kılımı (segment override) yukarıdaki gibi köşeli parantez içinde `BÖLÜT:ADRES` biçiminde yazılır.

Eklemeli Dizinle Adresleme Kipi

Eklemeli dizinle adresleme yönteminde yazmaçla dolaylı adresleme kipine benzer bir yol izlenir. Yazmacın yalnız başına adres olarak kullanılması yerine yazmaçtan okunan değer bir sayıyla toplanır.

```
MOV AL, [BX+20H]    ; AL <-- DS:[BX+20H]
MOV AL, [BP+03H]    ; AL <-- SS:[BP+03H]
MOV AL, [SI+04H]    ; AL <-- DS:[SI+04H]
MOV AL, [DI+07H]    ; AL <-- DS:[DI+07H]
```

Kod 3.6: Eklemeli dizinle adresleme örnekleri

Yukarıdaki kod parçasında ilk satırdaki `MOV AL, [BX+20H]` ifadesinde göreceli konum adresi BX yazmacının değerinin `20H` fazlasıdır ve bölüt adresi DS yazmacının o andaki değeridir.

İkinci satırdaki `MOV AL, [BP+03H]` ifadesinde göreceli konum adresi BP yazmacının değerinin `03H` fazlasıdır ve bölüt adresi değeri olarak SS yazmacının değeri kullanılır.

Üçüncü satırdaki adresleme yönteminde göreceli konum adresi olarak SI yazmacının değerinin `04H` fazlası, bölüt adresi değeri olarak da [BX] adreslemesinde olduğu gibi DS bölüt yazmacının değeri kullanılır.

Dördüncü satırdaki adreslemede ise göreceli konum adresi olarak DI yazmacının değerinin `07H` fazlası ve bölüt adresi değeri olarak DS kesim yazmacının değeri kullanılır.

Tabanlı Dizinle Adresleme Kipi

Tabanlı dizinle adresleme kipinde taban yazmaçları olarak adlandırılan BX ya da BP yazmaçlarının değerleri ilk taban değerini oluşturup, diğer bir yazmacın değerinin bu taban değerine eklenmesi ile göreceli konum adresi belirlenir.

Aşağıdaki kod parçasında tabanlı dizinle adresleme kipinin bazı örnekleri gösterilmiştir:

```
MOV AL, [BX+SI]     ; AL <-- DS:[BX+SI]
MOV AL, [BX+DI]     ; AL <-- DS:[BX+DI]
MOV AL, [BP+SI]     ; AL <-- SS:[BP+SI]
MOV AL, [BP+DI]     ; AL <-- SS:[BP+DI]
```

Kod 3.7: Tabanlı dizinle adresleme örnekleri

Yukarıdaki programın ilk satırında (`MOV AL, [BX+SI]`) göreceli konum adresi olarak BX ve SI yazmaçlarının değerlerinin toplamı kullanılır. BX yazmacı taban olarak kullanıldığından işlemci veri bölütünün adreslendiğini varsayacak ve bölüt yazmacı olarak DS kullanılacaktır.

Kod parçasının ikinci satırında (`MOV AL, [BX+DI]`) göreceli konum adresi olarak BX ve DI yazmaçlarının değerlerinin toplamı kullanılmaktadır. BX yazmacının kullanılıyor olması nedeniyle bölüt yazmacı olarak DS kullanılır.

Üçüncü satırda (`MOV AL, [BP+SI]`) göreceli konum adresi olarak BP ve SI yazmaçlarının değerlerinin toplamı kullanılır. Taban değeri olarak BP yazmacının değeri kullanıldığından işlemci yığıt bölütünün adreslendiğini varsayacağından bölüt yazmacı olarak SS yazmacı kullanılır.

Dördüncü satırda ise (`MOV AL, [BP+DI]`) göreceli konum adresi olarak BP ve DI yazmaçlarının değerlerinin toplamı kullanılır. BP yazmacı kullanıldığı için bölüt yazmacı olarak SS yazmacı kullanılır.

Tabanlı ve Eklemeli Dizinle Adresleme Kipi

Tabanlı ve eklemeli dizinle adresleme kipinde hem taban yazmaçlarının değerleri hem de eklemeler kullanılır.

```
MOV AL, [BX+SI+03H]     ; AL <-- DS:[BX+SI+03H]
MOV AL, [BX+DI+55H]     ; AL <-- DS:[BX+DI+55H]
MOV AL, [BP+SI+25H]     ; AL <-- SS:[BP+SI+25H]
MOV AL, [BP+DI+0FH]     ; AL <-- SS:[BP+DI+0FH]
```

Kod 3.8: Tabanlı ve eklemeli dizinle adresleme örnekleri

Programın ilk satırında (`MOV AL, [BX+SI+03H]`) göreceli konum adresi olarak BX yazmacının değeri, SI yazmacının değeri ve `03H` değerinin toplamı kullanılır.

İlk satırda göreceli konum adresinin hesaplanması için BX kullanıldığından bölüt yazmacı olarak DS kullanılır.

İkinci satırda (`MOV AL, [BX+DI+55H]`) göreceli konum adresi olarak BX yazmacının değeri, DI yazmacının değeri ve `55H` değerinin toplamı kullanılır. Bölüt yazmacı olarak DS yazmacı kullanılır.

Üçüncü satırda göreceli konum adresi olarak BP yazmacının değeri, SI yazmacının değeri ve `25H` değerinin toplamı kullanılır. Bölüt yazmacı olarak SS yazmacı kullanılır.

Dördüncü satırda ise göreceli konum adresi olarak BP yazmacının değeri, DI yazmacının değeri ve `0FH` değerinin toplamı kullanılır. Bölüt yazmacı olarak SS yazmacı kullanılır.

Bölüm Sonu Soruları

Alıştırma 3.1. 8086 mimarisinde hangi yazmaçlar vardır? Bu yazmaçlar arasında genel amaçlı kullanılanlar hangileridir? Program sayacı 8086'da nasıl tutulur? Hangi yazmaçların iki yarısı ayrı ayrı kullanılabilir? İki yarısı ayrı ayrı kullanılabilen yazmaçların bu özelliğini programcı nasıl kullanır?

Alıştırma 3.2. Aşağıdaki sanal adreslerin gerçek (fiziksel) adreslerini hesaplayın:

a) `1000H:0200H`
b) `ABCDH:1234H`
c) `0000H:FFFFH`

Alıştırma 3.3. Gerçek adresi `25A00H` olan bir bellek konumunu gösterebilecek iki farklı sanal adresi (bölüt:göreceli konum biçiminde) yazın.

Alıştırma 3.4. 8086 bayrak yazmacındaki CF, ZF, SF ve OF bitlerinin ne zaman 1 değerini aldığını açıklayın. Her bayrak için kısa bir örnek verin.

Alıştırma 3.5. Aşağıdaki her bellek erişimi için hangi adresleme kipinin kullanıldığını belirtin:

a) `MOV AX, [1234H]`
b) `MOV BL, [BX+SI+10H]`
c) `MOV CX, [BP]`
d) `MOV DL, [DI+05H]`

Alıştırma 3.6. 8086'da BIU (Veri Yolu Arabirimi Birimi) ile EU (Yürütme Birimi) arasındaki görev paylaşımını açıklayın. Komut ön yükleme (instruction prefetch) mekanizması başarıma nasıl katkı sağlar?

Alıştırma 3.7. Aşağıdaki komutlar çalıştırıldıktan sonra belirtilen yazmaçların değerleri ne olur?

```
MOV AX, 1234H
MOV BX, AX
MOV CH, BH
MOV CL, BL
```

AX, BX, BH, BL, CH, CL yazmaçlarının son değerlerini yazın.

Alıştırma 3.8. CS yazmacı neden `MOV CS, AX` gibi bir komutla doğrudan değiştirilemez? CS yazmacının değerini değiştirebilecek komutları listeleyin.

4

Çevirici Diline Giriş

Bu bölümde assembly diline kısa bir giriş yapılacaktır. Öncelikle veri türleri ve bölütler hakkında bilgiler verilecektir. Ardından çevirici, bağlayıcı, konumlandırıcı ve hata ayıklayıcıları tanıtılıp kullanımlarına örnekler verilecektir. Çevirici dil kodlarının makine diline nasıl çevirildiği hakkında bilgiler verildikten sonra 8086 Komut Kümesi'ni tanıtmak amacıyla kısa açıklamalar ve kısa örnekler verilecektir. Komutların ve örneklerin hepsinin anlaşılmaması bu aşamada sorun değildir. Gelecek bölümlerde komutlar kullanımları üzerine daha geniş açıklamalar ve örnekler verilecektir.

Günümüzde birçok yüksek düzeyli dil varken çevirici dilinin öğrenilmesinin ve kullanılmasının 3 nedeni vardır:

- Çevirici dili ile oluşturulmuş bir program daha az yer kaplar.
- Çevirici dili ile oluşturulmuş bir program yalnızca yapması gereken işlemleri yaptığından daha hızlıdır.
- Çevirici dili kullanılarak yüksek düzeyli dillerin yapamadığı bazı işlemler yapılabilir.

Günümüzde bilgisayarlar hız ve sığım (kapasite) yönünden ne kadar güçlü olsalar da daha hızlı ve daha geniş sığımlı bilgisayarlar piyasaya sürüldüğünde sistemler yenilenmektedir. Çevirici dili ile yazılan programların hız ve sığım açısından tasarruf sağladığı düşünülürse günümüzün yüksek hızlı ve yüksek belleğe gereksinim duyan bilgisayarlarında bile çevirici dilinin ne kadar önemli olduğu anlaşılacaktır.

Çevirici dili ilk bakışta çok karmaşık ve anlamsız gelebilir; ancak çevirici dilinin ana mantığı kavrandığı zaman yüksek düzeyli dillerden daha "mantıklı" olduğu anlaşılacaktır. Ayrıca iyi bir çevirici dili bilgisi yüksek düzeyli diller ile program yazarken yazılımcıya yardımcı olacaktır. İşlemcilerin çalışma mantıkları çok basittir. Çağdaş mikroişlemciler yapısal olarak çok karmaşık olsalar bile çevirici dilin ana mantığından ayrılmamaktadırlar. Tüm işlemciler bir sonraki komutu alır, işler ve diğer komuta geçerler. Çevirici dili ile program yazarken bu işlemler açıkça görülebilmektedir. Her komut işlenenleri ile birlikte işlenir ve sıradaki komuta geçilir.

4.1 Veri Türleri

İşlemcilerin değişik türlerde verileri kullanarak işlem yapabilir. Bu veri türleri daha çok, işlemcinin donanımının özelliklerine uygun olarak tanımlanır. Çevirici düzeyinde kodlama yapan programcının işlemci donanımın özelliklerini ve bu donanımla birlikte kullanılabilecek veri türlerini anlaması önemlidir.

En küçük veri türü bittir. Bir bit ya 0 ya da 1 değerine sahip olabilir. 4 tane bitin birlikte oluşturduğu veriye yarım bayt (nibble) denir. Bir yarım baytın göstereceği verinin değeri 0'dan 15'e kadardır. Bir bayt, 2 yarım bayttan ya da 8 tane bitten oluşur. Bir baytlık bir verinin alabileceği değerler 0 ile 255 arasında olabilir. Bir sözcük (word) iki bayttan oluşur ve alabileceği değerler 0 ile 65535 arasındadır. Ayrıca literatürde 32 bitten oluşan ikili sözcükler (double word) ve 64 bitten oluşan dörtlü sözcükler (quad word) de bulunmaktadır. Aşağıdaki tabloda veri türleri ve bu veri türlerinin alabileceği değerler özetlenmiştir.

Tablo 4.1: *Veri türleri ve değer aralıkları*

Adı	Bit Sayısı	Değer Aralığı
Bit	1	0 – 1
Yarım Bayt (nibble)	4	0 – 15
Bayt (byte)	8	0 – 255
Sözcük (word)	16	0 – 65.535
İkili Sözcük (double word)	32	0 – 4.294.967.295
Dörtlü Sözcük (quad word)	64	0 – 18.446.744.073.709.551.615

8086 işlemcisi 16 bitlik olduğundan kullanılan veri türleri bit, yarım bayt, bayt, sözcük ve bazen de ikili sözcüklerdir.

Çevirici dilinde bir değişken (bir bellek bölgesi) tanımlayıp ona değer atamak için `DB` (define byte – bayt tanımla) ve `DW` (define word – sözcük tanımla) kodları kullanılır. Bu kodlar aşağıdaki biçimde kullanılır:

```
ğı
Deisken_ad DB ğdeerğşı
Deiken_ad DW ğdeer
```

Kod 4.1: DB ve DW tanımlamaları

Değişken_adı bir harf ile başlamalıdır. Herhangi bir harf-sayı birleşimi değişken adı olarak kullanılabilir. Çevirici dili ile program yazılırken kullanılan bu değişkene ait değer'e bu Değişken_adı etiketi ile ulaşılabilir. Eğer Değişken_adı yazılmazsa herhangi bir etiket belirtilmemiş ve yalnızca belirtilen değer'i taşıyan bir bellek bölgesi ayrılmış olur. Değer olarak ikilik, onluk ya da onaltılık tabanda bir sayı yazılabilir. İkilik tabanda bir sayı yazıldığında sonuna "B" harfi (binary – ikilik) ve onaltılık tabanda bir sayı yazıldığında sonuna bir "H" harfi (hexadecimal – onaltılık) yazılarak kullanılan sayının tabanı belirtilmelidir. Onluk tabanda bir sayı kullanılırken sayının kendisinin yazılması yeterlidir.

Yazılan sayının değerinin, `DB` tanımlaması kullanıldığında bir bayta sığması, `DW` tanımlaması kullanıldığında ise bir sözcüğe sığması gerekir. Tanımlamanın ardından herhangi bir değer yazılması yerine "?" işareti kullanılırsa değişkenin ilk değeri olarak herhangi bir değer belirlenmez ancak bellekteki bir adrese Değişken_adı etiketi verilmiş olur.

Örnek 4.1

Aşağıda bazı değişken tanımlamaları gösterilmiştir:

```
degiskenim  DB 01110011B
degiskenim2 DW 0102H
degiskenim3 DB ?
degiskenim4 DW 12345
```

Kod 4.2: Değişken tanımlama örnekleri

Eğer bir değişken aşağıda gösterildiği gibi tanımlanırsa `05H` verisi 6 kez art arda yazılarak değişkenin 6 baytlık verileri tutacağı tanımlanmış olur.

```
degiskenim5 DB 6 DUP(05H)
```

Kod 4.3: DUP ile çoklu tanımlama

4.2 Bölütler

Çevirici dili ile yazılan programlarda en az 3 tane bölüt kullanılır: kod bölütü, veri bölütü ve yığıt bölütü. Bu bölütlerin dışında ayrıca bir de ek bölüt kullanılabilir. Kod bölütünde, işlenmek üzere yazılmış olan çevirici dili komutları bulunur. Veri bölütünde, kod bölütündeki komutlar tarafından işlenecek olan veriler yer alır. Yığıt bölütü geçici saklama işlemlerinde kullanılırken ek bölüt ise yedek bir veri bölütü gibi kullanılabilmektedir.

Bölütler bellekte 16 ile tam olarak bölünen bir adresten başlar ve bellekte 64KB boyutunda bir bellek alanını simgeleyebilirler. 64KB ile sınırlandırılmış olmalarının temel nedeni 8086'nın 16 bitlik bölüt yazmaçlarıdır; bir bölüt içindeki göreceli konum adresi 16 bit ile en fazla $2^{16} = 65536$ bayt (64KB) adresleyebilmektedir. 8086'nın 20 adres iğnesi bulunmasına ve bu adresleme bitleriyle 2^{20} baytlık (1MB) bellek alanını gösterebilmesine karşın tek bir bölüt içindeki erişim 64KB ile sınırlıdır. Bu nedenle kod, veri ve yığıt bölütlerinin her biri en fazla 64KB olabilir. Bazı yöntemler kullanılarak bu 64KB'ın da dışına çıkılabilmektedir. Bölütlerin nasıl kullanıldığının anlaşılabilmesi için gerçek adres, sanal adres ve göreceli konum adresi kavramlarının bilinmesi gerekir.

Bilgi Notu 4.1: Gerçek Adres, Sanal Adres ve Göreceli Konum Adresi

8086 işlemcisinde gerçek adres, 20 iğnelik adres kapısının iğnelerine atanan 20 bitlik adrestir. Bu 20 bitlik adres bellek arayüz devrelerindeki kod çözücüler tarafından çözülür. Böylece 8086'da `00000H`'dan başlayarak `FFFFFH`'ye kadar olan ROM ya da RAM'daki 1MB'lık bellek bölgesi adreslenebilmektedir. Göreceli konum adresi bir bölüt içinde kalan ve `0000H`'dan başlayarak `FFFFH`'ye kadar olan bölgeyi adreslemektedir. Sanal adres 2 kısımdan oluşur: birinci kısım bölüt adresini, ikinci kısım ise göreceli konum adresini belirtir. `0200H:0000H` sanal adresinde `0200H` bölüt adresi ve `0000H` göreceli konum adresidir. Sanal adres verildiğinde gerçek adres bulunabilir. Bunun için bölüt adresi bir basamak sola kaydırılır (ya da `10H` ile çarpılır) ve göreceli konum adresiyle toplanır. Örneğin `0200H:0000H` sanal adresine denk gelen gerçek adres:

$$\texttt{0200H} \times \texttt{10H} = \texttt{02000H}$$
$$\texttt{02000H} + \texttt{0000H} = \texttt{02000H}$$

şeklinde hesaplanır. Dikkat edilmesi gereken nokta iki ayrı sanal adresin aynı gerçek adresle eşleşebileceğidir.

4.2.1 Kod Bölütü

Yazılan bir programın çalıştırılabilmesi için programı oluşturan komutların bir yerden çekilmesi ve çalıştırılması gerekir. 8086 işlemcisi program kodunu CS yazmacı ile gösterilen kod bölütünden (code segment) çeker ve çalıştırır. Kod bölütünden bir komutun okunması için komutun bellekte bulunduğu gerçek adresin bilinmesi gerekir. Gerçek adres, bölüt adresi ve göreceli konum adreslerinden hesaplanabildiğinden, bir komut, belleğin bir bölütünden çekilirken bu bölütün neresinden çekildiğini belirtmek amacıyla göreceli konum adresinin de bölüt adresi ile birlikte belirtilmesi gerekir. Bu yüzden bir komutun bellekten getirilmesi için CS'nin yanında IP (instruction pointer – komut işaretçisi) adresi de verilmektedir. IP göreceli konum adresinin değerini tutmaya yarayan bir işaretçidir. Bir komut çekildikten sonra, çekilmiş olan komut aksini belirtmediği sürece, IP 1 artırılır ve sıradaki göreceli konum adresine geçilir. Örneğin `0200:0000` adresindeki komut çekilip çalıştırıldıktan sonra `0200:0001` adresine geçilir. `0200:0001` sanal adresine denk gelen gerçek adres:

$$\texttt{0200H} \times \texttt{10H} = \texttt{02000H}$$
$$\texttt{02000H} + \texttt{0001H} = \texttt{02001H}$$

şeklinde hesaplanarak `02001H` olarak bulunur. CS:IP `1234:0200` iken ise gerçek adres:

$$\texttt{1234H} \times \texttt{10H} = \texttt{12340H}$$
$$\texttt{12340H} + \texttt{0200H} = \texttt{12540H}$$

şeklinde hesaplanarak `12540H` olarak bulunur. Programın içerdiği komutlar bellekten getirilip çalıştırılırken bazı komutlar aracılığıyla IP'nin değeri değiştirilebilir. JMP (jump – atla), JE (jump if equal – eşit ise atla), JNE (jump if not equal – eşit değilse atla) gibi komutlar IP

adresinin değerini değiştirebilir. CS'nin değeri ise bölütler arası bir CALL (call – çağır) komutu uygulandığında IP adresiyle birlikte değişebilir. Ancak CALL komutuyla atlanan alt yordamda RET (return – geri dön) komutu çalıştırıldığında CS ve IP eski değerine dönecektir. CS'nin değerini olağan yollarla kalıcı olarak değiştirebilen bir komut yoktur. Çevirici dilinin komutlarına geçildiğinde bu kavramlara yeniden değinilecektir.

Aşağıdaki tablo bir dizi komutun bellekte bulunduğu biçimi göstermektedir:

Tablo 4.2: *Komutların bellekte gösterimi*

CS:IP	İşlem Komutu ve İşlenen	Çevirici Komutları
1234:0000	B3 53	`MOV BL, 'S'`
1234:0002	B4 43	`MOV AH, 'C'`
1234:0004	B9 12 00	`MOV CX, 12H`
1234:0007	BA 13 00	`MOV DX, 13H`
1234:000A	03 CA	`ADD CX, DX`

İşlemci, komutları işlemeye `1234:0000` adresinden başladığında `1234:0000` adresindeki `B3H` ile `1234:0001` adresindeki `53H` verisini yakalar ve bu verilerin `MOV BL, 'S'` çevirici dil koduna denk geldiğini algılar. Bir başka deyişle, `MOV BL, 'S'` çevirici kodu bir çevirici ile çevrilirse onaltılık tabandaki B3 ve 53 baytlarına dönüşecektir.

`1234:0002` adresinde AH yazmacına C harfinin onaltılık tabandaki ASCII değerini aktaran B4 işlem komutu ve `1234:0003` adresinde C harfinin onaltılık tabandaki değeri olan `43H` bulunmaktadır. MOV komutu BL yazmacına değişmez bir veri aktarırken `B3H` işlem komutuna çevrilirken, AH yazmacına değişmez bir veri aktarılırken `B4H` işlem komutuna çevrilmektedir. Bunun nedeni MOV komutunun yalnızca bir tane işlem kodu bulunması durumunda bu işlem kodu ile yalnızca tek bir yazmaca veri aktarabilmesidir.

`1234:0004` adresindeki veri okunduğunda `MOV CX, 12H` görülecektir. Bu kod bir işlem komutu ve iki tane işlenen olmak üzere 3 baytlık alan kaplamaktadır. `MOV BL, 'S'` 2 baytlık yer kaplarken, `MOV CX,12H` 3 baytlık yer kaplamasının nedeni BL yazmacının 8 bitlik olmasıdır. BL yazmacı 16 bit olan BX yazmacının 8 bitlik alt yazmacıdır. `MOV CX, 12H` kodu CX yazmacına bir baytlık `12H` verisini, CX yazmacının alt yarısına `12H` ve üst yarısına `00H` olarak yazar. Burada dikkat edilmesi gereken nokta CX yazmacına aktarılan `0012H` değerinin işlenen olarak 12 00 sırasıyla MOV komutuna verilmesidir. Bunun nedeni Intel mikroişlemcilerinin çoğunda "Küçüğü Başta (Little Endian)" Kuralı'nın kullanılmasıdır.

Örnekte `1234:0007`, `1234:0008` ve `1234:0009` adreslerindeki BA 13 00 baytları `MOV DX,13H` komutunu oluşturmaktadır. `1234:000A` ve `1234:000B` adreslerinde ise `ADD CX,DX` kodu bulunmaktadır. Bu kod CX ve DX yazmaçlarının değerlerini toplayıp sonucu CX yazmacına yazar.

4.2.2 Veri Bölütü

DS (Data Segment – Veri bölütü) yazmacının yerini gösterdiği veri bölütünün kullanımının sağladığı yararlar aşağıdaki örnek incelenerek anlaşılabilir.

Aşağıdaki kod parçasında BL yazmacına yazılan `FFH` sayısından ilk önce `10H` değeri çıkarılarak sonuç BL yazmacına kaydedilmekte, ardından BL yazmacının değeri `11H` ile toplanıp sonuç yeniden BL yazmacına yazılmaktadır.

```
1  ... MOV BL, 0FFH
```

```
2       SUB BL, 10H
3       ADD BL, 11H ...
```

Kod 4.4: Doğrudan anlık değer kullanan kod parçacığı

Bu kod parçacığının çok uzun bir programın küçük bir bölümü olduğu düşünülürse, BL yazmacına yüklenen `FFH` değeri `AAH` ile değiştirilmek istendiğinde bu program satırının çok sayıda satır arasından bulunup değiştirilmesi gerekir. Böyle bir zorlukla karşılaşmamak için, çevirici dilini kullanan programcı, programın içinde kullandığı değerleri veri bölütüne kaydettikten sonra, verinin kendisini kullanmak yerine verinin bulunduğu bellek adresini kullanabilir. Örneğin, eğer veri bölütü `0400H` göreceli konumundan başlıyor ise bellekte DS:0400 adresine `FFH`, DS:0401 adresine `10H` ve DS:0402 adresine `11H` verileri önceden kaydedilirse örnekte verilen kod parçası aşağıdaki biçimde yazılabilir:

```
1  ... MOV BL, [0400]
2      SUB BL, [0401]
3      ADD BL, [0402] ...
```

Kod 4.5: Bellek adresi kullanan kod parçacığı

Yukarıdaki program parçasının ilk satırında BL yazmacına MOV komutu ile doğrudan bir sayı yollamak yerine sayının bellekte bulunduğu konumun adresi olan `[0400]` yazılmıştır. Köşeli parantezler çeviriciye parantezin içindeki sayının kendisinin değil sayının gösterdiği bellek adresindeki verinin BL yazmacına yazılması gerektiğini anlatır. Bu tür bir adreslemede bölüt adresi olarak DS kullanılır. Göreceli konum adresi ise köşeli parantezlerin içine yazılmış olan `0400H` değeridir.

Aynı kodun yazmaç kullanan bir versiyonu şu şekilde yazılabilir:

```
1  ... MOV SI, 0400H
2      MOV BL, [SI]
3      INC SI
4      SUB BL, [SI]
5      INC SI
6      ADD BL, [SI] ...
```

Kod 4.6: SI yazmacı kullanan kod parçacığı

Birinci satırdaki `MOV SI,0400H` komutuyla SI (source index – kaynak dizini) yazmacına `0400H` sayısı yazılır. İkinci satırdaki `MOV BL,[SI]` komutu SI yazmacının değerini göreceli konum adresi olarak kullanarak belleğe erişir ve ilgili bellek adresindeki veriyi BL yazmacına aktarır. Üçüncü satırdaki `INC SI` komutu ise SI yazmacının değerini bir artırarak `0401H` yapar.

Görüldüğü gibi `[ ]` ayraçları arasına göreceli konum adresini yazmak yerine, bu ayraçların içine bir yazmaç adı yazılarak kullanılan yazmacın değerinin göreceli konum adresi olduğu belirtilebilir. Burada dikkat edilmesi gereken konu veri bölütünün yalnızca BX, SI ve DI yazmaçlarının değerlerini göreceli konum adresi olarak kabul etmesidir. DS:BX, DS:SI ve DS:DI kullanımları veri bölütünü adreslemek için uygun olmasına karşın DS:AX (ya da BX, SI ve DI dışında herhangi bir yazmaç) kullanımı kurallara aykırıdır.

Veri bölütüne erişim, 8086 işlemcisinin bellek erişim kurallarına uygun olarak yapılır. Örneğin veri bölütünün adresinin `0200H` ve veri bölütünün içinde erişilmek istenen verinin göreceli konum adresinin `0201H` olduğu varsayılırsa bellekte erişilmesi gereken gerçek adres aşağıdaki biçimde hesaplanır:

$$0200H \times 10H = 02000H$$
$$02000H + 0201H = 02201H$$

Yukarıdaki adres hesabı sonucu bellekte erişilecek bellek konumunun gerçek adresi `02201H` olarak bulunur.

4.2.3 Yığıt Bölütü

SS (stack segment – yığıt bölütü) yazmacının bellekteki konumunu gösterdiği yığıt bölütü hem okunabilen hem de yazılabilen bir bellek parçasıdır ve genellikle ana belleğin içinde yer alır. Yığıt belleğinde veriler geçici olarak saklanır. İşlemcide bulunan yazmaçların sayısının bazı teknik ve mali nedenlerden dolayı az olması programların yazılmasını zorlaştırdığından yığıt gibi geçici bir saklama alanın varlığı programlar yazılırken büyük kolaylık sağlamaktadır. Yığıt belleği sağladığı kolaylığa karşın işlemciden uzak olan ana bellekte yer alması ve yazmaçların işlemcinin içinde bulunması nedeniyle yazmaçlara göre daha yavaş erişim sağlar.

Yığıtın başındaki verinin adresi SS ve SP (stack pointer – yığın işaretçisi) yazmaçlarının birlikte kullanılmasıyla bulunur. Erişilen verinin gerçek adresi SS yazmacındaki veri bir basamak sola kaydırılıp SP yazmacının değeri ile toplanmasıyla bulunur. Örneğin SS yazmacı yığıtın bellekteki konumunu `0400H` olarak, SP yazmacı ise yığıt bölütünün içinde yığıtın başının göreceli konumunu `0000H` olarak gösteriyorsa `0400:0000` adresine erişmek için gereken gerçek bellek adresi aşağıdaki gibi bulunur:

$$0400H \times 10H = 04000H$$
$$04000H + 0000H = 04000H$$

Yığıt üzerinde herhangi bir işlem yapmadan önce MOV komutu ile SS ve SP yazmaçlarına ilk değerlerinin verilmesinde yarar vardır. Yığıta bir veri kaydetmek için PUSH komutu, yığıtta kayıtlı olan verinin yığıttan çekilmesi içinse POP komutu kullanılır. SP yazmacının değeri her PUSH komutundan sonra 2 azalır ve her POP komutundan sonra 2 artar; çünkü 8086'da PUSH ve POP komutları her seferinde 16 bitlik (2 baytlık) bir veri işler. SP yazmacının değerindeki değişim biçimi, işlemcinin işleyeceği bir sonraki komutun yerini gösteren ve işlenen her komuttan sonra 1 artırılan IP yazmacının değerinin değişme yönteminin tam tersidir.

Örnek 4.2

Aşağıdaki örnek yığıt komutlarının nasıl çalıştığını göstermektedir:

```
1:  MOV AX, 0200H
2:  MOV SS, AX
3:  MOV SP, 1238H
4:  MOV AX, 0AFFH
5:  MOV BH, 11H
6:  MOV BL, 12H
7:  MOV CX, 01H
8:  PUSH AX
9:  PUSH BX
```

```
10: POP AX
11: PUSH CX
12: POP BX
13: POP CX
```

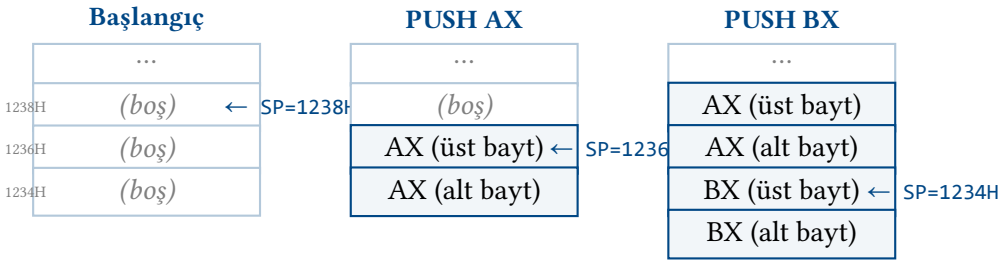
Kod 4.7: Örnek 4.2: PUSH ve POP komutları

Yukarıdaki örnek programın birinci satırında AX yazmacına `0200H` verisi yazılmakta, ikinci satırda ise bu saklanan veri SS yazmacına yüklenmektedir. SS yazmacına doğrudan veri aktaran bir MOV işlem kodu bulunmaması nedeniyle SS yazmacına yapılan yüklemeler her zaman örnekteki gibi başka bir yazmaç üzerinden dolaylı olarak yapılır. SP yazmacına ise doğrudan veri aktarılabilir. Programın üçüncü satırında SP'ye `1238H` verisi yüklenmektedir. Bu satırın ardından gelen 4, 5, 6 ve 7inci satırlarda yazmaçlara AX, BX ve CX yazmaçlarının değerleri MOV komutları aracılığıyla belirlenmektedir.

8inci satırdaki `PUSH AX` komutu ile AX yazmacının değeri `0200:1238` ve `0200:1236` adresleri arasına yazılır. AX yazmacının boyutunun 2 bayt olması nedeniyle bu saklama işleminin ardından SP yazmacının değeri 2 azalır. Ardından 9uncu satırdaki `PUSH BX` komutu ile BX yazmacının iki bayt uzunluğundaki değeri `0200:1236` ve `0200:1234` adreslerinin arasına yazılır. 10. satırdaki `POP AX` komutu SP'nin gösterdiği adresten başlayarak iki bayt okur ve AX yazmacına yazar. Böylece AX yazmacının değeri `1112H` olarak belirlenmiş olur. Sonraki satırdaki `PUSH CX` komutu ile `0200:1236` ve `0200:1234` adreslerinin arasına CX'in değeri olan `0001H` yazılır. 12inci satırdaki `POP BX` komutu ise SP yazmacı ile gösterilen yığıtın tepesinden iki baytlık bir veri çeker ve bu veriyi BX yazmacına yazar. 13üncü satırdaki `POP CX` komutu CX yazmacına `0200:1236` ve `0200:1238` adreslerinin arasındaki adreslerde bulunan verileri CX yazmacına atar.

Örnekte de görüldüğü gibi, PUSH komutu ile yığıta veri yazılmadan hemen önce SP'nin değeri 2 azaltılır. POP komutunda ise veri SP'nin o anda gösterdiği adresten okunur, veri okuma işleminin ardından SP'nin değeri 2 artar. Yığıttan veri okunurken ya da yığıta veri yazılırken SP işaretçisinin değerinde oluşan bu değişiklik sırasını programcı aklında bulundurmalıdır.

Aşağıdaki şekil, SP'nin başlangıçta `1238H` olduğu durumda art arda iki PUSH ve iki POP işleminin yığıt üzerindeki etkisini göstermektedir.



Şekil 4.1: Art arda PUSH komutlarında SP'nin azalması ve yığıtın aşağı doğru büyümesi.

Tamamı 16 bit (2 bayt) uzunluğunda olan yazmaçlar PUSH ve POP komutlarıyla birlikte kullanılırken bu yazmaçların parçaları ayrı ayrı 8 bitlik yazmaçlar olarak yığıttan okunamaz ya da yığıta geri yazılamaz. Bu yazmaçlarla işlem yapılırken 16 bitlik yazmacın tamamı kullanılmalıdır. Örneğin `POP AL` ya da `PUSH BH` gibi komutlar geçersizdir. Bu komutların yerine `POP AX` ya da `PUSH BX` kullanılmalıdır.

4.2.4 Ek Bölüt

Programlarda çok sık kullanılmayan ek bölütün bellekteki yeri ES (extra segment – ek bölüt) yazmacı ile tutulur. Çoğunlukla dizgi işlemleri için kullanılmaktadır. İleriki bölümlerde dizgi işlemleri ayrıntılı olarak açıklanırken bu konuya geri dönülecek ve ek bölüt daha ayrıntılı açıklanacaktır.

4.3 Bölütlerin Örtüşmesi

Bazı nedenlerden ötürü bölütlerin bellekte kapsadığı adres aralıkları örtüşebilir. Örneğin COM uzantılı dosyaların boyutları 64 KB'ı geçmeyecek biçimde tasarlanmak zorunda olduğu için bu dosyalarda bölütler birbirleriyle örtüşür. Programcı yazdığı programda bölütlere ayrı ayrı adresler vermek yerine tek bir bölüt adresi kullanarak hem kodunu hem de verilerini aynı bölüte yazabilir. Böylece bölüt içinde program kodundan önce, program kodunun arasında ya da program kodundan sonra programın kullandığı veriler konumlandırılabilir. Bu tür bir kullanım bölütlerin örtüşmesine örnektir.

4.4 Çevirici, Bağlayıcı, Konumlandırıcı ve Hata Ayıklayıcı

Çevirici diliyle yazılmış bir programın işlemcinin anlayabileceği biçimi alması için bir takım işlemlerden geçirilmesi gerekir. Program önce bir çeviriciyle (assembler) makine diline çevrilir, ardından bir bağlayıcı (linker) ile oluşturulan nesneler birbirine bağlanır ardından da bir konumlandırıcı (locator) kullanılarak bellek konumları ayarlanır. Bu kitap boyunca verilen örneklerde FASM çeviricisinin kodlama tarzı kullanılmıştır. Yazılan örnekler başlangıç aşamasında Windows ile birlikte gelen DEBUG.EXE programı kullanılarak denenebilir. Programların çevrilmesi için Intel ASM86 çeviricisi, bağlanması için Intel LINK86 bağlayıcısı ve konumlandırılması için Intel LOC86 konumlandırıcısı kullanılabilir. Bu programların, kitaptaki örneklerin denenmesi için gereken komutları ve bu komutlarla birlikte kullanılan terimler kısaca açıklanacaktır. Örnek programların doğrudan 8086 uyumlu bir işlemci üzerinde çalıştırılabilmesi için ise üretilen kodların Intel'in kullandığı 16lık tabandaki sayı düzenine çevrilmesi gerekir. Bu amaçla Intel Hex biçim dönüştürücüsü olarak OH86 programı kullanılabilir. Bütün bu programlar hakkında daha ayrıntılı bilgi Ek B bölümünde sunulmuştur. Ek B bölümünde ayrıca piyasada günümüzde çok kullanılan NASM, MASM gibi çeviriciler hakkında bilgiler bulunmaktadır. Yazılan programların Linux işletim sistemi kullanan bilgisayarlarda nasıl çevrileceği, bağlanacağı, konumlandırılacağı ve deneneceği Ek C bölümünde anlatılmaktadır.

Yazılan çevirici dil kodlarının herhangi bir 8086 tabanlı eğitim düzeneği ya da 8086 deneme kartı üzerinde denenebilmesi için oluşturulan `.hex` uzantılı dosyaların bir aracı yazılım aracılığıyla donanıma yüklenmesi gerekir. Bu kitapta verilen örnekler DOS işletim sisteminde COM uzantılı çalıştırılabilir dosyalara dönüştürülecektir. Windows işletim sisteminin sunduğu DEBUG programı bu COM dosyalarını işleyerek kullanıcıya yazmaç değerleri gibi bilgileri vermektedir. Çeviri dilinde yazılan kodlar FASM programı kullanılarak `.com` uzantılı dosyalara dönüştürülebilir. Bunun yanında 10. bölümdeki örneklerde `.bin` (ikilik düzende saf çıktı) uzantılı çıktı almak için FASM programı kullanılacaktır.

4.4.1 Çevirici

Intel firmasının sağladığı ASM86 çeviricisi `asm` uzantılı dosyaları `.obj` türünde dosyalara çevirmek için kullanılır. OBJ dosyası yeniden konumlandırılabilen nesne kodunu (relocatable object code) içerir ve bağlayıcı programı tarafından kullanılır. Çevirici dil kullanılarak yazılmış `deneme1.asm` isimli bir program aşağıdaki satır kullanılarak OBJ dosyasına dönüştürülür:

```
Asm86 deneme1.asm m1 db ep
```

Kod 4.8: ASM86 ile çevirme

Bu satırdaki `m1` terimi çevrilen kodun 80186 komut kümesini kullanacağını belirtmekte, `db` ve `ep` terimleri ise çevrilen kodun içinde herhangi bir hata bulunursa bu hatanın ekranda gösterilmesini sağlamaktadır.

4.4.2 Bağlayıcı

Intel firmasının sağladığı LINK86 bağlayıcısı çevirici tarafından oluşturulmuş olan bir ya da birden fazla OBJ dosyasını alır ve bu dosyaların birbirleriyle olan bağlantılarını sağlayarak `.lnk` uzantılı bir dosya yaratır. Yukarıdaki örnekte ASM86 çeviricisi ile çevrilen dosya aşağıdaki gibi bağlanır:

```
Link86 deneme1.obj to deneme1.lnk
```

Kod 4.9: LINK86 ile bağlama

Bu komut satırı çalıştırıldıktan sonra `deneme1.obj` dosyası `deneme1.lnk` dosyasına dönüştürülmüş olur.

4.4.3 Konumlandırıcı

Intel firmasının sağladığı LOC86 konumlandırıcısı bağlayıcı tarafından oluşturulmuş olan yeniden konumlandırabilen nesne kodu dosyasını alıp konumlandırarak yalın nesne koduna dönüştürür. Daha önce bağlayıcı tarafından bağlanan `deneme1.lnk` dosyası aşağıdaki gibi konumlandırılır:

```
Loc86 deneme1.lnk noic
```

Kod 4.10: LOC86 ile konumlandırma

Bu komut satırı çalıştırıldıktan sonra çıktı olarak `deneme1` adlı uzantısız bir dosya oluşacaktır. `Noic` (No initial conditions) terimi konumlandırıcıya olağan şartlarda programın başına eklenen başlangıç koşullarının yaratılmayacağını söyler.

4.4.4 Intel Hex Format Çeviricisi

Konumlandırıcının oluşturduğu 0 ve 1lerden oluşan ikilik düzendeki dosya (binary file) işlemcinin üzerinde doğrudan çalıştırılamaz. İşlemciye anlayabileceği 16lık taban kodu olan Intel hex biçiminde bir dosya yüklenmelidir. Yaratılan ikilik düzendeki dosyanın IHEX biçimine dönüştürülmesi için Intel firmasının sağladığı OH86 yazılımı kullanılabilir. Bu dönüşümü sağlamak için yalnızca dosya adının girilmesi yeterlidir:

```
Oh86 deneme1
```

Kod 4.11: OH86 ile Intel Hex biçimine dönüştürme

Yukarıdaki komut satırı çalıştırıldıktan sonra çıktı olarak `deneme1.hex` adlı IHEX biçiminde bir dosya elde edilir. `deneme1.hex` dosyası bu doğrudan işlemciye yüklenebilir.

Bilgi Notu 4.2: Intel Hex Biçimi

Intel Hex Kayıt Biçimi'nin oluşturulma nedeni programların kolay taşınabilmesinin sağlanması, dosya aktarımının kolaylaştırılması ve programların insanlar tarafından ikilik düzendeki dosyalara göre daha rahat okunabilen ASCII dosyası biçimine dönüştürülmesidir. Her hex kaydı 6 tane bölümden oluşur. Kayıt tipi olarak 6 tane seçenek vardır:

- `00` – Veri kaydı
- `01` – Kayıt dosyası sonu
- `02` – Ek bölüt adres kaydı
- `03` – Başlangıç bölütü adres kaydı
- `04` – Ek doğrusal adres kaydı
- `05` – Doğrusal başlangıç adres kaydı

Sağlama baytı hesaplanırken 2., 3., 4. ve 5. bölümdeki baytlar toplanır. Toplamdaki en az anlamlı bayt alınır, terslenir ve 1 eklenir.

4.4.5 Hata Ayıklayıcı

Hata ayıklayıcı olarak Windows ile birlikte gelen `debug.exe` programı DOS işletim sisteminin ilk yıllarında 8086 işlemcisi için yazılmış ve Intel x86 işlemci ailesi geriye dönük uyumlu olduğundan günümüze kadar fazla değişiklik geçirmeden gelmiştir. Bu hata ayıklayıcının DOS için yazılmıştır ve DOS için yazılan çalıştırılabilir programların COM ya da EXE dosyası olması gerekir. COM dosyalarında EXE dosyalarında olduğu gibi bir başlık kısmı olmadığından bu dosyaların oluşturulması daha kolaydır.

`.com` uzantılı bir dosya aşağıda gösterilen biçimde DEBUG programına terim olarak verilebilir:

```
debug C:\deneme1.com
```

Kod 4.12: DEBUG ile program yükleme

Program yukarıda gösterilen biçimde yüklendikten sonra debug programının içinden aşağıdaki komut ile çağırılabilir:

```
g =0100
```

Kod 4.13: DEBUG g komutu

`g` (Go), debug programına `0100` göreceli konum adresine gitmesini söyler ve program belirtilen adresten başlatılır.

```
d 0100
```

Kod 4.14: DEBUG d komutu

`d` komutu ise debug programına `0100` göreceli konum adresinden başlayarak bellekteki verilerin gösterilmesini söyler.

4.4.6 FASM

FASM programı hem bir çevirici hem de bir bağlayıcıdır. Eğer programla birlikte parametreler kullanılmazsa çıktı olarak `.com` dosyası ya da `.bin` dosyası verir. DOS işletim sisteminde COM dosyaları `0100H`'den başladığı için başında `ORG 0100H` komutu bulunan bütün programlar çevrildiğinde `.com` dosyasına dönüşür. Aksi halde çevrilme işlemi sonunda bir BIN dosyası ortaya çıkar. `.com` uzantılı dosyalar DEBUG programı ile denenebilir. Çevirici dil ile programlamanın temelleri tam olarak anlatılıp bütün komut kümesi açıklanana kadar verilecek örnekler `.com` çıktısı alacak biçimde yazılacak ve DEBUG programı ile denenebilecektir.

Unutulmaması gereken konu çeviricilerin kullandıkları asm kodunun söz diziminin farklı olabileceğidir. Örneğin ASM86 ile FASM arasında kod yazılırken bazı küçük farklar vardır. ASM86 Standard Intel sözdizimi (syntax) kullanırken FASM daha yalın bir sözdizimi kullanmaktadır. Aynı durum diğer çeviriciler için de geçerlidir.

4.5 Makine Diline Çevrim

Makine dili komutları 4 farklı bölümden oluşabilir. Bu bölümler aşağıdaki tabloda sıralanmıştır.

Tablo 4.3: *Makine dili komutu bölümleri*

Bölümün Adı	Uzunluğu (Bayt)
İşlem Kodu Baytı	1 – 2
Adresleme Baytı	0 – 1
Uzaklık Baytı	0 – 1
Anlık Veri Baytı	0 – 2

Bir komut en az bir işlem kodu baytından oluşur. Örneğin NOP komutunda yalnızca işlem kodu baytı bulunur. Bazı komutlar ise bu dört bölümün dördünü de bulundurur. Örneğin `MOV WORD PTR [BX + 0200H], 2000H` assembly kodu 4 bölümden oluşur ve toplamda 6 baytlık yer kaplar.

Anlık veri baytları ve uzaklık baytlarında küçüğü başta kuralının kullanıldığı unutulmamalıdır.

Yukarıdaki işlem kodu baytında işlem kodu `100010`'dur. `100010` işlem kodu MOV komutuna karşılık gelmektedir. D yön bitidir ve 1 olduğu durumunda yazmaç ya da bellekten (R/M'den) yazmaca (REG'e) bir veri aktarımı olur. W verinin bayt mı yoksa sözcük mü olduğunu belirtir. W örnekteki gibi 1 olduğunda verinin sözcük olduğu anlaşılır. Böylece yukarıdaki işlem koduyla bellek ya da yazmaçtan bir yazmaca bir sözcüklük bir veri aktarılacağı anlaşılır.

4.5.1 Yazmaç ve R/M Bellek Adreslemeleri

Yazmaç ve R/M bellek adreslemeleri aşağıdaki tablo göz önünde bulundurularak yapılır.

Tablo 4.4: *MOV komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ↔ REG	100010DW	MOD REG R/M	–	–
Anlık Veri → R/M	1100011W	MOD 000 R/M	Veri	W=1 ise Veri
Anlık Veri → REG	1011WREG	Veri	W=1 ise Veri	–
Bellek → Birikeç	1010000W	Adres (alt)	Adres (üst)	–
Birikeç → Bellek	1010001W	Adres (alt)	Adres (üst)	–
R/M → Bölüt Yazmacı	10001110	MOD 0 REG R/M	–	–
Bölüt Yazmacı → R/M	10001100	MOD 0 REG R/M	–	–

MOD'un 00 olması uzaklık baytının kullanılmayacağını belirtir. MOD'un 01 olması 1 baytlık bir uzaklık verisinin olacağını bildirir. MOD'un 10 olması 2 baytlık (ilk bayt alt uzaklık baytı, ikinci bayt üst uzaklık baytı olmak üzere) bir uzaklık verisinin olacağını belirtir. MOD'un 11 olması ise R/M'nin bir yazmaca eşit olduğunu belirtir.

Bilgi Notu 4.3: WORD PTR, BYTE PTR Kullanımı

Eğer çevirici dil kodunda işlenenin boyutu belli ise `BYTE PTR` ya da `WORD PTR` gibi eklerin kullanılmasına gerek yoktur. Örneğin aşağıdaki kodlarda kullanılan işlenenlerin boyutları belli olduğundan bunların kullanımı gereksizdir:

```
MOV [BX], AL
MOV [BX], CH
```

Ancak aşağıdaki komutta işlenen boyutu tam olarak belli değildir:

```
MOV [BX], 02H
```

Bu yüzden yukarıdaki komutun boyutunun belirlenmesi gerekir. Aktarılacak veri 2 bayt uzunluğunda ise `WORD PTR` ile `0002H` verisi, tek bayt uzunluğunda ise `BYTE PTR` ile `02H` verisi `[BX]` adresine yüklenecektir:

```
MOV WORD PTR [BX], 02H
MOV BYTE PTR [BX], 02H
```

Not: FASM kullanırken `WORD PTR` yerine yalnızca `WORD`, `BYTE PTR` yerine ise yalnızca `BYTE` yazmak yeterlidir.

4.5.2 Özel Adresleme

R/M'nin SS:[BP] adresleme kipine geçmesi özel bir durumdur ve makine koduna çevrim yapılırken normal işlemlerden farklı bir işlem yapılmaktadır. Bu özel durumda sabit bir bellek bölgesine erişim gerekir. `MOV [1000H], AL` assembly kodu bu özel duruma bir örnektir.

`100010` işlem kodu komutun bir MOV komutu olduğunu gösterir. D'nin 0 olması yazmaçtan (REG) yazmaca ya da belleğe (R/M) veri aktarılacağını belirtir. W'nun 0 olması 1 baytın aktarılacağı anlamına gelir. REG 000 ve W 0 olduğundan AL yazmacından R/M'ye veri aktarılacaktır. R/M değeri 110 olduğundan tablodan SS:BP adreslemesi kullanıldığı görülür. [BP] adresleme kipi uzaklık baytı olmadan kullanılamaz. Ancak MOD'un 00 olması uzaklık baytı

alınmayacağının bir göstergesidir. Buna karşın [BP] adreslemelerinde uzaklık verisi kullanılır ve yukarıdaki kodun uzaklık verisinde `1000H` bulunmaktadır. Bütün bu parçalar bir araya getirilerek `MOV [1000H], AL` komutu elde edilir.

MOD değeri 00 olsa bile [BP] adreslemesi kullanılırken [BP] → [BP+00] olarak değiştirilir ve 1 tane fazladan uzaklık baytı kullanılır. Eğer yukarıdaki makine kodunun bu özelliği göz ardı ederek çeviri dil koduna geri çevrilirse MOD'un değerinin 00 olması uzaklık baytının bulunmadığı anlamına geleceğinden ilk iki bayt `MOV [BP], AL` koduna denk olur. Son iki bayt ise `ADD DL, AL` koduna denk olacağından tersine çevrim yanlış bir sonuç çıkaracaktır.

Eğer özel durum kullanılmak istenmez ise yukarıdaki işlem 2 ayrı komutla yerine getirilebilir:

```
MOV SI, 1000H
MOV [SI], AL
```

Kod 4.15: Özel durumdan kaçınma

Böylece [BP+SI] adreslemesi kullanılarak özel durumun getirdiği karmaşıklık ortadan kaldırılabilir.

4.5.3 Anlık Veri Adreslemeleri

Anlık veri adreslemesine örnek olarak `MOV WORD PTR [BX + 2000H],4432H` çevirici dil komutu verilebilir. `4432` verisi doğrudan programdan gelmektedir ve komut bu veriyi `DS` × `10H` + `2000H` + BX bellek adresine (gerçek bellek adresine) yazacaktır.

Dikkat edilirse işlem kodu baytında yön belirten D'nin bulunmadığı görünür. İşlem kodu 7 bitliktir (MOV'un anlık veri kullanan özel bir şeklidir, REG kullanılmaz, en sonda anlık veri baytları bulunur) ve adresleme baytında REG de bulunmamaktadır. R/M değeri 111 olduğu için DS:[BX] adreslemesi kullanılacaktır. MOD değeri 10 olduğundan 2 baytlık bir uzaklık verisi bulunmaktadır. Ardından 2 baytlık anlık veri baytı gelmektedir.

4.5.4 Bölüt-Yazmaç Komutları

Bölüt yazmaçlarının değerleri PUSH, POP ve MOV gibi komutlarla değiştirildiği zaman REG'in değerleri farklı anlamlara gelir.

Tablo 4.5: *Bölüt yazmaç kodları*

Kod	Bölüt Yazmacı
000	ES
001	CS
010	SS
011	DS
100	FS
101	GS

CS yazmacının değeri yalnızca özel durumlarda değişebileceği için bu yazmaç POP ve MOV komutu ile birlikte kullanılamaz. Örneğin `MOV CS, AX` kural dışı bir komuttur. CS yazmacının değeri CALL FAR komutu ile değiştirilebilir.

CS yazmacının üzerine veri yazılamamasına karşın CS yazmacının değeri başka yazmaçlara aktarılabilir. Örneğin `MOV AX,CS` komutu kurallara uygun bir komuttur ve makine dilinde bir karşılığı vardır. MOV komutunun bu durumda kullanılacak biçiminin işlem kodu `10001100`'dır. R/M bir yazmaç olduğundan MOD değeri 11 olmalıdır. REG'in CS'yi simgelemesi gerektiğinden değeri 001 olmalıdır. R/M'nin AX için değeri 000'dır.

4. Bölüm Soruları

Alıştırma 4.1. Bölüt nedir? 8086 mimarisinde kaç tür bölüt vardır? Bu bölütler ne amaçla kullanılır?

Alıştırma 4.2. Aşağıdaki sanal adreslerin gerçek fiziksel adreslerini hesaplayın:

a) CS:IP = `1234H:0050H`
b) SS:SP = `0500H:1000H`
c) DS:`0200H` ile DS = `0300H`

Alıştırma 4.3. Başlangıçta SS = `0200H`, SP = `0010H` olsun. Aşağıdaki komut dizisi çalıştırıldıktan sonra SP'nin değeri ve yığıtın durumu ne olur?

```
MOV AX, 1234H
MOV BX, 5678H
PUSH AX
PUSH BX
POP CX
```

Her adımdan sonra SP'nin değerini ve yığıtta saklanan verileri tablolayın.

Alıştırma 4.4. `MOV BL, 0ABH` komutunun makine diline çevrilmiş halini bulun. (İpucu: BL yazmacına anlık veri aktarımı için `1011WREG` işlem kodu kullanılır; W=0, REG değeri BL için nedir?)

Alıştırma 4.5. Aşağıdaki veri tanımlamalarını yorumlayın; her tanımlama bellekte kaç bayt kaplar ve hangi değerleri içerir?

```
sayi1  DB 25H
sayi2  DW 1000H
dizi   DB 4 DUP(0FFH)
adsiz  DW ?
```

Alıştırma 4.6. 8086'da yalnızca 16 bitlik yazmaçların tamamı PUSH ve POP komutlarıyla kullanılabilirken `PUSH AL` ya da `POP BH` gibi komutlar neden geçersizdir? Bu kısıtlamanın donanımsal nedenini açıklayın.

Alıştırma 4.7. COM uzantılı çalıştırılabilir dosyalar ile EXE uzantılı çalıştırılabilir dosyalar arasındaki temel fark nedir? COM dosyasının neden `ORG 0100H` ile başlaması gerekir?

Alıştırma 4.8. Aşağıdaki komutun makine diline çevrilmesinde kullanılan MOD, REG ve R/M bitlerini belirleyin:

`MOV BX, CX`

(R/M ↔ REG türündeki MOV: işlem kodu `100010DW`; D=1 yazmaçtan yazmaca kopyalama; BX ve CX için REG ve R/M değerleri nedir?)

5

Veri Taşıma Komutları

5.1 MOV

MOV (move – taşı) komutu bir veriyi bir yerden alıp başka bir yere taşır. Aşağıda yapısı gösterilen MOV komutu işlenen2 verisini işlenen1'e kopyalar. Örneğin bellekte kayıtlı olan bir baytın bir yazmaca aktarılması için MOV komutu kullanılır.

```
MOV şilenen1, şilenen2
```

Kod 5.1: MOV komutu söz dizimi

Aşağıdaki tabloda MOV komutunun değişik türevleri gösterilmiştir.

Tablo 5.1: *MOV komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M* ↔ REG*	100010DW	MOD REG R/M	–	–
Anlık Veri → R/M	1100011W	MOD 000 R/M	Veri	W=1 ise Veri
Anlık Veri → REG	1011WREG	Veri	W=1 ise Veri	–
Bellek → Birikeç	1010000W	Adres (alt)	Adres (üst)	–
Birikeç → Bellek	1010001W	Adres (alt)	Adres (üst)	–
R/M → Bölüt Yazmacı	10001110	MOD 0 REG R/M	–	–
Bölüt Yazmacı → R/M	10001100	MOD 0 REG R/M	–	–

*REG = yazmaç R/M = yazmaç ya da bellek

Örnek 5.1

Aşağıdaki programda MOV komutunun yukarıdaki türevlerine örnekler verilmiştir.

```
ORG 0100H
MOV AX, BX              ; Yazmaç → Yazmaç
MOV [SI], BX            ; Yazmaç → Bellek
MOV BX, [SI]            ; Bellek → Yazmaç
```

```
MOV BX, 1000H           ; ıAnlk Veri → REG
MOV WORD [BX], 1000H    ; ıAnlk Veri → Bellek
MOV AX, [1000H]         ; Bellek → Birikeç
MOV [SI], AX            ; Birikeç → Bellek
MOV DS, BX              ; Yazmaç → Bölüt ıYazmac
MOV DS, [SI]            ; Bellek → Bölüt ıYazmac
MOV BX, DS              ; Bölüt ıYazmac → Yazmaç
MOV [SI], DS            ; Bölüt ıYazmac → Bellek
INT 03H                 ; ıProgramn ııdurdurulmasn ğsalar
```

Kod 5.2: Örnek 5.1: MOV komutu örnekleri

MOV komutu kullanılarak aşağıdaki 3 işlem yapılamaz:

- MOV komutu ile CS ve IP yazmaçlarının değerleri değiştirilemez.
- MOV komutu ile bir bölüt yazmacından diğer bir bölüt yazmacına veri aktarılamaz. Aracı bir genel amaçlı yazmaç kullanılmalıdır.
- MOV komutu ile bir bölüt yazmacına doğrudan anlık veri kaydedilemez. Aracı bir genel amaçlı yazmaç kullanılmalıdır.

5.2 PUSH

PUSH komutu yığıta bir verinin yazılması için kullanılır. PUSH komutunun ardından SP 2 azalır ve bellekteki SS:[SP] adresine işlenen1 kaydedilir.

```
PUSH şilenen1
```

Kod 5.3: PUSH komutu söz dizimi

80186 işlemcisi ve sonraki nesil x86 işlemciler PUSH ile bir anlık veriyi de yığıta yazabilir.

Tablo 5.2: *PUSH komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M	11111111	MOD 110 R/M	–	–
Yazmaç	01010REG	–	–	–
Bölüt Yazmacı	000REG110	–	–	–

5.3 POP

POP komutu ile yığıtın tepesinden bir veri alınır. POP komutunun ardından SP 2 artar ve işlenen1'e SS:[SP] bellek adresindeki veri kaydedilir.

```
POP şilenen1
```

Kod 5.4: POP komutu söz dizimi

Tablo 5.3: *POP komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M	10001111	MOD 000 R/M	–	–
Yazmaç	01011REG	–	–	–
Bölüt Yazmacı	000REG111	–	–	–

Örnek 5.2

Aşağıda PUSH ve POP komutlarının yazmaçlarla birlikte nasıl kullanıldığını gösteren bir program gösterilmiştir.

```
ORG 0100H
MOV AX, 1000H
MOV BX, 2000H
MOV CX, 3000H
MOV DX, 4000H
PUSH AX
PUSH BX
POP AX
PUSH DX
PUSH CX
POP BX
POP CX
POP DX
INT 03H
```

Kod 5.5: Örnek 5.2: PUSH ve POP ile yazmaç değiş tokuşu

Yukarıdaki kod FASM ile çevrilip DEBUG programı ile çalıştırıldığında yazmaç değerleri aşağıda gösterildiği biçimde değişir.

Tablo 5.4: *Örnek 5.2 sonrası yazmaç değerleri*

Yazmaç	Önce	Sonra
AX	1000H	2000H
BX	2000H	3000H
CX	3000H	4000H
DX	4000H	1000H

Örnek 5.3

Aşağıdaki program bellekteki `0100H` ve `0102H` göreceli konum adreslerindeki sözcükleri yığıta atar, ardından POP komutu kullanarak bu verileri AX ve BX yazmaçlarına aktarmaktadır.

```
ORG 0100H
PUSH WORD [0100H]
PUSH WORD [0102H]
POP AX
```

```
POP BX
INT 03H
```

Kod 5.6: Örnek 5.3: Bellekten yığıt ve yazmaçlara aktarma

Yukarıdaki program çalıştıktan sonra `0102H` göreceli konum adresinden aktarılan sözcük AX yazmacına, `0100H` göreceli konum adresinden aktarılan veri ise BX yazmacına yazılacaktır.

Örnek 5.4

Aşağıdaki program PUSH ve POP komutlarını kullanarak DS bölüt yazmacının değerini BX yazmacına ve AX yazmacının değerini de DS bölüt yazmacına aktarır.

```
ORG 0100H
MOV AX, 1234H
PUSH DS
POP BX
PUSH AX
POP DS
INT 03H
```

Kod 5.7: Örnek 5.4: Bölüt yazmacına PUSH/POP ile erişim

5.4 XCHG

XCHG (exchange – değiştirme) komutu işlenenlerin verilerini birbiriyle değiştirir. Komutun yürütülmesinin ardından işlenen2'nin değeri işlenen1'e, işlenen1'in değeri de işlenen2'ye aktarılmış olur.

```
XCHG şilenen1, şilenen2
```

Kod 5.8: XCHG komutu söz dizimi

Tablo 5.5: *XCHG komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	1000011W	MOD REG R/M	–	–
Yazmaç ve Birikeç	10010REG	–	–	–

Örnek 5.5

Aşağıdaki program XCHG komutunun kullanımına bir örnektir.

```
ORG 0100H
MOV AX, 1234H
MOV BX, 4321H
MOV CX, 3214H
MOV DX, 1243H
XCHG BX, CX          ; Yazmaç ↔ Yazmaç
XCHG BX, [0120H]     ; Yazmaç ↔ Bellek
```

```
XCHG AX, DX          ; Yazmaç ↔ Birikeç
INT 03H
```

Kod 5.9: Örnek 5.5: XCHG komutu örnekleri

5.5 IN

IN komutu seçilen giriş kapısında o anda bulunan veriyi AL ya da AX yazmacına aktarır. Eğer kapı numarası 255'den daha büyük ise kapı numarasını belirtmek için DX yazmacı kullanılmalıdır.

```
IN AL ya da AX, ıkap ınumaras
```

Kod 5.10: IN komutu söz dizimi

Tablo 5.6: *IN komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
255'den daha küçük kapı numarası	1110010W	Kapı numarası	–	–
255'den büyük kapı numarası	1110110W	–	–	–

Örnek 5.6

Aşağıdaki program `FFE4H` numaralı ve `04H` numaralı kapılardan IN komutuyla nasıl veri okunacağını göstermektedir.

```
ORG 0100H
MOV DX, 0FFE4H
IN AL, DX          ; FFE4H ınumaral ıkapdan verileri oku
IN AX, 04H         ; 04H ınumaral ıkapdan verileri oku
INT 03H
```

Kod 5.11: Örnek 5.6: IN komutu örnekleri

5.6 OUT

OUT komutu AL ya da AX yazmacındaki veriyi seçilen kapıya iletir. Eğer kapı numarası 255'den daha büyük ise kapı numarasını belirtmek için DX yazmacı kullanılmalıdır.

```
OUT ıkap ınumaras, AL ya da AX
```

Kod 5.12: OUT komutu söz dizimi

Tablo 5.7: *OUT komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
255'den daha küçük kapı numarası	1110011W	Kapı numarası	–	–
255'den büyük kapı numarası	1110111W	–	–	–

Örnek 5.7

Aşağıdaki program `FFE4H` numaralı ve `04H` numaralı kapılara OUT komutuyla nasıl veri gönderileceğini göstermektedir.

```
ORG 0100H
MOV DX, 0FFE4H
MOV AX, 1234H
OUT DX, AX        ; FFE4H ınumaral ıkapya AX'deki ğdeeri yaz
OUT 04H, AL       ; 04H ınumaral ıkapya AL'deki ğdeeri yaz
INT 03H
```

Kod 5.13: Örnek 5.7: OUT komutu örnekleri

5.7 XLAT

XLAT komutu AL yazmacına DS:[BX+AL] bellek adresindeki baytı aktarır. `MOV AL, [BX+AL]` komutu ile aynı işlemi yapar.

```
XLAT [BX]
```

Kod 5.14: XLAT komutu söz dizimi

Tablo 5.8: *XLAT komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
AL yazmacına bayt aktar	11010111	–	–	–

Örnek 5.8

Aşağıdaki program XLAT komutunun nasıl kullanıldığını göstermektedir.

```
ORG 0100H
MOV BX, 0105H
MOV AL, 04H
XLAT [BX]        ; DS:[BX+AL] (DS:0108) adresindeki ıbayt AL'ye kaydet
INT 03H
```

Kod 5.15: Örnek 5.8: XLAT komutu

5.8 XLATB

XLATB komutu AL yazmacına DS:[BX+AL] adresindeki baytı aktarır. XLAT komutunun işlenen almayan kısa şeklidir.

```
XLATB
```

Kod 5.16: XLATB komutu söz dizimi

Örnek 5.9

Aşağıdaki program XLATB komutunun nasıl kullanıldığını göstermektedir.

```
ORG 0100H
MOV BX, 0104H
MOV AL, 03H
XLATB            ; AL ıyazmacna DS:0107H adresindeki ıbayt aktar
INT 03H
```

Kod 5.17: Örnek 5.9: XLATB komutu

5.9 LEA

LEA (load effective address – etkin adresi yükle) komutu etkin adresi belirtilen yazmaca kaydeder. Aşağıda gösterilen biçimde iki işlenenle kullanılır ve işlenen1'e işlenen2'nin adresi kaydedilir.

```
LEA şilenen1, şilenen2
```

Kod 5.18: LEA komutu söz dizimi

Tablo 5.9: *LEA komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Yazmaca etkin adresi kaydet	10001101	MOD REG R/M	–	–

Örnek 5.10

Aşağıdaki örnek program LEA komutunun nasıl kullanıldığını göstermektedir.

```
ORG 0100H
MOV BX, 0100H
MOV DI, 0002H
LEA SI, [BX+DI]    ; SI ıyazmacna 0102H adresini kaydet
INT 03H
```

Kod 5.19: Örnek 5.10: LEA komutu

LEA komutu kullanılırken ikinci işlenenin bellekteki bir veriyi göstermesi gerekir. LEA bu verinin adresini hedef yazmacına kendisi atayacaktır.

5.10 LDS

LDS (load DS – DS'e yükle) komutu bellekteki çift sözcüğü bir genel amaçlı yazmaca ve DS bölüt yazmacına aktarır. İşlenen1'e çift sözcüğün ilk parçası olan sözcük, DS bölüt yazmacına da çift sözcüğün ikinci parçası olan sözcük yazılır. İşlenen2 bir bellek işaretçisi olmalıdır.

```
LDS şilenen1, şilenen2
```

Kod 5.20: LDS komutu söz dizimi

Tablo 5.10: *LDS komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Çift sözcüğü yazmaç ve DS'ye kaydet	11000101	MOD REG R/M	–	–

Örnek 5.11

Aşağıdaki örnek program LDS komutunun nasıl kullanıldığını göstermektedir.

```
ORG 0100H
MOV SI, [DEG1]      ; SI ıyazmacna DEG1'in göreceli adresini kaydet
LDS BX, [SI]        ; BX ıyazmacna 0100 adresindeki ğsözcüü,
INT 03H             ; DS'ye 0102 adresindeki ğsözcüü kaydet
DEG1 dw 0100H       ; DEG1 ğşdeikenine 0100H ğsözcüünü kaydet
```

Kod 5.21: Örnek 5.11: LDS komutu

5.11 LES

LES (load ES – ES'ye yükle) komutu LDS komutunun yaptığı işlemin yaklaşık olarak aynısını yapar. Tek farkı DS yazmacı yerine ES yazmacının değerini değiştirmesidir.

```
LES şilenen1, şilenen2
```

Kod 5.22: LES komutu söz dizimi

Tablo 5.11: *LES komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Çift sözcüğü yazmaç ve ES'ye kaydet	11000100	MOD REG R/M	–	–

Örnek 5.12

Aşağıdaki örnek program LES komutunun nasıl kullanıldığını göstermektedir.

```
ORG 0100H
MOV SI, [DEG1]      ; SI ıyazmacna DEG1'in göreceli adresini kaydet
LES BX, [SI]        ; BX ıyazmacna 0100 adresindeki ğsözcüü,
INT 03H             ; ES'ye 0102 adresindeki ğsözcüü kaydet
DEG1 dw 0100H       ; DEG1 ğşdeikenine 0100H ğsözcüünü kaydet
```

Kod 5.23: Örnek 5.12: LES komutu

5.12 LAHF

LAHF (load AH with flags – AH yazmacına bayrak bitlerini yükle) komutu SF, ZF, AF, PF ve CF bayrak bitlerini sırasıyla AH yazmacının 7, 6, 4, 2 ve 0 numaralı bitlerine kaydeder. Bayrak bitleri bu işlemden etkilenmez.

```
LAHF
```

Kod 5.24: LAHF komutu söz dizimi

Tablo 5.12: *LAHF komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
AH yazmacına bayrakları kaydet	10011111	–	–	–

Örnek 5.13

Aşağıdaki küçük program LAHF komutunun program içinde nasıl kullanıldığını göstermektedir.

```
ORG 0100H
LAHF
INT 03H
```

Kod 5.25: Örnek 5.13: LAHF komutu

5.13 SAHF

SAHF (store AH into flags – AH yazmacını bayrak bitlerine kaydet) komutu AH yazmacının 7, 6, 4, 2 ve 0 bitlerini sırasıyla SF, ZF, AF, PF ve CF bayrak bitlerine kaydeder.

```
SAHF
```

Kod 5.26: SAHF komutu söz dizimi

Tablo 5.13: *SAHF komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Bayraklara AH yazmacını kaydet	10011110	–	–	–

Örnek 5.14

Aşağıdaki SAHF komutunu kullanan küçük program çalıştırıldığında SF, ZF, AF, PF ve CF bayrak bitlerinin tamamı 1 olarak belirlenir.

```
ORG 0100H
MOV AH, 0FFH
```

```
SAHF
INT 03H
```

Kod 5.27: Örnek 5.14: SAHF komutu

5.14 PUSHF

PUSHF komutu PUSH komutu gibi çalışır. Tek farkı işlenen kullanmaması ve bayrak yazmacının tamamını yığıta yazmasıdır.

```
PUSHF
```

Kod 5.28: PUSHF komutu söz dizimi

Tablo 5.14: *PUSHF komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Bayrak yazmacını yığıta kaydet	10011100	–	–	–

Örnek 5.15

Aşağıdaki program çalıştırıldığında PUSHF komutu önce bayrak yazmacını yığıta yazar. Ardından bayrak yazmacı yığıttan POP komutu ile BX yazmacına aktarılır.

```
ORG 0100H
PUSHF               ; Bayrak ııyazmacn ığıyta kaydet
POP BX              ; ığıYttaki veriyi BX ıyazmacna kaydet
INT 03H
```

Kod 5.29: Örnek 5.15: PUSHF komutu

5.15 POPF

POPF komutu POP komutu gibi çalışır. Tek farkı yığıttan aldığı veriyi doğrudan bayrak yazmacına kaydetmesidir. Hedef olarak bayrak yazmacını kullandığından işleneni yoktur.

```
POPF
```

Kod 5.30: POPF komutu söz dizimi

Tablo 5.15: *POPF komutu türevleri*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Yığıttaki veriyi bayrak yazmacına kaydet	10011101	–	–	–

Örnek 5.16

Aşağıdaki program POPF komutunun nasıl kullanıldığını göstermektedir. Programın sonucunda bayrak yazmacına `1234H` değeri yazılır.

```
ORG 0100H
MOV BX, 1234H
PUSH BX              ; ığıYta 1234H ğdeerini yaz
POPF                 ; ığıYttaki veriyi bayrak ıyazmacna kaydet
INT 03H
```

Kod 5.31: Örnek 5.16: POPF komutu

5.16 Veri Taşıma Komutlarının Bayrak Etkileri

Aşağıdaki Tablo 5.16, bu bölümde ele alınan veri taşıma komutlarının OF, SF, ZF, AF, PF ve CF bayrak bitleri üzerindeki etkilerini özetlemektedir. Tabloda (—) simgesi ilgili bayrağın komuttan etkilenmediğini göstermektedir.

Tablo 5.16: *Veri taşıma komutlarının bayrak etkileri. (—) etkilenmez.*

Komut	OF	SF	ZF	AF	PF	CF
MOV	—	—	—	—	—	—
PUSH	—	—	—	—	—	—
POP	—	—	—	—	—	—
XCHG	—	—	—	—	—	—
IN	—	—	—	—	—	—
OUT	—	—	—	—	—	—
XLAT	—	—	—	—	—	—
XLATB	—	—	—	—	—	—
LEA	—	—	—	—	—	—
LDS	—	—	—	—	—	—
LES	—	—	—	—	—	—
LAHF	—	—	—	—	—	—
SAHF	*	*	*	*	*	*
PUSHF	—	—	—	—	—	—
POPF	*	*	*	*	*	*

* SAHF komutu AH yazmacının ilgili bitlerini SF, ZF, AF, PF ve CF bayraklarına yükler; OF etkilenmez. POPF komutu ise yığıttan aldığı sözcüğü bayrak yazmacının tamamına (OF dahil) yazar.

Bölüm Sonu Soruları

Alıştırma 5.1. SI yazmacında bir bellek konumunun adresinin bulunduğunu varsayarak, bu adresten başlayıp bellekten 100 tane 1 baytlık sayı okuyan ve bu sayıların en küçüğünü BL en

büyüğünü de BH yazmacına aktaran bir program yazın.

Alıştırma 5.2. AL yazmacında bir sayı olduğunu varsayarak o numaralı terime kadar olan tüm Fibonacci sayılarını yığıta yazan bir program yazın. [İpucu: Fib(0) = 0, Fib(1) = 1, a > 1 için Fib(a) = Fib(a-1) + Fib(a-2)]

Alıştırma 5.3. Aşağıdaki komut dizisi adım adım işlendiğinde SP, AX, BX ve CX yazmaçlarının değerlerini bulun. Başlangıç değerleri: SP = 0100H, AX = 1111H, BX = 2222H, CX = 3333H.

```
PUSH AX
PUSH BX
PUSH CX
POP  AX
POP  BX
POP  CX
```

Alıştırma 5.4. Aşağıdaki iki kod parçasının farkını açıklayın. Her ikisinin BX yazmacına bıraktığı değer nedir? (SAYI etiketi `0200H` göreceli adresinde tanımlıdır.)

```
; Kod 1
MOV BX, OFFSET SAYI

; Kod 2
LEA BX, [SAYI]
```

Alıştırma 5.5. `DS:0100H` bellek bölgesinde şu dört bayt bulunmaktadır: `34H, 12H, 78H, 56H`. Aşağıdaki komutlardan sonra BX ve DS yazmaçlarının değerlerini bulun.

```
MOV SI, 0100H
LDS BX, [SI]
```

Alıştırma 5.6. `DS:0100H` bellek bölgesinde şu dört bayt bulunmaktadır: `CDH, ABH, 01H, 00H`. Aşağıdaki komutlardan sonra DI ve ES yazmaçlarının değerlerini bulun.

```
MOV BX, 0100H
LES DI, [BX]
```

Alıştırma 5.7. Başlangıç değerleri AX = 1234H, BX = 5678H olan bir programda aşağıdaki komut dizisi uygulandıktan sonra AX ve BX yazmaçlarının değerlerini bulun. Yığıt kullanmadan XCHG ile aynı sonucu elde edebilir misiniz?

```
PUSH AX
PUSH BX
POP  AX
POP  BX
```

Alıştırma 5.8. `0105H` adresinden başlayan bir bayt tablosu şu değerleri içermektedir: `30H, 31H, 32H, 33H, 34H, 35H, 36H, 37H, 38H, 39H` (ASCII '0'–'9'). BX = 0105H ve AL = 07H iken XLAT komutunun ardından AL yazmacının değeri ne olur?

6 Aritmetik ve Mantık Komutları

6.1 Aritmetik Komutları

6.1.1 ADD ve ADC

ADD (add – topla) komutu ile işlenen1 ve işlenen2'nin değerleri toplanır ve sonuç işlenen1'e kaydedilir. İşlenen2'nin değeri değişmez.

```
ADD islenen1, islenen2
```

Kod 6.1: ADD komutunun sözdizimi

ADC (add with carry – artan ile topla) komutu ile işlenen1 ve işlenen2 toplanır ve sonuç işlenen1'e kaydedilir. İşlenen2'nin değeri ADD komutunda olduğu gibi değişmez. ADC'nin ADD komutundan tek farkı, bir önceki işlemden kalan CF bayrak bitinin değerini toplama işlemine dahil etmesidir: işlenen1 = işlenen1 + işlenen2 + CF. Bu sayede 8 ya da 16 bitten daha geniş sayıların çok adımlı olarak toplanması mümkün olur.

```
ADC islenen1, islenen2
```

Kod 6.2: ADC komutunun sözdizimi

Tablo 6.1: *ADD komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	`000000DW`	`MOD REG R/M`	–	–
Anlık Veri ve R/M	`100000SW`	`MOD 000 R/M`	Veri	W=1 ise Veri
Anlık Veri ve Birikeç	`0000010W`	Veri	W=1 ise Veri	–

Örnek

Aşağıda ADD komutunun değişik kullanım biçimlerini gösteren bir örnek verilmiştir.

```
1   ORG 0100H
2   MOV AL, 03H
```

```
3   MOV AH, 04H
4   ADD AL, AH                ; Yazmaç ve Yazmaç
5   ADD [0100], AH            ; Bellek ve Yazmaç
6   ADD AH, [0100]            ; Yazmaç ve Bellek
7   ADD AX, 1234H             ; Birikeç ve ıAnlk Veri
8   ADD WORD [0102H], 1234H   ; Bellek ve ıAnlk Veri
9   INT 03H
```

Kod 6.3: ADD komutunun farklı kullanım biçimleri (Örnek 6.1)

8. satırda `[0102H]` göreceli adresindeki verinin boyutu belli olmadığı için önüne bu verinin bir sözcük boyutunda olduğunu belirten `WORD` belirleyicisi yerleştirilmiştir.

Örnek

Aşağıdaki kodu FASM çevirici ile çevirip DEBUG programı ile deneyelim.

```
1   ORG 0100H
2   MOV AX, 01H
3   MOV BX, 00000010B
4   ADD AX, BX
5   MOV CX, 3
6   ADD BX, CX
7   INT 03H
```

Kod 6.4: Bayt toplama örneği (Örnek 6.2)

Programı çalıştırdıktan sonra yazmaçlar aşağıdaki değerlere sahip olacaktır:

Tablo 6.2: *Örnek 6.2 çıktısı – yazmaç değerleri.*

Yazmaç	Değer
AX	`0003H`
BX	`0005H`
CX	`0003H`

İkinci satırdaki kod ile AX yazmacına onaltılık tabandaki `01` değeri kaydedilmiştir. Üçüncü satırda BX yazmacına ikili `00000010` değeri yazılmıştır. Dördüncü satırdaki ADD AX,BX komutu işlendikten sonra toplam olan 3 değeri AX yazmacına kaydedilmiş olacaktır. Altıncı satırda BX ve CX yazmaçlarının değerleri toplanıp sonuç BX yazmacına yazılmıştır; bu komut işlendikten sonra BX yazmacının değeri ondalık 5 değerine eşit olacaktır.

Örnek

Kelime boyutundaki verileri toplamak için aşağıdaki örneği inceleyelim.

```
1   ORG 0100H
2   MOV AX, 0101H
3   MOV BX, 0000111100000010B
4   ADD AX, BX
5   MOV CX, 300
```

```
6   ADD BX, CX
7   INT 03H
```

Kod 6.5: Kelime toplama örneği (Örnek 6.3)

Program çalıştırıldıktan sonra yazmaçların yeni değerleri şunlar olacaktır:

Tablo 6.3: *Örnek 6.3 çıktısı – yazmaç değerleri.*

Yazmaç	Değer
AX	1003H
BX	102EH
CX	012CH

ADD komutu işlenen olarak en fazla bir kelime alabilir. İkili kelimeleri (veya daha uzun sayıları) toplayabilmek için ADC komutunu kullanmak gerekir.

Örnek

Aşağıda ADC komutunun nasıl kullanıldığını gösteren bir örnek verilmiştir.

```
1   ORG 0100H
2   MOV BX, 1234H
3   MOV CX, 0FFFFH
4   ADC BX, CX
5   MOV DX, 0FFFFH
6   ADC DX, 0FFFFH
7   ADC AX, 1234H
8   INT 03H
```

Kod 6.6: ADC komutu örneği (Örnek 6.4)

6. satırda DX yazmacı ve `FFFFH` sayısının toplanmasının ardından CF bayrak biti 1 olacaktır ve bu nedenle sonuca 1 eklenecek ve DX yazmacının değeri `FFFF` olarak belirlenecektir. Eğer 6. satırdaki ADC komutunun yerine ADD komutu kullanılırsa komutun ardından DX yazmacının alacağı değer `FFFE`'dir.

Örnek

İki ikili kelimemiz olsun: `01234567H` ve `1234F679H`. Bu sayıları ADC komutunu kullanarak toplayalım.
Sayıları kelimelere ayıralım. Önce her ikili kelimenin 2. kelimesini toplayacağız ve bir yazmaca kaydedeceğiz; ardından eğer ilk toplama işleminden artan varsa 1. kelimeleri toplarken bu artanı da ekleyeceğiz.

```
1   ORG 0100H
2   JMP BASLA
3   DB 23H, 01H, 67H, 45H
4   DB 34H, 12H, 79H, 0F6H
5   BASLA:
6   MOV SI, 0104H
```

```
7   MOV AX, WORD [SI]
8   MOV SI, 0108H
9   MOV BX, WORD [SI]
10  ADD AX, BX
11  DEC SI
12  DEC SI
13  MOV BX, WORD [SI]
14  MOV SI, 0102H
15  MOV CX, WORD [SI]
16  ADC CX, BX
17  XCHG AX, BX
18  XCHG AX, CX
19  INT 03H
```

Kod 6.7: İkili kelime toplama algoritması (Örnek 6.5)

On altıncı satırda ADC komutu kullanılarak bir önceki ADD'den artan olduysa bu artan toplama işlemine eklenmektedir. Sonuç olarak AX yazmacında `1358H` ve BX yazmacında `3BE0H` değeri bulunacaktır.

İşaretli Sayılarla ADD ve ADC Komutları

İşaretli sayılarda pozitif sayıların büyüklüğü 0 ile 127 arasında, negatif sayılarda ise -1 ile -128 arasındadır. Sayıları işaretli sayılara çevirmek için önce sayı yazılır, ardından her bit ters çevrilir ve sonuca 1 eklenir (ikiye tümleyen yöntemi).

Örneğin `-92` ile `-92` sayılarını topladığımızda 8 bitlik yazmaçlarla taşma oluşur; OF bayrak biti 1 olur. Doğru sonucu elde etmek için 16 bitlik yazmaçlar kullanılmalı ya da CBW komutuyla işaret genişletmesi yapılmalıdır.

```
1   ORG 0100H
2   MOV AL, -92
3   MOV BL, -92
4   ADD AL, BL
5   JO TASMA_VAR
6   JMP TASMA_YOK
7   TASMA_VAR:
8   MOV DX, 1111H
9   INT 03H
10  TASMA_YOK:
11  MOV DX, 0000H
12  INT 03H
```

Kod 6.8: İşaretli sayı taşmasını tespit etme (Örnek 6.5 devamı)

CBW komutu AL yazmacının 7. bitini alıp AH yazmacının tüm bitlerine kopyalar. Bu yöntemle `-92` ve `-92` sayıları doğru biçimde toplanabilir ve AX yazmacında `FF48H` (`-184`) sonucu elde edilir.

6.1.2 INC

INC (increment – artır) komutu işlenen1'in değerini 1 artırır. İşlem sonucunda elde çıksa da INC komutu CF bayrak bitinde değişikliğe neden olmaz.

```
INC islenen1
```

Kod 6.9: INC komutunun sözdizimi

Tablo 6.4: *INC komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M	1111111W	MOD 000 R/M	–	–
Yazmaç	01000REG	–	–	–

Örnek

Aşağıda INC komutunun nasıl kullanıldığını gösteren bir örnek verilmiştir.

```
1  ORG 0100H
2  MOV AX, 0FFFH
3  INC AX
4  INC BYTE [0120H]
5  INT 03H
```

Kod 6.10: INC komutu örneği (Örnek 6.6)

3. satırda AX yazmacının değeri bir artırılarak değeri `1000H` yapılmıştır. 4. satırda ise bellekte `0120H` göreceli konum adresindeki baytın değeri bir artırılmaktadır.

6.1.3 AAA

AAA (ASCII Adjust for ADD – ADD komutu için ASCII düzenlemesi) komutu AL yazmacında ikilik düzende saklanan değeri paketlenmemiş bir ondalık değere (BCD – Binary Coded Decimal düzenine) çevirir. Komut çalıştırıldığı sırada, eğer AL yazmacının değeri 9'dan büyükse ya da AF bayrak biti 1 ise, AH yazmacının değeri 1, AL yazmacının değeri 6 artırılır ve AF ile CF bayrak bitleri 1 yapılır. AAA komutu çalıştırıldığı sırada AL yazmacının değeri 9'dan büyük değilse AF ve CF bayrak bitleri sıfırlanır. Her iki durumda da AL yazmacının üst yarım baytı mutlaka sıfır değerini alır. AAA komutu herhangi bir işlenen almaz.

```
AAA
```

Kod 6.11: AAA komutunun sözdizimi

Tablo 6.5: *AAA komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
AAA	00110111	–	–	–

Örnek

Aşağıda AAA komutunun nasıl kullanıldığını gösteren bir örnek verilmiştir.

```
1  ORG 0100H
```

```
2   MOV AH, 00H
3   MOV AL, 08H
4   AAA
5   MOV BX, AX
6   MOV AH, 00H
7   MOV AL, 0EH
8   AAA
9   MOV CX, AX
10  INT 03H
```

Kod 6.12: AAA komutu örneği (Örnek 6.7)

İkinci ve üçüncü satırlarda AX yazmacının üst yarısına `00H` ve alt yarısına `08H` değerleri atanmıştır. AL yazmacının tuttuğu veri olan `08H` değeri 9'dan küçük olduğundan AL yazmacında herhangi bir değişiklik gerçekleşmez ve beşinci satırda BX yazmacına aktarılan değer `0008H`'dir.
Altıncı ve yedinci satırlarda AX yazmacının değeri `000EH` olarak belirlenmiştir. AL yazmacının sahip olduğu değer `0EH` (onluk tabanda 14) olduğundan sekizinci satırdaki AAA komutu çalıştırıldığında bu değeri onluk tabana dönüştürür. Dönüşüm için `0EH` sayısına `06H` eklenir, AL yazmacının üst yarım baytı sıfırlanır ve AH yazmacına 1 eklenir. Aşağıda AAA komutunun AX yazmacı üzerinde yaptığı işlemler adım adım gösterilmiştir:

Tablo 6.6: *AAA komutunun iç işlemleri.*

		AX	0	0	0	0	0	0	0	0	0	0	0	0	1	1	1	0
1. ADD AL, 6	→	AX	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0
2. AND AL, 00001111B	→	AX	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0
3. INC AH	→	AX	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0

Böylece programın dokuzuncu satırından sonra CX yazmacı AX yazmacının o anki değeri olan `0104H` değerini tutar. AH ve AL yazmaçlarının birlikte onluk tabandaki 14 sayısını gösterdiği dikkat çekmektedir.

6.1.4 DAA

DAA (Decimal Adjust for ADD – ADD komutu için ondalık düzenleme) komutu AL yazmacında saklanan ikilik tabandaki değeri, BCD düzeninde paketlenmiş bir ondalık değere çevirir. Eğer AL yazmacının alt yarım baytının değeri 9'dan büyükse ya da AF bayrak biti 1 ise AL yazmacına `06H` eklenir ve AF bayrak bitinin değeri 1 yapılır. Eğer AL yazmacının değeri `9F`'den büyükse ya da CF bayrak biti 1 ise AL yazmacına `60H` değeri eklenir ve CF bayrak bitinin değeri 1 olarak belirlenir.

```
DAA
```

Kod 6.13: DAA komutunun sözdizimi

Tablo 6.7: *DAA komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
DAA	`00100111`	–	–	–

Örnek

Aşağıda DAA komutunun nasıl kullanıldığını gösteren bir örnek verilmiştir.

```
1  ORG 0100H
2  MOV AL, 3EH
3  DAA
4  INT 03H
```

Kod 6.14: DAA komutu örneği (Örnek 6.8)

AL yazmacının değeri DAA komutundan önce `3EH` olarak belirlenmiştir. AL yazmacının alt yarım baytının değeri 9'dan büyük olan `E` (onluk tabanda 14) sayısı olduğu için AL yazmacına `06H` eklenecek ve yazmacın değeri `44H` olacaktır.

6.1.5 SUB ve SBB

SUB (subtract – çıkarma) komutu ile işlenen1'den işlenen2'nin değeri çıkarılır ve sonuç işlenen1'e yazılır. Bu komut sonrasında işlenen2'nin değeri değişmez.

```
SUB islenen1, islenen2
```

Kod 6.15: SUB komutunun sözdizimi

SBB (subtract with borrow – elde ile çıkarma) komutu işlenen1'den işlenen2'yi çıkarır ve sonucu işlenen1'e kaydeder. SBB komutunun SUB komutundan tek farkı elde ile işlem yapmasıdır. SBB komutu CF bayrak biti 1 ise, çıkarma işlemine ek olarak işlenen1'den 1 çıkarır; CF bayrak biti 0 olduğunda ise SUB komutu ile aynı işlemi yapar. CF bayrak bitinin değeri ne olursa olsun işlenen2'nin değeri değişmez.

```
SBB islenen1, islenen2
```

Kod 6.16: SBB komutunun sözdizimi

Tablo 6.8: *SUB komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç ya da tersi	`001010DW`	`MOD REG R/M`	–	–
R/M'den Anlık Veri	`100000SW`	`MOD 101 R/M`	Veri	SW=01 ise Veri
Birikeçten Anlık Veri	`0010110W`	Veri	W=1 ise Veri	–

Tablo 6.9: *SBB komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç ya da tersi	`000110DW`	`MOD REG R/M`	–	–
R/M'den Anlık Veri	`100000SW`	`MOD 011 R/M`	Veri	SW=01 ise Veri
Birikeçten Anlık Veri	`0001110W`	Veri	W=1 ise Veri	–

Örnek

Aşağıda SUB komutunun değişik kullanım biçimlerini gösteren bir örnek program verilmiştir.

```
1  ORG 0100H
2  MOV AX, 1234H
3  MOV BX, 4321H
4  MOV CX, 1243H
5  SUB BX, CX
6  SUB CX, WORD [0110H]
7  SUB AX, WORD [0101H]
8  INT 03H
```

Kod 6.17: SUB komutu örneği (Örnek 6.9)

Beşinci satırında içeriği `4321H` olan BX yazmacından içeriği `1234H` olan CX yazmacı çıkarılmaktadır. Çıkarma işlemi sonucunda bulunan fark olan `30EDH` değeri BX yazmacının yeni değeri olacaktır.
İkili kelimelerin çıkarılmasında da ADD/ADC çiftine benzer biçimde SUB/SBB çifti kullanılır.

Örnek

Aşağıda SBB komutunun nasıl kullanıldığını gösteren bir örnek program verilmiştir.

```
1   ORG 0100H
2   MOV AL, 03H
3   MOV BL, 02H
4   SBB AL, BL
5   MOV CL, 0FFH
6   ADD CL, 0FFH
7   MOV CL, 09H
8   MOV DL, 07H
9   SBB CL, DL
10  INT 03H
```

Kod 6.18: SBB komutu örneği (Örnek 6.10)

Dördüncü satırdaki SBB komutu CF bayrak biti 0 iken, dokuzuncu satırdaki SBB komutu ise CF bayrak biti 1 olması durumunda çıkarma işlemine örnektir. Dokuzuncu satırdaki SBB komutu CF bayrak biti 1 olduğu için çıkarma sonucunda bulunan sonuçtan 1 çıkaracaktır.

İşaretli Sayılarla SUB ve SBB

İşaretli sayılarla çıkarma işleminde de 8 bitlik yazmaçlar yeterli gelmeyebilir. Örneğin `-92` sayısından `92` sayısını çıkarınca sonuç `-184` olması gerekirken 8 bitlik alanda `+72` elde edilir. Bunun nedeni negatif sayıların 8 bitte `-128` ile `-1` arasında temsil edilmesidir. CBW komutu kullanılarak işaret genişlemesi yapılıp 16 bitlik yazmaçlarla işlem gerçekleştirildiğinde BX yazmacında `FF48H` (`-184`) doğru sonucu elde edilir.

6.1.6 DEC

DEC (decrement – azalt) komutu INC komutunun tersidir. İşlenen1'in değerini 1 azaltır.

```
DEC islenen1
```

Kod 6.19: DEC komutunun sözdizimi

Tablo 6.10: *DEC komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M	`1111111W`	`MOD 001 R/M`	–	–
Yazmaç	`01001REG`	–	–	–

Örnek

Aşağıda DEC komutunun nasıl kullanıldığını gösteren bir örnek program verilmiştir. Bu program sonlandığında BL yazmacının değeri `02H` olur.

```
1  ORG 0100H
2  MOV BL, 03H
3  DEC BL
4  INT 03H
```

Kod 6.20: DEC komutu örneği (Örnek 6.11)

6.1.7 NEG

NEG (negate – ters çevir) komutu işlenen1'in 2'ye tümleyenini alır ve sonucu yeniden işlenen1'e kaydeder.

```
NEG islenen1
```

Kod 6.21: NEG komutunun sözdizimi

Tablo 6.11: *NEG komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
2'ye tümleyenini al	`1111011W`	`MOD 011 R/M`	–	–

Örnek

Aşağıda NEG komutunun nasıl kullanıldığını gösteren bir örnek program verilmiştir. Bu program sonlandığında BL yazmacının değeri `FDH` (`11111101B`) olur.

```
1  ORG 0100H
2  MOV BL, 03H
3  NEG BL
4  INT 03H
```

Kod 6.22: NEG komutu örneği (Örnek 6.12)

6.1.8 CMP

CMP (compare – karşılaştır) komutu işlenen1 ile işlenen2'nin değerlerini karşılaştırır. Karşılaştırmanın sonucuna göre değiştirilmesi gereken bayrak bitlerinin (OF, SF, ZF, AF, PF, CF) değerleri CMP komutu tarafından değiştirilir. Bu komut işlenen1 ya da işlenen2'nin değerini değiştirmez.

```
CMP islenen1, islenen2
```

Kod 6.23: CMP komutunun sözdizimi

Tablo 6.12: *CMP komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	`001110DW`	`MOD REG R/M`	–	–
R/M ve Anlık Veri	`100000SW`	`MOD 111 R/M`	Veri	SW=01 ise Veri
Birikeç ve Anlık Veri	`0011110W`	Veri	W=1 ise Veri	–

Örnek

Aşağıdaki tabloda örnek olarak işlenen1 ve işlenen2 değerleri ile CMP komutu sonrasında değişmesi gereken bayrak bitlerinin durumları verilmiştir.

Tablo 6.13: *CMP komutu bayrak bit durumları (Örnek 6.13).*

Durum	İşlenen1	İşlenen2	CMP sonrasında Bayrak Bitleri
1	5	5	ZF=1; PF=1
2	5	6	CF=1; SF=1; PF=1; AF=1
3	5	4	–
4	5	-4	CF=1; PF=1; AF=1
5	-5	5	SF=1; PF=1
6	-5	-5	ZF=1; PF=1
7	-5	-6	–

CMP komutunun ardından genellikle JE (jump if equal – eşit ise atla) benzeri bir atlama komutu kullanıldığından hangi bayrak bitinin 1 hangisinin 0 olduğunun bilinmesine

genelde gerek yoktur.

Örnek

Aşağıda verilen örnek programda CMP komutu kullanılmıştır.

```
1  ORG 0100H
2  MOV AL, 13H
3  MOV BL, 13H
4  CMP AL, BL
5  INT 03H
```

Kod 6.24: CMP komutu örneği (Örnek 6.14)

AL ve BL yazmaçlarının değerleri 3. satırın ardından birbirine eşit olduğu için 4. satırdaki CMP komutu ZF bayrak bitini bire eşitler.

6.1.9 AAS

AAS (ASCII Adjust after Subtraction – Çıkarma işlemi sonrası ASCII düzenlemesi) komutu AL yazmacındaki değeri BCD biçiminde paketlenmiş bir ondalık değere çevirir. Eğer AL yazmacının değeri 9'dan büyükse ya da AF bayrak biti 1 ise AL yazmacının değeri 6, AH yazmacının değeri ise 1 azaltılarak AF ve CF bayrak bitleri bire eşitlenir. AL yazmacının değeri 9'dan büyük değilse AF ve CF bayraklarının değeri sıfırlanır. AF bayrağının değerinden bağımsız olarak AL yazmacının üst yarım biti AAS komutundan sonra her zaman sıfırlanır.

```
AAS
```

Kod 6.25: AAS komutunun sözdizimi

Tablo 6.14: *AAS komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
AAS	`00111111`	–	–	–

Örnek

Aşağıda verilen örnek programda AAS komutunun nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AX, 020AH
3  AAS
4  INT 03H
```

Kod 6.26: AAS komutu örneği (Örnek 6.15)

İkinci satırdaki MOV komutu AH yazmacının değerini `02H` ve AL yazmacının değerini `0A` olarak belirlemektedir. Üçüncü satırdaki AAS komutu çalıştırıldığında AL yazmacının değeri 9'dan büyük olan `A` (onluk tabanda 10) sayısına eşit olduğu için AL yazmacının değeri 6 azaltılarak `04H`, AH yazmacının değeri ise 1 azaltılarak `01H` yapılır. Böylece programın sonunda AX yazmacının değeri `0104H` olarak belirlenir.

6.1.10 DAS

DAS (Decimal Adjust after SUB – SUB komutu sonrası ondalık düzenleme) komutu AL yazmacındaki değeri, paketlenmiş bir ondalık değere çevirir. Eğer AL yazmacının alt yarım baytının değeri 9'dan büyükse ya da AF bayrak biti 1 ise AL yazmacından `06H` çıkarılır ve AF bayrak biti birlenir. Eğer AL yazmacının değeri `9F`'den de büyükse ya da CF bayrak biti 1 ise AL yazmacından ayrıca `60H` değeri çıkarılır ve CF bayrak biti birlenir.

```
DAS
```

Kod 6.27: DAS komutunun sözdizimi

Tablo 6.15: *DAS komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
DAS	`00101111`	–	–	–

Örnek

Aşağıda verilen örnek programda DAS komutunun nasıl kullanıldığı gösterilmiştir.

```
1   ORG 0100H
2   MOV AL, 0FFH
3   DAS
4   INT 03H
```

Kod 6.28: DAS komutu örneği (Örnek 6.16)

İkinci satırda AL yazmacının değeri `FF` olarak belirlenmiştir. Bu değerin alt yarım baytı olan `F`, 9'dan büyük; değerin tamamı da `9F`'den büyük olduğu için üçüncü satırdaki DAS komutuyla AL yazmacından `66H` (önce `06H` sonra `60H`) çıkarılarak değeri `99`'a eşitlenir ve CF bayrağı birlenir.

6.1.11 MUL ve IMUL

MUL (multiply – çarp) komutu işlenen1'in boyutu 1 bayt ise işlenen1'i AL yazmacı ile çarpıp sonucu AX yazmacına kaydeder. Eğer işlenen1 iki baytlık bir sözcük ise işlenen1, AX yazmacı ile çarpılır ve sonuç DX ve AX yazmaçlarına birlikte yazılır. CF ve OF bayrak bitleri sonucun üst baytları 0 olduğu sürece sıfıra eşittir.

```
MUL islenen1
```

Kod 6.29: MUL komutunun sözdizimi

IMUL komutu MUL komutunun işaretli işlem yapan türevidir. IMUL komutunun sonucunda bulunan çarpım işlenen1'e sığabiliyorsa CF ve OF bayrak bitleri sıfıra eşitlenir.

```
IMUL islenen1
```

Kod 6.30: IMUL komutunun sözdizimi

Tablo 6.16: *MUL komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
MUL	`1111011W`	`MOD 100 R/M`	–	–

Tablo 6.17: *IMUL komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
IMUL	`1111011W`	`MOD 101 R/M`	–	–

Örnek

Aşağıda verilen örnek programda MUL komutunun nasıl kullanıldığı gösterilmiştir.

```
1   ORG 0100H
2   MOV AL, 02H
3   MOV BL, 03H
4   MUL BL
5   MOV CX, AX
6   MOV BX, 1234H
7   MOV AL, 0FH
8   MUL BX
9   INT 03H
```

Kod 6.31: MUL komutu örneği (Örnek 6.17)

İkinci satırda AL yazmacının değeri 2, üçüncü satırında BL yazmacının değeri 3 olarak belirlenmiştir. Dördüncü satırdaki MUL komutu çalıştığında 2 ile 3 çarpılarak çarpım değeri olan 6, AX yazmacına yazılacaktır.
Sekizinci satırında 16 bitlik BX yazmacı MUL komutuna işlenen olarak verildiği için MUL komutu AX × BX işlemini yapacaktır. `1234H` × `000FH` işleminin sonucunda ortaya çıkan `1110CH` çarpımının üst 16 bitlik bölümü olan `0001H` değeri DX yazmacına, alt 16 bitlik bölümü olan `110CH` değeri ise AX yazmacına yazılacaktır.

Örnek

Bir bayt ile bir baytı çarpalım:

```
1   ORG 0100H
2   MOV AL, 03H
3   MOV BL, 04H
4   MUL BL
5   INT 03H
```

Kod 6.32: Bayt çarpma örneği (Örnek 6.18)

Yukarıdaki program `03H` (3) sayısı ile `04H` (4) sayısını çarpıp sonucu AX yazmacına kaydetmektedir. Programı DEBUG programı ile denersek AX yazmacında `000CH` (12) sayısının kayıtlı olduğunu göreceğiz.
İki kelimeyi birbirleriyle çarpmak için:

```
1  ORG 0100H
2  MOV AX, 0F234H
3  MOV BX, 0FEFEH
4  MUL BX
5  INT 03H
```

Kod 6.33: Kelime çarpma örneği (Örnek 6.18 devamı)

`F234H` ve `FEFEH` sayılarının çarpımının `F13FE798H` değerine eşit olduğu görülecektir. Sonucun ilk 4 hanesi DX yazmacına, son 4 hanesi ise AX yazmacına kaydedilmiştir.

Örnek

`12345679H` ve `012FFF67H` ikili kelimelerini çarpalım. İkili kelimeleri kelimelere ayırarak MUL komutunu tekrarlı biçimde uygulamak ve ara sonuçları toplamak gerekir. Aşağıda yapılan işlemler gösterilmiştir:

1. `5679H` × `FF67H` → `564551AFH` (0202H adresine kaydet)
2. `1234H` × `FF67H` → `12291EECH`
3. `1EECH` ile 0202H adresindeki kelimeyi topla, sonucu 0202H adresine yaz
4. `1229H` ile önceki artanı topla, 0200H adresine yaz
5. `5679H` × `012FH` → `00665937H` (0212H adresine kaydet)
6. `1234H` × `012FH` → `00158B8CH`
7. `8B8CH` ile 0212H adresindeki kelimeyi topla, sonucu 0212H adresine yaz
8. `0015H` ile önceki artanı topla, 0210H adresine yaz
9. 0200H ve 0210H adreslerine kaydedilmiş sayıları hane sırası gözetilerek topla, sonucu 0230H adresine yaz

Programı denediğimizde 0230H adresinde Little Endian kuralına uygun olarak yazılmış değerler bulunur. Gerçekte sonuç `00159E1BCE6851AFH`'dir; bu değer `12345679H` ve `012FFF67H` değerlerinin çarpımının sonucudur.

Örnek

Aşağıda verilen örnek programda IMUL komutunun nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AL, -02H
3  MOV BL, 03H
4  IMUL BL
5  INT 03H
```

Kod 6.34: IMUL komutu örneği (Örnek 6.19)

Bu programda AL yazmacının değeri sıfırdan küçük olan `-02H` değerine eşitlenmiştir. Program sonuçlandığında AX yazmacının değeri -2×3 işleminin sonucu olan `FFFAH` sayısına eşitlenecektir. OF ve CF bayrakları da işlemin sonucunda sıfırlanacaktır.

Örnek

IMUL komutunu örnekleyelim:

```
1   ORG 0100H
2   MOV AL, -4
3   MOV BL, -2
4   IMUL BL
5   MOV BX, AX
6   MOV AL, -4
7   MOV CL, 2
8   IMUL CL
9   MOV CX, AX
10  INT 03H
```

Kod 6.35: IMUL komutu çarpma örneği (Örnek 6.20)

Programı denediğinizde yazmaç değerlerinin aşağıdaki değerlere sahip olduklarını göreceksiniz:

Tablo 6.18: *Örnek 6.20 çıktısı – yazmaç değerleri.*

Yazmaç	Değer
BX	`0008H`
CX	`FFF8H`

CX yazmacında -4×2 sayısının çarpımının sonucu, BX yazmacında ise $-4 \times (-2)$ sayılarının çarpımının sonucu bulunmaktadır.

6.1.12 AAM

AAM (ASCII Adjust after Multiplication – Çarpma sonrası ASCII düzenlemesi) komutu AL yazmacının değerini alıp 10'a böler ve sonucu AH yazmacına kaydeder. Bölme işlemi sonunda kalan olursa, AL yazmacına kaydedilir.

```
AAM
```

Kod 6.36: AAM komutunun sözdizimi

Tablo 6.19: *AAM komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
AAM	`11010100`	`00001010`	–	–

Örnek

Aşağıda verilen örnek programda AAM komutunun nasıl kullanıldığı gösterilmiştir. Programın ikinci satırında AL yazmacının değeri `0AH`'ye (onluk tabanda 10'a) eşitlenir ve üçüncü satırdaki AAM komutunun ardından AH yazmacının değeri `01H` olarak belirlenir. Bölme işlemi sonucunda kalan oluşmadığı için AAM komutu çalıştıktan sonra

AX yazmacının aldığı değer `0100H` olur.

```
1  ORG 0100H
2  MOV AL, 0AH
3  AAM
4  INT 03H
```

Kod 6.37: AAM komutu örneği (Örnek 6.21)

6.1.13 DIV ve IDIV

DIV (divide – böl) komutu, işlenen1 bir bayt uzunluğunda ise, AX yazmacını işlenen1'e böler ve bölümü AL yazmacına, kalanı AH yazmacına kaydeder. İşlenen1 16 bitlik bir sözcük ise DX ve AX yazmaçlarının birlikte oluşturduğu değeri işlenen1'e böler ve bölümü AX yazmacına, kalanı DX yazmacına kaydeder.

```
DIV islenen1
```

Kod 6.38: DIV komutunun sözdizimi

IDIV, DIV komutunun işaretli sayılarla işlem yapan türevidir. DIV komutunun kullanıldığı biçimde kullanılır.

```
IDIV islenen1
```

Kod 6.39: IDIV komutunun sözdizimi

Tablo 6.20: *DIV komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
DIV	`1111011W`	`MOD 110 R/M`	–	–

Tablo 6.21: *IDIV komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
IDIV	`1111011W`	`MOD 111 R/M`	–	–

Örnek

Aşağıda verilen örnek programda DIV komutu kullanılarak nasıl bölme yapıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AX, 0034H
3  MOV BL, 03H
4  DIV BL
5  INT 03H
```

Kod 6.40: DIV komutu örneği (Örnek 6.22)

Programın başında AX yazmacının değeri `0034H` olarak, BL yazmacının değeri ise `03H`

olarak belirlenmiş ve dördüncü satırdaki DIV komutu ile 34/3 işlemi yapılmıştır. BL yazmacı bir bayt uzunluğunda olduğundan bu işlem sonucunda ortaya çıkan bölüm olan `11H` AL yazmacına, kalan olan `01H` ise AH yazmacına yazılır. Böylece AX yazmacının aldığı değer `0111H` olur.

Örnek

`56H` (86) sayısını `03H` (03) sayısına bölelim.

```
1  ORG 0100H
2  MOV AL, 86H
3  MOV BL, 03H
4  DIV BL
5  INT 03H
```

Kod 6.41: Örnek 6.23

AX yazmacında `021CH` değerinin bulunduğunu göreceğiz. AH'da bulunan `02H` (2) kalan ve AL'de bulunan `1CH` (28) bölümdür.

Örnek

`5656H` (22102) sayısını `0303H` (771) sayısına bölelim.

```
1  ORG 0100H
2  MOV DX, 0000H
3  MOV AX, 5656H
4  MOV BX, 0303H
5  DIV BX
6  INT 03H
```

Kod 6.42: Örnek 6.24

AX yazmacında `001CH` (28) ve DX yazmacında `0202H` (514) değerini göreceksiniz. AX yazmacındaki değer bölüm ve DX yazmacındaki değer kalandır.

Örnek

Aşağıdaki kodu deneyelim:

```
1  ORG 0100H
2  MOV AX, 5656H
3  MOV BX, 0000H
4  DIV BX
5  INT 03H
```

Kod 6.43: Sıfıra bölme hatası örneği (Örnek 6.25)

Kodu denediğinizde "Bölme Taşması" hatası ile karşılaşacaksınız. Bunun nedeni 0'a bölme yapılmasıdır. Bu tür kural dışı bir durum bölüm yazmaca sığmazsa da oluşabilir:

```
1  ORG 0100H
2  MOV DX, 0FFFFH
```

```
3   MOV AX, 0FFFFH
4   MOV BX, 0001H
5   DIV BX
6   INT 03H
```

Kod 6.44: Bölüm taşması örneği (Örnek 6.25 devamı)

Yukarıdaki programı denediğinizde yine "Bölme Taşması" kural dışı hata mesajını alacaksınız. Bunun nedeni DX ve AX'ten bulunan ikili kelimeyi (`FFFFFFFFH` sayısını) BX yazmacının değeri olan 1'e bölünce sonucun `FFFFFFFFH` bulunmasıdır; ancak bu sayı AX yazmacına kaydedilemeyecek kadar büyüktür. "Bölme Hatası" oluştuğunda tip 0 kesmesi oluşur.

Örnek

Aşağıda verilen örnek programda IDIV komutu kullanılarak işaretli sayılarla nasıl bölme yapıldığı gösterilmiştir.

```
1   ORG 0100H
2   MOV AX, -23H
3   MOV BL, 03H
4   IDIV BL
5   INT 03H
```

Kod 6.45: IDIV komutu örneği (Örnek 6.26)

Bu örnek program $-23/3$ işlemini AL yazmacındaki değeri BL yazmacındaki değere bölerek yapar. Bu işlem sonucunda bulunan bölüm -7, kalan ise 2'dir. Program çalıştıktan sonra AX yazmacının değeri `02F9H` olacaktır. -7 sayısı ikiye tümleyen yönteminde `F9` olarak gösterilmektedir.

Örnek

İşlenen olarak bir bayt girelim ve sonuçları inceleyelim.

```
1   ORG 0100H
2   MOV AX, -18
3   MOV BL, -2
4   IDIV BL
5   INT 03H
```

Kod 6.46: IDIV bayt işlenen örneği (Örnek 6.27)

Bu örnekte -18 sayısını -2 sayısına böldük. AL yazmacında `09H` (9) ve AH yazmacında `00H` (0) değerlerinin olduğunu göreceksiniz; kalan 0 ve bölüm 9'dur.
İşlenen olarak bir kelime girelim:

```
1   ORG 0100H
2   MOV DX, 0000H
3   MOV AX, 65535
4   MOV BX, -4660
5   IDIV BX
6   INT 03H
```

Kod 6.47: IDIV kelime işlenen örneği (Örnek 6.27 devamı)

Bu örnekte 65535 sayısını −4660 sayısına böldük. DX yazmacında `0127H` (295) kalan ve AX yazmacında `FFF2H` (−14) bölüm bulunmaktadır.

6.1.14 AAD

AAD (ASCII Adjust before Division – Bölme öncesi ASCII düzenleme) komutu bölme işleminde kullanılmak üzere AX yazmacında BCD düzeninde saklanan onluk tabandaki sayıyı ikilik tabana dönüştürür.

```
AAD
```

Kod 6.48: AAD komutunun sözdizimi

Tablo 6.22: *AAD komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
AAD	`11010101`	`00001010`	–	–

Örnek

Aşağıda verilen örnek programda AAD komutunun nasıl kullanıldığı gösterilmiştir.

```
1   ORG 0100H
2   MOV AH, 01H
3   MOV AL, 02H
4   AAD
5   INT 03H
```

Kod 6.49: AAD komutu örneği (Örnek 6.28)

Programda AH yazmacına `01H` değeri ve AL yazmacında `02H` değeri verilmiştir. Bu iki yazmacın değerleri BCD düzeninde onluk tabandaki 12 sayısını göstermektedir. AAD komutu çalıştırıldığında onluk tabandaki 12 sayısının ikilik tabandaki `0CH` değeri hesaplanır ve sonuç AL yazmacına kaydedilir.

6.1.15 CBW

CBW komutu (convert byte to word – baytı sözcüğe çevir) AL yazmacındaki baytı, iki baytlık bir sözcüğe dönüştürür. AH yazmacına, AL yazmacının en anlamlı biti bire eşit ise `FFH` değeri, değilse `00H` değeri yazılır. CBW komutu böylece bir baytlık verilerin 16 bitlik sözcüklere işaret genişlemesi yoluyla çevrilmesini sağlar. CBW her zaman AL yazmacıyla çalışır ve başka işlenen almaz.

```
CBW
```

Kod 6.50: CBW komutunun sözdizimi

Tablo 6.23: *CBW komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CBW	`10011000`	–	–	–

Örnek

Aşağıda verilen örnek programda önce AX yazmacının içeriği sıfırlanmakta, ardından bu yazmacın alt yarısına `FE` değeri yüklenmektedir.

```
1  ORG 0100H
2  MOV AX, 0000H
3  MOV AL, 0FEH
4  CBW
5  INT 03H
```

Kod 6.51: CBW komutu örneği (Örnek 6.29)

CBW komutu çalıştırıldığında AL yazmacının en anlamlı biti 1 olduğu için AX yazmacının programdan sonraki değeri `FFFEH` olacaktır.

6.1.16 CWD

CWD (convert word to double word – sözcüğü çift sözcüğe çevir) komutu AX yazmacındaki 16 bitlik sözcüğü 32 bitlik bir ikili sözcüğe çevirir. Dönüşüm sonrasında ortaya çıkan verinin üst yarısını gösteren sözcük DX yazmacına, alt yarısını gösteren sözcük ise AX yazmacına kaydedilir. AX yazmacının en anlamlı biti 1 ise DX yazmacına `FFFFH` değeri, sıfır ise DX yazmacına `0000H` değeri yazılır.

```
CWD
```

Kod 6.52: CWD komutunun sözdizimi

Tablo 6.24: *CWD komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CWD	`10011001`	–	–	–

Örnek

Aşağıda verilen örnek programda önce DX yazmacının içeriği sıfırlanmakta ve AX yazmacına `FFFEH` değeri yüklenmektedir. CWD komutunun ardından DX yazmacı AX yazmacının en anlamlı bitinin değeri ile doldurulacağından, DX yazmacı `FFFFH` değerini alır.

```
1  ORG 0100H
2  MOV DX, 0000H
3  MOV AX, 0FFFEH
4  CWD
5  INT 03H
```

Kod 6.53: CWD komutu örneği (Örnek 6.30)

AX yazmacına atanan başlangıç değerinin programda onaltılık tabanda 5 basamaklı bir sayı olarak gösterilmesine karşın işaret bitinin iki baytlık değerin en soldaki biti olduğuna dikkat edilmelidir.

6.2 Mantık Komutları

6.2.1 NOT

NOT komutu işlenen1'in her bitini tersine çevirir. NOT komutunun ardından işlenen1'in içinde yer alan her bir 0 biti 1'e, 1 biti ise sıfıra dönüşür.

```
NOT islenen1
```

Kod 6.54: NOT komutunun sözdizimi

NOT komutunun işlevi mantıksal devre tasarımında kullanılan DEĞİL kapısının işleviyle aynıdır.

Tablo 6.25: *NOT komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
NOT	`1111011W`	`MOD 010 R/M`	–	–

Örnek

Aşağıda NOT komutunun program içinde nasıl kullanıldığı gösterilmiştir. 3. satırdaki NOT komutunun ardından BL yazmacının değeri `11110000B` olacaktır.

```
1   ORG 0100H
2   MOV BL, 00001111B
3   NOT BL
4   INT 03H
```

Kod 6.55: NOT komutu örneği (Örnek 6.31)

6.2.2 SHL

SHL (shift logical left – mantık sola ötele) komutu işlenen1'i işlenen2'nin değeri kadar sola öteler. En soldaki bit öteleme sonucunda CF bayrak bitine yazılır, en sağdaki bit ise sıfırlanır. Eğer işlenen1'in işareti değişmez ise OF bayrağının değeri 0'a, değişirse 1'e eşitlenir.

```
SHL islenen1, islenen2
```

Kod 6.56: SHL komutunun sözdizimi

Tablo 6.26: *SHL komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
SHL	`110100VW`	`MOD 100 R/M`	–	–

Örnek

Aşağıda SHL komutunun program içinde nasıl kullanıldığı gösterilmiştir. 3. satırdaki SHL komutunun ardından BL yazmacının değeri iki basamak sola ötelenerek `00111100B` olacak, CF ve OF bayrakları sıfırlanacaktır.

```
1  ORG 0100H
2  MOV BL, 00001111B
3  SHL BL, 02H
4  INT 03H
```

Kod 6.57: SHL komutu örneği (Örnek 6.32)

6.2.3 SAL

SAL (shift arithmetic left – aritmetik sola ötele) komutu SHL komutu ile aynı işleve, işlem koduna ve adresleme koduna sahiptir. SHL komutunun kullanıldığı her programda SHL komutunun yerine SAL komutu yazılabilir.

```
SAL islenen1, islenen2
```

Kod 6.58: SAL komutunun sözdizimi

Tablo 6.27: *SAL komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
SAL	`110100VW`	`MOD 100 R/M`	–	–

Örnek

SHL komutu için verilen örnekte SHL yerine aşağıda gösterildiği gibi SAL yazılırsa programın işlevi değişmez.

```
1  ORG 0100H
2  MOV BL, 00001111B
3  SAL BL, 02H
4  INT 03H
```

Kod 6.59: SAL komutu örneği (Örnek 6.33)

6.2.4 SHR

SHR (shift logic right – mantık sağa ötele) komutu işlenen1'i işlenen2'nin değeri kadar sağa öteler. En sağdaki bit öteleme sonucunda CF bayrak bitine kaydırılır, en soldaki bit ise sıfırlanır.

Eğer işlenen1'in işareti değişmez ise OF bayrağının değeri sıfırlanır, değişirse birlenir.

```
SHR islenen1, islenen2
```

Kod 6.60: SHR komutunun sözdizimi

Tablo 6.28: *SHR komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
SHR	`110100VW`	`MOD 101 R/M`	–	–

Örnek

Aşağıdaki örnekte SHR komutu kullanılarak BL yazmacındaki verinin iki basamak sağa nasıl ötelendiği gösterilmektedir. Programın çalışmasının ardından BL yazmacının içeriği `00000011B` olacak ve OF bayrağı sıfırlanacaktır.

```
1   ORG 0100H
2   MOV BL, 00001111B
3   SHR BL, 02H
4   INT 03H
```

Kod 6.61: SHR komutu örneği (Örnek 6.34)

6.2.5 SAR

SAR (shift arithmetic right – aritmetik sağa ötele) komutu işlenen1'i işlenen2'nin değeri kadar sağa öteler. En sağdaki bit öteleme sonucunda CF bayrak bitine kaydedilir. En soldaki bit işaret biti kabul edildiğinden değeri değiştirilmez. SAR komutu aritmetik işlemlerde işaret biti yitirilmeden ötelenme işlemi yapılması için daha uygundur.

```
SAR islenen1, islenen2
```

Kod 6.62: SAR komutunun sözdizimi

Tablo 6.29: *SAR komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
SAR	`110100VW`	`MOD 111 R/M`	–	–

Örnek

Aşağıdaki örnekte BL yazmacının değeri başlangıçta ikilik tabandaki karşılığı `10000100B` olan `84H` olarak belirlenmiştir. BL yazmacında saklanan bu değerin en soldaki biti 1 olduğu için saklanan sayı sıfırdan küçüktür.
SAR komutu çalıştırıldığında BL yazmacının değeri iki basamak sağa ötelenecek, ancak soldaki işaret biti değiştirilmeyerek aynen kalacaktır. Öteleme işlemi sırasında her adımda en soldaki bitten sağa ötelenecek bit işaret biti olan 1'dir. Böylece programın ardından BL yazmacı ikilik tabandaki karşılığı `11100001B` olan `E1H` değerini tutacaktır.

```
1  ORG 0100H
2  MOV BL, 84H
3  SAR BL, 02H
4  INT 03H
```

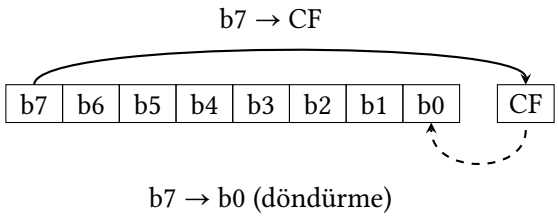
Kod 6.63: SAR komutu örneği (Örnek 6.35)

6.2.6 ROL

ROL (rotate left – sola döndür) komutu işlenen1'i işlenen2'nin değeri kadar sola doğru döndürerek işlenen1'deki değeri öteler. Öteleme öncesinde en solda olan bit, öteleme sonrasında en sağa geçer ve aynı zamanda CF bayrak bitine kaydedilir. ROL komutunun ardından eğer işlenen1'in işareti değişmez ise OF bayrak bitinin değeri 0 olur. İşlenen1'in işaretinin değerinin değişmesi durumunda OF bayrak biti birlenir.

```
ROL islenen1, islenen2
```

Kod 6.64: ROL komutunun sözdizimi



Şekil 6.1: ROL: En soldaki bit (b7) hem CF'ye hem de en sağ konuma (b0) taşınır; tüm bitler bir konum sola kayar.

Tablo 6.30: *ROL komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
ROL	`110100VW`	`MOD 000 R/M`	–	–

Örnek

Aşağıdaki örnekte BL yazmacının değeri başlangıçta ikilik tabandaki karşılığı `01000100B` olan `44H` olarak belirlenmiştir.
ROL komutu BL yazmacının içeriğini sola doğru 2 basamak döndürecektir. İşlem sonucunda BL yazmacının alacağı değer `00010001B` (`11H`) olacaktır. BL yazmacının işareti değişmediği için OF bayrak bitinin değeri sıfırdır.

```
1  ORG 0100H
2  MOV BL, 44H
3  ROL BL, 02H
4  INT 03H
```

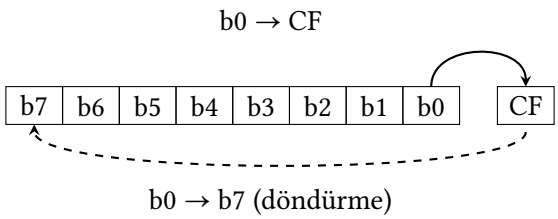
Kod 6.65: ROL komutu örneği (Örnek 6.36)

6.2.7 ROR

ROR (rotate right – sağa döndür) komutu işlenen1'i işlenen2'nin değeri kadar sağa döndürür. Öteleme öncesinde en sağda olan bit öteleme sonrasında en sola geçer. Bu bit aynı zamanda CF bayrak bitine kaydedilir. OF bayrak biti, eğer işlenen1'in işareti değişmez ise 0, aksi takdirde 1 değerini alır.

```
ROR islenen1, islenen2
```

Kod 6.66: ROR komutunun sözdizimi



Şekil 6.2: ROR: En sağdaki bit (b0) hem CF'ye hem de en sol konuma (b7) taşınır; tüm bitler bir konum sağa kayar.

Tablo 6.31: *ROR komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
ROR	`110100VW`	`MOD 001 R/M`	–	–

Örnek

Aşağıdaki örnekte BL yazmacının değeri başlangıçta ikilik tabandaki karşılığı `10000100B` olan `84H` olarak belirlenmiştir. BL yazmacında saklanan bu değerin en soldaki biti 1 olduğu için saklanan sayı sıfırdan küçüktür. ROR komutunun ardından BL yazmacının değeri 2 basamak sağa dönerek `00100001B` (`21H`) olur. İşlemin sonunda BL yazmacı işaret değiştirdiği için OF bayrak biti birlenir.

```
1   ORG 0100H
2   MOV BL, 84H
3   ROR BL, 02H
4   INT 03H
```

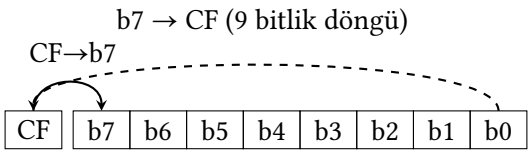
Kod 6.67: ROR komutu örneği (Örnek 6.37)

6.2.8 RCL

RCL (rotate with CF left – CF bayrak biti ile birlikte sola döndür) komutu işlenen1 ile CF'nin birlikte oluşturduğu değeri işlenen2'nin değeri kadar sola döndürerek öteler. Öteleme öncesinde en solda olan bit öteleme sonrasında en sağdaki bit olarak kullanılan CF bayrak bitine geçer. Bayrak bitindeki değer ise işlenen1'in en sağındaki bitine aktarılır. OF bayrak biti, eğer işlenen1'in işareti değişmez ise 0, aksi takdirde 1 değerini alır.

```
RCL islenen1, islenen2
```

Kod 6.68: RCL komutunun sözdizimi



Şekil 6.3: RCL: CF dahil 9 bitlik döngü sola döner. CF b7 konumuna, b7 yeni CF'ye geçer.

Tablo 6.32: *RCL komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
RCL	`110100VW`	`MOD 010 R/M`	–	–

Örnek

Aşağıdaki örnekte BL yazmacının değeri başlangıçta ikilik tabandaki karşılığı `01000100B` olan `44H` olarak belirlenmiş ve STC komutuyla CF bayrak biti birlenmiştir. RCL komutu ile BL yazmacı ile CF bayrak bitinin oluşturduğu değer sola doğru iki basamak döndürülerek, BL yazmacının değeri `00010010B` (`12H`) değerine, CF bayrak bitinin değeri ise 1'e getirilecektir.

```
1   ORG 0100H
2   MOV BL, 44H
3   STC          ; CF bayrak bitini 1 yapar
4   RCL BL, 02H
5   INT 03H
```

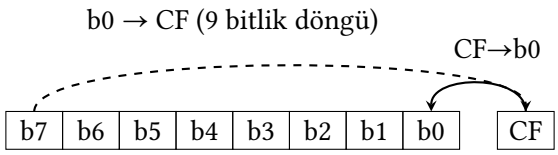
Kod 6.69: RCL komutu örneği (Örnek 6.38)

6.2.9 RCR

RCR (rotate with CF right – CF bayrak biti ile birlikte sağa döndür) komutu işlenen1 CF'nin birlikte oluşturduğu değeri işlenen2'nin değeri kadar sağa döndürerek öteler. Öteleme öncesinde en sağda olan bit, öteleme sonrasında en soldaki bit olarak kullanılan CF bayrak bitine geçer. Bayrak bitindeki değer ise işlenen1'in en solundaki bite aktarılır. OF bayrak biti eğer işlenen1'in işareti değişmez ise 0, aksi takdirde 1 değerini alır.

```
RCR islenen1, islenen2
```

Kod 6.70: RCR komutunun sözdizimi



Şekil 6.4: RCR: CF dahil 9 bitlik döngü sağa döner. CF b0 konumuna, b0 yeni CF'ye geçer.

Tablo 6.33: *RCR komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
RCR	`110100VW`	`MOD 011 R/M`	–	–

Örnek

Aşağıdaki örnekte BL yazmacının değeri başlangıçta ikilik tabandaki karşılığı `01000100B` olan `44H` olarak belirlenmiş ve STC komutuyla CF bayrak biti birlenmiştir. RCR komutunun ardından CF bayrak biti ile BL yazmacının oluşturduğu değer sağa doğru iki basamak döndürülerek, BL yazmacının değeri `00010001B` (`11H`) değerine, CF bayrak bitinin değeri ise 0'a getirilecektir.

```
1   ORG 0100H
2   MOV BL, 44H
3   STC          ; CF bayrak bitini 1 yapar
4   RCR BL, 02H
5   INT 03H
```

Kod 6.71: RCR komutu örneği (Örnek 6.39)

6.2.10 AND

AND komutu mantıksal devre tasarımında kullanılan VE kapısının görevini görür. İşlenen1 ve işlenen2'nin her bitinin değerine göre aşağıdaki tabloya uygun olarak bir sonuç üretilir ve üretilen sonuç işlenen1'e aktarılır.

```
AND islenen1, islenen2
```

Kod 6.72: AND komutunun sözdizimi

Tablo 6.34: *AND (VE) işleminin doğruluk tablosu.*

İşlenen1	İşlenen2	Sonuç
0	0	0
0	1	0
1	0	0
1	1	1

Tablo 6.35: *AND komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	`001000DW`	`MOD REG R/M`	–	–
R/M ve Anlık Veri	`1000000W`	`MOD 100 R/M`	Veri	W=1 ise Veri
Birikeç ve Anlık Veri	`0010010W`	Veri	W=1 ise Veri	–

Örnek

Aşağıda AND komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AL, 01010101B
3  MOV BL, 00001111B
4  AND AL, BL           ; Yazmaç VE Yazmaç
5  AND BL, 11110010B    ; Yazmaç VE ıAnlk Veri
6  AND BL, [0100H]      ; Yazmaç VE Bellek
7  AND AL, [0101H]      ; Birikeç VE Bellek
8  INT 03H
```

Kod 6.73: AND komutu örneği (Örnek 6.40)

Programın başında AL yazmacının değeri `01010101B`, BL yazmacının değeri ise `00001111B` olarak belirlenmiştir. 4. satırdaki AND komutu iki yazmacın tuttuğu değerleri kullanarak VE işlemi yapar ve sonuç değeri olan `00000101B`'yi AL yazmacına yazar. 5. satırdaki AND komutu BL yazmacının değerini doğrudan `11110010B` değeri ile VE işlemine sokmakta ve sonucu BL yazmacına yazmaktadır. 5. satırın ardından BL yazmacının değeri `00000010B` olur.

6.2.11 TEST

TEST komutu AND komutuna benzer biçimde işlenenler arasında VE işlemini gerçekleştiren bir komuttur. AND komutundan farklı olarak TEST komutunun ardından gerçekleştirilen VE işleminin sonucu herhangi bir yerde saklanmaz. Komutun ardından VE işleminin sonucuna bağlı olarak, yalnızca SF, ZF ve PF bayrak bitlerinin değerleri 1 ya da 0 olarak belirlenir. TEST komutu daha çok döngülerin koşullarını denetlemek amacıyla kullanılır.

```
TEST islenen1, islenen2
```

Kod 6.74: TEST komutunun sözdizimi

Tablo 6.36: *TEST komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	`1000010W`	`MOD REG R/M`	–	–
R/M ve Anlık Veri	`1111011W`	`MOD 000 R/M`	Veri	W=1 ise Veri
Birikeç ve Anlık Veri	`1010100W`	Veri	W=1 ise Veri	–

Örnek

Aşağıda TEST komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV BL, 00001111B
3  TEST BL, 11110010B
4  INT 03H
```

Kod 6.75: TEST komutu örneği (Örnek 6.41)

3. satırdaki TEST komutunun ardından BL yazmacının değeri değişmez. Ancak VE işleminin sonucu `00000010B` olduğu için SF bayrak bitinin değeri 0, ZF bayrak biti 0 ve PF bayrak biti de 0 olarak belirlenir. Eğer 3. satırdaki `11110010B` değeri yerine `11110000B` değeri kullanılırsa SF, ZF ve PF bayrak bitleri sırasıyla 0, 1 ve 1 değerlerini alır.

6.2.12 OR

OR komutu mantıksal devre tasarımında kullanılan VEYA kapısının görevini görür. İşlenen1 ve işlenen2'nin her bitinin değerine göre aşağıdaki tabloya uygun olarak bir sonuç üretilir ve üretilen sonuç işlenen1'e aktarılır.

```
OR islenen1, islenen2
```

Kod 6.76: OR komutunun sözdizimi

Tablo 6.37: *OR (VEYA) işleminin doğruluk tablosu.*

İşlenen1	İşlenen2	Sonuç
0	0	0
0	1	1
1	0	1
1	1	1

Tablo 6.38: *OR komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	`000010DW`	`MOD REG R/M`	–	–
R/M ve Anlık Veri	`1000000W`	`MOD 001 R/M`	Veri	W=1 ise Veri
Birikeç ve Anlık Veri	`0000110W`	Veri	W=1 ise Veri	–

Örnek

Aşağıda OR komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AL, 01010101B
3  MOV BL, 00001111B
4  OR AL, BL            ; Yazmaç VEYA Yazmaç
5  OR BL, 11110010B     ; Yazmaç VEYA ıAnlk Veri
```

```
6   OR BL, [0100H]      ; Yazmaç VEYA Bellek
7   OR AL, [0101H]      ; Birikeç VEYA Bellek
8   INT 03H
```

Kod 6.77: OR komutu örneği (Örnek 6.42)

Programın başında AL yazmacının değeri `01010101B`, BL yazmacının değeri ise `00001111B` olarak belirlenmiştir. 4. satırdaki OR komutu VEYA işlemi yapar ve sonuç değeri olan `01011111B`'yi AL yazmacına yazar. 5. satırın ardından BL yazmacının değeri `11111111B` olur.

6.2.13 XOR

XOR komutu mantıksal devre tasarımında kullanılan ÖZEL VEYA kapısının görevini görür. İşlenen1 ve işlenen2'nin her bitinin değerine göre aşağıdaki tabloya uygun olarak bir sonuç üretilir ve üretilen sonuç işlenen1'e aktarılır.

```
XOR islenen1, islenen2
```

Kod 6.78: XOR komutunun sözdizimi

Tablo 6.39: *XOR (ÖZEL VEYA) işleminin doğruluk tablosu.*

İşlenen1	İşlenen2	Sonuç
0	0	0
0	1	1
1	0	1
1	1	0

Tablo 6.40: *XOR komutunun makine dili kodlaması.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
R/M ve Yazmaç	`001100DW`	`MOD REG R/M`	–	–
R/M ve Anlık Veri	`1000000W`	`MOD 110 R/M`	Veri	W=1 ise Veri
Birikeç ve Anlık Veri	`0011010W`	Veri	W=1 ise Veri	–

Örnek

Aşağıda XOR komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
1   ORG 0100H
2   MOV AL, 01010101B
3   MOV BL, 00001111B
4   XOR AL, BL          ; Yazmaç ÖZEL VEYA Yazmaç
5   XOR BL, 11110010B   ; Yazmaç ÖZEL VEYA ıAnlk Veri
6   XOR BL, [0100H]     ; Yazmaç ÖZEL VEYA Bellek
7   XOR AL, [0101H]     ; Birikeç ÖZEL VEYA Bellek
8   INT 03H
```

Kod 6.79: XOR komutu örneği (Örnek 6.43)

Programın başında AL yazmacının değeri `01010101B`, BL yazmacının değeri ise `00001111B` olarak belirlenmiştir. 4. satırdaki XOR komutu ÖZEL VEYA işlemi yapar ve sonuç değeri olan `01011010B`'yi AL yazmacına yazar. 5. satırın ardından BL yazmacının değeri `11111101B` olur.

6.3 Mantık İşlemleri

Elimizde mantıksal işlemlerde kullanmak üzere NOT, SHL, SAL, SHR, SAR, ROL, ROR, RCL, RCR, AND, TEST, OR, XOR komutları bulunmaktadır. Bu komutları kullanarak donanım düzeyindeki mantıksal devreleri assembly diliyle modellemek mümkündür.

İlk olarak 8 bitlik 2 sayıyı toplayabilen bir devre tasarlayalım ve bu devreyi tasarladıktan sonra bilgisayarımızda yazacağımız program ile sınayalım. Aşağıda 2 tane 1 bitlik sayının toplamının doğruluk tablosu verilmiştir.

- X_0: Birinci sayı 1. bit
- Y_0: İkinci sayı 1. bit
- A_0: İki sayının 1. bitlerinin toplamından artan
- T_0: İki sayının 1. bitlerinin toplamı

Tablo 6.41: *1 bitlik tam toplayıcı doğruluk tablosu.*

X_0	Y_0	A_0	T_0
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

Tabloyu A_0 için Karnaugh haritasına aktarırsak:

Tablo 6.42: *A_0 için Karnaugh haritası.*

X_0 / Y_0	0	1
0	0	0
1	0	1

Karnaugh haritasını kullanarak A_0 için aşağıdaki denklemi elde edebiliriz:

$$A_0 = X_0 \text{ AND } Y_0$$

Aynı şekilde tabloyu T_0 için Karnaugh haritasına aktarırsak:

Tablo 6.43: *T_0 için Karnaugh haritası.*

X_0 / Y_0	0	1
0	0	1
1	1	0

Karnaugh haritasını kullanarak T_0 için aşağıdaki denklemi elde edebiliriz:

$$T_0 = (X_0' \cdot Y_0) + (Y_0' \cdot X_0) = X_0 \text{ XOR } Y_0$$

Bu denklemleri kullanarak artanları da hesaba katarak 16 bitlik tam toplayıcı tasarlamak gerekir. Bunun için önceki bit toplamından gelen artanın da toplamaya katılacağı bir mantıksal devrenin oluşturulması gerekir. Bu devre için gerekli doğruluk tablosu aşağıdadır:

- A_n: $(n-2)$'inci bitten gelen artan
- X_n: 1. sayı $(n-1)$'inci bit
- Y_n: 2. sayı $(n-1)$'inci bit
- T_n: 1. ve 2. sayının $(n-1)$'inci bitleri ve $(n-2)$'inci bitten gelen artanın toplamı
- A_{n+1}: Toplamdan artan

Tablo 6.44: *Tam toplayıcı doğruluk tablosu.*

A_n	X_n	Y_n	T_n	A_{n+1}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

T_n için Karnaugh haritası:

Tablo 6.45: *T_n için Karnaugh haritası.*

A_n / X_nY_n	00	01	11	10
0	0	1	0	1
1	1	0	1	0

Karnaugh haritasını kullanarak T_n için aşağıdaki denklemi yazabiliriz:

$$T_n = A_n \text{ XOR } (X_n \text{ XOR } Y_n)$$

A_{n+1} için Karnaugh haritası:

Tablo 6.46: *A_{n+1} için Karnaugh haritası.*

A_n / X_nY_n	00	01	11	10
0	0	0	1	0
1	0	1	1	1

$$A_{n+1} = (X_n \text{ AND } Y_n) \text{ OR } (A_n \text{ AND } Y_n) \text{ OR } (A_n \text{ AND } X_n)$$

Yukarıda tasarlanan devre 8 bitlik 2 sayının toplanması işlemi için uygun bir bitlik tam toplayıcıdır. Bu bir bitlik tam toplayıcıyı 8 kez kullanarak 8 bitlik toplayıcı elde edilebilir. İlk tam bit toplayıcısına artan olarak 0 verilir çünkü ilk bit toplamından önce herhangi bir bit toplamı yoktur. Son tam bit toplayıcısındaki A_8 1 olduğunda toplama işlemi artma hatası vermiş olacaktır. Sonuç $T_7T_6T_5T_4T_3T_2T_1T_0$ sayısıdır.

Yukarıda tasarlanan devrenin assembly ile modelini oluşturup devrenin düzgün çalışıp çalışmadığını doğrulayalım. `01010101B` ve `01000001B` değerlerini toplayalım.

Denklemler:

$$T_n = A_n \text{ XOR } (X_n \text{ XOR } Y_n)$$
$$A_{n+1} = (X_n \text{ AND } Y_n) \text{ OR } (A_n \text{ AND } Y_n) \text{ OR } (A_n \text{ AND } X_n)$$

```
1   ORG 0100H
2   MOV AH, 00000000B
3   MOV CH, 00000000B
4   MOV CL, 00000000B
5   MOV AL, 00000000B
6   MOV BH, 10000000B
7   MOV BL, 10000000B
8   DONGU:
9   ROL BH, 1
10  ROL BL, 1
11  PUSH BX
12  AND BH, 01010101B
13  AND BL, 01000001B
14  PUSH BX
15  XOR BH, BL
16  ROL AH, 1
17  XOR BH, AH
18  OR CH, BH
19  POP BX
20  MOV DH, BH
21  MOV DL, DL
22  AND DL, AH
23  AND DH, AH
24  AND BH, BL
25  OR BH, DL
26  OR BH, DH
27  MOV AH, BH
28  POP BX
29  CMP BX, 8080H
30  JE SON
31  JMP DONGU
```

```
32  SON:
33  INT 03H
```

Kod 6.80: 8 bitlik tam toplayıcı assembly modellemesi (Örnek 6.43 uygulaması)

İkinci, üçüncü ve dördüncü satırlarda değişkenlere ilk değerler veriliyor. Altıncı ve yedinci satırlarda BH ve BL yazmaçlarına `10000000B` değeri veriliyor; bu iki yazmaç sayıların sadece bir bitini yüklemek için kullanılacaktır. Dokuzuncu ve onuncu satırlarda işlenme biti 1 basamak sola kaydırılmaktadır. On ikinci ve on üçüncü satırlarda toplanacak olan sayılar ile AND işlemi yapılmakta ve sadece o bitin değeri BH ve BL yazmacına geçmektedir.

On beşinci satırda T_n denklemindeki (X_n XOR Y_n) bölümü hesaplanıp sonucu BH yazmacına kaydedilmektedir. On altıncı satırda AH yazmacındaki artan değeri bir sola kaydırılarak doğru bit konumuna getirilmektedir. On yedinci satırda artan ile BH yazmacındaki değer XOR işlemine tabi tutulmaktadır.

Yirmi ikinci ile yirmi altıncı satırlar arasında ikinci denklemdeki A_{n+1} hesabı yapılmaktadır. Yirmi yedinci satırda yeni artan AH yazmacına kaydedilmektedir.

Programı denediğimizde AH yazmacında `00H` değeri ve CH yazmacında `96H` değerinin olduğunu göreceğiz. AH yazmacındaki `00H` değeri toplama işlemimizden herhangi bir artan olmadığını göstermektedir.

Programımız ile `11110000B` ve `11001100B` değerlerini toplayalım. Programı DEBUG ile denediğimizde AH yazmacında `80H` değerinin bulunduğunu ve CH yazmacında `BCH` değerinin bulunduğunu göreceksiniz. Son bitlerin toplamından bir artma olmuştur ve toplama işlemimizin sonucu `1BCH`'dır.

6.4 Aritmetik ve Mantık Komutlarının Bayrak Etkileri

Aşağıdaki tablo bu bölümdeki komutların OF, SF, ZF, AF, PF ve CF bayrak bitleri üzerindeki etkilerini özetlemektedir. (—) etkilenmez; * komut sonucuna göre güncellenir; U tanımsız (undefined).

Tablo 6.47: *Aritmetik ve mantık komutlarının bayrak etkileri. (—) etkilenmez, (*) sonuca göre güncellenir, (U) tanımsız.*

Komut	OF	SF	ZF	AF	PF	CF
ADD	*	*	*	*	*	*
ADC	*	*	*	*	*	*
SUB	*	*	*	*	*	*
SBB	*	*	*	*	*	*
MUL	*	U	U	U	U	*
IMUL	*	U	U	U	U	*
DIV	U	U	U	U	U	U
IDIV	U	U	U	U	U	U
INC	*	*	*	*	*	—
DEC	*	*	*	*	*	—
NEG	*	*	*	*	*	*
CMP	*	*	*	*	*	*
DAA	U	*	*	*	*	*
DAS	U	*	*	*	*	*
AAA	U	U	U	*	U	*
AAS	U	U	U	*	U	*
AAM	U	*	*	U	*	U
AAD	U	*	*	U	*	U
SHL/SAL	*	*	*	U	*	*
SHR	*	*	*	U	*	*
SAR	0	*	*	U	*	*
ROL	*	—	—	—	—	*
ROR	*	—	—	—	—	*
RCL	*	—	—	—	—	*
RCR	*	—	—	—	—	*
AND	0	*	*	U	*	0
OR	0	*	*	U	*	0
XOR	0	*	*	U	*	0
NOT	—	—	—	—	—	—
TEST	0	*	*	U	*	0

6. Bölüm Soruları

Alıştırma 6.1. AX yazmacının içeriğini sıfırlamak için 3 ayrı yol gösterin.

Alıştırma 6.2. İşlemcinin giriş/çıkış kapısına bir kara kutu bağlanırsa ve bu kutuya `FFE4` numaralı kapıdan veri gönderildiğinde, kara kutu gönderilen verinin gösterdiği numaralı Fibonacci sayısını sağlıyor ve bu sayı `FFE5` numaralı kapıdan okunabiliyorsa, bu kutuyu kullanarak 100, 60 ve 5 numaralı Fibonacci sayılarının ortalamasını bulan programı yazın.

Alıştırma 6.3. AX = `00FFH` ve BX = `0001H` iken aşağıdaki komut dizisi adım adım işlendikten

sonra AX, CF ve OF bayrak bitlerinin değerlerini bulun.

```
ADD AX, BX   ; 1Adm 1
ADC AX, BX   ; 1Adm 2
```

Alıştırma 6.4. 32 bitlik iki işaretsiz sayıyı `AX:BX` ve `CX:DX` çiftlerinde saklayarak bu iki sayıyı toplayan (sonuç `AX:BX`'de) bir program yazın. ADD ve ADC komutlarını kullanın.

Alıştırma 6.5. BL = `01000100B` (`44H`) iken aşağıdaki komutların her birini ayrı ayrı uyguladıktan sonra BL ve CF bayrak bitinin değerlerini bulun:

(a) `ROL BL, 1`
(b) `ROR BL, 1`
(c) `RCL BL, 1` (CF başlangıçta 0)
(d) `RCR BL, 1` (CF başlangıçta 1)

Alıştırma 6.6. AL yazmacında bir işaretsiz sayı bulunduğunu varsayarak AL'deki sayıyı 10 ile çarpan (MUL kullanmadan, yalnızca SHL ve ADD komutlarıyla) bir kod parçası yazın. [İpucu: $10x = 8x + 2x$]

Alıştırma 6.7. Aşağıdaki mantık ifadesini x86 assembly kodu ile hesaplayın. Sonucu AL yazmacında bırakın.

$$\text{AL} = (\text{BL OR CL}) \text{ AND } (\text{NOT DL})$$

Alıştırma 6.8. AL yazmacında bir ASCII rakam karakteri (`30H–39H`) bulunduğunu varsayarak bu karakteri ikili BCD değerine dönüştüren bir kod parçası yazın.

7

Denetim Komutları

7.1 Denetim Aktarma Komutları

Programcı yazdığı programda bütün komutların art arda işlenmesini istemeyebilir. Programın belirli noktalarında kullanıcıdan alınan girdilerin ya da başka koşulların denetlenerek farklı kod parçalarının çalıştırılması ve program denetiminin programın değişik parçalarına aktarılması gerekebilir. Bu amaçla, işlendiği zaman işlemcinin bir sonraki adımda çalıştıracağı komutun yerini belirleyen "Denetim Aktarma Komutları" kullanılır. Bu komutlar bir koşula bağlı olabileceği gibi hiçbir koşula bağlı olmadan da çalışabilirler.

7.1.1 CALL

CALL komutu denetimin bir yordama aktarılmasını sağlar ve birkaç değişik adresleme çeşidi kullanır. CALL komutunun bölüt içi doğrudan adresleme, bölüt içi dolaylı adresleme, bölütler arası doğrudan adresleme ve bölütler arası dolaylı adresleme yapan türevleri bulunmaktadır. Bölüt içi adreslemenin kullanıldığı durumlarda, komut çalıştırıldığı sırada IP yazmacının değeri yığıta kaydedildikten sonra altyordama atlanır. İşlemci altyordamda bir RET komutu ile karşılaştığında, saklanan değerler yığıttan geri alınır ve program alt yordamın çağrıldığı noktaya geri döner. CALL komutunun bölütler arasında kullanılması durumunda IP yazmacına ek olarak CS yazmacı da yığıta kaydedilir.

```
CALL şilenen1
```

**OFF= Göreceli Adres SEG= Bölüt Adresi DISP= Uzaklık Verisi*

Tablo 7.1: *CALL komutu işlem kodları.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4	Kod 5
Bölüt içi doğrudan	11101000	DISP* (alt)	DISP* (üst)	–	–
Bölüt içi dolaylı	11111111	MOD 010 R/M	–	–	–
Bölütler arası doğrudan	10011010	OFF*(alt)	OFF*(üst)	SEG*(alt)	SEG*(üst)
Bölütler arası dolaylı	11111111	MOD 011 R/M	–	–	–

Örnek

Aşağıda CALL komutunun bölüt içinde doğrudan adreslemesine bir örnek verilmiştir.

```
1 ORG 0100H
2 CALL CAGRI
3 INT 03H
4 CAGRI:
5 MOV AX, 1234H
6 RET
```

Yukarıdaki program DEBUG yazılımı kullanılarak denendiğinde, program çalıştırıldıktan sonra AX yazmacının değerinin 1234H'ye eşit olduğu görülür. Programın ikinci satırı işlendikten sonra program dördüncü satırda başlayan CAGRI altyordamına atlar. Bu alt yordamda AX yazmacının değeri 1234H'ye eşitlendikten sonra altıncı satırdaki RET komutu programı altyordamın çağrıldığı noktaya geri döndürür. Program son olarak 3. satırın işlenmesiyle sonlanır.
DEBUG içinde "D" komutu kullanılarak programın bellekte saklandığı konumdaki verilere bakılırsa CALL komutunun karşılığının E8H olduğu görülür. Bellekte görülen bu kod yukarıdaki tablo yardımıyla çözüldüğünde kullanılan CALL komutunun "Bölüt içi doğrudan" adresleme türünü kullandığı anlaşılır.

Örnek

Aşağıda CALL komutunun bölüt içinde dolaylı adresleme türünü kullanmasına bir örnek verilmiştir.

```
1 ORG 0100H
2 MOV AX, 1234H
3 CALL AX
4 INT 03H
```

Bölüt içinde dolaylı adresleme yapılması istendiğinde yukarıdaki programda olduğu gibi göreceli adres bir yazmaca yazılır ve bu yazmaç işlenen olarak CALL komutuna verilir. Örnekte üçüncü satırdaki CALL komutu denetimi bellekte 1234H göreceli konumunda bulunan altyordama aktaracaktır.

7.1.2 RET/RETF

RET komutu, CALL komutu ile çağırılmış bir altyordamdan geri dönülmesini sağlar. RET komutu çalıştırıldığı anda IP yazmacını yığıttan geri yükler ve programın daha önce CALL komutu tarafından yığıta saklanan konumuna geri dönmesini sağlar. Bölütler arası bir CALL komutunun kullanılması durumunda RET yerine RETF komutu kullanılmalıdır. RETF komutu çağrıldığında RET komutundan farklı olarak hem CS hem IP yazmaçlarının değerleri yığıttan geri yüklenir.

Eğer RET ve RETF komutlarıyla birlikte işlenen olarak anlık bir değer kullanılmış ise bu anlık değer SP yazmacına eklenir. RET ve RETF komutlarıyla birlikte işlenen kullanılması gerekli değildir; bu komutlar yalnız başlarına da kullanılabilirler.

```
1 RET şilenen1
2 RETF şilenen1
```

Tablo 7.2: *RET/RETF komutu işlem kodları.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Bölüt içi	11000011	–	–	–
Bölüt içi ve anlık veri	11000010	Veri (alt)	Veri (üst)	–
Bölütler arası	11001011	–	–	–
Bölütler arası ve anlık veri	11001010	Veri (alt)	Veri (üst)	–

7.1.3 JMP

JMP komutu, programın işlenen1'de belirtilen adrese doğrudan, koşulsuz olarak, atlamasını sağlar.

```
JMP şilenen1
```

**OFF= Göreceli Adres SEG= Bölüt Adresi DISP= Uzaklık Verisi*

Tablo 7.3: *JMP komutu işlem kodları.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4	Kod 5
Bölüt içi doğrudan	11101001	DISP* (alt)	DISP* (üst)	–	–
Bölüt içi dolaylı	11111111	MOD 100 R/M	–	–	–
Bölütler arası doğrudan	11101010	OFF*(alt)	OFF*(üst)	SEG*(alt)	SEG*(üst)
Bölütler arası dolaylı	11111111	MOD 101 R/M	–	–	–
Bölüt içi doğrudan (kısa)	11101011	DISP*	–	–	–

Örnek

Aşağıda JMP komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AX, 0000H
MOV BX, 0000H
JMP AX_DOLDUR
MOV BX, 0FFFFH
AX_DOLDUR:
MOV AX, 0FFFFH
INT 03H
```

Yukarıdaki kod FASM ile çevrilip DEBUG ile denendiğinde BX yazmacının değerinin FFFFH olmadığı görülür. Bunun nedeni JMP komutu ile `MOV BX, 0FFFFH` komutunun üzerinden atlanmasıdır.
JMP komutuna işlenen olarak etiket girilmesi zorunlu değildir. Örneğin aşağıdaki kod doğrudan 1234H göreli adresine atlar.

```
ORG 0100H
JMP 1234H
INT 03H
```

7.1.4 JE/JZ

JE (jump if equal – eşit ise atla) ve JZ (jump if zero – sıfır ise atla) komutları ZF bayrak bitinin değerinin 1 olması durumunda işlenen1'de belirtilen adrese atlar. ZF bitinin değerinin 0 olması durumunda program bir sonraki komutun işlenmesiyle devam eder. JE ya JZ komutları işlenen olarak bir etiket alabileceği gibi doğrudan adres değeriyle kullanılabilir. JE ve JZ komutlarının, adları dışında, işlev açısından hiçbir farkları yoktur; program içinde biri diğerinin yerine kullanılabilir.

```
JE şilenen1
```

Tablo 7.4: *JE/JZ komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JE/JZ	01110100	Uzaklık Verisi	–	–

Örnek

Aşağıda JE komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 03H
MOV BL, 03H
CMP AL, BL
JE ESIT
JMP ESIT_DEGIL
INT 03H
ESIT:
MOV CX, 0FFFFH
INT 03H
ESIT_DEGIL:
MOV CX, 0000H
INT 03H
```

Yukarıdaki örnek programın başında AL ve BL yazmaçlarının değerleri deneme amacıyla 03H değerine eşitlenmiştir. Dördüncü satırdaki CMP komutu AL ve BL yazmaçlarının içerdiği değerleri karşılaştırır. CMP komutunun sonucunun hesaplanması için işlemci çıkarma işlemi yapar ve sonucun sıfır olup olmadığını ZF bayrak bitinde saklar. AL ve BL yazmaçlarının değerinin aynı olması durumunda CMP komutunun iç çıkarma işleminin sonucu sıfır olur ve ZF bayrak biti 1 değerini alır; beşinci satırdaki JE komutu ZF=1 koşulunu görünce programın sekizinci satırdaki ESIT etiketine atlamasını ve CX yazmacının FFFFH değerini almasını sağlar. Yazmaçların değerinin aynı olmaması durumunda program altıncı satırı işler ve JMP komutu koşulsuz olarak ESIT_DEGIL etiketine atlayarak CX yazmacının 0000H değerini almasını sağlar. Örnek programın başında AL ve BL yazmaçlarının değerleri eşitlendiği için programın çalışması bittiğinde CX yazmacının değeri FFFFH olacaktır.

7.1.5 JL/JNGE

JL (jump if less – az ise atla) ve JNGE (jump if not greater and equal – eşit ve büyük değilse atla) komutları SF ve OF bayrak bitlerinin değerlerinin aynı olmaması durumunda işlenen1'de belirtilen adrese atlar. CMP komutu kullanılırken, birinci işlenen ikinci işlenenden küçükse SF ve OF bayrak bitlerinin değeri eşitlenir. Böylece programın CMP komutunun ardından gelecek bir JL komutunun belirttiği adrese atlaması sağlanabilir. İşlenen olarak bir etiket ya da adres alan JL ve JNGE komutlarının işlev açısından hiçbir farkları yoktur.

```
JL şilenen1
JNGE şilenen1
```

Tablo 7.5: *JL/JNGE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JL/JNGE	01111100	Uzaklık Verisi	–	–

Örnek

Aşağıda JL komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 03H
MOV BL, 04H
CMP AL, BL
JL KUCUK
MOV CX, 0000H
INT 03H
KUCUK:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırdaki CMP komutu AL ve BL yazmaçlarının değerlerini karşılaştırarak SF ve OF bayrak bitlerinin değerlerini değiştirir. Beşinci satırdaki JL komutu AL yazmacının değerinin BL yazmacının değerinden daha az olması durumunda programın KUCUK etiketine atlamasını sağlar.

7.1.6 JLE/JNG

JLE (jump if less or equal – az veya eşit ise atla) ve JNG (jump if not greater – büyük değilse atla) komutları SF ve OF bayrak bitleri birbirlerine eşit değilse veya ZF bayrak biti 1 ise atlar. CMP komutu kullanılırken eğer birinci işlenen ikinci işlenenden küçük veya eşitse bu durum oluşmuş olur. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JLE şilenen1
JNG şilenen1
```

Tablo 7.6: *JLE/JNG komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JLE/JNG	01111110	Uzaklık Verisi	–	–

Örnek

Aşağıda JLE komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 03H
MOV BL, 04H
CMP AL, BL
JLE KUCUK_ESIT
MOV CX, 0000H
INT 03H
KUCUK_ESIT:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JLE komutu, programı SF ve OF bayrak bitlerinin değişimi nedeniyle KUCUK_ESIT etiketine atlatır.

7.1.7 JB/JNAE

JB (jump if below – az ise atla) ve JNAE (jump if not above and not equal – büyük ve eşit değilse atla) komutları CF bayrak biti 1 olduğu durumda atlar. CMP komutu kullanılırken eğer birinci işlenen ikinci işlenenden küçükse bu durum oluşmuş olur. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JB şilenen1
JNAE şilenen1
```

Tablo 7.7: *JB/JNAE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JB/JNAE	01110010	Uzaklık Verisi	–	–

Örnek

Aşağıda JB komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 03H
MOV BL, 04H
CMP AL, BL
JB KUCUK
MOV CX, 0000H
INT 03H
```

```
8  KUCUK:
9  MOV CX, 0FFFFH
10 INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JB komutu, programı CF bayrak bitinin 1 olması nedeniyle KUCUK etiketine atlatır.

7.1.8 JBE/JNA

JBE (jump if below or equal – az veya eşit ise atla) ve JNA (jump if not above – büyük değilse atla) komutları CF bayrak biti 1 veya ZF bayrak biti 1 olduğu durumda atlar. CMP komutu kullanılırken eğer birinci işlenen ikinci işlenenden küçük veya eşitse bu durum oluşmuş olur. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
1 JBE şilenen1
2 JNA şilenen1
```

Tablo 7.8: *JBE/JNA komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JBE/JNA	01110110	Uzaklık Verisi	–	–

Örnek

Aşağıda JBE komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AL, 03H
3  MOV BL, 04H
4  CMP AL, BL
5  JBE KUCUK_ESIT
6  MOV CX, 0000H
7  INT 03H
8  KUCUK_ESIT:
9  MOV CX, 0FFFFH
10 INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JBE komutu, programı CF bayrak bitinin 1 olması nedeniyle KUCUK_ESIT etiketine atlatır.

7.1.9 JP/JPE

JP (jump if parity – eşlik 1 ise atla) ve JPE (jump if parity bit even – eşlik çift ise atla) komutları PF bayrak biti 1 olduğu durumda atlar. İşlenen olarak bir etiket kullanılabileceği gibi bir adres de kullanılabilir.

```
1 JP şilenen1
2 JPE şilenen1
```

Tablo 7.9: *JP/JPE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JP/JPE	01111010	Uzaklık Verisi	–	–

Örnek

Aşağıda JP komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 00010100B
MOV BL, 00000000B
CMP AL, BL
JP ESLIK_CIFT
MOV CX, 0000H
INT 03H
ESLIK_CIFT:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JP komutu, programı PF bayrak bitinin 1 olması nedeniyle ESLIK_CIFT etiketine atlatır.

7.1.10 JO

JO (jump if overflow – taşma var ise atla) komutu OF bayrak biti 1 ise atlar. İşlenen olarak bir etiket kullanılabileceği gibi bir adres de kullanılabilir.

```
JO şilenen1
```

Tablo 7.10: *JO komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JO	01110000	Uzaklık Verisi	–	–

Örnek

Aşağıda JO komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, -125
MOV BL, 4
SUB AL, BL
JO TASMA_VAR
MOV CX, 0000H
INT 03H
TASMA_VAR:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnekte $-125 - 4 = -129$ işlemi taşma bitini 1 yapmıştır çünkü sonucun -128 ile 127 arasında olması gerekmektedir.

7.1.11 JS

JS (jump if signed – işaretli ise atla) komutu SF bayrak biti 1 olduğu zaman atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JS şilenen1
```

Tablo 7.11: *JS komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JS	01111000	Uzaklık Verisi	–	–

Örnek

Aşağıda JS komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, -125
MOV BL, 0H
CMP AL, BL
JS NEGATIF
MOV CX, 0000H
INT 03H
NEGATIF:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JS komutu, programı SF bayrak bitinin 1 olması nedeniyle NEGATIF etiketine atlatır.

7.1.12 JNE/JNZ

JNE (jump if not equal – eşit değilse atla) ve JNZ (jump if not zero – sıfır değilse atla) komutları ZF bayrak biti 0 ise atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JNE şilenen1
JNZ şilenen1
```

Tablo 7.12: *JNE/JNZ komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNE/JNZ	01110101	Uzaklık Verisi	–	–

Örnek

Aşağıda JNE komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 03H
MOV BL, 04H
CMP AL, BL
JNE ESIT_DEGIL
MOV CX, 0000H
INT 03H
ESIT_DEGIL:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNE komutu, programı ZF bayrak bitinin 0 olması nedeniyle ESIT_DEGIL etiketine atlatır.

7.1.13 JNL/JGE

JNL (jump if not lower – az değilse atla) ve JGE (jump if greater or equal – eşit veya büyük ise atla) komutları SF ve OF bayrak bitleri birbirine eşit ise atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir. CMP komutunun sonucu olarak birinci işlenen ikinci işlenenden büyük veya eşit ise JNL komutu atlar.

```
JNL şilenen1
JGE şilenen1
```

Tablo 7.13: *JNL/JGE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNL/JGE	01111101	Uzaklık Verisi	–	–

Örnek

Aşağıda JNL komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 02H
MOV BL, 01H
CMP AL, BL
JNL BUYUK_ESIT
MOV CX, 0000H
INT 03H
BUYUK_ESIT:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNL komutu, programı SF ve OF bayrak bitlerinin eşit olması nedeniyle BUYUK_ESIT etiketine atlatır.

7.1.14 JNLE/JG

JNLE (jump if not lower and not equal – eşit ve küçük değilse atla) ve JG (jump if greater – büyük ise atla) komutları SF ve OF bayrak bitleri birbirine eşit ve ZF bayrak biti 0 ise atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir. CMP komutunun sonucu olarak birinci işlenen ikinci işlenenden büyük ise JNLE komutu atlar.

```
JNLE şilenen1
JG şilenen1
```

Tablo 7.14: *JNLE/JG komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNLE/JG	01111111	Uzaklık Verisi	–	–

Örnek

Aşağıda JNLE komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 02H
MOV BL, 01H
CMP AL, BL
JNLE BUYUK
MOV CX, 0000H
INT 03H
BUYUK:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNLE komutu, programı SF ve OF bayrak bitlerinin eşit olması nedeniyle BUYUK etiketine atlatır.

7.1.15 JNB/JAE

JNB (jump if not below – az değilse atla) ve JAE (jump if above or equal – büyük veya eşit ise atla) komutları CF bayrak biti 0 ise atlar. CMP komutunun sonucunda birinci işlenen ikinci işlenenden küçük değilse bu durum oluşur. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JNB şilenen1
JAE şilenen1
```

Tablo 7.15: *JNB/JAE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNB/JAE	01110011	Uzaklık Verisi	–	–

Örnek

Aşağıda JNB komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 02H
MOV BL, 01H
CMP AL, BL
JNB BUYUK_ESIT
MOV CX, 0000H
INT 03H
BUYUK_ESIT:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNB komutu, programı CF bayrak bitinin sıfıra eşit olması nedeniyle BUYUK_ESIT etiketine atlatır.

7.1.16 JNBE/JA

JNBE (jump if not below and not equal – eşit ve az değilse atla) ve JA (jump if above – büyük ise atla) komutları ZF ve CF bayrak bitleri 0 ise atlar. CMP komutunun sonucu olarak birinci işlenen ikinci işlenenden büyükse atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JNBE şilenen1
JA şilenen1
```

Tablo 7.16: *JNBE/JA komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNBE/JA	01110111	Uzaklık Verisi	–	–

Örnek

Aşağıda JNBE komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 02H
MOV BL, 01H
CMP AL, BL
JNBE BUYUK
MOV CX, 0000H
INT 03H
BUYUK:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNBE komutu, programı ZF ve CF bayrak bitinin sıfıra eşit olması nedeniyle BUYUK etiketine atlatır.

7.1.17 JNP/JPO

JNP (jump if not parity – eşlik 0 ise atla) ve JPO (jump if parity odd – eşlik tek ise atla) komutları PF bayrak biti 0 olduğunda atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JNP şilenen1
JPO şilenen1
```

Tablo 7.17: *JNP/JPO komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNP/JPO	01111011	Uzaklık Verisi	–	–

Örnek

Aşağıda JNP komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, 00011100B
MOV BL, 00000000B
CMP AL, BL
JNP ESLIK_TEK
MOV CX, 0000H
INT 03H
ESLIK_TEK:
MOV CX, 0FFFFH
INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNP komutu, programı PF bayrak bitinin sıfıra eşit olması nedeniyle ESLIK_TEK etiketine atlatır.

7.1.18 JNO

JNO (jump if not overflow – taşma yok ise atla) komutu OF 0 ise atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JNO şilenen1
```

Tablo 7.18: *JNO komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNO	01110001	Uzaklık Verisi	–	–

Örnek

Aşağıda JNO komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
```

```
2  MOV AL, -1
3  MOV BL, 4
4  SUB AL, BL
5  JNO TASMA_YOK
6  MOV CX, 0000H
7  INT 03H
8  TASMA_YOK:
9  MOV CX, 0FFFFH
10 INT 03H
```

Yukarıdaki örnek programda dördüncü satırda $-1 - 4 = -5$ işlemi yapılmıştır. Sonuç -128 ile 127 arasında olduğundan 8 bitlik işaretli taşma gerçekleşmez ve OF bayrak biti 0 olur. Beşinci satırdaki JNO komutu, OF=0 koşulunu görünce programı TASMA_YOK etiketine atlatır ve CX yazmacına `0FFFFH` değerini yazar.
Not: -125 - 4 = -129 gibi 8 bitlik işaretli aralığı ($-128...127$) aşan bir işlem yapılsaydı OF=1 olurdu ve JNO atlamadan programın bir sonraki satırını işlerdi.

7.1.19 JNS

JNS (jump if not signed – işaretsiz ise atla) komutu SF bayrak biti 0 ise atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
1  JNS şilenen1
```

Tablo 7.19: *JNS komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JNS	01111001	Uzaklık Verisi	–	–

Örnek

Aşağıda JNS komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
1  ORG 0100H
2  MOV AL, -125
3  MOV BL, 0H
4  CMP AL, BL
5  JNS POZITIF
6  MOV CX, 0000H
7  INT 03H
8  POZITIF:
9  MOV CX, 0FFFFH
10 INT 03H
```

Yukarıdaki örnek programda dördüncü satırda AL ve BL'ye atanan değerler karşılaştırılmıştır. Beşinci satırdaki JNS komutu, programı SF bayrak bitinin sıfıra eşit olması nedeniyle POZITIF etiketine atlatır.

7.1.20 LOOP

LOOP (loop – döngü) komutu CX yazmacının değeri kadar işlenen1'in adresine atlar. CX her atlamada bir azaltılır. CX yazmacının değeri 0 olduğunda atlanmaz ve işlemlere bir sonraki komuttan devam edilir. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
LOOP şilenen1
```

Tablo 7.20: *LOOP komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
LOOP (CX kere)	11100010	Uzaklık Verisi	–	–

Örnek

Aşağıda LOOP komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV CX, 0005H
MOV BX, 0000H
YINELE:
INC BX
LOOP YINELE
INT 03H
```

LOOP komutu ile CX'e atanan beş sayısı kadar YINELE etiketine geri dönülür. CX her geri dönüşte bir azaltılır ve program dördüncü satır ile altıncı satır arasında döngüye girer.

7.1.21 LOOPZ/LOOPE

LOOPZ (loop if zero – sıfır ise atla) ve LOOPE (loop if equal – eşit ise atla) komutları CX yazmacının değeri 0'a eşit olmadığı ve ZF bayrak biti 1 olduğu sürece döngü devam eder. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
LOOPZ şilenen1
LOOPE şilenen1
```

Tablo 7.21: *LOOPZ/LOOPE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
LOOPZ/LOOPE	11100001	Uzaklık Verisi	–	–

Örnek

Aşağıda LOOPZ komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV CX, 0005H
MOV BX, 0000H
```

```
YINELE:
INC BX
LOOPZ YINELE
INT 03H
```

7.1.22 LOOPNZ/LOOPNE

LOOPNZ (loop if not zero – sıfır değilse atla) ve LOOPNE (loop if not equal – eşit değilse atla) komutları CX yazmacının değeri 0'a eşit olmadığı ve ZF bayrak biti 0 olduğu sürece döngü devam eder. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
LOOPNZ şilenen1
LOOPNE şilenen1
```

Tablo 7.22: *LOOPNZ/LOOPNE komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
LOOPNZ/LOOPNE	11100000	Uzaklık Verisi	–	–

Örnek

Aşağıda LOOPNZ komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV CX, 0005H
MOV BX, 0000H
YINELE:
INC BX
LOOPNZ YINELE
INT 03H
```

7.1.23 JCXZ

JCXZ komutu CX yazmacı 0 ise atlar. İşlenen olarak bir etiket girilebileceği gibi bir adres de girilebilir.

```
JCXZ şilenen1
```

Tablo 7.23: *JCXZ komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
JCXZ	11100011	Uzaklık Verisi	–	–

Örnek

Aşağıda JCXZ komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
MOV BX, 0000H
JCXZ ATLA1
SUB CX, 05H
JCXZ ATLA2
ATLA1:
MOV BX, 0FFFFH
INT 03H
ATLA2:
MOV BX, 0000H
INT 03H
```

Bu örnekte ilk durumda CX'e atanan bir değer olmadığı için program akışı devam eder. ATLA! etiketindeki INT 03H kesmesi ile program sonlanır.

7.1.24 INT

INT (interrupt – kesme) komutu işlenen olarak bir bayt alır. Bu bayt INT komutunun tipidir. INT komutu işlendiği zaman bayrak yazmaçları, CS kesim yazmacı ve IP yazmacı yığına kaydedilir. IF bayrak yazmacı sıfırlanır. Kesme yordamına denetim devredilir.

```
INT şilenen1
```

Tablo 7.24: *INT komutu işlem kodları.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
Tip belirtilen	11001101	Tip	–	–
Tip 3	11001100	–	–	–

Örnek

Aşağıda INT komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV BX, 1234H
INT 03H
```

INT komutu programın bulunduğu noktada durdurulup, komutun aldığı işlenene göre hareket etmesi şeklinde işler. Kesme türleri ve işlevleri ile ilgili daha fazla bilgi ilerleyen bölümlerde verilecektir.

7.1.25 INTO

INTO komutu OF bayrak biti 1 ise INT 04H görevi görür.

```
INTO
```

Tablo 7.25: *INTO komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
INTO	11001110	–	–	–

Örnek

Aşağıda INTO komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
MOV AL, -10
SUB AL, 127
INTO
INT 03H
```

Örnekte yapılan çıkarma işlemi sonucunda taşma olacağından OF biti 1 değerini alır. INTO komutu, INT 04H'nin işlevi olan taşma hatası bilgisini verir.

7.1.26 IRET

IRET komutu kesme yordamından geri dönmeyi sağlar. IP, CS ve bayrak bitlerini yığından geri alır.

```
IRET
```

Tablo 7.26: *IRET komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
IRET	11001111	–	–	–

7.1.27 Koşullu Atlama Komutları Özet Tablosu

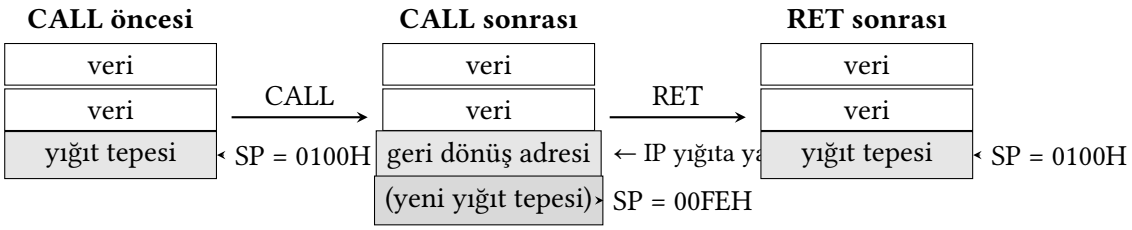
Aşağıdaki tablo, 8086'nın tüm koşullu atlama komutlarını atlar koşulunu, işaret türünü (işaretli/işaretsiz) ve test edilen bayrak bitlerini bir arada göstermektedir.

Tablo 7.27: *Koşullu atlama komutları özeti. İ = İşaretli, İS = İşaretsiz.*

Komut(lar)	Atlama koşulu	Tür	Bayrak testi
JE / JZ	Eşit / Sıfır	—	ZF=1
JNE / JNZ	Eşit değil / Sıfır değil	—	ZF=0
JL / JNGE	Küçük / Büyük-eşit değil	İ	SF≠OF
JLE / JNG	Küçük-eşit / Büyük değil	İ	ZF=1 veya SF≠OF
JG / JNLE	Büyük / Küçük-eşit değil	İ	ZF=0 ve SF=OF
JGE / JNL	Büyük-eşit / Küçük değil	İ	SF=OF
JB / JNAE / JC	Küçük / Büyük-eşit değil	İS	CF=1
JBE / JNA	Küçük-eşit / Büyük değil	İS	CF=1 veya ZF=1
JA / JNBE	Büyük / Küçük-eşit değil	İS	CF=0 ve ZF=0
JAE / JNB / JNC	Büyük-eşit / Küçük değil	İS	CF=0
JO	Taşma var	—	OF=1
JNO	Taşma yok	—	OF=0
JS	İşaret biti 1 (negatif)	—	SF=1
JNS	İşaret biti 0 (pozitif)	—	SF=0
JP / JPE	Eşlik biti 1 (çift eşlik)	—	PF=1
JNP / JPO	Eşlik biti 0 (tek eşlik)	—	PF=0
JCXZ	CX=0	—	CX=0

7.1.28 CALL ve RET Yığıt Diyagramı

CALL komutu çalıştırıldığında işlemci geri dönüş adresini (IP'nin o anki değerini) yığıta yazar ve SP'yi 2 azaltır. RET komutu çalıştırıldığında ise yığıttan geri dönüş adresini alır ve SP'yi 2 artırır. Aşağıdaki şekil bu süreci adım adım göstermektedir.



Şekil 7.1: CALL: IP yığıta yazılır, SP azalır. RET: IP yığıttan okunur, SP artar.

7.2 İşlemci Denetimi Komutları

7.2.1 CLC

CLC (clear CF – CF bayrak bitini sıfırla) komutu CF bayrak bitini 0 yapar.

```
CLC
```

Tablo 7.28: *CLC komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CLC	11111000	–	–	–

7.2.2 CMC

CMC (complement CF – CF bayrak bitini ters çevir) komutu CF bayrak bitini ters çevirir (CF'nin değeri 1 ise değerini 0, değeri 0 ise değerini 1 yapar).

```
CMC
```

Tablo 7.29: *CMC komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CMC	11110101	–	–	–

7.2.3 STC

STC (set CF – CF bayrak bitini 1 yap) komutu CF bayrak bitini 1 yapar.

```
STC
```

Tablo 7.30: *STC komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
STC	11111001	–	–	–

7.2.4 CLD

CLD (clear DF – DF bayrak bitini 0 yap) komutu DF bayrak bitini 0 yapar.

```
CLD
```

Tablo 7.31: *CLD komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CLD	11111100	–	–	–

7.2.5 STD

STD (set DF – DF bayrak bitini 1 yap) komutu DF bayrak bitini 1 yapar.

```
STD
```

Tablo 7.32: *STD komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
STD	11111101	–	–	–

7.2.6 CLI

CLI (clear IF – IF bayrak bitini 0 yap) komutu IF bayrak bitini 0 yapar. Bu bitin sıfırlanmasıyla donanım kesmeleri devre dışı bırakılmış olur.

```
CLI
```

Tablo 7.33: *CLI komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CLI	11111010	–	–	–

7.2.7 STI

STI (set IF – IF bayrak bitini 1 yap) komutu IF bayrak bitini 1 yapar. Donanım kesmeleri devrede olur.

```
STI
```

Tablo 7.34: *STI komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
STI	11111011	–	–	–

7.2.8 HLT

HLT (halt – dur) komutu sistemin durdurulmasını sağlar.

```
HLT
```

Tablo 7.35: *HLT komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
HLT	11110100	–	–	–

7.2.9 WAIT

WAIT (wait – bekle) komutu yardımcı işlemci işlemini bitirene kadar işlemciyi beklemeye alır. Bu şekilde yardımcı işlemci tarafından erişilmekte olan belleğe işlemcinin erişmeye çalışması engellenmiş olur.

```
WAIT
```

Tablo 7.36: *WAIT komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
WAIT	10011011	–	–	–

7.2.10 ESC

ESC (escape – çık) komutu diğer işlemcilere veriyoluna erişme hakkı tanır. İşlemci tarafından bir NOP komutu gibi algılanır ama bellek işlenenini bellek yoluna verir.

```
ESC ıanlk veri, yazmaç
ESC ıanlk veri, bellek
```

* *X = Keyfi değer alabilir*

Tablo 7.37: *ESC komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
ESC	11011XXX*	MOD XXX R/M	–	–

7.2.11 LOCK

LOCK (lock – kitle) komutu, çok işlemcili sistemlerde iki farklı işlemcinin aynı anda aynı bellek bölgesini değiştirmesini engeller. LOCK ön eki yalnızca bir bellek işlenenine sahip komutlarla birlikte geçerlidir; bellek adresleme içeren ADD, SUB, AND, OR, XOR, NOT, NEG, INC, DEC ve XCHG gibi komutların önüne getirilebilir. MOV, IN ve OUT gibi komutlarla birlikte LOCK ön eki kullanılamaz. XCHG komutu ise bellekle kullanıldığında 8086'da her durumda örtük olarak kilitleme sağlar.

```
LOCK
```

Tablo 7.38: *LOCK komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
LOCK	11110000	–	–	–

7.2.12 NOP

NOP (no operation – işlem yok) komutu işlemcinin herhangi bir işlem yapmadan bir komut döngüsü geçirmesini sağlar. İşlemci içinde, yazmaçların durumunda ya da bellekte herhangi bir değişiklik olmaz. NOP komutu, `XCHG AX, AX` komutuyla eşdeğerdir ve tek baytlık `90H` işlem koduna sahiptir. NOP komutu işlemcinin hiçbir işlem yapmadan gecikmesinin gerektiği durumlarda ya da daha sonra doldurulacak yer tutucu olarak kullanılır.

```
NOP
```

Tablo 7.39: *NOP komutu işlem kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
NOP	10010000	–	–	–

NOP'un işlem kodu `90H` = `10010000B`'dir ve tek bayttır. Bu değer `XCHG AX, AX` komutunun işlem koduyla özdeştir; dolayısıyla NOP, AX yazmacını kendisiyle değiştiren anlamsız bir işlem olarak da yorumlanabilir.

7. Bölüm Soruları

Alıştırma 7.1. Aşağıda verilen JAVA kodunu 8086 koduna dönüştürün.

```
a=5;
b=2;
while (a!=0) {
    a--;
    b=a*b;
}
```

Alıştırma 7.2. AX ve BX yazmaçlarında bulunan sayılar üzerinde CX yazmacında bulunan değere göre 00-özel veya (XOR), 01-ve (AND), 02-veya (OR), 03-toplama, 04-çıkarma, 05-çarpma, 06-bölme işlemi yapan bir program yazın. İşlemin sonucu DX yazmacına yazılmalıdır.

Alıştırma 7.3. Yığıt yapısını kullanarak verilen N sayısı kadar Fibonacci serisi öğesini yığıta koyan bir 8086 programı yazın. Örneğin N = 4 için programın sonunda yığıtın içeriği en alttan en üste doğru 0, 1, 1, 2, 3 olmalıdır. N sayısının CX yazmacında hazır olduğunu varsayabilirsiniz.

Alıştırma 7.4. Aşağıdaki Java kodunu 8086 çevirici diline çevirin:

```
int a = 16;
int b = 2;
int c = 0;
while (a < 300) {
    a *= b;
    c++;
}
```

Alıştırma 7.5. Bellekte SI yazmacının gösterdiği adreste 100 adet 2 bayt uzunluğunda sayı bulunmaktadır. Bu dizideki tüm elemanların yerine kendisi dahil kendinden önce gelen tüm elemanların toplamını yazan bir program yazın. Örneğin; eğer dizi 1, 2, 4, 2, 6, ... ise yeni dizi 1, 3, 7, 9, 15, ... olmalıdır. Program tamamlandığında bütün sayıların aritmetik ortalamasını AX yazmacına yazdırın.

Alıştırma 7.6. AX yazmacında verilen bir sayının asal sayı olup olmadığını bulan bir program yazın. Yığıta, sayının asal sayı olması durumunda 1, olmaması durumunda 0 değeri yazılmalıdır. Örneğin, AX yazmacının değeri 07H ise yığıta 1, 08H ise yığıta 0 yazılmalıdır.

Alıştırma 7.7. Çalıştırıldığında DS:SI adresinden başlayarak sonraki 100 baytı kendi içinde sıralayan bir 8086 çevirici dil program yazın. İstediğiniz sıralama algoritmasını kullanabilirsiniz.

Alıştırma 7.8. SI yazmacında bir bellek konumunun adresinin bulunduğunu varsayarak, bu adresten başlayıp bellekten 100 tane 1 baytlık sayı okuyan ve bu sayıların arasında değeri 05H değerine eşit olanların sayısını AX yazmacına yazan bir program yazın.

Alıştırma 7.9. Bellekte SI yazmacı ile gösterilen bir konumda bilinmeyen sayıda 1 baytlık sayı vardır. Bu sayı dizisinin sonunu 0FFH baytı belirtmektedir. Bellekteki bu sayısı belli olmayan sayıların en küçüğünü DL, en büyüğünü DH yazmacına yazan, okunan toplam değer sayısını BX yazmacında tutan ve dizinin sonu geldiğinde bütün sayıların ortalamasını alan bir program yazın.

Alıştırma 7.10. SI yazmacıyla gösterilen adreste sonunda '$' karakteri bulunan bir dizgi vardır. Bu dizgiyi ters çevirip aynı konuma yazdırın.

Alıştırma 7.11. Bellekte 0600H adresinden başlayarak sıralanmış bir miktar bir baytlık sayı olduğunu ve bu sayıların en sonuncusunun ardındaki adrese '$' baytı yazıldığını varsayın. Bu sayılardan kaç tanesinin tek, kaç tanesinin çift olduğunu bulan bir çevirici dil programı yazın. Program tamamlandığında bulduğu iki sonucu tek sayı sayısı önce okunacak biçimde yığıta yazmalıdır.

Alıştırma 7.12. SI yazmacıyla gösterilen bellek konumunda sonu $ karakteriyle işaretlenmiş bir dizgi vardır. Bu dizginin son 5 karakteri içindeki 'a' harfi sayısını bulan bir program yazın. Dizginin uzunluğu 5'ten küçükse program bütün dizgi içindeki 'a' harfi sayısını bulmalıdır.

Alıştırma 7.13. SI yazmacıyla gösterilen bellek konumunda 100 tane bir baytlık sayı vardır. Bu sayılardan ilk 50'si ile son 50'sinin yerlerini değiştiren bir program yazın. Programınızda sayıları başka bir bellek konumuna kopyalamanız yasaktır ancak yazmaçlar kullanılabilir.

Alıştırma 7.14. SI yazmacıyla gösterilen bellek konumunda 100 tane bir baytlık sayı vardır. Bu sayıları bulundukları bellek konumunda başı sonda olacak ve sayı dizisi tam olarak ters sıralanacak biçimde düzenleyin. Programınızda sayıları başka bir bellek konumuna kopyalamanız yasaktır ancak yazmaçlar kullanılabilir. Bu soruda Push ve Pop komutları kullanıldığında komut başına −5 puan alınacaktır.

Alıştırma 7.15. AX yazmacının değerinin 10'dan küçük olması durumunda 7, olmaması durumunda 17 kere dönen bir döngü içeren bir 8086 programı yazın. Döngülerin içine NOP komutu kullanılmalıdır.

Alıştırma 7.16. Bellekte SI yazmacıyla gösterilen bir konumda art arda dizilmiş 100 adet 1 baytlık sayı vardır. Bu sayıların en büyüğünü, en küçüğünü bulan bir program yazın. En küçük sayı AX yazmacının alt yarısında, en büyük sayı AX yazmacının üst yarısında tutulmalıdır.

Alıştırma 7.17. Bellekte SI yazmacıyla gösterilen bir konumda bulunan ve 150 karakterden oluşan bir dizgideki ”A” harfi sayısını bulan bir 8086 programı yazın.

Alıştırma 7.18. Bellekte SI yazmacıyla gösterilen konumda 100 adet sayı bulunmaktadır. Önce bütün bu sayıları yığıta atan daha sonra sayıların tamamını yığıttan okuyup bellekteki ilk sayının bulunduğu konumdan itibaren yazarak sayı dizisini tersine çeviren bir program yazın.

Alıştırma 7.19. Bellekte SI yazmacıyla gösterilen konumda 1 baytlık sayılar bulunmaktadır. Sayı dizisinin sonunu $ karakteri belirlemektedir. BL yazmacında bulunan bir baytlık sayıya en yakın değere sahip olan sayıyı bellekteki sayı dizisinde bulan, sayının değerini AL yazmacında, sayının adresini DX yazmacında saklayan bir program yazın. Örneğin, BL yazmacında 8 değeri varsa ve bellekteki dizi 2, 4, 7, 16, $ ise, program sonuç olarak 7 vermeli DX yazmacında ise 7 sayısının saklandığı adres bulunmalıdır.

Alıştırma 7.20. Bellekte SI yazmacıyla gösterilen bir konumda bir karakter dizisi vardır. Bu karakter dizisinin sonunu $ karakteri belirlemektedir. Bellekteki bu karakter dizisinde hangi karakterden kaç tane olduğunu bulan bir program yazın. Karakterler bellekte DS:1500 adresinden başlayarak bulunacak, ilgili karakterden dizgide kaç tane olduğu da DS:2500 adresinde tutulacaktır. Örneğin dizgide 25 tane ‘A’, 12 tane ‘B’ varsa DS:1500 adresinde ‘A’, DS:1501 adresinde ‘B’, DS:2500 adresinde 25 ve DS:2501 adresinde 12 değeri olmalıdır.

8

Dizgi İşleme Komutları

Dizgi işlemleri ya bir bayt ya da bir kelime üzerinde yapılır. Dizgi işlemlerinde dizgiler DI ve SI yazmaçları ile adreslenirler. DF bayrak biti 0 ise DI ve SI yazmaçları bayt için 1 ve kelime için 2 artırılır. DF bayrak biti 1 ise DI ve SI yazmaçları bayt için 1, kelime için 2 azaltılır.

Dizgi komutlarına bakılacak olursa her dizgi komutunun birkaç şeklinin olduğu görülecektir. Kısa şekillerinde işlenen belirtilmez ve otomatik olarak SI ve DI adresleri işlenen olarak kullanılır. Eğer bir kelime işlenen olarak kullanılacaksa SI yazmacı ile birlikte DS kesim yazmacı ve DI ile birlikte ES kesim yazmacı kullanılır.

8086 dizgi komutlarının listesi aşağıdaki gibidir:

- REP/REPE/REPNE/REPNZ/REPZ
- MOVSB/MOVSW
- CMPSB/CMPSW
- SCASB/SCASW
- LODSB/LODSW
- STOSB/STOSW

Aşağıdaki Tablo 8.1 yukarıdaki komutları özetlemektedir.

Tablo 8.1: *Dizgi komutları özeti.*

Komut	Kaynak	Hedef	Ön Ek
MOVSB	DS:SI	ES:DI	REP
MOVSW	DS:SI	ES:DI	REP
CMPSB	DS:SI	ES:DI	REPE/REPNE
CMPSW	DS:SI	ES:DI	REPE/REPNE
SCASB	AL	ES:DI	REPE/REPNE
SCASW	AX	ES:DI	REPE/REPNE
LODSB	DS:SI	AL	—
LODSW	DS:SI	AX	—
STOSB	AL	ES:DI	REP
STOSW	AX	ES:DI	REP

8.1 DF Bayrak Biti

Yukarıda DF bayrak bitinin DI ve SI yazmaçlarının artırılışını ve azaltılışını denetlediğinden bahsedilmişti. DF bayrak bitini değiştirmek için 2 tane assembly komutu bulunmaktadır.

CLD komutu DF bayrak bitini 0'a eşitler ve dizgi komutlarının SI ve DI yazmaçlarının değerlerini artırmasına neden olur.

STD komutu DF bayrak bitini 1'e eşitler ve dizgi komutlarının SI ve DI yazmaçlarının değerlerini azaltmasına neden olur.

8.2 REP/REPE/REPNE/REPNZ/REPZ

REP (repeat – yinele) komutu işlenen olarak verilen MOVSB, MOVSW, LODSB, LODSW, STOSB, STOSW komutlarından birinin CX yazmacındaki değer kadar tekrar edilmesini sağlar. Her yinelemede CX yazmacının değeri 01H kadar azaltılır. CX yazmacı 0 olduğunda sıradaki komuta atlar.

REPE (repeat if equal – eşit ise yinele) komutu işlenen olarak verilen CMPSB, CMPSW, SCASB, SCASW komutlarından birini ZF bayrak biti 1 olduğu ve CX yazmacındaki değer sıfırdan büyük olduğu sürece yineler. ZF bayrak biti 0 olduğu anda yineleme durdurulur ve bir sonraki komuta atlanır. Her yinelemede CX yazmacının değeri 01H kadar azaltılır.

REPZ (repeat if zero – sıfır ise yinele) komutu işlenen olarak verilen CMPSB, CMPSW, SCASB, SCASW komutlarından birini CX yazmacındaki değer 0 olana kadar, ZF bayrak biti 1 olduğu sürece yineler. ZF bayrak biti 0 olduğu anda yineleme durdurulur ve bir sonraki komuta atlanır. Her yinelemede CX yazmacının değeri 01H kadar azaltılır.

REPNE (repeat if not equal – eşit değilse yinele) komutu işlenen olarak verilen CMPSB, CMPSW, SCASB, SCASW komutlarından birini CX yazmacındaki değer 0 olana kadar, ZF bayrak biti 0 olduğu sürece yineler. ZF bayrak biti 1 olduğu anda yineleme durdurulur ve bir sonraki komuta atlanır. Her yinelemede CX yazmacının değeri 01H kadar azaltılır.

REPNZ (repeat if not zero – sıfır değilse yinele) komutu işlenen olarak verilen CMPSB, CMPSW, SCASB, SCASW komutlarından birini CX yazmacındaki değer 0 olana kadar, ZF bayrak biti 0 olduğu sürece yineler. ZF bayrak biti 1 olduğu anda yineleme durdurulur ve bir sonraki komuta atlanır. Her yinelemede CX yazmacının değeri 01H kadar azaltılır.

```
REP   islenen1
REPE  islenen1
REPZ  islenen1
REPNE islenen1
REPNZ islenen1
```

Tablo 8.2: *REP ön ekinin makine kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
REP	1111001Z	—	—	—

8.3 MOVSB/MOVSW

MOVSB komutu veri bölütünde bulunan DS:[SI] adresindeki bir baytı ek bölütte bulunan ES:[DI] adresine kaydeder. **MOVSW** komutu aynı işlemi yapar ancak bir bayt yerine iki baytlık bir sözcük aktarır. Okunan dizginin yönü DF bayrak bitinde 0 olarak gösterilmiş ise SI ve DI işlem sonunda 1 artırılır. DF bayrak bitinin 1 olması durumunda SI ve DI 1 azaltılır.

```
MOVSB
MOVSW
```

Tablo 8.3: *MOVS komutunun makine kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
MOVS	1010010W	—	—	—

Örnek

MOVSB ve REP ile bayt aktarımı Aşağıda MOVSB komutunun program içinde REP komutuyla birlikte nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
CLD              ;DF = 0
MOV SI, DEG1    ;SI'ya DEG1'in adresini kaydet
MOV DI, DEG2    ;DI'ya DEG2'nin adresini kaydet
MOV CX, 04H     ;CX = 4
REP MOVSB        ;MOVSB islemini 4 kez tekrarla
INT 03H
DEG1 DB 1, 2, 3, 4
DEG2 DB 5, 6, 7, 8
```

Programın başında DF bayrak biti sıfırlanmış, SI ve DI yazmaçlarına sırasıyla DEG1 ve DEG2'nin başlangıç adresleri atanmıştır. Bu işlemlerin ardından REP komutunun yardımıyla DEG1'in gösterdiği bellek konumundan DEG2'nin gösterdiği bellek konumuna 4 bayt art arda aktarılır. İşlemin sonucunda DEG2'nin bellekte bulunduğu bölgedeki bayt değerleri sırasıyla 1, 2, 3, 4 olur.

Örnek

MOVSB ve REP ile dizgi aktarımı MOVSB ile REP ön ekini kullanarak bir yere kaydettiğimiz "MERHABA DUNYA!" dizgisini her seferde bir bayt taşıyarak 0170H göreli adresine taşıyalım.

```
ORG 0100H
JMP BASLANGIC
VERI DB 'MERHABA DUNYA!'
BASLANGIC:
MOV CX, 14
MOV AX, CS
MOV DS, AX
MOV ES, AX
CLD
```

```
10 MOV SI, VERI
11 MOV DI, 0170H
12 REP MOVSB
13 INT 03H
```

3. satırda “MERHABA DUNYA!” dizgimizi tanımladık ve erişimi kolaylaştırmak için değişken adı olarak VERI seçtik.
5. satırda CX yazmacına “MERHABA DUNYA!” dizgisindeki harf sayısını girdik.
6., 7. ve 8. satırlarda DS ve ES kesim yazmaçlarının değerlerini CS yazmacına eşit hale getirdik.
9. satırda CLD komutunu kullanarak DF bayrak bitini 0’a eşitledik. Bu sayede her MOVSB komutunda DI ve SI yazmaçlarının değerleri 1 artırılacak.
10. satırda SI yazmacına “MERHABA DUNYA!” dizgisinin adresini yükledik. İşlenen olarak değişken adının yazılması “MERHABA DUNYA!” dizgisinin göreli konum adresinin yüklenmesine neden oldu.
11. satırda DI yazmacına 0170H değerini yükledik. DI yazmacı MOVSB komutunda hedef göreli adresi olarak kullanıldığından hedef olarak 0170H adresini seçtik.
12. satırda ise “MERHABA DUNYA!” dizgisinin taşınmasını sağladık. 4. satırda CX yazmacına yüklediğimiz 14 değeri sayesinde MOVSB komutu 14 kez tekrarlandı ve doğru bir şekilde 0170H adresine “MERHABA DUNYA!” dizgisi taşındı.
Yukarıdaki kodu DEBUG ile deneyip 0170H göreli konum adresindeki verilere bakarsak “MERHABA DUNYA!” dizgisinin tamamen taşınmış olduğunu göreceğiz.

Örnek

MOVSW ve REP ile dizgi aktarımı MOVSW ile REP ön ekini kullanarak bir yere kaydettiğimiz “MERHABA DUNYA!” dizgisini her seferde bir kelime (2 bayt) taşıyarak 0170H göreli adresine taşıyalım.

```
1  ORG 0100H
2  JMP BASLANGIC
3  VERI DB 'MERHABA DUNYA!'
4  BASLANGIC:
5  MOV CX, 7          ; 14 bayt / 2 bayt/kelime = 7 yineleme
6  MOV AX, CS
7  MOV DS, AX
8  MOV ES, AX
9  CLD
10 MOV SI, VERI
11 MOV DI, 0170H
12 REP MOVSW
13 INT 03H
```

MOVSW her adımda 2 bayt taşıdığından, 14 baytlık “MERHABA DUNYA!” dizgisini aktarmak için CX’e 14/2 = 7 değeri yüklenmiştir. Yukarıdaki kodu DEBUG ile deneyip 0170H göreli konum adresindeki verilere bakarsak “MERHABA DUNYA!” dizgisinin tamamen taşınmış olduğunu göreceğiz.

8.4 CMPSB/CMPSW

CMPSB komutu ES:[DI] ve DS:[SI] adreslerinde bulunan iki baytı karşılaştırır. **CMPSW** komutu ise aynı bellek konumlarındaki iki baytlık iki sözcüğü birbiriyle karşılaştırır. OF, SF, ZF, AF, PF ve CF bayrak bitleri bu iki karşılaştırma işleminin sonucunda değişir. DF bayrak bitinin değeri 0 ise SI ve DI 1 artırılır, 1 ise SI ve DI 1 azaltılır.

```
CMPSB
CMPSW
```

Tablo 8.4: *CMPS komutunun makine kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
CMPS	1010011W	—	—	—

Örnek

CMPSB ve REPE ile bayt karşılaştırması Aşağıda CMPSB komutunun program içinde REPE komutuyla birlikte nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
CLD                  ;DF = 0
MOV SI, DEG1        ;SI'ya DEG1'in adresini kaydet
MOV DI, DEG2        ;DI'ya DEG2'nin adresini kaydet
MOV CX, 04H         ;CX = 4
REPE CMPSB           ;Baytlari karsilastir
JNZ ESIT_DEGIL      ;ZF = 0 ise ESIT_DEGIL'e atla
MOV AX, 1234H
INT 03H
ESIT_DEGIL:
MOV AX, 0000H
INT 03H
DEG1 DB 1, 2, 3, 4
DEG2 DB 1, 2, 3, 4
```

Programın başında DF bayrak biti sıfırlanmış, SI ve DI yazmaçlarına sırasıyla DEG1 ve DEG2'nin bellekteki başlangıç adresleri atanmıştır. Bu işlemlerin ardından REPE komutunun yardımıyla DEG1'in gösterdiği bellek konumundaki 4 bayt DEG2'nin gösterdiği bellek konumundaki 4 bayt ile art arda karşılaştırılır. REPE komutu birbirinden farklı iki baytı karşılaştırdığında döngü durur ve yedinci satırdaki JNZ komutu çalıştırılır. İki farklı bayt birbiriyle karşılaştırıldığında ZF'nin değeri 0 olacağından program onuncu satırdaki ESIT_DEGIL satırına atlar ve AX yazmacının içeriğini sıfırlar. Eğer REPE komutu birbirinden farklı iki bayt bulamazsa dört kere döner ve ZF bitinin değeri 1 olur. Bu durumda AX yazmacının değeri 1234H olarak belirlenir.

Örnek

CMPSB ve REPE ile dizgi karşılaştırması CMPSB komutunu ve REPE ön ekini kullanarak "MERHABA DUNYA!" ve "MERABA DUNYA!" dizgilerini karşılaştıralım. Birbirlerine eşit iseler BX yazmacına 1234H değeri, değilseler 4321H değeri yazdıralım.

```
1  ORG 0100H
2  JMP BASLANGIC
3  VERI1 DB 'MERHABA DUNYA!'
4  VERI2 DB 'MERABA DUNYA!'
5  BASLANGIC:
6  MOV CX, 14
7  MOV AX, CS
8  MOV DS, AX
9  MOV ES, AX
10 CLD
11 MOV SI, VERI1
12 MOV DI, VERI2
13 REPE CMPSB
14 JE AYNI
15 JMP FARKLI
16 AYNI:
17 MOV BX, 1234H
18 INT 03H
19 FARKLI:
20 MOV BX, 4321H
21 INT 03H
```

3. satırda VERI1 değişkenini tanımladık ve 4. satırda VERI2 değişkenini tanımladık.
11. satırda SI yazmacına VERI1 değişkeninin adresini yükledik ve 12. satırda DI yazmacına VERI2 değişkeninin adresini yükledik.
13. satırda REPE CMPSB işleminin sonucuna göre AYNI etiketine atlanıp atlanılmayacağı kararlaştırılmaktadır.
15. satırda ise JMP komutu AYNI etiketine 13. satırda atlanmamışsa FARKLI etiketine atlanmasını sağlamaktadır.
Yukarıdaki kodu denediğimizde BX yazmacında 4321H değerinin olduğunu göreceğiz. Eğer VERI2'yi "MERHABA DUNYA!" şekline getirip bir daha denersek BX yazmacında 1234H değerini göreceğiz.
REPNE CMPSB, REPE CMPSW ve REPNE CMPSW için benzer programlar yazılabilir.

8.5 SCASB/SCASW

SCASB komutu AL yazmacındaki değer ile ES:[DI] adresindeki baytı karşılaştırır. Karşılaştırma sonucunda OF, SF, ZF, AF, PF ve CF bayrak bitlerinin değerleri değişir. Komut çalıştırıldığı sırada DF bayrak biti 0 ise komutun işlenmesinin ardından DI yazmacının değeri bir artırılır; DF bayrak bitinin değerinin 1 olması durumunda ise DI yazmacının değeri bir azaltılır. Benzer biçimde **SCASW** komutu AX yazmacındaki 16 bitlik sözcük ile ES:[DI] adresinde bulunan sözcüğü karşılaştırır. SCASW komutu çalıştırıldığında DF bayrak biti 0 ise DI yazmacının değeri 2 artırılır; DF bayrak bitinin değerinin 1 olması durumunda ise DI yazmacının değeri 2 azaltılır.

```
1 SCASB
2 SCASW
```

Tablo 8.5: *SCAS komutunun makine kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
SCAS	1010111W	—	—	—

Örnek

SCASB ve REPE ile bayt tarama Aşağıda SCASB komutunun program içinde REPE komutuyla birlikte nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
CLD                  ;DF = 0
MOV DI, DEG1        ;DI'ya DEG1'in adresini kaydet
MOV CX, 04H         ;CX = 4
MOV AL, 02H
REPE SCASB           ;Baytlari karsilastir
JNZ ESIT_DEGIL      ;ZF = 0 ise ESIT_DEGIL'e atla
MOV AX, 1234H
INT 03H
ESIT_DEGIL:
MOV AX, 0000H
INT 03H
DEG1 DB 02H, 02H, 02H, 02H
```

Yukarıdaki programın başında DF bayrak biti CLD komutu kullanılarak sıfırlanmış, DI yazmacına DEG1'in bellekteki başlangıç adresi atanmıştır. Beşinci satırda AL yazmacının değeri 02H olarak belirlenmiştir. Bu işlemlerin ardından REPE komutunun yardımıyla AL yazmacının değeri DEG1'in gösterdiği bellek konumundaki 4 bayt ile art arda karşılaştırılır. REPE komutu birbirinden farklı iki baytı karşılaştırdığında döngü durur ve yedinci satırdaki JNZ komutu çalıştırılır. İki farklı bayt birbiriyle karşılaştırıldığında ZF'nin değeri 0 olacağından program onuncu satırdaki ESIT_DEGIL satırına atlar ve AX yazmacının içeriğini sıfırlar. Eğer REPE komutu, örnekteki program olduğu gibi, birbirinden farklı iki bayt bulamazsa dört kere döner ve ZF bitinin değeri 1 olur. Bu durumda AX yazmacının değeri 1234H olarak belirlenir. Bu örnek programda AL yazmacının değeri olan 02H ile DEG1 değişkeninin gösterdiği bellek konumunda art arda gelen 4 baytın değeri aynı olduğundan programın sonucunda AX yazmacının değeri 1234H olarak belirlenecektir.

Örnek

SCASB ve REPNE ile dizgide harf değiştirme SCASB komutu ile REPNE ön ekini kullanarak "MERHABA DUNYA!" dizgisindeki "D" harfini "S" harfi ile değiştirelim.

```
ORG 0100H
JMP BASLANGIC
VERI DB 'MERHABA DUNYA!'
BASLANGIC:
MOV CX, 14
MOV AX, CS
MOV ES, AX
CLD
```

```
9  MOV DI, VERI
10 MOV AL, 'D'
11 REPNE SCASB
12 JNE SON
13 DEC DI
14 MOV BYTE [DI], 'S'
15 SON:
16 INT 03H
```

11. satırda AL yazmacındaki değer ES:DI tarafından işaret edilen bellek bölgesindeki değer ile karşılaştırılıyor ve eşit olmadığı sürece DI yazmacının değeri 1 artırılarak karşılaştırmalara devam ediliyor.
12. satırda eğer eşit olan bir değer bulunamaz ise SON etiketine atlanıyor.
13. satırda eğer eşit bir harf bulunmuş ise DI yazmacının değeri 1 azaltılıyor çünkü her SCASB komutu işlemini yaptıktan sonra DI yazmacını 1 artırıyor. Doğru harfe işaret etmek için DI yazmacının değerini 1 azaltmamız gerekiyor.
14. satırda DI göreli konum adresindeki bayt değeri "S" harfinin değeri ile değiştiriliyor.
REPE SCASB, REPNE SCASW ve REPE SCASW için benzer programlar yazılabilir.

8.6 LODSB/LODSW

LODSB komutu DS:[SI] adresindeki baytı AL yazmacına aktarır. Komut çalıştırıldığında DF bayrak bitinin değeri 0 ise SI yazmacının değeri 1 artırılır; DF bayrak bitinin değerinin 1 olması durumunda ise SI yazmacının değeri 1 azaltılır. **LODSW** komutu, LODSB komutuna benzer biçimde, DS:[SI] adresindeki 16 bitlik bir sözcüğü AX yazmacına aktarır. LODSW komutu çalıştırıldığında DF bayrak bitinin değeri 0 ise SI yazmacının değeri 2 artırılır; DF bayrak bitinin değerinin 1 olması durumunda ise SI yazmacının değeri 2 azaltılır.

```
LODSB
LODSW
```

Tablo 8.6: *LODS komutunun makine kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
LODS	1010110W	—	—	—

Örnek

LODSB ile bellekten ardışık bayt yükleme Aşağıda LODSB komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
CLD                  ;DF = 0
MOV SI, DEG1        ;SI'ya DEG1'in adresini kaydet
MOV CX, 3           ;CX = 3
REP LODSB            ;AL'ye 3 kere bayt yukle
INT 03H
DEG1 DB 01H, 02H, 03H, 04H, 05H
```

Programın başında SI yazmacına DEG1'in bellekteki başlangıç adresi atanmıştır. Bu bellek adresinde sırasıyla 01H, 02H, 03H, 04H ve 05H değerleri bulunmaktadır. CX yazmacının değeri 3 olarak belirlendiği için beşinci satırdaki REP komutu LODSB komutunu 3 kere çalıştıracaktır. DF bayrak biti ikinci satırdaki CLD komutu ile sıfırlandığı için LODSB komutu her çalıştığında SI yazmacının değeri bir artacaktır. Böylece REP komutunun 3 kere çalışması sırasında AL yazmacına sırasıyla 01H, 02H ve 03H değerleri aktarılacaktır.
Uyarı: REP ile LODSB/LODSW birlikte kullanılması genellikle önerilmez. Her yinelemede AL (ya da AX) üzerine yazıldığından döngünün sonunda yalnızca son okunan değer kalır; önceki değerler kaybolur. LODSB'yi döngü içinde STOSB veya başka bir komutla birleştirmek gerçek veri işleme için çok daha kullanışlıdır.

Örnek

LODSB ile dizgi kopyalama Şimdi yazacağımız örnekte LODSB komutunu kullanarak bir adreste bulunan bir dizgiyi başka bir adrese taşıyacağız.

```
1  ORG 0100H
2  JMP BASLANGIC
3  YAZI DB 'MERHABA DUNYA!'
4  BASLANGIC:
5  MOV AX, CS
6  MOV DS, AX
7  MOV ES, AX
8  CLD
9  MOV CX, 14
10 MOV SI, YAZI
11 MOV DI, 0200H
12 OKU:
13 LODSB
14 MOV [DI], AL
15 INC DI
16 LOOP OKU
17 INT 03H
```

5. satırda CS kesim yazmacının değerini AX yazmacına aktardık. 6. ve 7. satırlarda ise bu değeri DS ve ES kesim yazmaçlarına yazdık.
8. satırda DF bayrak bitini 0'a eşitledik.
9. satırda "MERHABA DUNYA!" dizgimizin uzunluğu olan 14 sayısını CX yazmacına yazdırdık. Bu değeri yazmamızın nedeni LOOP OKU'nun 14 kez okuma yapmasını sağlamaktır.
10. satırda "MERHABA DUNYA!" dizgimizin adresini SI yazmacına yazdırıyoruz.
11. satırda DI yazmacına 0200H değerini yazdırıyoruz. "MERHABA DUNYA!" dizgimizi 0200H adresine kopyalamak istediğimizden bu değeri yükledik.
13. satırda LODSB komutu ile SI yazmacının adresindeki veri AL yazmacına aktarılıyor.
14. satırda DI yazmacının adresine AL yazmacının değeri aktarılıyor ve 15. satırda DI yazmacının değeri bir sonraki bayta geçmek için 1 artırılıyor.
16. satırdaki LOOP komutu ise 14 kez 13., 14. ve 15. satırdaki işlemlerin tekrarlanmasını sağlıyor.

8.7 STOSB/STOSW

STOSB komutu AL yazmacındaki baytı ES:[DI] adresine yazar. Komut çalıştırıldığı sırada DF bayrak bitinin değeri 0 ise DI yazmacının değeri 1 artar; DF bayrak bitinin değerinin 1 olması durumunda ise DI yazmacının değeri 1 azaltılır. Benzer biçimde **STOSW** komutu AX yazmacındaki 16 bitlik sözcüğü ES:[DI] adresine yazar. STOSW komutu çalıştığı sırada DF bayrak bitinin değeri 0 ise DI yazmacının değeri 2 artırılır; DF bayrak bitinin değerinin 1 olması durumunda ise DI yazmacının değeri 2 azaltılır.

```
STOSB
STOSW
```

Tablo 8.7: *STOS komutunun makine kodu.*

Tanım	Kod 1	Kod 2	Kod 3	Kod 4
STOS	1010101W	—	—	—

Örnek

STOSB ve REP ile bellek bölgesini doldurma Aşağıda STOSB komutunun program içinde nasıl kullanıldığı gösterilmiştir.

```
ORG 0100H
CLD                  ;DF = 0
MOV DI, DEG1        ;DI'ya DEG1'in adresini kaydet
MOV AL, 0FFH        ;AL = FF
MOV CX, 4           ;CX = 4
REP STOSB
INT 03H
DEG1 DB 01H, 02H, 03H, 11H
```

Programın başında DI yazmacına DEG1'in bellekteki başlangıç adresi atanmıştır. Program çalışmadan önce bu bellek adresinde sırasıyla 01H, 02H, 03H ve 11H değerleri bulunmaktadır. CX yazmacının değeri 4 olarak belirlendiği için altıncı satırdaki REP komutu STOSB komutunu 4 kere çalıştıracaktır. DF bayrak biti ikinci satırdaki CLD komutu ile sıfırlandığı için STOSB komutu her çalıştığında DI yazmacının değeri bir artacaktır. Böylece REP komutunun 4 kere çalışması sırasında AL yazmacındaki FFH verisi DI yazmacının bellekte gösterdiği art arda sıralanmış 4 baytın üzerine yazılacaktır. Böylece programın çalışmasından önce bellekte 01H, 02H, 03H ve 11H değerlerinin bulunduğu baytların değerlerinin tamamı FFH olarak değişecektir.

Örnek

STOSB ve REP ile dizgi kopyalama STOSB komutu ile birlikte REP ön ekini kullanarak bir adreste bulunan "MERHABA DUNYA!" dizgisini başka bir adrese taşıyacağız.

```
ORG 0100H
JMP BASLANGIC
YAZI DB 'MERHABA DUNYA!'
BASLANGIC:
```

```
5  MOV AX, CS
6  MOV DS, AX
7  MOV ES, AX
8  CLD
9  MOV SI, YAZI
10 MOV DI, 0200H
11 MOV CX, 14
12 OKU:
13 PUSH CX
14 MOV AL, [SI]
15 MOV CX, 1
16 REP STOSB
17 INC SI
18 POP CX
19 LOOP OKU
20 INT 03H
```

11. satırda CX yazmacına 14 değerini aktardık. Bu değeri LOOP OKU döngümüzde kullanacağız.
13. satırda bu yüklediğimiz değeri yığına kaydettik.
14. satırda SI yazmacının değerinin adresindeki veriyi AL yazmacına kaydettik.
15. satırda REP STOSB komutunun ne kadar kez tekrarlanacağını belirtmek amacıyla CX yazmacına 1 değerini yükledik.
16. satırda REP STOSB komutunu kullandık. Bu sayede DI yazmacının değerinin adresine AL yazmacındaki veri kaydedildi. DI yazmacının değeri 7. satırda DF bayrak biti 0'a eşitlendiğinden 1 artırıldı.
17. satırda "MERHABA DUNYA!" dizgisinin bir sonraki harfini işaret etmek için SI yazmacının değerini 1 artırdık.
18. satırda LOOP OKU döngümüzde kullandığımız CX yazmacının değerini yığından geri CX yazmacına yükledik.

8.8 BCD-ASCII İşlemleri

x86 işlemcileri assembly komutlarının çoğu hem BCD hem de ASCII sayılarını işlenen olarak kabul edebilir. ASCII sistemi Amerika Birleşik Devletleri ve birçok ülkede standart olarak kabul edilmesine rağmen BCD sayıları tüm dünyada genel geçerdir. Ayrıca klavyeler, yazıcılar ve monitörler ASCII ile çalışmaktadır ve BCD sayıların gösteriminin yapılması gerekmektedir. Bunun için x86 işlemcileri bu iki sayı sistemi arasında geçiş yapabilmektedir. Komutların BCD ve ASCII sayıları ile kullanımlarını örneklendirmeden önce BCD sayı sistemi ve ASCII sayı sistemini inceleyelim.

8.8.1 BCD Sayı Sistemi

BCD (binary coded decimal – ikili kodlanmış onluk sayı) sayı sisteminin oluşturulma nedeni insanların onluk tabanda bir sayı gösterirken bir haneye 0 ile 9 arasında sayıları yazmasıdır. Tek veya çift haneli bir onluk sayının gösterimi BCD sayı sistemi ile diğer sistemlere göre çok daha kolaydır. BCD sayı sistemi 2 çeşittir:

- Paketlenmemiş BCD (Unpacked BCD) Sayı Sistemi
- Paketlenmiş BCD (Packed BCD) Sayı Sistemi

Paketlenmemiş BCD Sayı Sistemi

Paketlenmemiş BCD sayı sisteminde 0 ile 9 arası sayılar bir baytın alt 4 bitine kodlanır. Üst 4 bit 0'a eşitlenir. Bu düzende her BCD sayısı 1 baytlık yer kaplar. Aşağıdaki Tablo 8.8 paketlenmemiş BCD sayı sisteminde 0'dan 9'a kadar olan sayıları göstermektedir.

Tablo 8.8: *Paketlenmemiş BCD sayı sistemi (0–9).*

Ondalık	Paketlenmemiş BCD
0	0000 0000
1	0000 0001
2	0000 0010
3	0000 0011
4	0000 0100
5	0000 0101
6	0000 0110
7	0000 0111
8	0000 1000
9	0000 1001

Paketlenmiş BCD Sayı Sistemi

Paketlenmemiş BCD sayı sisteminde üst 4 bitin 0'a eşit olduğunu söylemiştik. Paketlenmiş BCD sayı sisteminde bu üst 4 bit bir üst basamağın BCD değerini kaydetmek için kullanılır. 8 bitin alt 4 biti sayının ilk hanesi için, üst 4 bit ise sayının ikinci hanesi için kullanılmaktadır. Bu düzende 2 basamak sadece 1 baytlık yer kaplamaktadır. Bu yüzden paketlenmiş BCD sayıları paketlenmemiş BCD sayılarına tercih edilmektedir. Aşağıdaki Tablo 8.9 bazı ondalık sayılar için paketlenmiş BCD sayı değerlerini göstermektedir.

Tablo 8.9: *Paketlenmiş BCD sayı sistemi.*

Ondalık	Paketlenmiş BCD
10	0001 0000
21	0010 0001
32	0011 0010
43	0100 0011
54	0101 0100
65	0110 0101
76	0111 0110
87	1000 0111
98	1001 1000
19	0001 1001

8.8.2 ASCII Sayılar

ASCII sayılarını örneklendirmek için 0'dan 9'a kadar olan sayıların ASCII değerlerini gösterelim. Aşağıdaki Tablo 8.10 bu değerleri listelemektedir.

Tablo 8.10: *ASCII rakam kodları (0–9).*

Ondalık	ASCII (İkili)	ASCII (Onaltılık)
0	011 0000	30
1	011 0001	31
2	011 0010	32
3	011 0011	33
4	011 0100	34
5	011 0101	35
6	011 0110	36
7	011 0111	37
8	011 1000	38
9	011 1001	39

Dikkat edilirse yukarıdaki tabloda ASCII sayıları gösterirken ikili gösterimde 7 bit kullandık. ASCII kodlama sisteminde anlamlı olan bitler alt 7 bittir. 8. bit ise 0 veya 1 olabilir. Bazı uygulamalarda 8. bit eşlik biti olarak kullanılmaktadır.

8.8.3 Paketlenmemiş BCD – ASCII Çevirimi

Örnek olarak 0000 0101 paketlenmemiş BCD sayısını (ondalık 5 sayısını) ASCII sayı sistemine çevirelim. 5 sayısının ASCII gösterimi 011 0101'dir.

Yapmamız gereken paketlenmemiş BCD sayısının alt 4 bitini alıp üst 4 biti de 0011 olarak tamamlamaktır. Aşağıda yukarıdaki işlemi yapan program verilmiştir:

```
1  ORG 0100H
2  JMP BASLA
3  BCD_SAYI   DB 00000101B
4  ASCII_SAYI DB 00H
5  BASLA:
6  MOV SI, BCD_SAYI
7  MOV AL, [SI]
8  MOV SI, ASCII_SAYI
9  AND AL, 00001111B
10 XOR AL, 00110000B
11 MOV [SI], AL
12 INT 03H
```

6. satırda SI yazmacına BCD_SAYI değişkeninin adresi kaydediliyor.
7. satırda SI yazmacının değerinin işaretlediği adresteki veriler AL yazmacına aktarılıyor.
8. satırda SI yazmacına ASCII_SAYI değişkeninin adresi kaydediliyor.
9. satırda AL yazmacının alt 4 biti korunurken üst 4 biti 0'a eşitleniyor.
10. satırda üst 4 bite 0011 yazılıyor.
11. satırda ise AL yazmacının değeri ASCII_SAYI konumuna geri yükleniyor.

8.8.4 Paketlenmiş BCD – ASCII Çevirimi

Örnek olarak 0111 0101 paketlenmiş BCD sayısını (ondalık 75 sayısını) ASCII sayı sistemine çevirelim. 75 sayısının ASCII gösterimi 0011 0111 0011 0101'dir.

Yapmamız gereken paketlenmiş BCD sayısının alt 4 bitini ve üst 4 bitini ayrı ayrı alıp ASCII sayı sistemine uygun hale getirmektir. Aşağıda yukarıdaki işlemi yapan program verilmiştir:

```
1  ORG 0100H
2  JMP BASLA
3  BCD_SAYI   DB 01110101B
4  ASCII_SAYI DW 0000H
5  BASLA:
6  MOV SI, BCD_SAYI
7  MOV AL, [SI]
8  MOV AH, [SI]
9  AND AL, 00001111B
10 AND AH, 11110000B
11 XOR AL, 00110000B
12 SHR AH, 4
13 XOR AH, 00110000B
14 MOV SI, ASCII_SAYI
15 MOV [SI], AX
16 INT 03H
```

6. satırda SI yazmacına BCD_SAYI değişkeninin adresi kaydediliyor.

7. ve 8. satırlarda AL ve AH yazmaçlarına SI yazmacının değerinin işaretlediği veriler aktarılıyor.

9. satırda AL yazmacının alt 4 biti korunuyor, üst 4 bit 0'a eşitleniyor.

10. satırda AH yazmacının üst 4 biti korunuyor, alt 4 biti ise 0'a eşitleniyor.

11. satırda AL yazmacının üst 4 bitine 0011 yazdırılıyor.

12. satırda AH yazmacının değeri 4 bit sağa kaydırılıyor. (Paketlenmiş BCD sayısının üst 4 bitinde bir sonraki basamağın değeri bulunduğundan bu değeri alt 4 bite çekmemiz gerekiyor.)

13. satırda AH yazmacının üst 4 bitine 0011 yazdırılıyor.

14. satırda SI yazmacına ASCII_SAYI değişkeninin adresi yazılıyor.

15. satırda AX yazmacının değeri SI yazmacının değerinin işaretlediği bellek konumuna aktarılıyor.

8.8.5 ASCII – Paketlenmemiş BCD Çevirimi

Örnek olarak 0011 0010 ASCII sayısını (2 sayısını) paketlenmemiş BCD sayısına çevirelim. 2 sayısının paketlenmemiş BCD gösterimi 0000 0010'dur.

0011 0010 sayısının üst 4 bitini 0'a eşitleme işlemi gerekmektedir. Aşağıda bu işlemi yapan program verilmiştir:

```
ORG 0100H
JMP BASLA
BCD_SAYI   DB 00H
ASCII_SAYI DB 32H
BASLA:
MOV SI, ASCII_SAYI
MOV AL, [SI]
AND AL, 00001111B
```

```
9  MOV SI, BCD_SAYI
10 MOV [SI], AL
11 INT 03H
```

6. satırda SI yazmacına ASCII_SAYI değişkeninin adresi kaydediliyor.
7. satırda AL yazmacına SI yazmacının değerinin işaretlediği veriler aktarılıyor.
8. satırda AL yazmacının alt 4 biti korunup üst 4 biti 0'a eşitleniyor.
9. satırda SI yazmacına BCD_SAYI değişkeninin adresi kaydediliyor.
10. satırda SI yazmacının değerinin işaretlediği bellek bölgesine AL yazmacının değeri aktarılıyor.

8.8.6 ASCII – Paketlenmiş BCD Çevirimi

Örnek olarak 0011 0010 0011 0001 ASCII sayısını (21 sayısını) paketlenmiş BCD sayısına çevirelim. 21 sayısının paketlenmiş BCD gösterimi 0010 0001'dir.

0011 0010 0011 0001 sayısının ilk baytını paketlenmiş BCD sayısının üst 4 bitine, ikinci baytını ise paketlenmiş BCD sayısının alt 4 bitine yazmalıyız. Aşağıda bu işlemi yapan program verilmiştir:

```
1  ORG 0100H
2  JMP BASLA
3  BCD_SAYI   DB 00H
4  ASCII_SAYI DW 3231H
5  BASLA:
6  MOV SI, ASCII_SAYI
7  MOV AL, [SI]
8  MOV AH, [SI]
9  AND AL, 00001111B
10 AND AH, 00001111B
11 SHL AH, 4
12 MOV SI, BCD_SAYI
13 MOV [SI], AX
14 INT 03H
```

5. satırda SI yazmacına ASCII_SAYI değişkeninin adresini kaydettik.
6. ve 7. satırlarda ise AL ve AH yazmaçlarına SI yazmacının değerinin işaretlediği bellek bölgesindeki değeri aktardık.
8. ve 9. satırlarda ise AL ve AH yazmaçlarının üst 4 bitlerini 0'a eşitledik. Bu işlemi yaparken alt 4 bitlerde herhangi bir değişikliğe neden olmamak için 1 ile AND'ledik.
10. satırda AH yazmacını 4 bit sola kaydırdık. Bu şekilde paketlenmiş BCD sayısının üst 4 biti için verilmesi gereken bit değerlerini doğru bitlere kaydırmış olduk.
11. satırda SI yazmacına BCD_SAYI değişkeninin adresini kaydettik.
12. satırda SI yazmacının değerinin işaretlediği bellek bölgesine AX yazmacının değerini kaydettik.

8. Bölüm Soruları

Alıştırma 8.1. Paketlenmiş BCD biçimindeki bir sayıyı ASCII biçimine çeviren bir 8086 programı yazın.

Alıştırma 8.2. ASCII biçimindeki bir sayıyı paketlenmemiş BCD biçimine çeviren bir 8086 programı yazın.

Alıştırma 8.3. ASCII biçimindeki bir sayıyı paketlenmiş BCD biçimine çeviren bir 8086 programı yazın.

Alıştırma 8.4. DS:SI adresinden başlayan ve uzunluğu CX yazmacında belirtilen bir bayt dizisini ES:DI adresine MOVSB komutu ile kopyalayan bir program yazın. Kopyalama işleminin ardından kaynak ve hedef dizilerin içeriklerinin aynı olduğunu CMPSB komutu kullanarak doğrulayın; eşitlerse AX = 1, değilse AX = 0 olsun.

Alıştırma 8.5. AL yazmacında bir karakter (ASCII değeri) ve ES:DI'nın gösterdiği adreste uzunluğu CX yazmacında belirtilen bir bayt dizisi bulunmaktadır. SCASB ve REPNE komutlarını kullanarak bu karakterin dizide kaçıncı konumda bulunduğunu BX yazmacına yazan bir program yazın. Karakter bulunamazsa BX = 0FFFFh olsun.

Alıştırma 8.6. ES:DI'nın gösterdiği adresten başlayarak CX yazmacındaki değer kadar bellek konumunu AL yazmacındaki değerle dolduran (silen) STOSB tabanlı bir program yazın. Ardından bu bellek bölgesini SCASB komutuyla tarayarak AL değerini içeren ilk konumun adresini BX yazmacına kaydedin.

Alıştırma 8.7. DS:SI adresinden başlayan ve sonunu `0` baytı ile işaretlenmiş bir dizginin (C tarzı dizgi) uzunluğunu SCASB komutu yardımıyla bulan ve sonucu CX yazmacına yazan bir program yazın. (İpucu: AL = 0, REPNE SCASB kullanın; döngü bittiğinde CX değerinden geriye doğru uzunluğu hesaplayın.)

Alıştırma 8.8. DS:SI adresinden başlayan ve CX bayt uzunluğunda olan bir dizginin içindeki büyük harf sayısını (A–Z, ASCII 41H–5AH) sayan ve sonucu AX yazmacına yazan bir program yazın. LODSB ve koşullu atlama komutlarını kullanın.

Alıştırma 8.9. DS:SI adresindeki uzunluğu CX olan bir bayt dizisini **ters sırada** ES:DI adresine kopyalayan bir program yazın. Ters kopyalama için STD komutu ile DF=1 yapın ve MOVSB kullanın.

9

Kesmeler ve Çevre Birimlerinin Kullanımı

9.1 Kesmeler

x86 işlemcisinde bir kesme oluştuğu anda o an çalışan program durdurulur (kesilir) ve işlemci ISR (Interrupt Service Routine – Kesme Hizmeti Yordamı) adlı özel bir programa atlar. Her ISR aslında bellekte bulunan normal bir programdır. ISR işini bitirdiğinde işlemci kesildiği noktaya genellikle (bazı kesmelerde dönmez) geri döner.

ISR yukarıdan da anlaşılacağı gibi belli bir bellek bölgesine atlamaya dayalı bir sistemdir. Bunun dışında PIC (Programmable Interrupt Controller – Programlanabilir Kesme Denetleyicisi) veya APIC (Advanced Programmable Interrupt Controller – Gelişmiş Programlanabilir Kesme Denetleyicisi) kullanılabilir. Bu iki sistem donanım düzeyindedir. İşlemcinin kesme iğnesine ve kesme yapan donanıma aynı anda bağlıdırlar ve kesmeleri bu şekilde denetlemektedirler.

Kesmeler 3 ana bölüme ayrılabilir:

- Donanım Kesmeleri
- Yazılım Kesmeleri
- Kural Dışı Kesmeler

9.1.1 Donanım Kesmeleri

Donanım kesmeleri adından da anlaşılacağı gibi donanımlar tarafından oluşturulmaktadırlar. Klavyenizde bir tuşa bastığınızda klavyeniz bir donanım kesmesi oluşturur. İşlemci bu kesmeyi işlemek için klavye veri giriş kodunu çalıştırır ve bastığınız tuşun kodu bir bellek bölgesine kaydedilir. Donanım kesmeleri asenkrondur; kesme oluştuğu anda çalışmakta olan programdan bağımsızdır.

9.1.2 Yazılım Kesmeleri

Yazılım kesmeleri yazılımların içinden oluşturulabilen kesmelerdir. Yazılım kesmelerine örnek olarak şu ana kadar çokça kullandığımız INT 03H kesmesini verebiliriz. İleride göreceğimiz INT 10H kesmesi kullanılarak ekrana yazı yazdırabiliriz.

9.1.3 Kural Dışı Kesmeler

Kural dışı kesmeler çalışmakta olan programda kural dışı bir sonuç oluştuğunda ortaya çıkar. Mesela 0'a bölme veya izin olmayan bir bellek bölgesine erişmeye çalışmak kural dışı kesmelere bir örnektir.

Birçok işletim sisteminde kesmeler kullanılır. Mesela DOS, Linux gibi işletim sistemlerinde kesmeler kullanılabilir. Windows işletim sisteminde ise kesmeler yerine API (Application Programming Interface – Yazılım Programlama Ara Yüzü) kullanılır. API bir bakıma kesmelerin ara yüz yapılarak kullanımlarını kolaylaştıran bir halidir.

9.1.4 BIOS Kesmeleri

BIOS kesmeleri bir bilgisayarda alt seviye programlar yazarken çok önemlidir. Aşağıdaki Tablo 9.1'nde BIOS kesmelerinin bir listesi ve kısaca tanımları verilmiştir. Daha ayrıntılı bir liste için Ek D'ye bakınız.

Tablo 9.1: *BIOS kesme listesi.*

Kesme	Tanım
INT 00H	İşlemci: Sıfıra Bölme
INT 01H	İşlemci: Hata ayıklama için adım adım yürütme
INT 02H	İşlemci: NMI (Non-Maskable Interrupt – Maskelenemeyen Kesme)
INT 03H	İşlemci: Hata ayıklama için duraksama noktası
INT 04H	İşlemci: Taşma hatası
INT 05H	Ekranı Yazdır (Print Screen)
INT 08H	IRQ0: Sistem zamanlayıcısı tarafından çağrılır
INT 09H	IRQ1: Klavye tarafından çağrılır
INT 0BH	IRQ3: COM2 (2. seri kapı) tarafından çağrılır
INT 0CH	IRQ4: COM1 (1. seri kapı) tarafından çağrılır
INT 0DH	IRQ5: Sabit disket denetleyicisi (PC/XT) veya 2. paralel kapı (AT)
INT 0EH	IRQ6: Disket sürücü denetleyicisi tarafından çağrılır
INT 0FH	IRQ7: 1. paralel kapı (yazıcı) tarafından çağrılır
INT 10H	Video Hizmetleri
INT 11H	Çevre birimi yüklendi
INT 12H	Geleneksel bellek boyutunu al
INT 13H	Alt Seviye Disk Hizmetleri
INT 14H	Seri G/Ç (Giriş/Çıkış)
INT 15H	Çeşitli Kesmeler
INT 16H	Klavye
INT 17H	Yazıcı Hizmetleri
INT 18H	INT 19H başarısızlıkla sonuçlanırsa kullanılır
INT 19H	POST'tan sonra işletim sistemini yüklemek için kullanılır
INT 1AH	RTC (Real Time Clock – Gerçek Zamanlı Saat) Hizmetleri
INT 1BH	"Break" tuşuna basıldığında çağrılır
INT 1CH	Yazılım zamanlayıcı kesmesi. INT 08H tarafından çağrılır
INT 1DH	Adres işaretçisi: VPT (Video Parameter Table)
INT 1EH	Adres işaretçisi: DPT (Diskette Parameter Table)
INT 1FH	Adres işaretçisi: VGCT (Video Graphics Character Table)
INT 41H	Adres işaretçisi: FDPT 1 (1. Sabit Disket Parametre Tablosu)
INT 46H	Adres işaretçisi: FDPT 2 (2. Sabit Disket Parametre Tablosu)
INT 4AH	RTC tarafından alarm için çağrılır
INT 70H	IRQ8: RTC tarafından çağrılır
INT 74H	IRQ12: Fare tarafından çağrılır
INT 75H	IRQ13: Matematiksel ek işlemci tarafından çağrılır
INT 76H	IRQ14: Birinci IDE denetleyicisi tarafından çağrılır
INT 77H	IRQ15: İkinci IDE denetleyicisi tarafından çağrılır

Bilgi Notu 9.1

Yukarıdaki tabloda listelenen tüm kesmeler ve bazı diğer kesmeler numaraları ile çağırabileceğimiz, maskelenebilir (geçici olarak ihmal edilebilen) kesmelerdir. Yalnız NMI (Non-Maskable Interrupt – Maskelenemeyen Kesme) adından da anlaşıldığı gibi maskelenemez; aşırı önemli durumlarda ve genellikle büyük sistem hatalarının sonucunda ortaya çıkar. NMI'lerin ihmal edilmesi mümkün değildir, oluştukları zaman işlenirler. NMI oluşturacak bir sistem hatasına örnek olarak bellekten gelen verilerde bir eşlik hatasının oluşması verilebilir.

Şimdi de yukarıdaki kesmelerden bazılarını örnekleyelim.

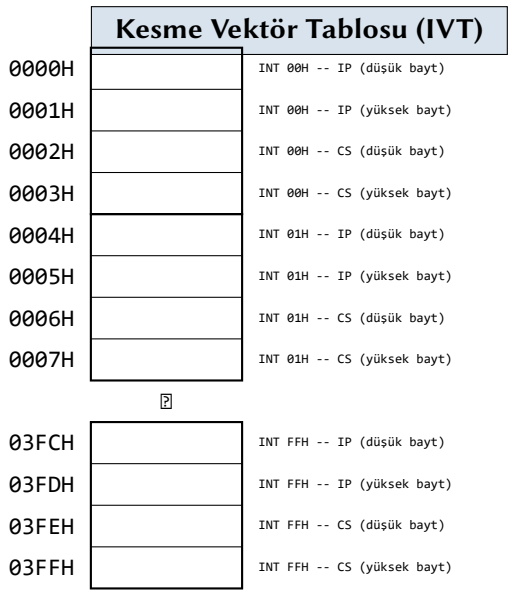
9.1.5 Kesme Vektör Tablosu

x86 sistemlerde her kesme numarasına karşılık gelen ISR adresini bulmak için **Kesme Vektör Tablosu** (Interrupt Vector Table – IVT) kullanılır. IVT, bellekte `0000H:0000H` ile `0000H:03FFH` arasındaki 1024 baytlık alanda saklanır.

Tabloda her kesme için 4 bayt yer ayrılır: ilk 2 bayt ISR'nin ofset adresi (IP), son 2 bayt ise kesim adresi (CS). Bu nedenle N numaralı kesmenin vektör adresi şu formülle bulunur:

$$N \times 4$$

Örneğin INT 09H (klavye kesmesi) için vektör adresi 09H × 4 = 24H'tir; yani bellekteki `0000H:0024H` adresinde CS:IP çifti saklanır.



Şekil 9.1: IVT bellek düzeni. 0000H:0000H–0000H:03FFH arasında 256 adet 4 baytlık kesme vektörü saklanır.

INT N komutu çalıştırıldığında işlemci şu adımları sırasıyla gerçekleştirir:

1. FLAGS yazmacı yığına itilir.
2. IF (Kesme Bayrağı) ve TF (Tuzak Bayrağı) sıfırlanır.

3. CS yazmacı yığına itilir.
4. IP yazmacı yığına itilir.
5. IVT'den $N \times 4$ adresindeki ofset değeri IP'ye, $N \times 4 + 2$ adresindeki kesim değeri CS'ye yüklenir.
6. ISR'nin ilk komutu çalıştırılmaya başlanır.

ISR tamamlandığında `IRET` komutu çalıştırılır: yığından sırasıyla IP, CS ve FLAGS geri alınarak kesilen programa dönülür.

Bilgi Notu 9.2: Kesme Vektörlerini Değiştirme

Bir kesme vektörü değiştirilerek özel bir ISR yüklenebilir. Bunun için ilgili IVT girişine yeni CS:IP çifti yazılır. Örneğin INT 09H (klavye) vektörü değiştirilerek özel klavye sürücüsü kurulabilir. İşlem bitince özgün vektörün geri yüklenmesi gerekir; aksi halde sistem davranışı değişir.

9.2 INT 00H

INT 00H, 0'a bölme hatası oluştuğu zaman veya DIV ya da IDIV komutlarıyla yapılan bölme işleminin bölümünün yazmaca sığmadığı zaman çağırılır.

8086, 8088, V20 ve V30 işlemcileri hata veren DIV veya IDIV komutunun yığına kaydettikten sonra CS:IP adresini yığına kaydederken 80286 ve 80386 gibi işlemciler sadece hata veren DIV veya IDIV komutunun CS:IP adresini yığına kaydeder.

9.3 INT 10H ve Monitör

INT 10H kesmesini kullanarak ekrana yazı yazdırabiliriz. Bu yazımız için farklı bir arka plan rengi seçebilir ve pek çok özelliğini denetleyebiliriz. INT 10H kesmesinin alt yordamlarına ulaşabilmek için AH yazmacına INT 10H'ı çağırmadan önce farklı değerler vermeliyiz. Aşağıdaki Tablo 9.2'da bu değerler ve işlevleri listelenmiştir.

Tablo 9.2: *INT 10H alt yordamları.*

AH	İşlev
00H	Video tipini ayarla. AL = tip.
01H	İmleç şeklini ayarla. CH = başlama satırı, CL = bitiş satırı. Not: Satır değerleri 0–15 arasında olduğundan imleci yok etmek için CH yazmacına 20H yüklenebilir.
02H	İmleç pozisyonunu ayarla. BH = sayfa numarası, DH = y koordinatı, DL = x koordinatı.
03H	İmleç pozisyonunu ve şeklini al. Girdi: BH = imlecin bulunduğu sayfa numarası. Çıktı: CH = başlama satırı, CL = bitiş satırı, DL = x koordinatı, DH = y koordinatı.
05H	Görüntü sayfasını ayarla. AL = görüntü sayfası.
06H	Temizle veya ekranı yukarı kaydır. AL = kaydırılacak satır sayısı, BH = görüntü özelliği, CL/CH = sol-üst köşe, DL/DH = sağ-alt köşe. AL 00H ise temizleme görevi görür.
07H	Temizle veya ekranı aşağı kaydır. Parametreler 06H ile aynıdır.
08H	İmleç pozisyonundaki karakteri ve özelliklerini oku. Girdi: BH = sayfa numarası. Çıktı: AL = karakter, AH = özellikler.
09H	İmleç pozisyonuna karakter yaz. AL = karakter, BH = sayfa numarası, BL = karakter özelliği, CX = tekrarlanma sayısı.
0AH	İmleç pozisyonuna karakteri ve özellikleri yaz. AL = karakter, BH = sayfa numarası.
0BH	Kenarlık rengini ayarla. BH = 00H, BL = renk.
0EH	Karakter yazdır (özelliklerini kullanarak ve imleci yeniden konumlandırarak). AL = karakter, BH = sayfa numarası.
0FH	Video tipi ayarlarını al. Çıktı: AH = kolon sayısı, AL = video tipi, BH = sayfa numarası.
13H	Dizgi yazdır. AL = yazma kipi, BH = sayfa numarası, BL = özellik, CX = dizgi uzunluğu, DH = başlama satırı, DL = başlama sütunu, ES:BP = dizginin adresi.

Şimdi INT 10H kesmesini kullanarak ekrana “M” harfini yazdıralım:

```
1  ORG 0100H
2  MOV AH, 0EH
3  MOV AL, 'M'
4  INT 10H
5  RET
```

2. satırda AH yazmacına 0EH değerini yükledik. Yukarıdaki tablodan bakılacak olursa AL yazmacına yazılacak karakteri ve BH yazmacına sayfa numarasını girmemiz gerektiğini göreceğiz. BH yazmacını olduğu değerde bırakmamız herhangi bir sorun teşkil etmeyecektir çünkü zaten başlangıçta o sayfanın değerine sahip olacaktır.

3. satırda AL yazmacına M harfinin ASCII değerini yazdık.

4. satırda ise INT 10H komutunu çağırdık ve karakterin ekrana yazılmasını sağladık.

5. satırdaki RET komutu ile de programımızı sonlandırdık.

Şimdi de bu M harfini kırmızı yazı rengi ile yazdıralım. Bunun için yukarıdaki tablodan da görülebileceği gibi AH = 09H durumunu kullanmamız gerekiyor:

```
1  ORG 0100H
2  MOV AH, 09H
3  MOV AL, 4DH       ;M harfinin ASCII kodu
4  MOV BL, 4         ;Kirmizi renk
5  MOV CX, 1         ;1 kez tekrarla
6  INT 10H
7  RET
```

2. satırda AH yazmacına 09H değerini yükledik.

3. satırda AL yazmacına yazılacak olan karakterin ASCII kodunu yükledik: M harfinin ASCII karşılığı olan 4DH değerini AL yazmacına yükledik.

4. satırda BL yazmacına yazının özellik değerini yazdık. Özellik değerleri için aşağıdaki Tablo 9.3'ye bakınız.

5. satırda ise CX yazmacına karakterin ne kadar kez tekrarlanacağını belirttik. Sadece 1 tane M harfi istediğimizden 1 yazdık.

Tablo 9.3: *INT 10H renk kodları.*

Onaltılık	İkili	Renk
0	0000	Siyah
1	0001	Mavi
2	0010	Yeşil
3	0011	Camgöbeği
4	0100	Kırmızı
5	0101	Eflatun
6	0110	Kahverengi
7	0111	Açık gri
8	1000	Koyu gri
9	1001	Açık mavi
A	1010	Açık yeşil
B	1011	Açık camgöbeği
C	1100	Açık kırmızı
D	1101	Açık eflatun
E	1110	Sarı
F	1111	Beyaz

Şimdi de ekrana "MERHABA DUNYA!" yazısını yazdıralım:

```
1  ORG 0100H
2  MOV AH, 03H                    ;imlec konumunu al
3  INT 10H
4  MOV AL, 01H                    ;yaziyi yaz
5  MOV BH, 00H
6  MOV BL, 01001111B
7  MOV CX, MESAJ_SON - MESAJ_BAS
8  PUSH CS
9  POP ES
10 MOV BP, MESAJ_BAS
```

```
11  MOV AH, 13H
12  INT 10H
13  JMP SON
14  MESAJ_BAS DB 'MERHABA DUNYA!'
15  MESAJ_SON DB 00H
16  SON:
17  RET
```

Yukarıdaki kodu çevirip DEBUG ile denediğinizde ekrana yazdırılan "MERHABA DUNYA!" yazısının kırmızı arka plan ve beyaz yazı rengine sahip olduğunu göreceksiniz. Kodu incelersek:

2. satırda AH yazmacına 03H değerini yüklüyoruz; bu şekilde INT 10H'ın 03H alt yordamını kullanmak istiyoruz. INT 10H tablosuna bakarsak bu alt yordamının imleç pozisyonunu ve şeklini almaya yaradığını göreceğiz. CH yazmacına imlecin başlama satırı, CL yazmacına imlecin bitiş satırı, DL yazmacına imlecin x koordinatı ve DH yazmacına imlecin y koordinatı yazılacaktır.

3. satırda INT 10H kesmemizi çağırıyoruz.

4. satırda AL yazmacına 01H değerini kaydediyoruz. Bu değer ileride çağıracağımız INT 10H kesmesinin 13H numaralı alt yordamında yazma kipi olarak kullanılacaktır.

5. satırda sayfa numarası olarak kullanılacak olan BH yazmacının değeri atanıyor.

6. satırda yazılacak olan yazının özellikleri ayarlanıyor. Kırmızı arka plan ve beyaz yazı rengi.

7. satırda CX yazmacına "MERHABA DUNYA!" dizgisinin uzunluğu hesaplanıp kaydediliyor.

8. satırda yığına CS yazmacının değeri kaydediliyor.

9. satırda 8. satırda kaydedilmiş olan CS yazmacı değeri yığından çağrılıyor ve ES yazmacına yazdırılıyor. PUSH-POP ikilisini kullanmak yerine 2 tane MOV komutu kullanabilirdik ama başarım açısından PUSH-POP kullanmak daha verimlidir.

10. satırda BP yazmacına "MERHABA DUNYA!" dizgisinin ilk harfinin göreli konum adresini yazdırıyoruz. INT 10H kesmesinin 13H numaralı alt yordamına bakacak olursak yazılacak olan dizginin ES:BP adresinden başlanarak yazılacağını görürüz. 9. ve 10. satırlarda ES:BP adresinin bizim "MERHABA DUNYA!" dizgimize işaret etmesini sağladık.

11. satırda AH yazmacına 13H değerini yazıyoruz ve hemen ardından 12. satırda INT 10H kesmesini çağırıyoruz. Bu şekilde INT 10H kesmesinin 13H numaralı alt yordamına erişmiş olduk.

9.4 INT 15H

INT 15H kesmesi ilk senelerde bilgisayarlara bağlı kasetleri yönetmek için kullanılmaktaydı. Kasetler günümüzde kullanımda olmadığından INT 15H kesmesi birçok farklı işlevi yerine getirmek için kullanılmaktadır. Mesela C0H alt yordamı sayesinde bilgisayarın alt-model numarasına erişilebilir. Ayrıntılı bilgi için Ek D'ye bakınız.

9.5 INT 09H ve Klavye

Klavyenin her tuşuna basılışında ve bırakılışında INT 09H kesmesi çağırılır. Bu kesme donanım kesmesidir (IRQ 1). ROM'da bulunan INT 09H yordamı aşağıdaki şekilde klavyeden gelen bilgileri işler:

- Herhangi bir ASCII karakter tuşuna basıldığında basılan tuşun ASCII kodu ve tarama kodu 32 baytlık klavye tampon belleğine kaydedilir. Tampon belleğe 0000:041E adresinden ulaşılabilir. Gelen ASCII ve tarama kodları 0000:041C adresindeki klavye imleç tamponunun işaretlediği bellek bölgesine kaydedilir. Her kayıttan sonra klavye imleç tamponunun değeri 2 artırılır. Eğer klavye imleç tamponunun değeri klavye tamponunun sonundan daha ileriye işaret ediyorsa geri başa döner.
- "Pause/Duraklama" tuşuna basıldığında sonsuz bir döngüye girilir ve döngü geçerli bir ASCII tuşuna basılana kadar dönmeye devam eder.
- "Print Screen" tuşuna basıldığında INT 05H kesmesi çağırılır.
- "Break" tuşuna basıldığında INT 1BH kesmesi çağırılır.
- "Ctrl", "Alt" veya "Shift" tuşlarına basılırsa Shift Durumu (0000:0417 adresindeki) ve Genişletilmiş Shift Durumu (0000:0418 adresindeki) baytlar güncellenir.
- "Ctrl+Alt+Del" tuşlarının hepsine birden basılırsa 0000:0472 adresindeki baştan başlatma bayrağının değeri 1234H'a eşitlenir ve denetim POST (Power On Self Test) ye devredilir.

9.6 INT 16H ve Klavye Servisleri

INT 16H kesmesini kullanarak klavyeden karakter alma gibi işlemleri çok basit bir şekilde yapabiliriz. INT 16H kesmesinde INT 10H kesmesinde olduğu gibi alt yordamlar vardır. Aşağıdaki Tablo 9.4'da bu alt yordamlara ulaşmak için gerekli olan AH yazmacı değerleri ve işlevleri listelenmiştir.

Tablo 9.4: *INT 16H alt yordamları.*

AH	İşlev
00H	Klavyeden basılan karakteri okur. Ekrana herhangi bir çıktı vermez. Çağırıldığı zaman bir tuşa basılana kadar bilgisayarı dondurur. Çıktı olarak AL (basılan tuşun ASCII değeri) ve AH (klavye tarama değeri) yazmaçları verilir.
01H	Girdi durumunu okur. Basılan tuşun ASCII değerini herhangi bir yazmaca kaydetmez. Sadece bir tuşa basılıp basılmadığını denetlemek için kullanılır. Bilgisayarı dondurmaz.
02H	Kontrol tuşlarının durumunu okur. Çıktı olarak AL yazmacı verilir; AL yazmacının bitleri hangi kontrol tuşlarına basıldığını gösterir.
10H	Genişletilmiş karakter oku.
11H	Genişletilmiş girdi durumunu oku.
12H	Genişletilmiş kontrol karakter tuşlarının durumunu oku.

Yukarıdaki alt yordamlardan sadece ilk üçünü inceleyeceğiz.

9.6.1 00H Alt Yordamı

Öncelikle "t" harfine basılana kadar kapanmayacak bir program yazalım. Klavyeden bastığımız her tuş ekrana yazdırılsın.

```
ORG 0100H
KARAKTER_OKU:
MOV AH, 00H
INT 16H
CMP AL, 74H     ;'t' harfi
JE SON
MOV AH, 0EH
INT 10H
JMP KARAKTER_OKU
SON:
RET
```

Yukarıdaki koda baktığımızda karakterin önce okunduğunu, ardından karşılaştırıldığını ve "t" harfine eşit olup olmama durumuna göre programın sonlandırdığını veya tekrar bir harf okuduğunu göreceğiz. Programı denediğimizde ise "t" dışındaki bir harfe bastığımız anda ekrana yazdırıldığını göreceğiz.

9.6.2 01H Alt Yordamı

AH yazmacının değeri 00H olduğu durumda klavyeden karakter okunurken karakter basılana kadar kesme devam etmektedir; başka bir deyişle yazdığımız bir program bir tuşa basılana kadar kilitlenmiştir. Bunun yerine AH yazmacının değerini 01H yaparak INT 16H kesmesini çağırırsak sadece anlık bir tuşa basılıp basılmadığını denetleyebiliriz. Bir örnek ile 01H alt yordamını inceleyelim.

```
ORG 0100H
XOR CX, CX
MOV AH, 03H                    ;Ekrana mesaji yazdir
INT 10H
MOV AL, 01H
MOV BH, 00H
MOV BL, 01001111B
MOV CX, MESAJ1_SON - MESAJ1_BAS
PUSH CS
POP ES
MOV BP, MESAJ1_BAS
MOV AH, 13H
INT 10H
OKU:                            ;Klavyeyi oku
MOV AH, 1
INT 16H
JZ ARTIR
MOV AH, 00H
INT 16H
CMP AL, 61H                    ;'a' harfi
JE SON
CMP AL, 64H                    ;'d' harfi
```

```
23  JNE ARTIR
24  PUSH CX
25  CMP BYTE [SI], 0FFH
26  JE YAZ
27  YAZ_ILK:
28  MOV SI, MESAJ2_SON
29  MOV BYTE [SI], 0FFH
30  INC DH
31  MOV DL, 00H
32  MOV AL, 01H
33  MOV BH, 00H
34  MOV BL, 01001111B
35  MOV CX, MESAJ2_SON - MESAJ2_BAS
36  PUSH CS
37  POP ES
38  MOV BP, MESAJ2_BAS
39  MOV AH, 13H
40  INT 10H
41  POP CX
42  MOV AL, CH
43  MOV AH, 0EH
44  INT 10H
45  MOV AL, CL
46  MOV AH, 0EH
47  INT 10H
48  JMP ARTIR
49  YAZ:
50  POP CX
51  MOV AH, 02H
52  INT 10H
53  MOV AL, CH
54  MOV AH, 0EH
55  INT 10H
56  MOV AL, CL
57  MOV AH, 0EH
58  INT 10H
59  ARTIR:
60  INC CX
61  JMP OKU
62  JMP SON
63  MESAJ1_BAS DB 'CX YAZMACININ ASCII DEGERINI GORMEK ICIN D, CIKMAK ICIN A HARFINE BASINIZ!'
64  MESAJ1_SON DB 00H
65  MESAJ2_BAS DB 'CX DEGERI:'
66  MESAJ2_SON DB 00H
67  SON:
68  RET
```

3. ile 13. satır arasındaki kodlar ile ekrana “CX YAZMACININ ASCII DEGERINI GORMEK ICIN D, CIKMAK ICIN A HARFINE BASINIZ!” dizgisi yazdırılıyor.

16. satırda AH yazmacına 01H değeri yükleniyor. Bu şekilde 17. satırda çağıracağımız INT 16H kesmesinin 01H alt yordamına erişiyoruz. INT 16H kesmesinin 01H numaralı alt yordamı çağırıldığı anda bir tuşa basılmış ise ZF bayrak bitini 1, basılmamış ise ZF bayrak bitini 0 değerinde tutar.

18. satırda eğer ZF bayrak biti 1 ise (bir tuşa basılmamış ise) ARTIR etiketine atlanır. Aksi

takdirde (bir tuşa basılmış ise) devam edilir.

19. ve 20. satırdaki kodlar ile INT 16H kesmesinin 00H alt yordamına erişiyoruz. Bu işlemi yapmamızın nedeni klavyenin belleğinden karakterin ASCII kodunun AL yazmacına aktarılmasını sağlamak çünkü INT 16H kesmesinin 01H alt yordamı AL yazmacına klavyenin belleğindeki veriyi yazmaz.

21. satırda AL yazmacının değerini ASCII 'a' harfi ile karşılaştırıyoruz. Eğer eşit iseler 22. satırda belirtildiği gibi SON etiketine atlıyoruz.

23. satırda ise AL yazmacının değerini ASCII 'd' harfi ile karşılaştırıyoruz. Eğer eşit değilseler ARTIR etiketine 24. satırda belirtildiği gibi atlıyoruz. Eğer AL yazmacının değeri 'd' harfinin ASCII değerine eşit ise 25. satırdan devam ediliyor.

63. satırda ise CX yazmacının değerini 1 artırıyoruz ve ardından herhangi bir tuşa basılıp basılmadığını denetlemek amacıyla OKU etiketine atlıyoruz.

9.6.3 02H Alt Yordamı

INT 16H kesmesinin 02H alt yordamı çağırıldığında AL yazmacı ile hatırlayacağınız gibi basılan kontrol tuşları hakkında bilgi vermektedir.

Bu alt yordamın nasıl çalıştığını görmek için aşağıdaki kodu kullanabiliriz:

```
1  ORG 0100H
2  MOV AH, 02H
3  INT 16H
4  INT 03H
```

Kodu denemeden önce "Caps Lock" veya diğer kontrol tuşlarına basılacak olursa AL yazmacı üzerindeki etkisi bu şekilde gözlemlenebilir.

9.7 INT 14H ve Seri Kapı

INT 14H kesmesi kullanılarak seri kapı üzerinde işlemler yapılabilir. Veri alınıp gönderilebilir. Alt yordamları aşağıdaki Tablo 9.5'da verilmiştir.

Tablo 9.5: *INT 14H alt yordamları.*

AH	İşlev
00H	Seri kapı başlatılması. Girdi: AL yazmacı. AL yazmacının değerine göre seri kapı ayarlanır ve başlatılır.
01H	Seri kapıdan karakter gönder. Girdi: AL (gönderilecek veri), DX (seri kapı numarası 0–3). Çıktı: AH = 00H ise başarılı; AH'ın 7. biti 1 ise hata oluştu.
02H	Seri kapıdan karakter al. Girdi: DX (seri kapı numarası). Çıktı: AL (alınan veri).
03H	Seri kapı durumu. Girdi: DX (seri kapı numarası). Çıktı: AX (durum bilgisi).

Bilgi Notu 9.3

Hangi seri kapının kullanıldığı yukarıdaki alt yordamlar çağırıldığı andaki DX yazmacı değerine bağlıdır. DX yazmacı 0 ile 3 arasında değerler alabilir: COM1, COM2, COM3 veya COM4 olabilir.

RS232 standardı hakkında kısa bir bilgi verelim. RS232'yi incelememizin nedeni günümüzde bilgisayarlarda genellikle seri kapı olarak RS232 bulunmasıdır.

1969 yılında EIA (Electronic Industries Alliance) tarafından seri iletişim için RS232-C standardı oluşturulmuştur. Bu standartta voltaj değerleri, sinyaller, kapasitans, kablo uzunluğu, kapıların tipleri ve iğne bilgileri belirlenmiştir. Standartta hiçbir şekilde veri iletim için herhangi bir karakter kodlama sınırı konmamıştır. Hata belirme hakkında herhangi bir kural yoktur. Ayrıca veri sıkıştırma üzerine de herhangi bir şey belirtilmemiştir. Aynı şekilde veri iletişim hızı hakkında da herhangi bir kısıtlama yapılmamıştır.

Veri alışverişinde karakter kodlaması, hız gibi bilgiler seri kapı donanımı tarafından denetlenmektedir. Genellikle bu donanım tek bir UART (Universal Asynchronous Receiver/Transmitter) entegre devresinden oluşmaktadır. Bu devre paralel olarak gelen veriyi seriye çevirmektedir.

Bu kesmeyi denemek için 2 farklı program yazalım. Bu programlardan biri bir bilgisayara yüklensin, diğeri de başka bir bilgisayara yüklensin. Bu iki bilgisayarı bir seri kablo ile birbirlerine bağlayalım.

İlk olarak biri dinleyici biri yollayıcı olmak üzere 2 tane program yazalım. Dinleyici program seri kapıdan gelen verileri ekrana yazacak, yollayıcı ise sürekli olarak "S" harfini seri kapıdan yollayacaktır. Dinleyici COM1'e, yollayıcı ise COM2'ye bağlanmalıdır.

İlk olarak veri yollayıcıyı inceleyelim:

```
1  ORG 0100H                    ;YOLLAYICI PROGRAM
2  MOV DX, 0001H                ;COM2
3  MOV AH, 00H                  ;SERI KAPI AYARLAMA
4  MOV AL, 00000010B            ;110 BAUD - 7 BIT VERI - PARITE BITI YOK
5  INT 14H                      ;1 TANE DURAKLAMA BITI
6  SERI_YOLLA:
7  MOV AL, 'S'                  ;AL yazmacina S harfini yaz
8  MOV AH, 01H                  ;veri yollama icin gerekli AH degeri
9  INT 14H
10 TEST AH, 80H                 ;hata olusup olusmadigini kontrol et (bit 7 = hata bayragi)
11 JZ SERI_YOLLA                ;olusmadilysa SERI_YOLLA ya geri don
12 SON:
13 RET
```

1. ve 6. satır arasındaki işlemler ile seri iletişimin kapısı, hızı, veri biti sayısı, eşlik biti sayısı ve duraklama biti sayısı ayarlandı. 00H alt yordamının parametrelerine bakacak olursak 110 baud veri iletişim hızı, 7 bit veri, 0 eşlik biti ve 1 tane duraklama biti ayarlarını yaptığımızı göreceğiz. Seri iletişim kapısı olarak COM2 ayarlanmıştır.

7. satırda AL yazmacına 'S' harfinin ASCII değerini kaydediyoruz.

8. satırda AH yazmacına 01H değerini yükleyerek 9. satırda atlayacağımız INT 14H kesmesinin AL yazmacındaki veriyi ayarlara göre iletmesini sağlıyoruz.

10. satırda AH yazmacının 7. bitine (hata bayrağına) bakıyoruz. INT 14H kesmesi AH yazmacının 7. bitini hata oluştuğunda 1, oluşmadığında 0 yapar. TEST AH,80H komutu bu

biti sınayarak ZF bayrağını ayarlar; ZF=1 ise hata oluşmamış demektir. Hata oluşmadıysa SERI_YOLLA etiketine atlanır. Hata oluştuysa program sonlandırılır.

Şimdi de dinleyiciyi inceleyelim:

```
1  ORG 0100H                    ;DINLEYICI PROGRAM
2  MOV DX, 0000H                ;COM1
3  MOV AH, 00H
4  ;SERI KAPI AYARLAMA
5  MOV AL, 00000010B            ;110 BAUD - 7 BIT VERI - PARITE BITI YOK
6  INT 14H                      ;1 TANE DURAKLAMA BITI
7  XOR AL, AL                   ;AL yazmacinin degerini 00H yap
8  SERI_DINLE:
9  MOV AH, 02H                  ;AH yazmacina 02H degerini kaydet
10 INT 14H                      ;INT 14H kesmesine atla
11 TEST AH, 80H                 ;hata olusup olusmadigini kontrol et (bit 7 = hata bayragi)
12 JZ MESAJ_YAZ                 ;MESAJ_YAZ etiketine atla
13 SON:
14 RET
15 MESAJ_YAZ:
16 PUSH DX                      ;DX yazmacini yigina at
17 MOV AH, 0EH                  ;AH yazmacina 0EH degerini yaz
18 INT 10H                      ;INT 10H kesmesine atla
19 POP DX                       ;DX yazmacina yigindaki veriyi kaydet
20 JMP SERI_DINLE               ;SERI_DINLE etiketine geri atla
```

Karakter yollayıcı programımızda olduğu gibi 1. ve 6. satır arasındaki işlemler ile seri iletişimin ayarlarını yaptık. Seri iletişim kapısı olarak COM1 ayarlanmıştır. Veri iletişim hızı 110 baud, duraklama biti sayısı 1, eşlik biti sayısı 0 ve veri biti sayısı 7 olarak ayarlanmıştır.

6. satırdaki işlemle AL yazmacının değeri 00H değerine eşitlenmiştir. Bunun yerine MOV AL,00H komutu da kullanılabilirdi. Ama XOR AL,AL, MOV AL,00H'a göre daha hızlıdır.

10. ve 11. satırdaki işlemlerle, AH yazmacına 02H değerini atayıp INT 14H kesmesine atlayarak seri kapıdan veri almak istediğimizi belirttik.

12. satırdaki TEST AH,80H komutu ile AH yazmacının 7. bitini (hata bayrağını) sınadık. INT 14H, AH'ın 7. bitini hata oluştuğunda 1 yapar. Bit 7 sıfır ise hata oluşmamış demektir ve ZF=1 olur; bu durumda JZ MESAJ_YAZ komutu 15. satırdaki MESAJ_YAZ etiketine atlayacak.

17. satırdaki PUSH DX komutu ile DX yazmacının değerini yığına kaydettik. Bunu yapmamızın nedeni 19. satırdaki INT 10H komutunun DX yazmacının değerini değiştirmesidir.

18. satırda AH yazmacına 0EH değerini kaydettik ve 19. satırdaki INT 10H komutu ile ekrana AL yazmacındaki veriyi yazdırdık.

20. satırdaki POP ile DX yazmacına eski değeri geri kaydettik ve 21. satırdaki kod sayesinde SERI_DINLE etiketine geri atladık.

İlerideki bölümlerde girdi/çıktı komutları ile seri iletişimi işledikten sonra daha ayrıntılı bir veri iletişim örneği yapacağız. Burada seri iletişime fazla değinilmemesinin nedeni bu bölümün amacının kesme mantığının ve kesmelerin kullanımlarının açıklanmasıdır.

9.8 INT 17H ve Yazıcı

INT 17H BIOS kesmesi ile yazıcı denetlenebilir. Aşağıdaki Tablo 9.6'da bu kesmenin alt yordamları ve işlevleri listelenmiştir.

Tablo 9.6: *INT 17H alt yordamları.*

AH	İşlev
00H	Bir karakter yazdırır. Girdi: AL (yazdırılacak karakter). Çıktı: AH (yazıcı durum bilgisi).
01H	Yazıcıyı hazırlar.
02H	Yazıcının durum bilgilerini okur. Çıktı: AH (yazıcı durum bilgisi).

9.9 INT 19H

INT 19H kesmesini kullanarak sistem baştan başlatılabilir. INT 19H'yi ayrıntılı anlatmadan önce başlatılma şekillerini inceleyelim. Sıcak ve soğuk başlangıç olmak üzere iki tip başlatılma şekli vardır.

Bilgisayar kapalı iken bilgisayarı açarsak "soğuk başlangıç" yapmış oluruz.

Ctrl+Alt+Del tuşlarına basarak bilgisayar yeniden başlatılırsa "sıcak başlangıç" yapılmış olur. Bu başlangıç şeklinde bellek sınamaları dışında bütün sistem sınamaları (Power On, POST, vb.) yapılmaktadır. Kesme vektörleri ve sistem zamanlayıcıları yeniden yüklenir. Bu şekilde sistem işletim sistemi başlatmaya uygun hale getirilir.

Bilgisayarın hangi şekilde başlatılacağı BIOS ROM'undaki bir yeniden başlatılma bayrağının değerine bağlıdır. Bayrağın değeri 1234H olursa sıcak başlangıç yapılır. Diğer tüm değerler soğuk başlangıç yapılmasına neden olur.

INT 19H'nin tek başına kullanılması ile sadece işletim sistemi yeniden yüklenir ve bazı durumlarda işletim sistemi yüklenmeye hazır olmayabilir. INT 19H çağırıldığında kesme vektörleri yeniden yüklenmez. Eğer bazı kesmeler üzerinde işlemler yapılmış ise INT 19H ile işletim sistemi yüklendikten sonra bu değiştirilmiş kesmeler çağırıldığında istenmeyen sonuçlar elde edilebilir. Ayrıca zamanlayıcıların ve bazı donanımlar baştan yüklenmediğinden sorunlar oluşabilir. Bu yüzden INT 19H tek başına kullanılmamalıdır.

9.10 INT 1AH ve RTC

INT 1AH ile RTC (Real Time Clock – Gerçek Zamanlı Saat) ve sistem zamanlayıcısı yönetilebilir. Sistem zamanlayıcısı (PIT, INT 08H) saniyenin 1/18'inde bir 1 artar (saniyede yaklaşık 18,2 kez) ve bilgisayar açıldığında veya yeniden başlatıldığında sıfırlanır. RTC ise sistem zamanlayıcısından farklı bir devreye dayanır: 32.768 kHz'lik bir kristal osilatörle çalışır ve saniyeyi, dakikayı, saati, tarihi doğrudan saklar. RTC bir CMOS pili ile beslendiğinden bilgisayar kapatılsa bile zaman değerlerini korur. INT 1AH hem sistem zamanlayıcısına (AH=00H, 01H) hem de RTC'ye (AH=02H–07H) erişim sağlar; bunlar birbirinden bağımsız devrelerdir ve birinin değerini değiştirmek diğerini etkilemez. Aşağıdaki Tablo 9.7'da INT 1AH'ın alt yordamları listelenmiştir.

Tablo 9.7: *INT 1AH alt yordamları.*

AH	İşlev
00H	Sistem zamanlayıcı zamanını oku. Çıktı: CX (üst bitler), DX (alt bitler), AL (24 saat geçmediyse 0).
01H	Sistem zamanlayıcı zamanını ayarla. Girdi: CX (üst bitler), DX (alt bitler).
02H	RTC zamanını oku. Çıktı: CH (saat, BCD), CL (dakika, BCD), DH (saniye, BCD), CF (zamanlayıcı durumu).
03H	RTC zamanını ayarla. Girdi: CH (saat, BCD), CL (dakika, BCD), DH (saniye, BCD), DL (güneş ışığından yararlanma).
04H	RTC tarihini oku. Çıktı: CH (yüzyıl, BCD), CL (yıl, BCD), DH (ay, BCD), DL (gün, BCD).
05H	RTC tarihini ayarla. Girdi: CH (yüzyıl, BCD), CL (yıl, BCD), DH (ay, BCD), DL (gün, BCD).
06H	RTC alarmını ayarla. Girdi: CH (saat), CL (dakika), DH (saniye). Alarm INT 4AH kesmesinin çağırılmasıyla çalmaya başlar.
07H	RTC alarmını kapat.
08H	Bilgisayarın otomatik açılma zamanı. Girdi: CH (saat), CL (dakika), DH (saniye).
09H	Alarm zamanını ve durumunu oku. Çıktı: CH (saat), CL (dakika), DH (saniye), CF (RTC durumu).
0AH	Sistem zamanlayıcı tarihini oku. Çıktı: CX (1 Ocak 1980'den beri geçen gün sayısı).
0BH	Sistem zamanlayıcı tarihini ayarla. Girdi: CX (1 Ocak 1980'den beri geçen gün sayısı).

Bilgi Notu 9.4

Yukarıdaki alt yordamlardan bazıları sistemin özelliklerine bağlı olarak bulunmayabilir. İlk 8 alt yordam çoğu sistemde bulunmaktadır.

RTC zamanı ve tarih bilgilerini aşağıdaki şekilde okuyabiliriz:

```
;RTC zamani okuma
ORG 0100H
MOV AH, 02H     ;RTC ızaman okuma
INT 1AH
INT 03H
```

```
;RTC tarihi okuma
ORG 0100H
MOV AH, 04H     ;RTC tarihi okuma
INT 1AH
INT 03H
```

9.11 INT 11H

INT 11H yüklü sistem birimlerini listeler. Girdi olarak herhangi bir yazmaç almazken çıktı olarak AX yazmacını verir. Aşağıdaki Tablo 9.8'nda AX yazmacına yazılan bilgilerin anlamları verilmiştir.

Tablo 9.8: *INT 11H çıktı AX yazmacı anlamları.*

AX Bitleri	Anlam
15–14	Bağlı yazıcı sayısı
13	1: Seri yazıcı bağlı; 0: Seri yazıcı bağlı değil
12	1: Oyun adaptörü bağlı; 0: Oyun adaptörü bağlı değil
11–9	Yüklü RS232 kartı sayısı
8	1: DMA var; 0: DMA yok
7–6	Bağlı disket sürücü sayısı
5–4	Video kipi
3–2	Kullanımda değil
1	1: Matematiksel yardımcı işlemci mevcut; 0: Mevcut değil
0	1: Disket sürücüsünde disket var; 0: Disket yok

Bilgi Notu 9.5

AX yazmacına yazılan yukarıdaki bilgilere aynı zamanda kesme çağırıldıktan sonra 0000:0410 adresindeki kelime okunarak da ulaşılabilir.

9.12 INT 13H – Alt Seviye Disk Servisleri

INT 13H kesmesi ile diskler üzerinde işlemler yapılabilir. INT 13H kesmesi ilk yazıldığında yaklaşık 8 GB'lık disk boyutu limiti vardı. Böyle bir limit bulunmasının nedeni CHS (Silindir-Başlık-Sektör) adresleme düzeninde en fazla 1024 silindir, 255 başlık ve 63 sektör kullanılabilmesidir (DH yazmacı 8 bit olup 00H–FEH arası geçerli başlık numarası verir; 256 değil 255 benzersiz başlık adreslenir). Her sektörde 512 bayt olduğundan:

$$1024 \times 255 \times 63 \times 512 = 8.422.686.720 \text{ bayt} \approx 7{,}844 \text{ GB}$$

Günümüzde piyasada 8 GB'lık sabit diskler bulunamamaktadır. Peki 8 GB üzerine erişim nasıl sağlanmaktadır? 1990'lı yıllarda bu INT 13H kesmesine ek bazı yordamlar (INT 13H EXT – Genişletilmiş) eklendi. Bu yordamlar sayesinde yüksek boyutlarda sabit diskler üzerinde işlemler yapılabilmektedir. Daha ayrıntılı bilgi için Ek D'ye bakınız.

Şimdi INT 13H alt yordamlarını kullanarak bir disket sürücü biçimlendiricisi yazalım. Tabii ki sürücü biçimlendiriciyi yazmadan önce disket sürücüyü, kullanacağımız alt yordamları ve biçimlendirme işlemlerini inceleyelim.

9.12.1 Disket Sürücü

Disket sürücü manyetik bir depolama birimidir; kutuplamalara göre 0 veya 1 (kuzey-güney) değerleri depolanmaktadır. Disket sürücüden bir okuma/yazma başlığı ile manyetik endüksiyon kullanılarak kutuplamalar okunabilir veya manyetikleştirme işlemleri ile veri yazılabilir.

Disketlerin içi pistlere ve sektörlere ayrılmışlardır. Pistler çevresel bölmelerdir. Sektörler ise disket üzerinde tanımlanan açısal bölmelerdir. Disketler bir veya iki yüzlü olabilirler. İki yüzlü olduklarında doğal olarak kapasiteleri ikiye katlanmaktadır. Bir pist ve sektörün kesişmesiyle oluşan bölmelere göreli sektörler denir.

Pistler dıştan başlayarak numaralandırılırlar. Toplam pist sayısı n ise en dıştaki pistin numarası 0, en içteki pistin numarası $n-1$'dir. m sayısı biçimlendirilmiş sektör sayısı ise sektörler 1'den başlayarak m'e kadar numaralandırılabilir.

Bu bilgiler sayesinde bir disketin kapasitesini rahatça hesaplayabiliriz:

$$\text{Disketin kapasitesi} = C \times n \times m \times A$$

- C: Disketteki yüz sayısı
- n: Bir yüzde bulunan pist sayısı
- m: Pist başına sektör sayısı
- A: Göreli sektör boyutu

Göreli sektör sayısını ise aşağıdaki şekilde hesaplayabiliriz:

$$\text{Göreli sektör sayısı} = C \times n \times m$$

Günümüzde piyasada bulunan disket sürücüler genellikle şu özelliklere sahiptir:

- 3"1/2 HD (High Density – Yüksek Yoğunluk) tipinde
- 2 yüzlü
- Pist başına 18 sektörlü
- Yüz başına 80 pistli
- Disket başına 2880 sektörlü
- 1440 kB kapasitesinde

Bu tip disketlerin tanımlama kodları F0H'dır.

9.12.2 INT 13H Alt Yordamları

Disket sürücüleri yönetmek için kullanabileceğimiz INT 13H alt yordamları aşağıdaki Tablo 9.9'da verilmiştir.

Tablo 9.9: *INT 13H alt yordamları.*

AH	İşlev
00H	Yeniden başlat. Disket sürücüyü başlatmak veya hata durumunda yeniden başlatmak için kullanılır. Girdi: DL (sürücü numarası). Çıktı: AH, AL ve CF (hata varsa CF=1).
01H	Sürücü durumunu oku. Girdi: DL (sürücü numarası). Çıktı: AH, AL ve CF.
02H	Disket sürücüyü oku. Girdi: AL (sektör sayısı), CH (silindir numarası alt 8 bit), CL (sektör numarası), DH (başlık numarası), DL (sürücü numarası), ES:BX (veri tamponu). Çıktı: AH, AL ve CF.
03H	Disket sürücüye yaz. Girdi parametreleri 02H ile aynıdır. Çıktı: AH ve CF.
04H	Disket sürücü sektör sınama. Girdi parametreleri 02H ile aynıdır. Çıktı: AH ve CF.
05H	Formatla. Pistleri formatlar. Girdi: AL (sektör sayısı), CH (pist numarası), DH (başlık numarası), DL (sürücü numarası), ES:BX (adres tamponu). Çıktı: AH ve CF.

İlerideki bölümlerde INT 13H kesmesinin kullanımını ayrıntılı olarak örnek yazılımlar ile göreceğiz. Aşağıda günümüzde yaygın olarak kullanılan FAT dosyalama sistemi hakkında bilgilendirici bir yazı bulunmaktadır. Disketlerde ve sabit disklerde verilerin nasıl düzenlendiği hakkında bilgi edinmek amacıyla okunabilir.

9.12.3 FAT Dosyalama Sistemi

Günümüzde kullanımda olan FAT dosyalama sistemleri (File Allocation Table – Dosya Ayırma Tablosu) FAT12, FAT16 ve FAT32'dir. Bu dosyalama sistemlerinin isimlerindeki numaralar tablolarında kullandıkları bit sayılarını göstermektedir. Tablo kullanılmasının nedeni kullanımda olan ve kullanımda olmayan disket bölgelerini ayırabilmek ve bu sayede verilerin üzerine yanlışlıkla başka verilerin yazılmasını engellemek, disketi düzenlemektir. Ayrıca bir de VFAT (Virtual FAT – Sanal FAT) adıyla geçen bir dosyalama sistemi daha vardır. Bu dosyalama sisteminde uzun dosya adları da kullanılabilmektedir.

Şimdi FAT dosyalama sisteminin bölümlerini inceleyelim.

Küme (Cluster) FAT biriminde bir grup sektör bir küme oluşturur. Sadece veri bölgelerinde bulunan sektörler kümelere katılabilirler. Dosyaların ve klasörlerin verileri bu kümelerin içinde tutulur. Her kümenin boyutu (içinde bulundurduğu sektör sayısı) önyükleme kaydında tutulur. Bu değer 1 ile 128 sektör arasında olabilir.

Önyükleme Kaydı (Boot Record) FAT12 ve FAT16 dosyalama sistemlerinde önyükleme kaydı için 1 sektör ayrılırken FAT32'de 32 sektör ayrılır. Önyükleme kaydının boyutu 1 sektördür ve 0000H sektöründe bulunur.

Dosya Ayırma Tablosu (File Allocation Table) FAT12 için 12 bit, FAT16 için 16 bit ve FAT32 için 32 bit uzunluğunda veri içeren bir tablodur. Dosya sisteminde hangi bölümlerin

hangi dosyalara ait olduğunu belirtir. Genellikle bir dosya ayırma tablosunun kopyası bazı hatalı durumlarda kurtarma yapabilmek için disket içinde tutulur. Dosya ayırma tablosunun birincisi 0001H adresinden başlar ve boyutu 9 sektördür. Birincisinin kopyası olan ikincisi 000AH adresinden başlar ve boyutu 9 sektördür.

Kök Dizini Kök dizininin diğer dizinlerden farkı "`.`" ve "`..`" girişlerini bulundurmamasıdır. FAT12 ve FAT16'da kök dizini dosya ayırma tablosunun hemen ardındadır. Kök dizini 0013H adresinden başlar ve boyutu 14 sektördür.

Veri Bölgesi Dosyaların ve klasörlerin verileri veri bölgesinde tutulur. Veri bölgesi kümelere ayrılmıştır ve her kümenin boyu birbirine eşit olmalıdır. Veri bölgesi 0033H adresinden başlar ve boyutu 2847 sektördür.

Klasörler ve Dosyalar Kök klasör dışındaki tüm klasörler FAT12 dosyalama sisteminde dosya gibi tutulurlar. Her dosyanın boyutu klasörde tutulurken klasörlerin boyutları herhangi bir yerde tutulmaz. Klasörlerin boyutları sahip olduğu küme sayısına bağlıdır ve küme boyutlarının tam katıdır. Bir klasöre yeni bir dosya eklendiyse ve klasörün kümesi dolmuş ise klasörün boyutu 1 küme genişletilir. Kök klasörün boyutu sabit olduğundan kök klasörde böyle bir genişletme durumu yoktur. Klasörler klasör girişlerine ayrılır. Her klasör girişinde klasörün içinde bulunan dosya veya klasörün adı, özellikleri, boyutu, tarihi, zamanı ve küme numarası bulunur ve klasör girişleri 32 bayt uzunluğundadır.

9. Bölüm Soruları

Alıştırma 9.1. CX yazmacında bir N değeri bulunduğunu varsayarak fibonacci(N) değerini AX yazmacına atan bir program yazın. Bu programın içinde TOPLA ve EKRANA_YAZDIR adlı iki altyordam bulunmalı ve yapılacak tüm toplama işlemleri TOPLA altyordamına yaptırılırken işlemin adımları EKRANA_YAZDIR alt yordamı ile ekrana yazdırılmalıdır. EKRANA_YAZDIR altyordamı yazılan her sayıdan sonra "*" karakterini ekrana basmalıdır (İpucu: ekrana veri yazdıran BIOS kesmesi INT 10H'dir. INT 10H'nin 0EH altyordamı AL'deki veriyi ekrana yazdırır).

Alıştırma 9.2. Klavyeden bir dizgi okuyan ve okunan dizginin palindrom olup olmadığını ekrana yazan bir program yazın. Program sonsuz döngüde sürekli dizgi sormalı, boş dizgi girildiğinde program sonlandırılmalıdır.

Alıştırma 9.3. Ekranda rastgele bir konumdan başlayıp çapraz ilerleyerek ekranın kenarlarından 45 derece açıyla seken yeşil renkte bir top gösteren 8086 çevirici dil programı yazın.

Alıştırma 9.4. Bir önceki sorudaki gibi hareket eden, kızıl, sarı ve mavi renkte 3 ayrı topun birbirinden bağımsız olarak farklı noktalardan başlamasını ve toplardan sarı olanın saatin tersi yönünde diğerlerinin saat yönünde dönerek hareket etmesini sağlayın.

Alıştırma 9.5. Aşağıda bir 8086 assembly kodunun başı ve sonu verilmiştir. Boşluk bırakılan yeri tamamlayarak `veri` ile tanımlanmış dizginin başından ve sonundan okunduğunda aynı olması durumunda (palindrom) ekrana `P`, olmaması durumunda ekrana `D` yazan bir program

yazın. Dizginin sonundaki $ karakteri dizginin sonunu göstermektedir ve dizginin içinde değildir.

```
1  org 0100h
2  jmp basla
3  veri db 'ey edip adanada pide ye$'
4  basla:
5  mov cx, 0
6  mov ax, cs
7  mov es, ax
8  mov ds, ax
9  mov si, 0102h
10 dec si
11 dongu:
12 inc si
13 inc cx
14 cmp [si], '$'
15 jne dongu
16
17 push cx
18
19 mov di, 0200h
20 tasi:
21 mov ax, [si]
22 mov [di], ax
23 inc di
24 dec si
25 loop tasi
26
27 mov di, 0201h
28 inc si
29 pop cx
30 dec cx
31
32 ;;;; KODUNUZ BU BOLGEYE YAZILMALIDIR
33 ;
34 ;
35 ;;;; KODUNUZ BU BOLGEYE YAZILMALIDIR
36
37 ayni:
38 mov ah, 0eh
39 mov al, 'P'
40 int 010h
41 jmp son
42
43 farkli:
44 mov ah, 0eh
45 mov al, 'D'
46 int 010h
47 son:
48 hlt
```

Alıştırma 9.6. Ekrana 100 kere "8086 ile kodlamayı öğrendim" yazan bir 8086 çevirici dil programı yazın.

Alıştırma 9.7. Çalıştığı zaman ekrana "Adınızı girin: " yazıp kullanıcıdan girdi bekleyen, klavyeden "Burak" girildiğinde ekrana "Bu işleri bırak" yazıp sonlanan, başka bir girdi olduğunda ise "Çalışmaya devam" yazarak yeniden başa dönen bir 8086 çevirici dil programı yazın.

Alıştırma 9.8. Klavyeden bir sayı (n) okuyan ve ekrana n adet satıra ilk satırda 1, ikinci satırda 2 ve n. satıra n tane yıldız yazan bir 8086 çevirici dil programı yazın. Program bir döngüyle kullanıcıdan sayı beklemeye devam edecek ve 'S' harfine basıldığında program sonlanacaktır.

Alıştırma 9.9. Bir sayı (n) okuyan ve ekrana uygun biçimde çıktı veren 8086 çevirici dil programını yazın.

Alıştırma 9.10. Çalıştırıldığında ekranın tam ortasından başlayarak ekranın üst ve alt sınırları arasında '*' karakterini dikey olarak hareket ettiren bir program yazın.

Alıştırma 9.11. Tek basamaklı bir sayı (n) okuyan ve ekrana uygun biçimde çıktı veren 8086 çevirici dil programı yazın. 'S' girildiğinde program sona ermelidir.

Alıştırma 9.12. Klavyeden bir sayı okuyup ekrana o sayı kadar yıldız yazan bir program yazın.

Alıştırma 9.13. CX yazmacındaki değeri kullanarak bu yazmacın değeri kadar karakteri çapraz olarak ekrana basan bir program yazın. Bir alt satıra geçmek için 0DH değerini kullanarak satır sonu karakterini ekrana basabilirsiniz. (Örnek: CX=5 ise her satırda bir sağa kayarak 5 yıldız.)

Alıştırma 9.14. Klavyeden '1' tuşuna basıldığında 5 kere, '2' tuşuna basıldığında 10 kere ekrana "Baver arabayı satma" yazan program yazın. 't' tuşuna basıldığında program sonlanmalı, diğer tuşlarda tepki vermemelidir.

Alıştırma 9.15. Klavyeden girilen tek basamaklı bir sayı değerini kullanarak bu değer kadar '*' karakterini yan yana basan ve her yıldız öbeğinin arasında bir boşluk karakteri bulunan bir program yazın. (Örnek: Klavyeden girilen değer 5 ise çıktı: ***** **** *** ** *.)

Alıştırma 9.16. Bir Dudeney sayısı basamakları toplamının küpü kendisine eşit olan sayma sayısıdır. Toplamda 6 adet Dudeney sayısı vardır ve bu sayıların tamamı 16 bit ile ifade edilebilecek büyüklüktedir. İşlemcinin giriş/çıkış kapısuna bağlanan bir giriş aygıtının FFE4H numaralı kapıdan onaltılık tabanda 3E8 ile 270F arasında bir sayı gönderdiğini varsayarak, sürekli FFE4H kapısını dinleyen ve girilen değerin Dudeney sayısı olup olmadığını FFE5H kapısından çıktı olarak veren bir program yazın. Sayı Dudeney sayısı ise 01H, değilse 00H çıktısı verilmelidir.

Alıştırma 9.17. Klavyeden 1 tuşuna basıldığında "Aferin Tunç", 2 tuşuna basıldığında "Haydi bakalım" yazan, diğer tuşlarda işlem yapmayan bir 8086 programı yazın. Program sonsuz döngüde dönmelidir.

Alıştırma 9.18. () ve [] ayraçlarından oluşan bir dizgi klavyeden girilecektir. Bu dizgi aşağıdaki koşulları sağladığında doğru kabul edilir: (a) Eğer dizgi boş ise doğrudur. (b) Eğer A ve B dizgileri doğru ise AB dizgisi doğrudur. (c) Eğer A dizgisi doğru ise (A) ve [A] dizgileri

doğrudur. Ayraçlardan oluşan bir dizgiyi klavyeden girdi olarak alıp bu dizginin doğru olup olmadığını belirleyen bir program yazın. Program her Enter tuşuna basıldığında girilen dizginin doğru ya da yanlış olduğunu kullanıcıya yazılı olarak bildirmelidir.

Alıştırma 9.19. Bellekte herhangi bir adreste art arda boşluklarla ayrılmış çok sayıda sözcük bulunmaktadır. Sözcük dizisinin sonunu `$` karakteri göstermektedir. Klavyeden girilen bir sözcüğün bu sözcüklerden kaç tanesiyle aynı olduğunu bulan ve sonucu ekranda görüntüleyen bir program yazın. (Örnek: Klavyeden "müdür" sözcüğü girilip Enter tuşuna basıldığında bellekte "müdür müdür müdür ?$" bilgisi saklanıyorsa ekrana 3 yazılmalıdır.)

10

Girdi/Çıktı İşlemleri

Bölüm 3'de mikroişlemcideki girdi/çıktı iğnelerinden bahsetmiştik. Bölüm 4'nde ise girdi/çıktı işlemlerinin hangi x86 komutlarıyla yapılabileceğini göstermiştik. Bu bölümde ise girdi/çıktı işlemlerinin nasıl yapılması gerektiği hakkında bilgi edineceğiz. Girdi/çıktı işlemlerini incelerken seri iletişim, paralel iletişim ve ekran kartına girdi/çıktı yapmayı göreceğiz.

10.1 Seri İletişim

Bir önceki bölümde seri iletişim için `INT 14H` kesmesini kullanmıştık. Şimdi ise seri iletişimi kesme kullanmadan, doğrudan girdi/çıktı komutları ile gerçekleyeceğiz. Bunun için bilgisayarda bulunan 8250 Seri İletişim Çipi'ne erişmemiz gerekmektedir.

10.2 8250

8250 çipinde mikroişlemcilerde olduğu gibi bazı yazmaçlar bulunmaktadır ve bizim yapmamız gereken bu yazmaçları denetlemektir. 8250 üzerinde 9 tane girdi/çıktı yazmacı bulunmaktadır. COM1 seri kapısının adresi `03F8H` ve COM2 seri kapısının adresi `02F8H`'dır.

Aşağıdaki tabloda COM1 ve COM2 seri kapıları için 8250 yazmaç adresleri ve yazmaçların işlevleri verilmiştir.

Tablo 10.1: *8250 yazmaç adresleri.*

Adres (COM1)	Adres (COM2)	İşlev
03F8H	02F8H	Veri alış-veriş yazmacı. (Baud Rate Divisor Latch yazmacı alt baytı olarak da kullanılır.) Bu yazmaç gerçekte biri veri girişi, diğeri de veri çıkışı için kullanılan 2 tane alt yazmaçtan oluşmaktadır.

Adres (COM1)	Adres (COM2)	İşlev
03F9H	02F9H	Kesme yönetim yazmacı. (Baud Rate Divisor Latch yazmacı üst baytı olarak da kullanılır.) 8250 çipi 4 tane kesme oluşturabilir: (1) Karakter alındı kesmesi (0. bit), (2) Karakter verildi kesmesi (1. bit), (3) İletişim hatası kesmesi (2. bit), (4) Durum değişikliği kesmesi (3. bit).
03FAH	02FAH	Kesme tanımlama yazmacı. Salt okunur olup oluşan kesme hakkında bilgi almak için kullanılır. 0. bit 0 ise kesme oluştuğu, 1 ise herhangi bir kesme oluşmadığı anlaşılır.
03FBH	02FBH	Hat denetim yazmacı. Seri iletişimin özellikleri bu yazmaç üzerinden belirlenir.
03FCH	02FCH	Durum denetim yazmacı.
03FDH	02FDH	Hat durum yazmacı. İletim durumu hakkında bilgi verir; salt okunurdur.
03FEH	02FEH	Modem durum yazmacı. Salt okunurdur; modem sinyalleri ve el sıkışma işlemi hakkında bilgi verir.
03FFH	02FFH	Shadow Receive Register (salt okunur; özgün PC'lerde bulunmaz).

10.2.1 Örnek

İlk örneğimizde COM1 kapısını ve loopback kullanarak seri iletişimi gerçekleyeceğiz. Seri iletişim kesmelerini kullanmayacağız.

```
        ORG 0100H
        PUSH CS
        PUSH CS
        POP DS
        POP ES
        CALL ILETISIM_AYARLA
        MOV AL,'A'
        CALL VERI_YOLLA
DINLE:
        CALL DURUM_OKU
        CMP AL,61H
        JE VERI_ALINABILIR
        CMP AL,60H
        JE VERI_ALINAMAZ
        CMP AL,40H
        JE VERI_ALINAMAZ
SON:
        INT 03H
VERI_ALINABILIR:
        CALL VERI_AL
        JMP SON
VERI_ALINAMAZ:
        JMP DINLE
; AL yazmacindaki veriyi yollar
```

```
25  VERI_YOLLA:
26          MOV DX,03F8H
27          OUT DX,AL
28          RET
29  ; Veriyi alir ve AL yazmacina kaydeder
30  VERI_AL:
31          MOV DX,03F8H
32          IN AL,DX
33          RET
34  ; Hat durumunu okur ve AL yazmacina kaydeder
35  DURUM_OKU:
36          MOV DX,03FDH
37          IN AL,DX
38          RET
39  ; Seri iletisimi ayarlar
40  ILETISIM_AYARLA:
41          MOV DX,03FBH
42          MOV AL,10000010B
43          OUT DX,AL
44          MOV DX,03F9H
45          MOV AL,04H
46          OUT DX,AL
47          DEC DX
48          MOV AL,17H
49          OUT DX,AL
50          MOV DX,03FBH
51          MOV AL,00000010B
52          OUT DX,AL
53          MOV DX,03FCH
54          MOV AL,00010011B
55          OUT DX,AL
56          RET
```

Kod 10.1: Örnek 10.1 – COM1 loopback seri iletişim

10.2.2 Paralel İletişim

Tablo 10.2: *Paralel port iğne işlevleri.*

İğne	İşlev
1	Veri bulunup bulunmadığını belirtir. Veri bulunduğunda 0, bulunmadığında 1 değerini alır. Çıkış olarak kullanılır.
2–9	8 bitlik veri yolu. 2. pin 0. veriyolu, 9. pin 7. veriyoludur. Çıkış olarak kullanılır.
10	Yollanan veri yerine ulaştığı zaman alıcı donanım tarafından gönderilen kabul sinyalini bulundurur. Kabul edildiğinde 0, edilmediğinde 1 değerindedir. Giriş olarak kullanılır.
11	Veri yollanacak donanım meşgul ise bu donanım tarafından 0 değeri verilir. Meşgul değilse 1 değerini alır. Giriş olarak kullanılır.

İğne	İşlev
12	Yazıcı tarafından kağıt bittiğinde 1 değeri verilir; aksi halde 0'dır. Giriş olarak kullanılır.
13	1 değerine sahip iken yazıcı seçilidir; aksi halde seçili değildir. Giriş olarak kullanılır.
14	0 olduğunda yazıcı otomatik olarak her CR'den sonra yeni satıra geçer. Çıkış olarak kullanılır.
15	0 olması yazıcıda hata oluştuğunu gösterir. Giriş olarak kullanılır.
16	50 ms'den uzun süre 0 olduğunda yazıcı kendini hazırlar. Çıkış olarak kullanılır.
17	1 olduğunda yazıcı kendini kapar. Çıkış olarak kullanılır.
18–25	Toprak.

Paralel port adresleri bilgisayara göre değişiklik gösterebilir. Tipik adresler şunlardır: `0378H` (en yaygın LPT1 adresi), `0278H` (LPT2) ve `03BCH` (MDA kartına bağlı paralel port). Aşağıdaki tabloda LPT yazmaç adresleri `0378H` üzerinden gösterilmektedir; farklı bir adres kullanılıyorsa buna göre uyarlanmalıdır.

Tablo 10.3: *LPT yazmaç adresleri (temel adres `0378H` için).*

Adres (LPT1, temel=0378H)	İşlev
0378H	Veri yazmacı. Okunabilir veya yazılabilir.
0379H	Durum yazmacı.
037AH	Denetim yazmacı.

Bilgi Notu 10.1: El sıkışma protokolü

Paralel iletişimde el sıkışma (handshaking) protokolü şu adımlardan oluşur:
Alıcı taraf:

1. Meşgul hattını kapat.
2. Tetikleme hattı açılana kadar bekle.
3. Veriyi veri hattından oku.
4. Kabul hattını aç.
5. Tetikleme hattı kapanana kadar bekle.
6. Kabul hattını kapat.
7. Yukarıdaki adımları her bayt için tekrarla.

Verici taraf:

1. Meşgul hattını denetle; meşgul ise kapanana kadar bekle.
2. Veriyi veri hattından yolla.
3. Tetikleme hattını aç.
4. Kabul hattı açılana kadar bekle.
5. Tetikleme hattını kapat.
6. Kabul hattı kapanana kadar bekle.
7. Yukarıdaki adımları her bayt için tekrarla.

10.2.3 Tek Yönlü İletişim

Tek yönlü iletişimde veri yalnızca bir yönde aktarılır; alıcı ve verici rolleri sabittir. Paralel port üzerinden tek yönlü iletişimde verici veri baytını veri yazmaçlarına yazar ve el sıkışma protokolüyle alıcıya iletir.

10.2.4 Vidyo Belleği Girdi/Çıktı İşlemleri

Aşağıdaki tabloda 20×5 piksellik bir görüntüde piksel numaralandırması gösterilmektedir.

Tablo 10.4: *20×5 piksel ızgara numaralandırması.*

Y\X	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
1	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39
2	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59
3	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79
4	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99

Yazı modunda vidyo belleği `B800H:0000H` adresinden başlar. Her karakter iki baytla temsil edilir: ilk bayt ASCII karakter kodu, ikinci bayt ise öznitelik baytıdır.

```
1        MOV AX,0B800H
2        MOV ES,AX
3        MOV AH,0FH
```

```
4        MOV AL,61H
5        MOV SI,0000H
6        ES MOV [SI],AX
```

Kod 10.2: Örnek 10.2 – Yazı modunda vidyo belleğine doğrudan erişim

10.2.5 BIOS Vidyo Kipleri

`INT 10H` kesmesinin `00H` alt yordamı ile vidyo kipi değiştirilebilir; AL yazmacına aşağıdaki tablodaki değerler yazılır.

Tablo 10.5: *`INT 10H` vidyo kipleri.*

AL Değeri	Vidyo Kipi	Çözünürlük	Renk Sayısı
01H	Yazı	40×25	16
03H	Yazı	80×25	16
04H	CGA	320×200	4
06H	CGA	640×200	2
0DH	EGA	320×200	16
0EH	EGA	640×200	16
10H	EGA	640×350	16
11H	VGA	640×480	2 (tek renkli)
12H	VGA	640×480	16
13H	VGA	320×200	256

Yazı modunda öznitelik baytının bit anlamları:

Tablo 10.6: *Yazı modu öznitelik baytı bit anlamları.*

Bit	Anlam
7	1 ise yazı yanıp söner; 0 ise sabittir.
6–4	Arka plan rengi.
3–0	Yazı rengi.

VESA SVGA vidyo kipleri `INT 10H` kesmesinin `4F02H` alt yordamı ile seçilir; BX yazmacına aşağıdaki tablodaki değer yazılır.

Tablo 10.7: *VESA SVGA vidyo kipleri.*

BX Değeri	Vidyo Kipi	Çözünürlük	Renk
0100H	Grafik	640×400	256
0101H	Grafik	640×480	256
0102H	Grafik	800×600	16
0103H	Grafik	800×600	256
0104H	Grafik	1024×768	16

BX Değeri	Vidyo Kipi	Çözünürlük	Renk
0105H	Grafik	1024×768	256
0106H	Grafik	1280×1024	16
0107H	Grafik	1280×1024	256
0108H	Yazı	80×60	16
0109H	Yazı	132×25	16
010AH	Yazı	132×43	16
010BH	Yazı	132×50	16
010CH	Yazı	132×60	16
010DH	Grafik	320×200	32768
010EH	Grafik	320×200	65536
010FH	Grafik	320×200	16,7 milyon
0110H	Grafik	640×480	32768
0111H	Grafik	640×480	65536
0112H	Grafik	640×480	16,7 milyon
0113H	Grafik	800×600	32768
0114H	Grafik	800×600	65536
0115H	Grafik	800×600	16,7 milyon
0116H	Grafik	1024×768	32768
0117H	Grafik	1024×768	65536
0118H	Grafik	1024×768	16,7 milyon
0119H	Grafik	1280×1024	32768
01AH	Grafik	1280×1024	65536
01BH	Grafik	1280×1024	16,7 milyon

Tablo 10.8: *VESA `INT 10H` işlevleri.*

AX Değeri	İşlev
4F00H	SVGA bilgilerini çıktı olarak verir.
4F01H	SVGA vidyo kipi bilgilerini çıktı olarak verir.
4F02H	SVGA veya VGA vidyo kiplerini ayarlamak için kullanılır; BX yazmacına vidyo kipinin kodu girilmelidir.
4F03H	Kullanımdaki SVGA veya VGA vidyo kipini BX yazmacında çıktı olarak verir.
4F04H	SVGA vidyo durumunu kaydetme veya geri yüklemede kullanılır.
4F05H	Bellek penceresi seçimi. Yüksek çözünürlüklü grafik kiplerinde 64 KB'lık bellek pencerelerini yönetmek için kullanılır.
4F06H	Lojik ekran genişliğini ayarlamaya yarar.
4F07H	Lojik ekranın gerçek ekranda nerede başlayacağını ayarlamak için kullanılır.
4F08H	Kullanımdaki paletin bit genişliğini almak veya değiştirmek için kullanılır.

10.2.6 Vidyo Kartı Yazmaçları

VGA kartı, doğrudan girdi/çıktı komutlarıyla erişilebilen çeşitli yazmaç gruplarından oluşur. Bu yazmaçlar genellikle bir dizin (index) yazmacı ve bir veri yazmacı çiftiyle denetlenir: önce dizin yazmacına erişmek istenen iç yazmaç numarası yazılır, ardından veri yazmacından okunur veya yazılır.

Tablo 10.9: *VGA yazmaç grupları ve temel girdi/çıktı adresleri.*

Grup	Dizin / Veri Adresi	İşlev
Genel yazmaçlar	03C2H / 03CCH	Giriş/çıkış adresi seçimi, senkronizasyon
Sıralayıcı	03C4H / 03C5H	Bellek erişim kipini ve yazma düzlemlerini denetler
CRT Denetleyicisi	03D4H / 03D5H	Ekran tarama parametrelerini ayarlar
Grafik Denetleyicisi	03CEH / 03CFH	Bit düzlemi seçimi ve çizim kipini ayarlar
Öznitelik Denetleyicisi	03C0H / 03C1H	Palet rengi ve vidyo kipi özniteliklerini ayarlar
DAC Palet	03C8H / 03C9H	256 renk paletini RGB değerleriyle programlar

VGA'nın doğrudan yazmaç düzeyinde programlanması, BIOS'un sunmadığı ince ayar imkânı sağlar. Ancak bu işlem karttan karta değişkenlik gösterebileceğinden taşınabilirlik sorunu yaratabilir. Günümüzde bu düzeyde programlama yerine donanım soyutlama katmanları (HAL) veya grafik sürücüleri tercih edilmektedir.

10.2.7 Dikey Tekrar Bekleme

Vidyo belleğine yazmadan önce dikey tekrar (vertical sync) sinyalini beklemek ekranda titreme oluşmasını engeller. `03DAH` adresindeki durum yazmacı bu amaçla kullanılır.

```
DIKEY_TEKRAR_BEKLE:
        MOV DX,03DAH
DONGU1:
        IN AL,DX
        TEST AL,08H
        JNZ DONGU1
DONGU2:
        IN AL,DX
        TEST AL,08H
        JZ DONGU2
        RET
```

Kod 10.3: Örnek 10.3 – Dikey tekrar bekleme yordamı

10.2.8 Yazı Modunda Metin Yazdırma

```
        ORG 0100H
        PUSH CS
        POP DS
        JMP BASLA

VIDYOKIPI  DB 01H
M1_B       DB 'MERHABA DUNYA!'
M1_S       DB ' '

BASLA:
        MOV AX,0B800H
        MOV ES,AX
        CALL VIDYO_KIPI_SEC
        MOV CX,M1_S - M1_B
        MOV SI,0FFFEH
        MOV AH,0FH
        MOV DI,M1_B -1

DONGU:
        INC SI
        INC SI
        INC DI
        MOV AL,[DI]
        ES MOV [SI],AX
        LOOP DONGU

BEKLE:
        MOV AH,1
        INT 16H
        JZ BEKLE
        MOV AH,00H
        INT 16H
        CMP AL,61H
        JE SON
        JMP BEKLE
SON:
        CALL VIDYO_KIPI_SEC
        INT 03H
```

```
VIDYO_KIPI_SEC:
        MOV AH,0FH
        INT 10H
        MOV BL,AL
        MOV AH,00H
        MOV AL,[VIDYOKIPI]
        MOV [VIDYOKIPI],BL
        INT 10H
        RET
```

Kod 10.4: Örnek 10.4 – Yazı modunda metin yazdırma

10.2.9 VGA Grafik Modunda Çizim

VGA grafik kipinde (`13H`, 320×200, 256 renk) vidyo belleği `A000H:0000H` adresinden başlar.

```
        ORG 0100H
        PUSH CS
        POP DS
        JMP BASLA

VIDYOKIPI DB 13H

BASLA:
        MOV AX,0A000H
        MOV ES,AX
        CALL VIDYO_KIPI_SEC
        MOV CX,0C8H
        MOV AL,66H
        MOV SI,0000H
        MOV DI,013FH

DONGU:
        ES MOV BYTE [SI],AL
        ADD SI,0141H
        ES MOV BYTE [DI],AL
        ADD DI,013FH
        LOOP DONGU

BEKLE:
        MOV AH,1
        INT 16H
        JZ BEKLE
        MOV AH,00H
        INT 16H
        CMP AL,61H
        JE SON
        JMP BEKLE
SON:
        CALL VIDYO_KIPI_SEC
        INT 03H

VIDYO_KIPI_SEC:
        MOV AH,0FH
```

```
        INT 10H
        MOV BL,AL
        MOV AH,00H
        MOV AL,[VIDYOKIPI]
        MOV [VIDYOKIPI],BL
        INT 10H
        RET
```

Kod 10.5: Örnek 10.5 – VGA 320×200 grafik modunda X şekli çizme

```
        ORG 0100H
        PUSH CS
        POP DS
        JMP BASLA

VIDYOKIPI DB 13H

BASLA:
        MOV AX,0A000H
        MOV ES,AX
        CALL VIDYO_KIPI_SEC
        MOV CX,0C8H
        MOV AL,66H
        MOV SI,0000H
        MOV DI,013FH

DONGU:
        ES MOV BYTE [SI],AL
        ADD SI,0141H
        ES MOV BYTE [DI],AL
        ADD DI,013FH
        LOOP DONGU

        MOV CX,1111H
LP1:
        PUSH CX
        MOV CX,0FFFFH
LP2:
        LOOP LP2
        POP CX
        LOOP LP1

        CALL DIKEY_TEKRAR_BEKLE
        MOV DX,03C8H
        MOV AL,66H
        OUT DX,AL
        INC DX
        MOV AL,0FFH
        OUT DX,AL
        MOV AL,00H
        OUT DX,AL
        MOV AL,00H
        OUT DX,AL

BEKLE:
        MOV AH,1
```

```
47          INT 16H
48          JZ BEKLE
49          MOV AH,00H
50          INT 16H
51          CMP AL,61H
52          JE SON
53          JMP BEKLE
54  SON:
55          CALL VIDYO_KIPI_SEC
56          INT 03H
57
58  DIKEY_TEKRAR_BEKLE:
59          MOV DX,03DAH
60  DONGU1:
61          IN AL,DX
62          TEST AL,08H
63          JNZ DONGU1
64  DONGU2:
65          IN AL,DX
66          TEST AL,08H
67          JZ DONGU2
68          RET
69
70  VIDYO_KIPI_SEC:
71          MOV AH,0FH
72          INT 10H
73          MOV BL,AL
74          MOV AH,00H
75          MOV AL,[VIDYOKIPI]
76          MOV [VIDYOKIPI],BL
77          INT 10H
78          RET
```

Kod 10.6: Örnek 10.6 – VGA palet yazmacı ile renk değiştirme

```
1           ORG 0100H
2           PUSH CS
3           POP DS
4           JMP BASLA
5
6   VIDYOKIPI DB 13H
7
8   BASLA:
9           MOV AX,0A000H
10          MOV ES,AX
11          CALL VIDYO_KIPI_SEC
12          MOV CX,0C8H
13          MOV AL,00H
14          MOV SI,009BH
15
16  DONGU_DIKEY:
17          PUSH CX
18          MOV CX,0AH
19  DONGU_YATAY:
20          ES MOV BYTE [SI],AL
21          INC SI
```

```
22          LOOP DONGU_YATAY
23          POP CX
24          INC AL
25          ADD SI,0136H
26          CALL GECIKME
27          LOOP DONGU_DIKEY
28
29  BEKLE:
30          MOV AH,1
31          INT 16H
32          JZ BEKLE
33          MOV AH,00H
34          INT 16H
35          CMP AL,61H
36          JE SON
37          JMP BEKLE
38  SON:
39          CALL VIDYO_KIPI_SEC
40          INT 03H
41
42  GECIKME:
43          PUSH CX
44          MOV CX,0111H
45  LP1:
46          PUSH CX
47          MOV CX,0FFFFH
48  LP2:
49          LOOP LP2
50          POP CX
51          LOOP LP1
52          POP CX
53          RET
54
55  VIDYO_KIPI_SEC:
56          MOV AH,0FH
57          INT 10H
58          MOV BL,AL
59          MOV AH,00H
60          MOV AL,[VIDYOKIPI]
61          MOV [VIDYOKIPI],BL
62          INT 10H
63          RET
```

Kod 10.7: Örnek 10.7 – VGA dikey renk bantları animasyonu

Alıştırmalar

1. **8250 baud hızı hesabı:** 8250 çipinde baud hızı bölücü (divisor) değeri şu formülle hesaplanır: bölücü = 115200/baud hızı. Aşağıdaki baud hızları için bölücü değerlerini onaltılık olarak hesaplayınız ve 03F8H/03F9H çiftine nasıl yazılacağını gösteriniz:
   (a) 9600 baud
   (b) 19200 baud

   (c) 38400 baud

2. **Seri iletişim çerçeve yapısı:** “8N1” ve “7E2” parametrelerinin her birinin anlamını açıklayınız (veri biti sayısı, eşlik biti türü, duraklama biti sayısı). Her parametreye göre 03FBH hat denetim yazmacına yazılması gereken değeri ikili sayı olarak gösteriniz.
3. **Paralel port veri aktarımı:** LPT1 üzerinden `0101_1010B` baytını bir yazıcıya göndermek istiyorsunuz. El sıkışma protokolünü kullanarak veriyi göndermek için gereken adımları BIOS kullanmadan, doğrudan 0378H veri ve 037AH denetim yazmaçlarına erişerek kısaca açıklayınız.
4. **VGA kipi seçimi:** INT 10H kesmesini kullanarak:
   (a) 80×25 karakterlik yazı kipine geçen,
   (b) 320×200, 256 renkli grafik kipine geçen

   iki ayrı kod parçacığı yazınız.
5. **Vidyo belleği adresi (13H kipi):** VGA 13H kipinde (320×200, 256 renk, A000H:0000H) (x, y) koordinatındaki pikselin vidyo belleğindeki göreli adresini veren formülü yazınız. $(x, y) = (100, 80)$ koordinatı için adresi hesaplayınız.
6. **Piksel rengi belirleme:** 13H kipinde A000H segmentinde çalışırken $(50, 30)$ koordinatına renk kodu `0FH` (beyaz) olan bir piksel yazan assembly kod parçacığı yazınız.
7. **Yazı modu öznitelik baytı:** Yazı modunda öznitelik baytının bit yapısını açıklayınız. Aşağıdaki görünümlere karşılık gelen öznitelik bayt değerlerini onaltılık olarak hesaplayınız:
   (a) Mavi arka plan üzerine beyaz yazı (sabit)
   (b) Siyah arka plan üzerine kırmızı yazı (yanıp sönen)
8. **Vidyo belleği organizasyonu:** Yazı modunda (B800H:0000H) 80×25 karakterlik ekranda (c, s) sütun ve satır koordinatındaki karakterin karakter baytının vidyo belleğindeki göreli adresini veren formülü yazınız. $(c, s) = (20, 10)$ koordinatı için adres değerini hesaplayınız.

11

Basit Bir İşletim Sistemi

Bu bölümde basit bir işletim sistemi yazacağız. Yazmaya başlamadan önce işletim sistemleri hakkında kısa bir bilgi edinmemiz gerekmektedir.

11.1 İşletim Sistemi Nedir?

İşletim sistemi bilgisayarın açıldıktan sonra belleğe yüklenen ve bilgisayar donanımının yönetimi ile yazılımların çalışmasını sağlayacak ortamın hazırlanmasında görevli özel bir yazılımdır.

Ayrıca işletim sistemi yazılımların belleğe ve girdi/çıktı donanımına erişimini sağlamakla sorumludur. Bu görevini her zaman yapmakla yükümlü değildir; girdi/çıktı işlemlerini serbest de bırakabilir.

11.2 Basit İşletim Sistemimiz

İşletim sistemimiz aşağıdaki parçalardan oluşacak:

- Önyükleme yazılımı
- Genel menü yazılımı
- Hesap makinası yazılımı
- Basit bir metin düzenleyici yazılım
- Seri iletişim yazılımları
- 1024×768 piksel çözünürlük ve 24 bit renk vidyo kipinde ekrana resim çizdirme yazılımı

İşletim sistemimiz ve yazdığımız programlar diskete kaydedilecektir. İlk olarak önyükleme programı belleğe yüklenecek ve ardından önyükleme kodumuz sayesinde genel menü yazılımımız belleğe yüklenip çalıştırılacaktır.

Önyükleme yazılımımız ilk yüzün, ilk pistinin, ilk sektöründe bulunacaktır. BIOS disketin ilk sektörünü okuyup `07C0H:0000H` bellek adresine kaydeder ve bu adresten başlayarak kodu çalıştırır. Önyükleme programımız bazı ayarlamaları yaptıktan sonra disketten genel menü programını okutup belleğin `1000H:0000H` adresine kaydedecek ve bu adrese atlayacaktır.

Genel menü yazılımımız disketin ikinci sektöründe bulunacaktır. Diğer yazılımları disketten okutup belleğe yükletip çalıştırabileceğiz. Genel menü yazılımı disketten okuduğu yazılımları `1000H:0420H` adresine kaydedecek ve buraya atlayacaktır. Her yazılım sonlandığında geri genel menü yazılımına (`1000H:0000H` adresine) atlayacaktır.

Hesap makinası yazılımımız disketin dördüncü sektöründe bulunacaktır. `W`, `A`, `S`, `D` ve `ENTER` tuşları ile ekrandan sayılar seçilebilecek ve toplanabilecektir.

Metin düzenleyici yazılımımız disketin beşinci sektöründe bulunacaktır. Yazılım açıldığında boş bir sayfa açılacak ve bu sayfaya klavyeden karakterler girilebilecektir.

Ekrana resim çizdirme yazılımımız disketin altıncı sektöründe bulunacaktır. Çizdireceği resim disketin ikinci yüzünün ilk pistinin ilk sektöründen başlayarak diskette saklanacaktır.

Yazılımların hepsini diskete yazdırmak için DOS ortamında çalışan bir `.com` yazılımı yazmamız gerekmektedir. Bu yazılım `.bin` dosyalarımızı kullanarak disketimizi dolduracaktır.

11.2.1 Disket Yükleyici

Disket yükleyici yazılımımızın kodu aşağıda bulunmaktadır. Yazılımlarımızın dosya adlarının aşağıdaki tabloyla uyumlu olması gerekmektedir.

Tablo 11.1: *İşletim sistemi dosyaları.*

Dosya Adı	Dosya Tanım
p0.bin	Önyükleme yazılımı
p1.bin	Genel menü yazılımı
p2.bin	Hesap makinası yazılımı
p3.bin	Metin düzenleyici yazılımı
p4.bin	Resim gösterici yazılımı
p5.bin	Seri iletişim verici yazılımı
p6.bin	Seri iletişim alıcı yazılımı
resim.bin	Resim verisi

Dosyaların `C:` sürücüsünde bulunması gerekmektedir. Gerekli açıklamalar yorum olarak kodun içinde yapıldığından burada fazladan açıklama yapmaya gerek yoktur.

```
        ORG 0100H
        JMP BASLANGIC
DOSYA1  DB "C:\resim.bin", 00H
DOSYA2  DB "C:\p0.bin", 00H
DOSYA3  DB "C:\p1.bin", 00H
DOSYA4  DB "C:\p2.bin", 00H
DOSYA5  DB "C:\p3.bin", 00H
DOSYA6  DB "C:\p4.bin", 00H
DOSYA7  DB "C:\p5.bin", 00H
DOSYA8  DB "C:\p6.bin", 00H
DOSYAIMLECI DW 0000H
GECICI   DW 0000H
MESAJ1  DB "* Diskette resim dosyasi yazildi.          *",0AH,0DH,00H
MESAJ2  DB "* On yukleme kodu diskete yazildi.         *",0AH,0DH,00H
MESAJ3  DB "* Genel menu kodu diskete yazildi.         *",0AH,0DH,00H
MESAJ4  DB "* Hesap makinasi kodu diskete yazildi.     *",0AH,0DH,00H
MESAJ5  DB "* Metin duzenleyici kodu diskete yazildi.  *",0AH,0DH,00H
```

```
18  MESAJ6  DB "* Resim gosterme kodu diskete yazildi.     *",0AH,0DH,00H
19  MESAJ7  DB "* Seri iletisim alici kodu diskete yazildi.*",0AH,0DH,00H
20  MESAJ8  DB "*Seri iletisim verici kodu diskete yazildi.*",0AH,0DH,00H
21  TAMPON  DB 512 DUP(00H)
22
23  BASLANGIC:
24          PUSH CS
25          POP DS
26          PUSH CS
27          POP ES
28  ; Resim dosyasini diskete yaz
29          MOV CL,01H
30          MOV CH,00H
31          PUSH CX
32          MOV SI,DOSYA1
33          CALL DOSYA_AC
34  DONGU1:
35          MOV AH,3FH
36          MOV DX,TAMPON
37          MOV CX,0200H
38          MOV SI,DOSYAIMLECI
39          MOV BX,[SI]
40          INT 21H
41          JC HATA3
42          CMP AX,CX
43          JNE DOSYA_SONU
44          POP CX
45          MOV AH,03H
46          MOV AL,01H
47          MOV DH,01H
48          MOV DL,00H
49          MOV BX,TAMPON
50          INT 13H
51          JC HATA4
52          INC CL
53          CMP CL,13H
54          JNE DEVAM
55          INC CH
56          MOV CL,01H
57  DEVAM:
58          PUSH CX
59          JMP DONGU1
60  DOSYA_SONU:
61          CALL DOSYA_KAPAT
62          MOV SI,MESAJ1
63          CALL SATIR_YAZ
64  ; Onyukleme kodunu diskete yaz
65          MOV CL,01H
66          MOV CH,00H
67          MOV SI,GECICI
68          MOV [SI],CX
69          MOV SI,DOSYA2
70          CALL DOSYA_AC
71          CALL DISKETE_YAZ
72          MOV SI,MESAJ2
73          CALL SATIR_YAZ
```

```
74  ; Genel Menu yazilimini diskete yaz
75          MOV CL,02H
76          MOV CH,00H
77          MOV SI,GECICI
78          MOV [SI],CX
79          MOV SI,DOSYA3
80          CALL DOSYA_AC
81          CALL DISKETE_YAZ
82          MOV SI,MESAJ3
83          CALL SATIR_YAZ
84  ; ...diger dosyalar benzer sekilde yazilir...
85          JMP SON
86
87  DISKETE_YAZ:
88  DONGU2:
89          MOV AH,3FH
90          MOV DX,TAMPON
91          MOV CX,0200H
92          MOV SI,DOSYAIMLECI
93          MOV BX,[SI]
94          INT 21H
95          JC HATA3
96          CMP AX,CX
97          JNE DOSYA_SONU2
98          MOV SI,GECICI
99          MOV CX,[SI]
100         MOV AH,03H
101         MOV AL,01H
102         MOV DH,00H
103         MOV DL,00H
104         MOV BX,TAMPON
105         INT 13H
106         JC HATA4
107         INC CL
108         CMP CL,13H
109         JNE DEVAM2
110         INC CH
111         MOV CL,01H
112 DEVAM2:
113         MOV SI,GECICI
114         MOV [SI],CX
115         JMP DONGU2
116 DOSYA_SONU2:
117         CALL DOSYA_KAPAT
118         RET
119
120 DOSYA_KAPAT:
121         MOV AH,3EH
122         MOV SI,DOSYAIMLECI
123         MOV BX,[SI]
124         INT 21H
125         JC HATA2
126         RET
127
128 DOSYA_AC:
129         MOV AH,3DH
```

```
130         MOV AL,00H
131         MOV DX,SI
132         INT 21H
133         JC HATA1
134         MOV SI,DOSYAIMLECI
135         MOV [SI],AX
136         RET
137
138 HATA1:  MOV AL,31H
139         CALL EKRANA_YAZDIR
140         JMP SON
141 HATA2:  MOV AL,32H
142         CALL EKRANA_YAZDIR
143         JMP SON
144 HATA3:  MOV AL,33H
145         CALL EKRANA_YAZDIR
146         JMP SON
147 HATA4:  MOV AL,34H
148         CALL EKRANA_YAZDIR
149         JMP SON
150
151 EKRANA_YAZDIR:
152         PUSH AX
153         MOV AH,0EH
154         INT 10H
155         POP AX
156         RET
157
158 SON:
159         MOV AX,4C00H
160         INT 21H
161
162         INCLUDE 'yordamlar.inc'
```

Kod 11.1: Örnek 11.1 – Disket yükleyici yazılımı

11.2.2 Önyükleme Yazılımı

Önyükleme yazılımımızın disketin ilk sektöründe bulunması gerekir. Bilgisayar açıldığında disket sürücüde disketten önyükleme için gerekli BIOS ayarları yapılmışsa disketin ilk sektörü okunur ve `07C0H:0000H` adresine kaydedilir.

Önyükleme programımız sırasıyla şu işlemleri yapacaktır:

1. Vidyo kipini ayarla.
2. Bir açılış mesajı göster.
3. Bir süre bekle.
4. Disketin birinci sektörünü oku ve `1000H:0000H` adresine kaydet.
5. `1000H:0000H` adresine atla.

```
1         ORG 0000H
2         JMP FAR 07C0H:BASLANGIC
3
4 ; DEGISKEN TANIMLAMALARI
```

```
5   M_ACILIS0 DB "***********************************",0AH,0DH,00H
6   M_ACILIS1 DB "* BASIT ISLETIM SISTEMI - 1. ASAMA *",0AH,0DH,00H
7   M_ACILIS2 DB "*                                 *",0AH,0DH,00H
8   M_ACILIS3 DB "* HOSGELDINIZ!                    *",0AH,0DH,00H
9
10  BASLANGIC:
11          PUSH CS
12          POP DS
13  ; Vidyo kipini degistir
14          MOV AH,00H
15          MOV AL,03H
16          INT 10H
17  ; Acilis mesajlarini yaz
18          MOV SI,M_ACILIS0
19          CALL SATIR_YAZ
20          MOV SI,M_ACILIS1
21          CALL SATIR_YAZ
22          MOV SI,M_ACILIS2
23          CALL SATIR_YAZ
24          MOV SI,M_ACILIS2
25          CALL SATIR_YAZ
26          MOV SI,M_ACILIS3
27          CALL SATIR_YAZ
28          MOV SI,M_ACILIS0
29          CALL SATIR_YAZ
30  ; Bir sure bekle
31          MOV CX,0FFFFH
32          CALL GECIKME
33          MOV CX,0FFFFH
34          CALL GECIKME
35          MOV CX,0FFFFH
36          CALL GECIKME
37  ; Diskette sektor 1'i 1000H:0000H adresine yukle
38          MOV AH,02H
39          MOV AL,02H
40          MOV CH,00H
41          MOV CL,02H
42          MOV DH,00H
43          MOV DL,00H
44          MOV BX,1000H
45          MOV ES,BX
46          MOV BX,0000H
47          INT 13H
48  ; Genel menu programina atla
49          JMP 1000H:0000H
50
51          DB 510 - ($ - $$) DUP (00H)  ; Dolgu baytlari (512 baytlik sektoru doldurur)
52          DB 55H, 0AAH                 ; Onyukleme imzasi (son 2 bayt)
53
54          INCLUDE 'yordamlar.inc'
```

Kod 11.2: Örnek 11.2 – Önyükleme yazılımı (p0.bin)

Bilgi Notu 11.1: Önyükleme İmzası

BIOS, bir disketin önyüklenebilir olup olmadığını anlamak için ilk sektörün son iki baytını denetler. Bu baytlar sırasıyla `55H` ve `0AAH` olmalıdır. Bu imza bulunmazsa BIOS disket önyüklemesini reddeder ve bir sonraki önyükleme aygıtına geçer.

`yordamlar.inc` dosyasında tanımlı iki yordam bulunmaktadır:

```
; SATIR_YAZ
; [SI] adresindeki dizgiyi 00H degerine ulasana kadar ekrana yazdirir
SATIR_YAZ:
MESAJ_YAZ_HARF_AL:
        MOV AL,[SI]
        INC SI
        OR AL,AL
        JZ MESAJ_YAZ_SON
        MOV BX,00H
        MOV AH,0EH
        INT 10H
        JMP MESAJ_YAZ_HARF_AL
MESAJ_YAZ_SON:
        RET

; GECIKME
; CX yazmacininin degeri kadar gecikme olusturur
GECIKME:
GECIKME_LP1:
        PUSH CX
        MOV CX,0FFFFH
GECIKME_LP2:
        LOOP GECIKME_LP2
        POP CX
        LOOP GECIKME_LP1
GECIKME_SON:
        RET
```

Kod 11.3: Örnek 11.3 – yordamlar.inc içeriği

11.2.3 Genel Menü Yazılımı

Genel menü yazılımımız disketin ikinci ve üçüncü sektöründe bulunmaktadır. Önyükleme yazılımı tarafından `1000H:0000H` bellek adresine kaydedilmekte ve denetim bu yazılıma geçirilmektedir.

```
        ORG 0000H
        PUSH CS
        POP DS
        JMP BASLANGIC
M_MENU0  DB "****************************************",0AH,0DH,00H
M_MENU1  DB "*- MENU--------------------------------*",0AH,0DH,00H
M_MENU2  DB "*1 - MENU'YU YENIDEN YAZ               *",0AH,0DH,00H
M_MENU3  DB "*2 - HESAP MAKINASI YAZILIMI           *",0AH,0DH,00H
M_MENU4  DB "*3 - METIN DUZENLEYICI YAZILIMI        *",0AH,0DH,00H
M_MENU5  DB "*4 - EKRANA RESIM CIZDIRME YAZILIMI    *",0AH,0DH,00H
```

```
M_MENU6  DB "*5 - SERI ILETISIM - ALICI YAZILIMI     *",0AH,0DH,00H
M_MENU7  DB "*6 - SERI ILETISIM - VERICI YAZILIMI    *",0AH,0DH,00H
M_MENU8  DB "*7 - BILGISAYARI YENIDEN BASLAT         *",0AH,0DH,00H
M_MENU9  DB "*8 - BILGISAYARI KAPAT                  *",0AH,0DH,00H
M_MENU10 DB "*----------------------------------------*",0AH,0DH,00H
M_MESAJ1 DB "*Bilgisayar Kapatiliyor...*",0AH,0DH,00H
M_MESAJ2 DB "*Bilgisayar Yeniden Baslatiliyor...*",0AH,0DH,00H

BASLANGIC:
        MOV AH,00H
        MOV AL,03H
        INT 10H

        MOV SI,M_MENU0
        CALL SATIR_YAZ
        MOV SI,M_MENU1
        CALL SATIR_YAZ
        MOV SI,M_MENU2
        CALL SATIR_YAZ
        MOV SI,M_MENU3
        CALL SATIR_YAZ
        MOV SI,M_MENU4
        CALL SATIR_YAZ
        MOV SI,M_MENU5
        CALL SATIR_YAZ
        MOV SI,M_MENU6
        CALL SATIR_YAZ
        MOV SI,M_MENU7
        CALL SATIR_YAZ
        MOV SI,M_MENU8
        CALL SATIR_YAZ
        MOV SI,M_MENU9
        CALL SATIR_YAZ
        MOV SI,M_MENU10
        CALL SATIR_YAZ
        MOV SI,M_MENU0
        CALL SATIR_YAZ

KLAVYE_OKU:
        MOV AH,01H
        INT 16H
        JZ KLAVYE_OKU
        MOV AH,00H
        INT 16H
        CMP AL,31H
        JE SECENEK_1
        CMP AL,32H
        JE SECENEK_2
        CMP AL,33H
        JE SECENEK_3
        CMP AL,34H
        JE SECENEK_4
        CMP AL,35H
        JE SECENEK_5
        CMP AL,36H
        JE SECENEK_6
```

```
67          CMP AL,37H
68          JE SECENEK_7
69          CMP AL,38H
70          JE SECENEK_8
71          JMP KLAVYE_OKU
72
73  SECENEK_1: JMP BASLANGIC
74  SECENEK_2: MOV CL,04H
75             JMP YUKLE
76  SECENEK_3: MOV CL,05H
77             JMP YUKLE
78  SECENEK_4: MOV CL,06H
79             JMP YUKLE
80  SECENEK_5: MOV CL,07H
81             JMP YUKLE
82  SECENEK_6: MOV CL,08H
83             JMP YUKLE
84  SECENEK_7: JMP YENIDEN_BASLAT
85  SECENEK_8: JMP KAPAT
86
87  YENIDEN_BASLAT:
88          MOV SI,M_MESAJ2
89          CALL SATIR_YAZ
90  ; Sicak baslangic
91          MOV AX,0040H
92          MOV ES,AX
93          MOV BX,0072H
94          MOV WORD [ES:BX],1234H
95          JMP 0FFFFH:0FFF0H
96
97  KAPAT:
98          MOV SI,M_MESAJ1
99          CALL SATIR_YAZ
100 ; APM ile bilgisayari kapat
101         MOV AX,5300H
102         MOV BX,0000H
103         INT 15H
104         PUSH AX
105         MOV AX,5308H
106         MOV BX,1
107         MOV CX,1
108         INT 15H
109         MOV AX,5308H
110         MOV BX,0FFFFH
111         MOV CX,1
112         INT 15H
113         MOV AX,5301H
114         MOV BX,0000H
115         INT 15H
116         MOV AX,530EH
117         MOV BX,0000H
118         POP CX
119         INT 15H
120         MOV AX,530DH
121         MOV BX,1
122         MOV CX,1
```

```
123         INT 15H
124         MOV AX,530FH
125         MOV BX,1
126         MOV CX,1
127         INT 15H
128         MOV AX,5307H
129         MOV BX,1
130         MOV CX,3
131         INT 15H
132         JMP KAPAT
133
134 YUKLE:
135         MOV AH,02H
136         MOV AL,01H
137         MOV CH,00H
138         MOV DH,00H
139         MOV DL,00H
140         MOV BX,1000H
141         MOV ES,BX
142         MOV BX,0420H
143         INT 13H
144
145 SON:
146         DB 00EAH          ; JMP FAR
147         DW 0420H, 1000H   ; 1000:0420 -> Program
148         DB 151 DUP (00H)
149
150         include 'yordamlar.inc'
```

Kod 11.4: Örnek 11.4 – Genel menü yazılımı (p1.bin)

Bilgi Notu 11.2: Tam Kod

Bu yazılımın tam kaynak kodu, kitabın çevrim içi deposunda bulunabilir.

11.2.4 Hesap Makinası Yazılımı

Hesap makinası yazılımımız belleğin `0420H` göreli adresine yerleştirilecektir. `W`, `A`, `S`, `D` ve `ENTER` tuşları kullanılarak ekrandan sayılar seçilebilecek ve toplanabilecektir. Diğer aritmetik işlemlerin de eklenmesi oldukça basittir.

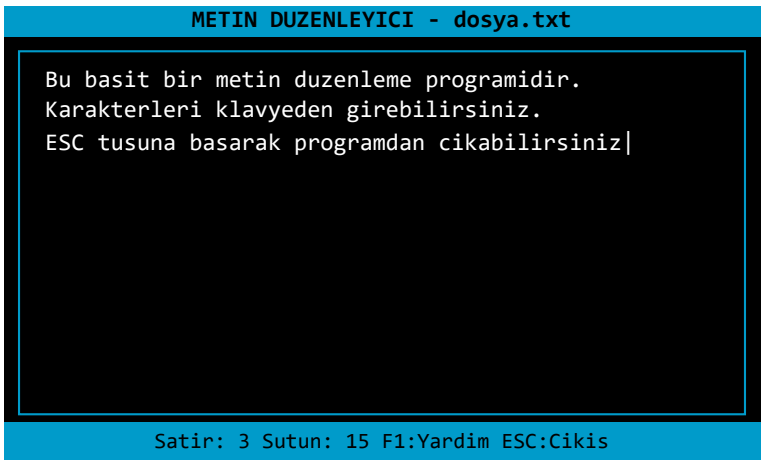
Şekil 11.1: Hesap makinası ekran görüntüsü. DOS metin kipinde çalışan basit bir hesap makinası arayüzü.

Bilgi Notu 11.3: Tam Kod

Bu yazılımın tam kaynak kodu, kitabın çevrim içi deposunda bulunabilir.

11.2.5 Metin Düzenleyici Yazılımı

Metin düzenleyici yazılımımız ekrana bir çerçeve çizecek ve bu çerçeve içine klavyeden karakterler girilebilecektir. Metin düzenleyici yazılımımız da hesap makinası yazılımımız gibi belleğin `0420H` göreli adresine kaydedilecek ve oradan çalıştırılacaktır.



Şekil 11.2: Metin düzenleyici ekran görüntüsü. DOS metin kipinde çalışan basit bir metin düzenleyici arayüzü.

Bilgi Notu 11.4: Tam Kod

Bu yazılımın tam kaynak kodu, kitabın çevrim içi deposunda bulunabilir.

11.2.6 Resim Çizdirme Yazılımı

Çizdirmek istediğimiz resim 400×400 çözünürlükte bir resim olsun. Bellekte 1024×768 çözünürlük ve 24 bit renkte her piksel için dört bayt kullanıldığından bize $400 \times 400 \times 4 = 640\,000$ bayt gerekmektedir. Bu değer yaklaşık 0,6 MB alana karşılık gelir. Resmimizin doğrusal olmaması nedeniyle ekrana doğru biçimde yazdırabilmek için $1024 \times 399 \times 4 + 400 \times 4 = 1\,635\,904$ bayt $\approx$ 1,56 MB yol katetmemiz gerekmektedir.

8086 mikroişlemcisinin adresleme için 20 iğne kullandığından tam olarak 1 MB adresleme yapabildiğinden daha önce bahsetmiştik. 1,56 MB'lık adreslemenin üstesinden gelmek için vidyo belleğinin öbek (bank) özelliğini kullanacağız. Vidyo belleği önceden belirlenmiş parçalara bölünmüştür ve bu öbeklere ayrı ayrı resim yazabiliriz.

Bilgi Notu 11.5: Tam Kod

Bu yazılımın tam kaynak kodu, kitabın çevrim içi deposunda bulunabilir.

11.2.7 Seri İletişim Alıcı ve Verici Yazılımları

Seri iletişim alıcı yazılımı ve seri iletişim verici yazılımı önceki bölümde anlatılan 8250 çipini kullanarak iki bilgisayar arasında veri alışverişi yapar. Bu yazılımlar da `0420H` adresine yüklenerek çalıştırılır.

Bilgi Notu 11.6: Tam Kod

Bu yazılımın tam kaynak kodu, kitabın çevrim içi deposunda bulunabilir.

Bölüm Sonu Soruları

Alıştırma 11.1. Önyükleme sektörü yapısı: BIOS, disketten önyükleme yaparken hangi sektörü okur ve bu sektörü belleğin hangi adresine yükler? İmza baytlarının bu sektördeki konumu (bayt numaraları) nedir ve bu baytlar doğru değilse ne olur?

Alıştırma 11.2. Bellek haritası: Bu bölümdeki basit işletim sisteminin bellek düzenini açıklayınız. Önyükleme yazılımı, genel menü ve uygulama yazılımları bellekte hangi adreste bulunmaktadır? Her bölge için segment:ofset çiftini yazınız.

Alıştırma 11.3. Yeni menü seçeneği ekleme: Genel menü yazılımına (p1.bin) "9 – ZAMAN GOSTER" adlı yeni bir seçenek ekleyiniz. Bu seçenek seçildiğinde INT 1AH kesmesini kullanarak ekrana geçerli saati yazacak bir yordam çağırılmalıdır. Genel menü yazılımında yapılması gereken değişiklikleri listeleyiniz.

Alıştırma 11.4. Hesap makinası genişletme: Hesap makinası yazılımı şu an yalnızca toplama işlemi yapmaktadır. Çıkarma işlemini de desteklemek için yazılımda yapılması gereken değişiklikleri açıklayınız. Kullanıcı arabiriminde hangi ek tuş veya menü öğesi eklenmelidir?

Alıştırma 11.5. Metin düzenleyici klavye işleme: Metin düzenleyici yazılımı klavyeden karakter okumak için INT 16H kesmesini kullanmaktadır. Geri silme (Backspace) tuşuna basıldığında imleç bir sola gitmeli ve önceki karakter silinmelidir. Bu işlevi gerçekleştirmek için kullanılacak tarama kodu nedir ve vidyo belleğinde hangi değer yazılmalıdır?

A

x86 Komutlarının Listesi

Burada 8086 mikroişlemcisinin komut kümesini ve ardından da diğer x86 mikroişlemcilerinin getirdiği yeni komutları bulabilirsiniz.

8086 Komutları

Tablo A.1: *8086 komut kümesi.*

Komut	İşlenen(ler)	Tanım
MOV	İŞLENEN1, İŞLENEN2	İşlenen2'yi işlenen1'e kopyalar. CS ve IP değiştirilemez; kesim yazmaçları arası doğrudan aktarım yapılamaz.
PUSH	İŞLENEN1	SP 2 azaltılır; SS:[SP]'ye işlenen1 kaydedilir.
POP	İŞLENEN1	SS:[SP] işlenen1'e kaydedilir; SP 2 artırılır.
XCHG	İŞLENEN1, İŞLENEN2	İki işlenenin değerlerini karşılıklı olarak değiştirir.
IN	AL/AX, KAPI	Belirtilen kapıdan AL veya AX yazmacına veri aktarır.
OUT	KAPI, AL/AX	AL veya AX yazmacındaki veriyi belirtilen kapıya iletir.
XLAT	[BX]	AL'ye DS:[BX+AL] adresindeki baytı aktarır.
XLATB	–	XLAT komutunun işlenen almayan kısa biçimi.
LEA	İŞLENEN1, İŞLENEN2	İşlenen2'nin etkin adresini işlenen1'e kaydeder.
LDS	İŞLENEN1, İŞLENEN2	Bellekteki ikili kelimeyi bir genel amaçlı yazmaç ve DS'ye kaydeder.
LES	İŞLENEN1, İŞLENEN2	LDS gibi çalışır; DS yerine ES kullanır.

Komut	İşlenen(ler)	Tanım
LAHF	–	SF, ZF, AF, PF, CF bitlerini AH'ye kaydeder.
SAHF	–	AH'deki bayrak bitlerini bayrak yazmacına kaydeder.
PUSHF	–	Bayrak yazmacını yığına atar.
POPF	–	Yığından bayrak yazmacını geri yükler.
ADD	İŞLENEN1, İŞLENEN2	İşlenen1 + işlenen2 ⟶ işlenen1.
ADC	İŞLENEN1, İŞLENEN2	İşlenen1 + işlenen2 + CF ⟶ işlenen1.
INC	İŞLENEN1	İşlenen1'in değerini 1 artırır; CF'yi etkilemez.
AAA	–	AL'deki değeri paketlenmemiş ondalık değere çevirir.
DAA	–	AL'deki değeri paketlenmiş ondalık değere çevirir.
SUB	İŞLENEN1, İŞLENEN2	İşlenen1 − işlenen2 ⟶ işlenen1.
SBB	İŞLENEN1, İŞLENEN2	İşlenen1 − işlenen2 − CF ⟶ işlenen1.
DEC	İŞLENEN1	İşlenen1'in değerini 1 azaltır.
NEG	İŞLENEN1	İşlenen1'in 2'ye tümleyenini alır.
CMP	İŞLENEN1, İŞLENEN2	İşlenenleri karşılaştırır; bayrakları günceller, değerleri değiştirmez.
AAS	–	Çıkarma sonrası ASCII düzenleme.
DAS	–	Çıkarma sonrası ondalık düzenleme.
MUL	İŞLENEN1	İşaretsiz çarpma: AL × işlenen1 (bayt) ⟶ AX; AX × işlenen1 (kelime) ⟶ DX:AX.
IMUL	İŞLENEN1	İşaretli çarpma.
AAM	–	Çarpma sonrası ASCII düzenleme; AL'yi 10'a böler, bölüm AH, kalan AL.
DIV	İŞLENEN1	İşaretsiz bölme: AX ÷ işlenen1 (bayt) ⟶ AL bölüm, AH kalan.
IDIV	İŞLENEN1	İşaretli bölme.
AAD	–	Bölme öncesi ASCII düzenleme.
CBW	–	AL'deki baytı AX'e işaret genişlemeli olarak kopyalar.
CWD	–	AX'deki kelimeyi DX:AX'e işaret genişlemeli kopyalar.
NOT	İŞLENEN1	İşlenen1'in tüm bitlerini tersine çevirir.
SHL/SAL	İŞLENEN1, İŞLENEN2	Lojik/Aritmetik sola öteleme.
SHR	İŞLENEN1, İŞLENEN2	Lojik sağa öteleme.
SAR	İŞLENEN1, İŞLENEN2	Aritmetik sağa öteleme (işaret bitini korur).

Komut	İşlenen(ler)	Tanım
ROL	İŞLENEN1, İŞLENEN2	Sola döndürme.
ROR	İŞLENEN1, İŞLENEN2	Sağa döndürme.
RCL	İŞLENEN1, İŞLENEN2	CF ile birlikte sola döndürme.
RCR	İŞLENEN1, İŞLENEN2	CF ile birlikte sağa döndürme.
AND	İŞLENEN1, İŞLENEN2	Mantıksal VE; sonuç işlenen1'e aktarılır.
TEST	İŞLENEN1, İŞLENEN2	AND gibi çalışır; sonucu kaydetmez, bayrakları günceller.
OR	İŞLENEN1, İŞLENEN2	Mantıksal VEYA.
XOR	İŞLENEN1, İŞLENEN2	Özel VEYA.
REP	İŞLENEN1	MOVSB/W, LODSB/W, STOSB/W komutlarını CX kez tekrarlar.
REPE/REPZ	İŞLENEN1	ZF=1 olduğu sürece tekrarlar.
REPNE/REPNZ	İŞLENEN1	ZF=0 olduğu sürece tekrarlar.
MOVSB/MOVSW	–	DS:[SI]'daki bayt/kelimeyi ES:[DI]'ya kopyalar; SI ve DI güncellenir.
CMPSB/CMPSW	–	DS:[SI] ile ES:[DI]'yi karşılaştırır; bayrakları günceller.
SCASB/SCASW	–	AL/AX ile ES:[DI]'yi karşılaştırır.
LODSB/LODSW	–	DS:[SI]'dan AL/AX'e yükler; SI güncellenir.
STOSB/STOSW	–	AL/AX'i ES:[DI]'ya kaydeder; DI güncellenir.
CALL	İŞLENEN1	Yordama dallanır; IP (kesimler arası: CS:IP) yığına kaydedilir.
JMP	İŞLENEN1	Koşulsuz dallanma.
RET/RETF	[İŞLENEN1]	Yordamdan geri dönüş; RETF kesimler arası.
JE/JZ	ETİKET	ZF=1 ise atla.
JL/JNGE	ETİKET	SF≠OF ise atla.
JLE/JNG	ETİKET	SF≠OF veya ZF=1 ise atla.
JB/JNAE	ETİKET	CF=1 ise atla.
JBE/JNA	ETİKET	CF=1 veya ZF=1 ise atla.
JP/JPE	ETİKET	PF=1 ise atla.
JO	ETİKET	OF=1 ise atla.
JS	ETİKET	SF=1 ise atla.
JNE/JNZ	ETİKET	ZF=0 ise atla.
JNL/JGE	ETİKET	SF=OF ise atla.
JNLE/JG	ETİKET	SF=OF ve ZF=0 ise atla.
JNB/JAE	ETİKET	CF=0 ise atla.

Komut	İşlenen(ler)	Tanım
JNBE/JA	ETİKET	CF=0 ve ZF=0 ise atla.
JNP/JPO	ETİKET	PF=0 ise atla.
JNO	ETİKET	OF=0 ise atla.
JNS	ETİKET	SF=0 ise atla.
LOOP	ETİKET	CX kez tekrarlar; her atlamada CX azaltılır.
LOOPZ/LOOPE	ETİKET	CX≠0 ve ZF=1 olduğu sürece döngü.
LOOPNZ/LOOPNE	ETİKET	CX≠0 ve ZF=0 olduğu sürece döngü.
JCXZ	ETİKET	CX=0 ise atla.
INT	İŞLENEN1	Yazılım kesmesi; bayraklar, CS, IP yığına kaydedilir.
INTO	–	OF=1 ise INT 04H.
IRET	–	Kesmeden geri dönüş; IP, CS ve bayraklar yığından geri yüklenir.
CLC	–	CF=0.
CMC	–	CF'yi ters çevirir.
STC	–	CF=1.
CLD	–	DF=0.
STD	–	DF=1.
CLI	–	IF=0; donanım kesmeleri devre dışı.
STI	–	IF=1; donanım kesmeleri devrede.
HLT	–	İşlemciyi durdurur.
WAIT	–	Yardımcı işlemci işini bitirene kadar bekler.
ESC	ANLIK VERİ, YER	Diğer işlemcilere veriyoluna erişme hakkı tanır.
LOCK	–	Birden fazla işlemcinin aynı bellek bölgesini değiştirmesini engeller.
NOP	–	İşlem yok (XCHG AL,AL ile eşdeğer; 3 saat darbesi).

80186/80188 ile Eklenen Komutlar

Tablo A.2: *80186/80188 ile eklenen komutlar.*

Komut	İşlenenler	Tanım
BOUND	KAYNAK, SINIR	KAYNAK dizin değerini belirtilen alt/üst sınırlarla karşılaştırır; taşılırsa INT 5.
ENTER	AYIR, SEVİYE	Yüksek seviyeli dil yordamı için yığını ayarlar.
LEAVE	–	ENTER ile ayrılan yığıt bölgesini serbest bırakır.
INS	HEDEF, KAPI	KAPI'dan ES:[DI] adresine veri aktarır.
OUTS	KAPI, KAYNAK	DS:[SI]'dan belirtilen kapıya veri iletir.

Komut	İşlenenler	Tanım
POPA	–	DI, SI, BP, SP, DX, CX, BX, AX yazmaçlarını yığından geri yükler.
PUSHA	–	AX, BX, CX, DX, SP, BP, SI, DI yazmaçlarını yığına atar.

80286 ile Eklenen Komutlar

Tablo A.3: *80286 ile eklenen komutlar.*

Komut	İşlenenler	Tanım
ARPL	HEDEF, KAYNAK	HEDEF'in RPL bitlerini KAYNAK'ın RPL bitleri ile karşılaştırır; gerekirse günceller.
CLTS	–	MSR'deki TS bayrağını sıfırlar.
LAR	HEDEF, KAYNAK	Erişim yetkilerini HEDEF'e yükler.
STR	HEDEF	Görev yazmacını HEDEF'e yazar.
SLDT	HEDEF	LDT yazmacını HEDEF'e kaydeder.
SGDT	HEDEF	GDT yazmacını HEDEF'e kaydeder.
LTR	KAYNAK	Görev yazmacına KAYNAK'ı yazar.
LSL	HEDEF, SEÇİCİ	Bölüt sınırını HEDEF'e yükler.
LMSW	KAYNAK	KAYNAK'ı MSW'ye atar.
SMSW	HEDEF	MSW'yi HEDEF'e yazar.
SIDT	HEDEF	IDT yazmacını HEDEF'e kaydeder.
VERR	SEÇİCİ	Seçicinin okuma yetki seviyesini denetler.
LIDT	KAYNAK	IDT yazmacına KAYNAK'ı yazar.
LLDT	KAYNAK	LDT yazmacına KAYNAK'ı yazar.
VERW	SEÇİCİ	Seçicinin yazma yetki seviyesini denetler.
LGDT	KAYNAK	GDT yazmacına KAYNAK'ı yazar.

80386 ile Eklenen Komutlar

Tablo A.4: *80386 ile eklenen komutlar (seçilmiş).*

Komut	İşlenenler	Tanım
BSF/BSR	HEDEF, KAYNAK	İleri/Geri yönde bit taraması.
BT/BTC/BTR/BTS	HEDEF, KAYNAK	Bit sınaması ve değiştirme işlemleri.
CDQ	–	EAX'i EDX:EAX'e işaret genişlemeli dönüştürür.
CWDE	–	AX'i EAX'e işaret genişlemeli kopyalar.

Komut	İşlenenler	Tanım
IRETD	–	32 bitlik kesmeden geri dönüş.
JECXZ	ETİKET	ECX=0 ise atla.
LFS/LGS/LSS	HEDEF, KAYNAK	32 bitlik bellek işaretçisini ilgili yazmaç çiftine yükler.
MOVSX	HEDEF, KAYNAK	İşaret genişlemeli taşıma.
MOVZX	HEDEF, KAYNAK	Sıfır genişlemeli taşıma.
POPF/POPFD	–	Kelime/ikili kelimeyi yığından bayrak yazmacına yazar.
PUSHF/PUSHFD	–	Bayrak yazmacını yığına atar.
LODSD	–	DS:[SI]'dan EAX'e 32 bit yükler.
STOS	–	Akümülatördeki veriyi ES:[DI] adresine yükler.
SETxx	HEDEF	Koşul sağlanırsa HEDEF baytını 1, aksi halde 0 yapar (SETAE, SETB, SETE ...).
SHLD/SHRD	HEDEF, KAYNAK, SAYAÇ	Çift hassasiyetli sola/sağa kaydırma.

80486 ile Eklenen Komutlar

Tablo A.5: *80486 ile eklenen komutlar.*

Komut	İşlenenler	Tanım
BSWAP	YAZMAÇ	32 bitlik yazmacın bayt sırasını tersine çevirir (big-endian ↔ little-endian).
CMPXCHG	HEDEF, KAYNAK	Akümülatör ile HEDEF'i karşılaştırır; eşitse HEDEF'e KAYNAK yazar.
INVD	–	İşlemci iç önbelleğini boşaltır.
INVLPG	–	TLB'nin bir sayfasını geçersizleştirir.
WBINVD	–	Önbelleği diske geri yazar ve boşaltır.

Pentium ile Eklenen Komutlar

Tablo A.6: *Pentium ile eklenen komutlar.*

Komut	İşlenenler	Tanım
CMPXCHG8B	HEDEF	64 bitlik karşılaştırma ve değiştirme.
RDMSR	–	ECX ile belirtilen MSR'yi EDX:EAX'e okur.
RDPMC	–	Performans sayaçlarını okur.
RDTSC	–	Zaman damgası sayacını EDX:EAX'e yazar.

Komut	İşlenenler	Tanım
WRMSR	–	EDX:EAX değerini ECX ile belirtilen MSR'ye yazar.
CPUID	GİRDİ	İşlemci tanımlama bilgisi verir (EAX, EBX, ECX, EDX).

Pentium Pro (P6) ile Eklenen Komutlar

Tablo A.7: *Pentium Pro (P6) ile eklenen koşullu taşıma komutları.*

Komut	İşlenenler	Tanım
CMOVA	HEDEF, KAYNAK	CF=0 ve ZF=0 ise KAYNAK → HEDEF.
CMOVAE	HEDEF, KAYNAK	CF=0 ise KAYNAK → HEDEF.
CMOVB	HEDEF, KAYNAK	CF=1 ise KAYNAK → HEDEF.
CMOVE	HEDEF, KAYNAK	ZF=1 ise KAYNAK → HEDEF.
CMOVG	HEDEF, KAYNAK	ZF=0 ve SF=OF ise KAYNAK → HEDEF.
CMOVGE	HEDEF, KAYNAK	SF=OF ise KAYNAK → HEDEF.
CMOVL	HEDEF, KAYNAK	SF≠OF ise KAYNAK → HEDEF.
CMOVLE	HEDEF, KAYNAK	ZF=1 veya SF≠OF ise KAYNAK → HEDEF.
CMOVNA	HEDEF, KAYNAK	CF=1 veya ZF=1 ise KAYNAK → HEDEF.
CMOVNAE	HEDEF, KAYNAK	CF=1 ise KAYNAK → HEDEF.
CMOVNB	HEDEF, KAYNAK	CF=0 ise KAYNAK → HEDEF.
CMOVNBE	HEDEF, KAYNAK	CF=0 ve ZF=0 ise KAYNAK → HEDEF.
CMOVNC	HEDEF, KAYNAK	CF=0 ise KAYNAK → HEDEF.
CMOVNE	HEDEF, KAYNAK	ZF=0 ise KAYNAK → HEDEF.
CMOVNG	HEDEF, KAYNAK	ZF=1 veya SF≠OF ise KAYNAK → HEDEF.
CMOVNGE	HEDEF, KAYNAK	SF≠OF ise KAYNAK → HEDEF.
CMOVNL	HEDEF, KAYNAK	SF=OF ise KAYNAK → HEDEF.
CMOVNLE	HEDEF, KAYNAK	ZF=0 ve SF=OF ise KAYNAK → HEDEF.
CMOVNZ	HEDEF, KAYNAK	ZF=0 ise KAYNAK → HEDEF.
CMOVNP	HEDEF, KAYNAK	PF=0 ise KAYNAK → HEDEF.
CMOVNS	HEDEF, KAYNAK	SF=0 ise KAYNAK → HEDEF.
CMOVO	HEDEF, KAYNAK	OF=1 ise KAYNAK → HEDEF.
CMOVP	HEDEF, KAYNAK	PF=1 ise KAYNAK → HEDEF.
CMOVPE	HEDEF, KAYNAK	PF=1 ise KAYNAK → HEDEF.
CMOVPO	HEDEF, KAYNAK	PF=0 ise KAYNAK → HEDEF.
CMOVS	HEDEF, KAYNAK	SF=1 ise KAYNAK → HEDEF.
CMOVZ	HEDEF, KAYNAK	ZF=1 ise KAYNAK → HEDEF.
FCOMI	ST(0), ST(i)	FPU karşılaştırması; bayrakları EFLAGS'a yazar.
FCOMIP	ST(0), ST(i)	FCOMI + ST(0)'ı yığından atar.
FUCOMI	ST(0), ST(i)	Sırasız FPU karşılaştırması.
FUCOMIP	ST(0), ST(i)	FUCOMI + ST(0)'ı yığından atar.

Pentium III (SSE) ile Eklenen Komutlar

Pentium III ile birlikte SSE (Streaming SIMD Extensions) komut kümesi eklenmiştir. Bu komutlar 128 bitlik XMM yazmaçlarını kullanarak tek hassasiyetli kayan noktalı sayılar üzerinde SIMD işlemleri yapar.

Tablo A.8: *Pentium III ile eklenen SSE komutlarının listesi.*

Komut	Açıklama
Veri aktarımı komutları	
`MOVAPS`	Hizalanmış tam paket tek hassasiyetli kayan noktalı değerleri XMM yazmacına/yazmacından taşır.
`MOVUPS`	Hizalanmamış tam paket tek hassasiyetli kayan noktalı değerleri taşır.
`MOVHPS`	Paketin üst 64 bitini bellek ile XMM yazmacı arasında taşır.
`MOVLPS`	Paketin alt 64 bitini bellek ile XMM yazmacı arasında taşır.
`MOVMSKPS`	Paketin her elemanının işaret bitini tamsayı yazmacına aktarır.
`MOVSS`	Tek bir tek hassasiyetli kayan noktalı değeri taşır.
Aritmetik komutlar	
`ADDPS`	Paket tek hassasiyetli kayan noktalı değerleri toplar.
`ADDSS`	Tek bir tek hassasiyetli kayan noktalı değeri toplar.
`SUBPS`	Paket tek hassasiyetli kayan noktalı değerleri çıkarır.
`SUBSS`	Tek bir tek hassasiyetli kayan noktalı değeri çıkarır.
`MULPS`	Paket tek hassasiyetli kayan noktalı değerleri çarpar.
`MULSS`	Tek bir tek hassasiyetli kayan noktalı değeri çarpar.
`DIVPS`	Paket tek hassasiyetli kayan noktalı değerleri böler.
`DIVSS`	Tek bir tek hassasiyetli kayan noktalı değeri böler.
`RCPPS`	Paket elemanlarının yaklaşık tersini hesaplar.
`RCPSS`	Tek bir elemanın yaklaşık tersini hesaplar.
`SQRTPS`	Paket elemanlarının karekökünü hesaplar.
`SQRTSS`	Tek bir elemanın karekökünü hesaplar.
`MAXPS`	Paket elemanları arasından büyük olanı seçer.
`MAXSS`	Tek iki eleman arasından büyük olanı seçer.
`MINPS`	Paket elemanları arasından küçük olanı seçer.
`MINSS`	Tek iki eleman arasından küçük olanı seçer.
`RSQRTPS`	Paket elemanlarının karekökünün yaklaşık tersini hesaplar.
`RSQRTSS`	Tek bir elemanın karekökünün yaklaşık tersini hesaplar.
Karşılaştırma komutları	
`CMPPS`	Paket tek hassasiyetli değerleri karşılaştırır; sonucu maske olarak yazar.

Tablo A.8 — devamı

Komut	Açıklama
`CMPSS`	Tek bir tek hassasiyetli değeri karşılaştırır; sonucu maske olarak yazar.
`COMISS`	Tek hassasiyetli değerleri sıralı karşılaştırır; EFLAGS güncellenir.
`UCOMISS`	Tek hassasiyetli değerleri sırasız karşılaştırır; EFLAGS güncellenir.
Mantıksal komutlar	
`ANDPS`	Paket kayan noktalı değerlere bit düzeyinde VE işlemi uygular.
`ANDNPS`	Hedefe bit düzeyinde DEĞİL-VE işlemi uygular.
`ORPS`	Paket kayan noktalı değerlere bit düzeyinde VEYA işlemi uygular.
`XORPS`	Paket kayan noktalı değerlere bit düzeyinde XOR işlemi uygular.
Karıştırma ve açma komutları	
`SHUFPS`	Paket elemanlarını belirtilen düzene göre karıştırır.
`UNPCKHPS`	İki XMM yazmacının üst 64 bitlerini çift katmanlı olarak açar.
`UNPCKLPS`	İki XMM yazmacının alt 64 bitlerini çift katmanlı olarak açar.
Dönüştürme komutları	
`CVTPI2PS`	Paket 32-bit tamsayıları paket tek hassasiyetli kayan noktaya dönüştürür.
`CVTSI2SS`	Tek bir 32-bit tamsayıyı tek hassasiyetli kayan noktaya dönüştürür.
`CVTPS2PI`	Paket tek hassasiyetli kayan noktalıyı paket tamsayıya dönüştürür.
`CVTTPS2PI`	Paket tek hassasiyetliyi paket tamsayıya keserek dönüştürür.
`CVTSS2SI`	Tek hassasiyetli kayan noktalıyı tamsayıya dönüştürür.
`CVTTSS2SI`	Tek hassasiyetli kayan noktalıyı tamsayıya keserek dönüştürür.
Durum ve önbellek komutları	
`LDMXCSR`	MXCSR denetim/durum yazmacını bellekten yükler.
`STMXCSR`	MXCSR denetim/durum yazmacını belleğe kaydeder.
`PREFETCHT0`	Veriyi T0 düzey önbelleklere önceden getirir.
`PREFETCHT1`	Veriyi T1 ve daha üst düzey önbelleklere önceden getirir.
`PREFETCHT2`	Veriyi T2 ve daha üst düzey önbelleklere önceden getirir.

Tablo A.8 — devamı

Komut	Açıklama
`PREFETCHNTA`	Veriyi önbellek kirliliğini en aza indirecek biçimde önceden getirir.
`SFENCE`	Tüm yazmadan sonraki yazma işlemlerini sıralı hale getirir (depo önbellek önlemi).

B

Derleme Araçları

Burada FASM ve NASM derleyicilerinin, ilki Windows ikincisi Linux ortamında olmak üzere, bu kitaptaki örnekleri uygulamaya yetecek kadar temelleri gösterilecektir. Derleyicilerin bu temel özellikler dışında birçok özelliği bulunmaktadır. Yüksek seviyeli dillerde bulunan IF-ELSE dallanmalarına benzer dallanmaları çevirici dili koduna uygulamak mümkündür. Daha ayrıntılı bilgi için derleyicilerin kullanma kılavuzlarını inceleyiniz.

FASM Kullanımı

FASM (Flat Assembler) bedava dağıtılan bir x86 çevirici-bağlayıcısıdır. Kullanımı çoğu çeviriciye göre kolaydır; girilmesi gereken çeviriciye özel kodlar yoktur. Bağlayıcısını kendi içinde bulundurduğundan fazladan bir bağlayıcı kullanımı gerektirmez. Intel söz dizimi kullanılır.

Küçük bir kod yazalım ve bunu FASM yardımıyla çevirip bağlayalım. Kodumuz DOS ekranına "Merhaba Dünya!" yazısını yazdırsın.

```
        ORG 0100H
        JMP BASLANGIC
MESAJ           DB 'MERHABA DUNYA!'
MESAJ_UZUNLUK   DW $-MESAJ
BASLANGIC:
        MOV AH,03H              ; Imleç konumunu al
        INT 10H                 ; INT 10H çagir
        MOV AH,13H              ; Dizgi yazdir
        MOV AL,01H              ; Yazma kipi 01
        MOV BH,00H              ; Sayfa 0
        MOV BL,0FH              ; Beyaz yazi - siyah arkaplan
        MOV SI,MESAJ_UZUNLUK
        MOV CX,[SI]             ; Mesaj uzunlugu
        PUSH CS
        POP ES
        MOV BP,MESAJ            ; Tampon adresi ES:BP
        INT 10H                 ; INT 10H çagir
SON:
        JMP SON
```

Kod B.1: Örnek B.1 – FASM ile "Merhaba Dünya!" (DOS, .com)

Bu kodu FASM programının düzenleyicisine yazalım ve `File→Save` seçeneğiyle `ekb1.asm` olarak kaydedelim. Ardından `Run→Run` seçeneğiyle programımızı çevirelim ve çalıştıralım. DOS ekranına "Merhaba Dünya!" yazısı yazdırılacaktır. Dosyanın bulunduğu klasörde `ekb1.com` adlı çalıştırılabilir bir dosya oluşturulduğu görülebilir.

Linux'ta NASM Kullanımı

NASM (Netwide Assembler) bedava dağıtılan bir x86 derleyicisidir ve Intel söz dizimini kullanır. AT&T söz dizimi kullanılmak istenirse Linux ile dağıtılan `as` çeviricisi kullanılabilir.

Linux'ta kod yazmadan önce birkaç konudan bahsetmemiz gerekiyor. Linux, korumalı kipte çalışan 32 bitlik bir işletim sistemidir; DOS ise gerçek kipte çalışan 16 bitlik bir işletim sistemidir.

32 bitlik x86 işlemcilerde yazmaçlar genişletilmiştir: AX yazmacının 32 bitlik biçimi EAX'tir. EAX'e veri yazıldığında hem üst hem de alt 16 bit yazılır; alt 16 bit AX'tir. EBX, ECX ve EDX için de aynı durum geçerlidir.

Linux korumalı kipte çalıştığından BIOS kesmelerine doğrudan erişilememektedir. Bunun yerine `INT 80H` kesmesine parametreler verilerek Linux sistem çağrıları kullanılır. Linux sistem çağrıları 16 bitlik yazmaçlar yerine EAX, EBX, ECX, EDX (ayrıca ESI, EDI ve EBP) gibi 32 bitlik yazmaçları parametre olarak alır.

```
1  SECTION .DATA
2          mesaj:          DB 'Merhaba Dunya!',0AH
3          mesaj_uzunluk:  EQU $-mesaj
4  SECTION .TEXT
5          GLOBAL _start
6
7  _start:
8          MOV EAX,04H         ; "sys_write"
9          MOV EBX,01H         ; Dosya tanim., 01 = ekran
10         MOV ECX,mesaj       ; Yazilacak mesajin adresi
11         MOV EDX,mesaj_uzunluk ; Uzunluk
12         INT 80H             ; Sistem cagrisi
13
14         MOV EAX,01H         ; "sys_exit"
15         MOV EBX,00H         ; 00 hata kodu
16         INT 80H             ; Sistem cagrisi
```

Kod B.2: Örnek B.2 – NASM ile "Merhaba Dünya!" (Linux 32-bit)

Kodu NASM ile çevirmek için terminale şu komutu yazınız:

```
nasm -f elf ekb2.asm
```

Çevrim tamamlandığında `ekb2.o` adlı nesne dosyası oluşturulur. Bu dosyayı bağlamak için `ld` bağlayıcısını kullanınız:

```
ld -s -o ekb2 ekb2.o
```

Çalıştırmak için:

```
./ekb2
```

Terminal ekranına "Merhaba Dünya!" yazısı yazdırılacaktır. `_start` etiketi, `ld` bağlayıcısının istediği başlangıç noktasını açıkça belirtir.

FAT12 Dosya Sistemi

Bilgi Notu B.1: İlgili Bölüm

Bu ek, Bölüm 11'deki basit işletim sistemi projesinde kullanılan FAT12 dosya sistemi hakkında ek bilgi sunmaktadır.

Disketlerde kullanılan FAT12 dosya sisteminde her klasör girdisinde bir öznitelik baytı bulunur:

Tablo B.1: *FAT12 klasör girdisi öznitelik baytı.*

7	6	5	4	3	2	1	0
paylaşılabilir	kullanımda değil	arşiv	klasör	veri birimi etiketi	sistem	gizli	salt okunur

Zaman ve tarih alanlarının bit düzeyinde yapısı:

Tablo B.2: *FAT12 zaman alanı (16 bit).*

15–11	10–5	4–0
Saat	Dakika	Saniye (÷2)

Tablo B.3: *FAT12 tarih alanı (16 bit).*

15–9	8–5	4–0
Yıl (1980'den itibaren)	Ay	Gün

C

DOS ve Linux Kesmeleri

Burada çok kullanılan DOS kesmeleri ve Linux sistem çağrıları hakkında bilgi vereceğiz.

DOS Kesmeleri

INT 21H – Girdi/Çıktı Kesmesi

AH = 01H: Standart giriş biriminden (klavye) bir karakter okur. Karakter ASCII kodu AL yazmacına kaydedilir ve konsol ekranında görüntülenir.

AH = 02H: Standart çıkış birimine (ekran) bir karakter yazar. DL yazmacına yazılacak karakter girilir.

AH = 05H: Yazıcıya karakter yazar. Yazılacak karakter DL yazmacına girilir.

AH = 06H (çıkış): DL yazmacındaki karakteri konsola yazdırır (DL ≠ FFH).

AH = 06H (giriş): DL = FFH ise konsoldan karakter okur; karakter yoksa ZF=1 ve AL=00H, aksi halde ZF=0 ve AL'ye ASCII değer yazılır.

AH = 07H: Klavyeden karakter okur; ekrana basmaz.

AH = 08H: 07H ile aynı; girilen karakter ekranda görünmez.

AH = 09H: DS:DX adresindeki dizgiyi '$' karakterine kadar ekrana yazar.

AH = 0AH: Klavyeden girilen karakterleri DS:DX adresindeki tampona kaydeder.

AH = 0BH: STDIN durumunu öğrenir; AL=00H basılmış karakter yok, AL=FFH karakter var.

AH = 0CH: Tamponu temizler ve STDIN okuma işlevini çalıştırır.

AH = 0DH: Diski yeniden başlatır.

AH = 0EH: DL yazmacındaki koda göre varsayılan sürücüyü seçer (A=00H, B=01H vb.).

AH = 19H: Varsayılan sürücüyü AL yazmacına yazar.

AH = 25H: Kesme vektörünü ayarlar; AL=kesme numarası, DS:DX=kesme kodu.

AH = 2AH: Sistem tarihini alır; CX=yıl, DH=ay, DL=gün, AL=haftanın günü.

AH = 2BH: Sistem tarihini ayarlar; CX=yıl, DH=ay, DL=gün.

AH = 2CH: Sistem zamanını alır; CH=saat, CL=dakika, DH=saniye, DL=saniyenin yüzdesi.

AH = 2DH: Sistem zamanını ayarlar; CH=saat, CL=dakika, DH=saniye, DL=saniyenin yüzdesi.

AH = 2EH: Denetleme bayrağını ayarlar; AL=00H kapalı, AL=01H açık.
AH = 30H: DOS sürümünü alır; AL=ana sürüm, AH=alt sürüm.
AH = 35H: Kesme vektörünü alır; AL=kesme numarası, ES:BX=kesme kodu.
AH = 36H: Boş disk alanını alır; DL=sürücü numarası.
AH = 39H: Alt klasör oluşturur (MKDIR); DS:DX=klasör adresi.
AH = 3AH: Alt klasörü siler (RMDIR); DS:DX=klasör adresi (klasör boş olmalı).
AH = 3BH: Klasör değiştirir (CHDIR); DS:DX=hedef klasör adresi.
AH = 3CH: Yeni dosya oluşturur veya boyutu sıfırlar; CX=özellikler, DS:DX=dosya adresi.
AH = 3DH: Dosya açar; AL=erişim kuralları, DS:DX=dosya adresi. Başarılıysa AX=dosya etiketi.
AH = 3EH: Açılmış dosyayı kapatır; BX=dosya etiketi.
AH = 3FH: Açılmış dosyayı okur; BX=dosya etiketi, CX=okunacak bayt sayısı, DS:DX=hedef adres.
AH = 40H: Açılmış dosyaya yazar; BX=dosya etiketi, CX=yazılacak bayt sayısı, DS:DX=kaynak adres.
AH = 41H: Dosya siler; DS:DX=dosya adresi. (FAT tablosundaki girişi siler; dosya içeriği korunabilir.)
AH = 42H: Dosya konumunu ayarlar; AL=başlangıç (00H=mevcut, 01H=başlangıç, 02H=son), BX=dosya etiketi, CX:DX=uzaklık.
AH = 43H: Dosya özelliklerini okur (AL=00H) veya yazar (AL=01H); CX=özellikler, DS:DX=dosya adresi.
AH = 47H: Kullanımda olan klasörü öğrenir; DL=sürücü numarası, DS:SI=klasör adres tamponu.
AH = 4CH: Yazılımı geri dönüş koduyla sonlandırır; AL=geri dönüş kodu.
AH = 4DH: Geri dönüş kodunu alır; AH=sonlandırılma tipi, AL=geri dönüş kodu.
AH = 54H: Denetleme bayrağını alır; AL=00H kapalı, AL=01H açık.
AH = 56H: Dosyayı yeniden adlandırır; DS:DX=eski ad, ES:DI=yeni ad.
AH = 57H: Dosyanın son yazım tarihini ve saatini öğrenir (AL=00H) veya değiştirir (AL=01H); BX=dosya etiketi, CX=zaman, DX=tarih.

`INT 33H` – Fare Kesmesi

AX = 00H: Fare sürücüsü yüklü mu? AX=FFFFH yüklü, AX=0000H yüklü değil; BX=düğme sayısı.
AX = 01H: Fare imlecini göster.
AX = 02H: Fare imlecini gizle.
AX = 03H: Fare konumu ve düğme durumunu öğren; CX=yatay pozisyon, DX=dikey pozisyon.
Düğme durum sözcüğü bit anlamları:

Tablo C.1: *Fare düğme durum sözcüğü.*

15–2	1	0
Kullanımda değil	Sağ düğme	Sol düğme

AX = 04H: Fare imlecini konumlandır; CX=yatay, DX=dikey.

AX = 05H: Düğme basılma durumunu öğren; BX=0 sol düğme, BX=1 sağ düğme.
AX = 06H: Düğme bırakılma durumunu öğren.
AX = 07H: Yatay hareket sınırlarını ayarla; CX=minimum, DX=maximum.
AX = 08H: Dikey hareket sınırlarını ayarla; CX=minimum, DX=maximum.

Linux Kesmeleri (Sistem Çağrıları)

Linux 32 bitlik korumalı kipte çalıştığından BIOS kesmelerine doğrudan erişilememektedir. Bunun yerine `INT 80H` kesmesi sistem çağrısı mekanizması olarak kullanılır. Sistem çağrısı numarası EAX yazmacına, parametreler ise EBX, ECX, EDX, ESI, EDI ve EBP yazmaçlarına yazılır.

`INT 80H` Sistem Çağrıları

EAX = 01H (`sys_exit`): Yazılımı sonlandırır; EBX=hata kodu.

EAX = 03H (`sys_read`): Dosyadan okur; EBX=dosya tanımlayıcısı, ECX=tampon adresi, EDX=bayt sayısı.

EAX = 04H (`sys_write`): Dosyaya yazar; EBX=dosya tanımlayıcısı (1=ekran), ECX=tampon adresi, EDX=bayt sayısı.

EAX = 05H (`sys_open`): Dosya açar; EBX=dosya adı adresi, ECX=erişim bayrakları, EDX=yetki bayrakları.

Sık kullanılan dosya erişim bayrakları:

- O_RDONLY = 0 (salt okunur)
- O_WRONLY = 1 (salt yazılır)
- O_RDWR = 2 (okuma/yazma)
- O_CREAT = 64 (dosya oluştur)
- O_TRUNC = 512 (varsa sil yeniden oluştur)
- O_APPEND = 1024 (sona ekle)

Sık kullanılan dosya yetki bayrakları:

- S_IRUSR = 400 (sahibi okuyabilir)
- S_IWUSR = 200 (sahibi yazabilir)
- S_IRGRP = 040 (grup okuyabilir)
- S_IROTH = 004 (diğerleri okuyabilir)

EAX = 06H (`sys_close`): Dosya kapatır; EBX=dosya tanımlayıcısı.

EAX = 08H (`sys_creat`): Dosya oluşturur; EBX=dosya adı adresi, ECX=yetki bayrakları.

EAX = 0CH (`sys_chdir`): Klasör değiştirir; EBX=hedef klasör adresi.

EAX = 0DH (`sys_time`): Sistem zamanını alır (epoch'tan saniye); EBX=zaman değişkeni adresi (NULL olabilir), EAX=saniye sayısı.

EAX = 13H (`sys_lseek`): Dosya konumunu ayarlar; EBX=dosya tanımlayıcısı, ECX=uzaklık, EDX=başlangıç (0=başlangıç, 1=mevcut konum, 2=son).

EAX = 19H (`sys_stime`): Sistem zamanını ayarlar; EBX=epoch'tan saniye sayısının adresi.

EAX = 26H (`sys_rename`): Dosyayı yeniden adlandırır; EBX=eski ad adresi, ECX=yeni ad adresi.

EAX = 27H (`sys_mkdir`): Klasör oluşturur; EBX=klasör adı adresi, ECX=yetki bayrakları.
EAX = 28H (`sys_rmdir`): Klasör siler; EBX=klasör adı adresi.

D

BIOS Kesmeleri

Aşağıda çokça kullanılan BIOS kesmeleri hakkında bilgi bulabilirsiniz.

BIOS ve Donanım Kesme Vektörleri

Tablo D.1: *BIOS ve donanım kesme vektörleri.*

Kesme	Tanım
INT 00H	İşlemci: Sıfıra bölme
INT 01H	İşlemci: Hata ayıklama için adım adım yürütme
INT 02H	İşlemci: NMI (Non-Maskable Interrupt – Maskelenemeyen Kesme)
INT 03H	İşlemci: Hata ayıklama için duraksama noktası
INT 04H	İşlemci: Taşma hatası
INT 05H	Ekranı yazdır (Print Screen)
INT 08H	IRQ0: Sistem zamanlayıcısı tarafından çağrılır
INT 09H	IRQ1: Klavye tarafından çağrılır
INT 0BH	IRQ3: COM2 (2. seri kapı) tarafından çağrılır
INT 0CH	IRQ4: COM1 (1. seri kapı) tarafından çağrılır
INT 0DH	IRQ5: Sabit disket denetleyicisi (PC/XT) veya 2. paralel kapı (AT) tarafından çağrılır
INT 0EH	IRQ6: Disket sürücü denetleyicisi tarafından çağrılır
INT 0FH	IRQ7: 1. paralel kapı (yazıcı) tarafından çağrılır
INT 10H	Video hizmetleri
INT 11H	Çevre birimi yüklendi
INT 12H	Konvansiyonel bellek boyutunu al
INT 13H	Alt seviye disk hizmetleri
INT 14H	Seri I/O
INT 15H	Çeşitli kesmeler
INT 16H	Klavye

Kesme	Tanım
INT 17H	Yazıcı hizmetleri
INT 18H	INT 19H başarısız olursa kullanılır
INT 19H	POST'tan sonra işletim sistemini yüklemek için kullanılır
INT 1AH	RTC (Real Time Clock – Gerçek Zamanlı Saat) hizmetleri
INT 1BH	"Break" tuşuna basıldığında çağrılır
INT 1CH	Yazılım zamanlayıcı kesmesi; INT 08H tarafından çağrılır
INT 1DH	Adres işaretçisi: VPT (Video Parameter Table)
INT 1EH	Adres işaretçisi: DPT (Diskette Parameter Table)
INT 1FH	Adres işaretçisi: VGCT (Video Graphics Character Table)
INT 41H	Adres işaretçisi: FDPT 1 (1. Sabit Disk Parametre Tablosu)
INT 46H	Adres işaretçisi: FDPT 2 (2. Sabit Disk Parametre Tablosu)
INT 4AH	RTC tarafından alarm için çağrılır
INT 70H	IRQ8: RTC tarafından çağrılır
INT 74H	IRQ12: Fare tarafından çağrılır
INT 75H	IRQ13: Matematiksel ek işlemci tarafından çağrılır
INT 76H	IRQ14: Birinci IDE denetleyicisi tarafından çağrılır
INT 77H	IRQ15: İkinci IDE denetleyicisi tarafından çağrılır

Bilgi Notu D.1: NMI – Maskelenemeyen Kesme

Yukarıdaki tabloda listelenen tüm kesmeler ve bazı diğer kesmeler maskelenebilir (geçici olarak ihmal edilebilen) kesmelerdir. Yalnız NMI (Non-Maskable Interrupt – Maskelenemeyen Kesme) adından da anlaşıldığı gibi maskelenemez; aşırı önemli durumlarda ve genellikle büyük sistem hatalarının sonucunda ortaya çıkar. NMI'lerin ihmal edilmesi mümkün değildir. Bellekten gelen verilerde eşlik hatasının oluşması buna bir örnektir.

`INT 10H` – Video Hizmetleri

Tablo D.2: *`INT 10H` video hizmetleri alt yordamları.*

AH	İşlev
00H	Video tipini ayarla. AL=tip.
01H	İmleç şeklini ayarla. CH=başlama satırı, CL=bitiş satırı. İmleci yok etmek için CH'ye 20H yazılabilir.
02H	İmleç pozisyonunu ayarla. BH=sayfa numarası, DH=y, DL=x.
03H	İmleç pozisyonunu ve şeklini al. BH=sayfa. Çıktı: CH=başlama satırı, CL=bitiş satırı, DL=x, DH=y.
05H	Görüntü sayfasını ayarla. AL=görüntü sayfası.
06H	Ekranı yukarı kaydır veya temizle. AL=kaydırılacak satır sayısı (0=temizle), BH=özellik, CL/CH=sol-üst, DL/DH=sağ-alt.
07H	Ekranı aşağı kaydır veya temizle. AL=kaydırılacak satır sayısı (0=temizle).

AH	İşlev
08H	İmleç pozisyonundaki karakteri ve özelliklerini oku. BH=sayfa. Çıktı: AL=karakter, AH=özellik.
09H	İmleç pozisyonuna karakter yaz. AL=karakter, BH=sayfa, BL=özellik, CX=tekrar sayısı.
0AH	İmleç pozisyonuna karakteri yaz (özelliği koruyarak). AL=karakter, BH=sayfa.
0BH	Kenarlık rengini ayarla. BH=00H, BL=renk.
0EH	Karakter yazdır (imleci sonra yeniden konumlandırarak). AL=karakter, BH=sayfa.
0FH	Video tipi ayarlarını al. Çıktı: AH=sütun sayısı, AL=video tipi, BH=sayfa.
13H	Dizgi yazdır. AL=yazma kipi, BH=sayfa, BL=özellik, CX=uzunluk, DH=satır, DL=sütun, ES:BP=dizgi adresi.

VGA renk kodları:

Tablo D.3: *VGA 16 renk kodları.*

Hex	İkili	Renk	Hex	İkili	Renk
0	0000	Siyah	8	1000	Koyu gri
1	0001	Mavi	9	1001	Açık mavi
2	0010	Yeşil	A	1010	Açık yeşil
3	0011	Camgöbeği	B	1011	Açık camgöbeği
4	0100	Kırmızı	C	1100	Açık kırmızı
5	0101	Eflatun	D	1101	Açık eflatun
6	0110	Kahverengi	E	1110	Sarı
7	0111	Açık gri	F	1111	Beyaz

`INT 11H` – Çevre Birimi Durumu

AX yazmacındaki dönen değerin bit anlamları:

Tablo D.4: *`INT 11H` çıktı AX yazmacı bit anlamları.*

AX Bitleri	Anlam
15–14	Bağlı yazıcı sayısı
13	1: Seri yazıcı bağlı; 0: Seri yazıcı yok
12	1: Oyun adaptörü bağlı; 0: Yok
11–9	Yüklü RS232 kartı sayısı
8	1: DMA var; 0: DMA yok
7–6	Bağlı disket sürücü sayısı
5–4	Video kipi
3–2	Kullanımda değil
1	1: Matematiksel yardımcı işlemci mevcut
0	1: Disket sürücüsünde disket var

`INT 13H` – Alt Seviye Disk Hizmetleri

Tablo D.5: *`INT 13H` disk hizmetleri alt yordamları.*

AH	İşlev
00H	Yeniden başlat. DL=sürücü numarası. Hata var ise CF=1.
01H	Sürücü durumunu oku. DL=sürücü numarası. Çıktı: AH=durum, CF=hata.
02H	Diskten oku. AL=sektör sayısı, CH=silindir, CL=sektör, DH=başlık, DL=sürücü, ES:BX=tampon. Çıktı: AH=durum, CF=hata.
03H	Diske yaz. AL=sektör sayısı, CH=silindir, CL=sektör, DH=başlık, DL=sürücü, ES:BX=tampon. Çıktı: AH=durum, CF=hata.
04H	Sektör sınama. Parametreler 02H ile aynı.
05H	Formatla. AL=sektör sayısı, CH=pist, DH=başlık, DL=sürücü, ES:BX=tampon.

`INT 14H` – Seri Kapı Hizmetleri

Tablo D.6: *`INT 14H` seri kapı hizmetleri alt yordamları.*

AH	İşlev
00H	Seri kapıyı başlat. AL=ayar değeri, DX=kapı numarası.
01H	Seri kapıdan karakter gönder. AL=veri, DX=kapı numarası. Çıktı: AH=durum (00H=başarılı).
02H	Seri kapıdan karakter al. DX=kapı numarası. Çıktı: AL=alınan veri.
03H	Seri kapı durumunu oku. DX=kapı numarası. Çıktı: AX=durum.

INT 16H – Klavye Hizmetleri

Tablo D.7: *INT 16H klavye hizmetleri alt yordamları.*

AH	İşlev
00H	Klavyeden basılan karakteri okur; bir tuşa basılana kadar bekler. Çıktı: AL=ASCII değeri, AH=tarama kodu.
01H	Girdi durumunu okur; beklemez. ZF=0 ise tuşa basılmış, ZF=1 ise basılmamış.
02H	Denetim tuşlarının durumunu okur. Çıktı: AL yazmacının bitleri hangi denetim tuşlarına basıldığını belirtir.
10H	Genişletilmiş karakter oku.
11H	Genişletilmiş girdi durumunu oku.
12H	Genişletilmiş denetim karakter tuşlarının durumunu oku.

INT 17H – Yazıcı Hizmetleri

Tablo D.8: *INT 17H yazıcı hizmetleri alt yordamları.*

AH	İşlev
00H	Bir karakter yazdırır. AL=karakter. Çıktı: AH=yazıcı durum bilgisi.
01H	Yazıcıyı hazırlar.
02H	Yazıcının durum bilgilerini okur. Çıktı: AH=yazıcı durum bilgisi.

INT 1AH – RTC Hizmetleri

Tablo D.9: *INT 1AH gerçek zamanlı saat hizmetleri alt yordamları.*

AH	İşlev
00H	Sistem zamanlayıcı zamanını oku. Çıktı: CX=üst bitler, DX=alt bitler, AL=24 saat aşıldı mı (1=evet).
01H	Sistem zamanlayıcı zamanını ayarla. CX=üst bitler, DX=alt bitler.
02H	RTC zamanını oku. Çıktı: CH=saat (BCD), CL=dakika (BCD), DH=saniye (BCD), CF=hata.
03H	RTC zamanını ayarla. CH=saat (BCD), CL=dakika (BCD), DH=saniye (BCD), DL=yaz saati (1=açık).
04H	RTC tarihini oku. Çıktı: CH=yüzyıl (BCD), CL=yıl (BCD), DH=ay (BCD), DL=gün (BCD).

AH	İşlev
05H	RTC tarihini ayarla. CH=yüzyıl (BCD), CL=yıl (BCD), DH=ay (BCD), DL=gün (BCD).
06H	RTC alarmını ayarla. CH=saat (BCD), CL=dakika (BCD), DH=saniye (BCD). CF=hata.
07H	RTC alarmını kapat.
08H	Otomatik açılma zamanını ayarla. CH=saat (BCD), CL=dakika (BCD), DH=saniye (BCD).
09H	Alarm zamanını ve durumunu oku.
0AH	Sistem zamanlayıcı tarihini oku. Çıktı: CX=1 Ocak 1980'den beri geçen gün sayısı.
0BH	Sistem zamanlayıcı tarihini ayarla. CX=1 Ocak 1980'den beri geçen gün sayısı.

E

ASCII Kodları ve Klavye Tarama Kodları

ASCII Tablosu

Tablo E.1: *ASCII karakter tablosu. Satır: onaltılık üst basamak (0x...), Sütun: alt basamak.*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT	LF	VT	FF	CR	SO	SI
1	DLE	DC1	DC2	DC3	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS	RS	US
2	SP	!	”	#	$	%	&	’	(	)	*	+	,	-	.	/
3	0	1	2	3	4	5	6	7	8	9	:	;	<	=	>	?
4	@	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
5	P	Q	R	S	T	U	V	W	X	Y	Z	[	\	]	^	_
6	‘	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
7	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	DEL

Denetim Karakterlerinin Tanımları

Tablo E.2: *ASCII denetim karakterlerinin tanımları.*

Karakter	Tanım
NUL (null – sıfır)	Herhangi bir karaktere basılmadığında iletilen denetim karakteri.
SOH (start of heading)	Veri iletiminde başlığın başını simgeler.
STX (start of text)	Veri iletiminde başlığın sonu ve metinin başını simgeler.
ETX (end of text)	Veri iletiminde metinin sonunu simgeler.
EOT (end of transmission)	Veri iletiminde bir iletim bloğunun sonunu simgeler.
ENQ (enquiry – sorgu)	Veri iletiminde uç birimlerin durumlarını sorgulamada kullanılır.

Karakter	Tanım
ACK (acknowledge – alındılama)	Veri iletiminde bir verinin alındığını belirtmek için kullanılır.
BEL (bell – zil)	Uç birimini zil ile uyarmada kullanılır.
BS (backspace – geri adım)	İmleci bir sola kaydırmada kullanılır.
TAB (horizontal tab – yatay sekme)	İmleci yatay eksende önceden belirlenmiş bir konuma göndermede kullanılır.
LF (line feed – satır besle)	Yeni bir satıra geçilmesini sağlar.
VT (vertical tab – dikey sekme)	İmleci dikey eksende önceden belirlenmiş bir konuma göndermede kullanılır.
FF (form feed – sayfa besle)	Yeni bir sayfa ile beslenmesini sağlar.
CR (carriage return – satır başı)	İmlecin satır başına gitmesini sağlar.
SO (shift out – özel koda geçiş)	ASCII karakterleri yerine özel karakterlerin kullanılmasını sağlar.
SI (shift in – özel koddan çıkış)	Özel karakterlerin yerine ASCII karakterlerini geri getirir.
DLE (data link escape – veri bağı çıkış)	İki birim arasındaki veri iletişiminin sonlandırılmasının istendiğini belirtir.
DC1 (device control 1)	Bir sistemin uzaktan kontrolünü sağlar.
DC2 (device control 2)	Bir sistemin uzaktan kontrolünü sağlar.
DC3 (device control 3)	Bir sistemin uzaktan kontrolünü sağlar.
DC4 (device control 4)	Bir sistemin uzaktan kontrolünü sağlar.
NAK (negative acknowledge – negatif alındılama)	Bir verinin alınamadığını belirtmek için kullanılır.
SYN (synchronous idle – eşzamanlama)	Sistemlerin eşzamanlanmasına yardımcı olan denetim karakteri.
ETB (end of transmission block)	Bir iletim bloğunun sonuna ulaşıldığını belirtir.
CAN (cancel – iptal)	İşlenmekte olan bir komutu sonlandırır.
EM (end of medium – ortam sonu)	Veri ortamının sonuna gelindiğini belirtir.
SUB (substitute – yedek)	Hatalı bir veri alındığında hatalı veriyi değiştirmede kullanılır.
ESC (escape – çıkış)	Sistemden çıkış veya sonraki karakterlerin anlamlarını değiştirmede kullanılır.
FS (file separator – dosya ayırıcı)	Dosya olarak tanımlanan verileri diğer verilerden ayırmak için kullanılır.
GS (group separator – grup ayırıcı)	Grup olarak tanımlanan verileri ayırmak için kullanılır.
RS (record separator – tutanak ayırıcı)	Tutanak olarak tanımlanan verileri ayırmak için kullanılır.
US (unit separator – birim ayırıcı)	Birim olarak tanımlanan verileri ayırmak için kullanılır.

Karakter	Tanım
SP (space – boşluk)	İmleci bir sağa kaydırır.
DEL (delete – silmek)	Yazılmış bir karakterin silinmesinde kullanılır.

Klavye Tarama Kodları

Aşağıdaki tabloda tuşların Normal, Shift, Ctrl ve Alt tuşuyla birlikte basıldığında oluşan onaltılık tarama kodları verilmektedir. Çizgi (–) o bileşimin geçerli bir tarama kodu üretmediğini gösterir.

Tablo E.3: *Klavye tarama kodları (onaltılık).*

Tuş	Normal	Shift	Ctrl	Alt	Tuş	Normal	Shift	Ctrl	Alt
A	1E	1E	1E	1E	]	1B	1B	1B	–
B	30	30	30	30	;	27	27	–	–
C	2E	2E	2E	2E	’	28	28	–	–
D	20	20	20	20	‘	29	29	–	–
E	12	12	12	12	\	2B	2B	2B	–
F	21	21	21	21	,	33	33	–	–
G	22	22	22	22	.	34	34	–	–
H	23	23	23	23	/	35	35	–	–
I	17	17	17	17	F1	3B	54	5E	68
J	24	24	24	24	F2	3C	55	5F	69
K	25	25	25	25	F3	3D	56	60	6A
L	26	26	26	26	F4	3E	57	61	6B
M	32	32	32	32	F5	3F	58	62	6C
N	31	31	31	31	F6	40	59	63	6D
O	18	18	18	18	F7	41	5A	64	6E
P	19	19	19	19	F8	42	5B	65	6F
Q	10	10	10	10	F9	43	5C	66	70
R	13	13	13	13	F10	44	5D	67	71
S	1F	1F	1F	1F	F11	85	87	89	8B
T	14	14	14	14	F12	86	88	8A	8C
U	16	16	16	16	SAĞ OK	4D	4D	74	9D
V	2F	2F	2F	2F	SOL OK	4B	4B	73	9B
W	11	11	11	11	YUKARI OK	48	48	8D	98
X	2D	2D	2D	2D	AŞAĞI OK	50	50	91	A0
Y	15	15	15	15	BS	0E	0E	0E	0E
1	02	02	–	78	DEL	53	53	93	A3
2	03	03	03	79	END	4F	4F	75	9F
3	04	04	–	7A	ESC	01	01	01	01
4	05	05	–	7B	HOME	47	47	77	97
5	06	06	–	7C	INS	52	52	92	A2
6	07	07	–	7D	PG UP	49	49	84	99
7	08	08	–	7E	PG DN	51	51	76	A1
8	09	09	–	7F	ENTER	1C	1C	1C	A6
9	0A	0A	–	80	SPACE	39	39	39	39
0	0B	0B	–	81	TAB	0F	0F	94	A5
-	0C	0C	–	82					
=	0D	0D	–	83					
[	1A	1A	1A	–					